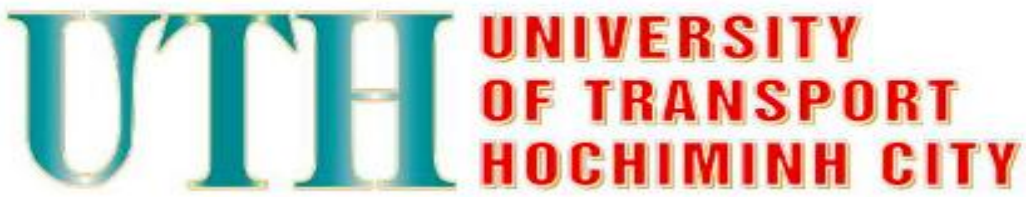
BỘ XÂY DỰNG

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM

VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO



HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

Mã học phần: 012012100304

Tên đề tài: QUẢN LÝ HỒ SƠ SINH VIÊN

Giảng viên hướng dẫn: Trần Anh Quân

Sinh viên thực hiện

Họ tên sinh viên	*Mã số sinh viên*
Nguyễn Hồ Diễm My	075306001575
Nguyễn Hồ Hùng Phương	083206006476
Hồ Thị Ngọc Mỹ	075306009542
Huỳnh Gia Huy	079205020248
Huỳnh Thái Vỹ	052206006365

2026

LỜI MỞ ĐẦU

Trong thời đại công nghệ thông tin phát triển mạnh mẽ, việc ứng dụng công nghệ vào công tác quản lý dữ liệu ngày càng trở nên cần thiết và đóng vai trò quan trọng trong nhiều lĩnh vực của đời sống. Đặc biệt, trong lĩnh vực giáo dục, số lượng sinh viên ngày càng lớn khiến việc quản lý thông tin bằng phương pháp thủ công trở nên khó khăn, tốn nhiều thời gian và dễ xảy ra sai sót. Vì vậy, việc xây dựng một hệ thống cơ sở dữ liệu khoa học, chính xác và thuận tiện là một yêu cầu cần thiết đối vớicác cơ sở giáo dục.

Xuất phát từ những yêu cầu thực tế trên, nhóm chúng em lựa chọn đề tài **"Quản lý hồ sơ sinh viên"** để thực hiện trong môn **Hệ quản trị cơ sở dữ liệu**. Đề tài hướng đến việc xây dựng một hệ thống cơ sở dữ liệu phục vụ cho việc quản lý thông tin sinh viên một cách khoa học và hiệu quả. Hệ thống tập trung quản lý các thông tin cơ bản của sinh viên, đồng thời tổ chức dữ liệu liên quan đến khoa, lớp, môn học, quá trình đăng ký học và kết quả học tập.

Thông qua đề tài, nhóm chúng em có cơ hội vận dụng linh hoạt các kiến thức lý thuyết đã học vào bài toán thực tế - từ khâu phân tích, thiết kế mô hình dữ liệu quan hệ cho đến việc ứng dụng các kỹ thuật lập trình nâng cao như Stored Procedure, Function, Trigger, xử lý giao tác (Transaction) và giải quyết tranh chấp dữ liệu. Qua đó, nhóm không chỉ hiểu rõ hơn về quy trình xây dựng và quản lý một cơ sở dữ liệu chuẩn hóa, mà còn rèn luyện kỹ năng phát triển phần mềm, tạo nền tảng vững chắc cho việc học tập và thực hiện các dự án trong tương lai.

Mặc dù đã cố gắng hoàn thành đề tài một cách đầy đủ và chính xác, nhưng do thời gian thực hiện và kiến thức thực tế còn hạn chế, bài tiểu luận khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể hoàn thiện đề tài tốt hơn.

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện bài tiểu luận môn Hệ quản trị cơ sở dữ liệu với đề tài "Quản lý hồ sơ sinh viên", nhóm chúng em xin gửi lời cảm ơn chân thành đến giảng viên "Trần Anh Quân" môn "Hệ quản trị cơ sở dữ liệu" đã tận tình giảng dạy, hướng dẫn và truyền đạt những kiến thức cần thiết trong suốt quá trình học tập. Những kiến mà giảng viên cung cấp đã giúp nhóm có nền tảng để thực hiện đề tài này.

Trong quá trình thực hiện đề tài "Quản lý hồ sơ sinh viên", nhóm chúng em đã có cơ hội vận dụng những kiến thức lý thuyết vào một bài toán thực tế. Qua đó, nhóm hiểu rõ hơn về cách phân tích yêu cầu, xác định các đối tượng cần quản lý, xây dựng mô hình dữ liệu, thiết kế các bảng và tổ chức dữ liệu sao cho hợp lý. Đây là những kiến thức và kỹ năng rất hữu ích đối với quá trình học tập cũng như công việc sau này.

Mặc dù nhóm đã cố gắng hoàn thành bài tiểu luận với tinh thần nghiêm túc và trách nhiệm, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên bài làm chắc chắn vẫn còn một số thiếu sót. Nhóm chúng em rất mong nhận được sự nhận xét, góp ý và hướng dẫn thêm từ giảng viên để có thể khắc phục những hạn chế và nâng cao kiến thức của bản thân.

Cuối cùng, nhóm chúng em xin kính chúc giảng viên luôn mạnh khỏe, thành công trong công việc và xin chân thành cảm ơn sự hướng dẫn, hỗ trợ trong suốt quá trình học tập và thực hiện đề tài.

Nhóm chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI MỞ ĐẦU 2
LỜI CẢM ƠN 3
MỤC LỤC 4
BẢNG PHÂN CÔNG NHIỆM VỤ 7
CHƯƠNG 1: TỔNG QUAN HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU VÀ NGÔN NGỮ T-SQL 8
1.1. TỔNG QUAN VỀ HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU 8
1.1.1. Một số khái niệm cơ bản 8
1.1.2. Góc nhìn dữ liệu 8
1.1.3. Yêu cầu đối với dữ liệu trong CSDL 8
1.1.4. Chức năng và lợi ích của Hệ quản trị CSDL 9
1.1.5. Kiến trúc và các thành phần của một HQTCSDL 10
1.1.6. Phân loại HQTCSDL 10
1.2. PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ MÔ HÌNH DỮ LIỆU 11
1.2.1. Phân tích bài toán và xác định các đối tượng (Thực thể) 11
1.2.2. Sơ đồ thực thể kết hợp và cấu trúc các bảng dữ liệu 12
1.2.3. Phân tích chức năng và nhiệm vụ cần thực hiện của hệ thống 13
CHƯƠNG 2: QUẢN LÝ GIAO TÁC 14
1. GIAO TÁC (TRANSACTION) 14
1.1. Khái niệm 14
1.2. Tính chất ACID của giao tác 15
1.3. Đơn vị dữ liệu 15
1.4. Các thao tác cơ bản của giao tác 16
1.5. Các trạng thái của giao tác 16
1.6. Thực thi chức năng trên giao diện và code 17
2. CÁC ĐỐI TƯỢNG TRONG SQL CỦA MỘT HỆ QUẢN TRỊ 30
2.1. VIEW 30
2.1.1. Khái niệm 30
2.1.2. Đặc điểm 30

2.1.3. Cách sử dụng 31
2.1.4. Ưu điểm và nhược điểm 32
2.1.5. Thực thi chức năng trên giao diện và code 33
2.2. STORED PROCEDURE 53
2.2.1 Khái niệm 53
2.2.2. Đặc điểm 53
2.2.3. Cách sử dụng 53
2.2.4. Ưu điểm và nhược điểm 54
2.2.5. Thực thi chức năng trên giao diện và code 54
2.3. FUNCTION 151
2.3.1. Khái niệm 151
2.3.2. Đặc điểm 151
2.3.3. Cách sử dụng 152
2.3.4. Ưu điểm và nhược điểm 153
2.3.5. Thực thi chức năng trên giao diện và code 154
2.4. TRIGGER 171
2.4.1. Khái niệm 171
2.4.2. Đặc điểm 171
2.4.3. Cách sử dụng 172
2.4.4. Ưu điểm và nhược điểm 173
2.4.5. Thực thi chức năng trên giao diện và code 174
CHƯƠNG 3: ĐIỀU KHIỂN TƯƠNG TRANH 191
3.1. Giới thiệu 191
3.2. Mất dữ liệu cập nhật (Lost Update) 191
3.2.1. Mô phỏng hiện tượng Lost Update 192
3.2.2. Khắc Phục Lost Update 209
3.3. Đọc dữ liệu rác (Dirty Read) 217
3.3.1. Mô phỏng hiện tượng Dirty Read 218
3.3.2. Khắc Phục Dirty Read 249
3.4. Không đọc lại được dữ liệu (Non-Repeatable Read) 262
3.4.1. Mô phỏng hiện tượng Non-Repeatable Read 262

3.4.2. Khắc phục Non-Repeatable Read 296
3.5. Hiện tượng bóng ma (Phantom Read) 321
3.5.1. Mô phỏng hiện tượng Phantom Read 321
3.5.2. Khắc phục Phantom Read 344
3.6. DEADLOCK 365
3.6.1. Mô phỏng DEADLOCK 366
3.6.2. Khắc phục DEADLOCK - ORDERING ALL THE ITEMS PROTOCOL 371
TÀI LIỆU THAM KHẢO 381

BẢNG PHÂN CÔNG NHIỆM VỤ

Họ tên sinh viên	Mã số sinh viên	Nhiệm vụ	Đánh giá và mức độ hoàn thành
Nguyễn Hồ Diễm My	075306001575	Phân công, Chương 2-Các đối tượng trong SQL: Function, Trigger. Làm word, powerpoint.	Đánh giá: Thực hiện đầy đủ theo sự phân công Mức độ hoàn thành: 100%
Nguyễn Hồ Hùng Phương	083206006476	Chương 3: Không đọc lại được dữ liệu (Non-repeatable Read), Hiện tượng bóng ma (Phantom). Làm word, powerpoint.	Đánh giá: Thực hiện đầy đủ theo sự phân công Mức độ hoàn thành: 100%
Hồ Thị Ngọc Mỹ	075306009542	Chương 2-Các đối tượng trong SQL: View, Stored procedure. Làm word, powerpoint.	Đánh giá: Thực hiện đầy đủ theo sự phân công Mức độ hoàn thành: 100%
Huỳnh Gia Huy	079205020248	Chương 3: Mất dữ liệu cập nhật (Lost Update), Đọc dữ liệu rác (Dirty Read). Làm word, powerpoint.	Đánh giá: Thực hiện đầy đủ theo sự phân công Mức độ hoàn thành: 100%
Huỳnh Thái vỹ	079205020248	Chương 2: Giao tác (transaction), Chương 3: Deadlock. Làm word, powerpoint.	Đánh giá: Thực hiện đầy đủ theo sự phân công Mức độ hoàn thành: 100%

CHƯƠNG 1: TỔNG QUAN HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU VÀ NGÔN NGỮ T-SQL

1.1. TỔNG QUAN VỀ HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

1.1.1. Một số khái niệm cơ bản

- Cơ sở dữ liệu (CSDL/Database): là tập hợp các thông tin có liên quan với nhau, được thiết kế nhằm làm giảm thiểu sự lặp lại dữ liệu.
- Hệ quản trị cơ sở dữ liệu (HQTCSDL/DBMS): là hệ thống gồm một CSDL và các thao tác trên CSDL đó, được xây dựng trên một nền tảng phần cứng, phần mềm và kiến trúc nhất định.
- Hệ cơ sở dữ liệu (HCSDL/DBS): là phần mềm cho phép xây dựng một HQTCSDL.

1.1.2. Góc nhìn dữ liệu

Dữ liệu trong CSDL được thể hiện các mức độ trừu tượng khác nhau, giúp tăng tính độc lập giữa cách thức lưu trữ, thiết kế và chương trình sử dụng dữ liệu:

- Mức vật lý (physical level): mức thấp nhất, mô tả cách dữ liệu được lưu trữ trên thiết bị lưu trữ.
- Mức luận lý (logic level): mô tả những thông tin được lưu trong CSDL và mối quan hệ giữa chúng.
- Mức view (view level): mức cao nhất, mô tả phần dữ liệu mà một nhóm người dùng cụ thể được quyền truy cập.

1.1.3. Yêu cầu đối với dữ liệu trong CSDL

Một CSDL tốt cần đảm bảo các đặc trưng: ít hoặc không trùng lắp dữ liệu; chia sẻ được cho nhiều người dùng mà không gây xung đột; đảm bảo an ninh, bảo mật; có khả năng khôi phục khi có sự cố; và đảm bảo tính độc lập dữ liệu.

Tính độc lập dữ liệu gồm hai loại:

- Độc lập luận lý: thay đổi lược đồ mức luận lý mà không ảnh hưởng đến lược đồ ngoài cũng như chương trình ứng dụng.
- Độc lập vật lý: thay đổi tổ chức vật lý của CSDL mà không ảnh hưởng đến lược đồ luận lý.

Nhờ đó, người dùng có thể thêm/bớt thuộc tính, bảng, thay đổi thiết bị lưu trữ hay cấu trúc chỉ mục mà không cần viết lại toàn bộ chương trình.

1.1.4. Chức năng và lợi ích của Hệ quản trị CSDL

HQTCSDL cung cấp các công cụ để xây dựng, khai thác và quản lý CSDL, thông qua:

- Xây dựng (sử dụng ngôn ngữ định nghĩa dữ liệu DDL - Data Definition Language): Để định nghĩa cấu trúc CSDL.
- Khai thác (sử dụng ngôn ngữ thao tác dữ liệu DML - Data Manipulation Language): Để truy vấn và thao tác dữ liệu.
- Quản lý: đảm bảo an toàn, bảo mật, điều khiển truy xuất đồng thời và khôi phục khi có sự cố.

Một số HQTCSDL phổ biến: MS SQL Server, Oracle, MySQL,…

Một số lợi ích của Hệ quản trị CSDL gồm:

- Độc lập dữ liệu
- Truy cập hiệu quả
- Toàn vẹn dữ liệu
- An ninh dữ liệu
- Hỗ trợ truy xuất đồng thời
- Khôi phục sau sự cố
- Giảm thời gian phát triển ứng dụng.

1.1.5. Kiến trúc và các thành phần của một HQTCSDL

Về kiến trúc, một HQTCSDL tiếp nhận yêu cầu từ nhiều nhóm người dùng (Naive Users, Casual Users, Application Programmers, Database Administrator) thông qua các giao diện khác nhau (Forms, DML Interface, CLI, DDL), sau đó xử lý qua bộ máy đánh giá truy vấn (Query Evaluation Engine gồm Parser, Optimizer, Plan Executor, Operator Evaluator) và bộ điều khiển truy xuất đồng thời (Transaction Manager, Lock Manager) trước khi thao tác trực tiếp với tệp dữ liệu, bộ đệm và ổ đĩa lưu trữ.

Các thành phần chính của một HQTCSDL bao gồm:

- Quản lý giao diện lập trình: cung cấp giao diện dòng lệnh hoặc đồ họa cùng ngôn ngữ lập trình CSDL như SQL, T-SQL.
- Quản lý dữ liệu ngữ nghĩa: khai báo lược đồ CSDL bằng các câu lệnh DDL và lưu trữ trong catalog.
- Quản lý an toàn và bảo mật: chứng thực, phân quyền người dùng và mã hóa dữ liệu để chống tấn công.
- Quản lý truy xuất đồng thời: đảm bảo nhiều giao tác (transaction) chạy đồng thời không làm mất tính nhất quán dữ liệu, sử dụng cơ chế khóa (lock) và bộ lập lịch (scheduler); đồng thời phải xử lý tình huống deadlock khi các giao tác chờ đợi lẫn nhau.
- Tối ưu hóa câu truy vấn: biên dịch câu truy vấn SQL thành cây phân tích, kiểm tra ngữ nghĩa, chuyển sang đại số quan hệ và sắp xếp lại các phép toán để tối ưu hiệu năng.
- Quản lý khôi phục sau sự cố: ghi nhận các thao tác trên CSDL để có thể khôi phục khi xảy ra sự cố.
- Quản lý lưu trữ: điều khiển việc đọc/ghi dữ liệu giữa bộ nhớ và thiết bị lưu trữ.

1.1.6. Phân loại HQTCSDL

HQTCSDL có thể được phân loại theo nhiều tiêu chí khác nhau:

- Theo mô hình dữ liệu: mô hình phân cấp (hierarchical), mô hình mạng (network), mô hình quan hệ (relational) và mô hình hướng đối tượng (object-oriented).
- Theo kiến trúc tính toán: tập trung (centralized), khách/chủ (client-server) và phân tán (distributed).
- Theo đặc tính: HQTCSDL thời gian thực, HQTCSDL chịu lỗi cao và HQTCSDL đa phương tiện.

Về lịch sử phát triển, HQTCSDL đã trải qua nhiều giai đoạn:

- Mô hình phân cấp và mạng vào thập niên 1960 (IBM IMS, IDS)
- Mô hình quan hệ thống trị vào thập niên 1970-1980 (Oracle, DB2, SQL Server, mysql)
- Mô hình hướng đối tượng vào thập niên 1990
- Sự bùng nổ của nosql và điện toán đám mây vào giai đoạn 2000-2010 (mongodb, Redis, AWS dynamodb)
- Các hệ newsql, AI-Driven Database trong giai đoạn hiện nay.

1.2. PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ MÔ HÌNH DỮ LIỆU

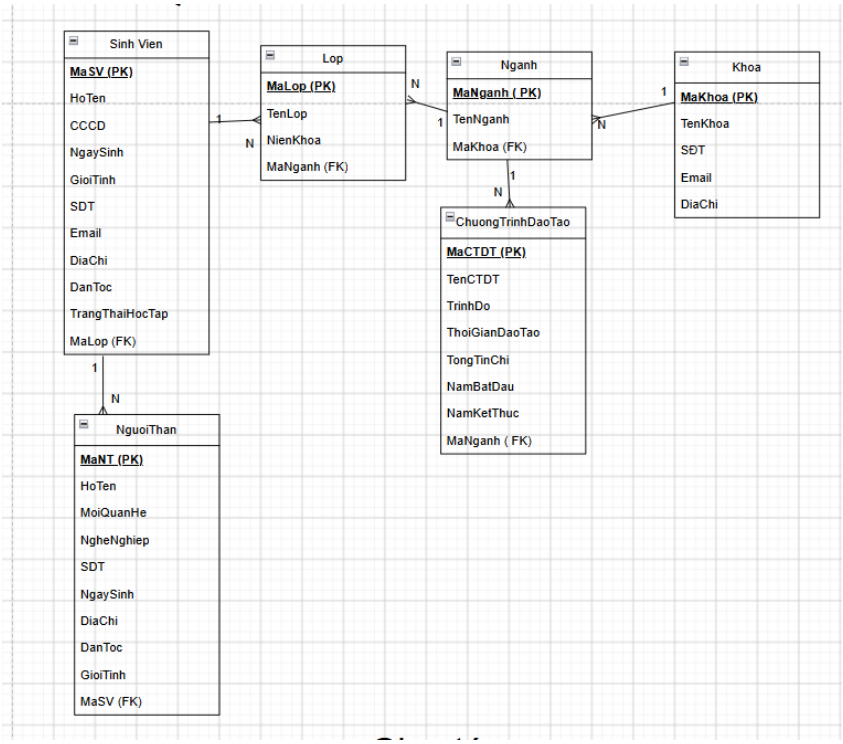
1.2.1. Phân tích bài toán và xác định các đối tượng (Thực thể)

Hệ thống **"Quản lý Hồ sơ Sinh viên"** được xây dựng nhằm tự động hóa công tác lưu trữ, tra cứu và quản lý toàn bộ thông tin liên quan đến sinh viên trong một cơ sở giáo dục. Dựa trên yêu cầu thực tế, hệ thống bao gồm các đối tượng (thực thể) chính sau đây:

1. **Khoa (Khoa):** Quản lý thông tin chung về các đơn vị khoa trực thuộc trường.
2. **Ngành (Nganh):** Quản lý các chuyên ngành đào tạo thuộc về từng Khoa cụ thể.
3. **Chương trình đào tạo (ChuongTrinhDaoTao):** Quản lý khung chương trình học chuẩn áp dụng cho từng Ngành.
4. **Lớp (Lop):** Quản lý thông tin về các lớp hành chính của sinh viên thuộc một Ngành nhất định.

5. **Sinh viên (Sinh Viên):** Quản lý hồ sơ cá nhân, thông tin liên lạc và trạng thái học tập của từng sinh viên.
6. **Người thân (NguoiThan):** Quản lý thông tin về gia đình/người giám hộ của sinh viên để phục vụ công tác liên lạc, quản lý hồ sơ nhân thân.

1.2.2. Sơ đồ thực thể kết hợp và cấu trúc các bảng dữ liệu



Từ sơ đồ thiết kế thực thể, cấu trúc các bảng dữ liệu được xác định như sau:

- Khoa (MaKhoa, TenKhoa, SDT, Email, DiaChi)
- Nganh (MaNganh, TenNganh, MaKhoa)
- ChuongTrinhDaoTao (MaCTDT, TenCTDT, TrinhDo, ThoiGianDaoTao, TongTinChi, NamBatDau, NamKetThuc, MaNganh)
- Lop (MaLop, TenLop, NienKhoa, MaNganh)
- SinhVien (MaSV, HoTen, CCCD, NgaySinh, GioiTinh, SDT, Email, DiaChi, DanToc, TrangThaiHoTap, MaLop)
- NguoiThan (MaNT, HoTen, MoiQuanHe, NgheNghiep, SDT, NgaySinh, DiaChi, DanToc, GioiTinh, MaSV)

1.2.3. Phân tích chức năng và nhiệm vụ cần thực hiện của hệ thống

Để hệ thống hoạt động hiệu quả, môt số chức năng cốt lõi được liệt kê như sau:

Quản lý danh mục cơ cấu tổ chức:

- **Quản lý Khoa/Ngành:** Thêm, sửa, xóa, tìm kiếm thông tin các khoa và ngành đào tạo.
- **Quản lý Chương trình đào tạo:** Thiết lập khung chương trình, số tín chỉ và thời gian đào tạo cho từng ngành.
- **Quản lý Lớp học:** Phân chia lớp theo khóa và ngành học.

Quản lý hồ sơ nhân sự (Sinh viên & Người thân):

- **Quản lý sinh viên:** Thêm mới hồ sơ sinh viên khi nhập học, cập nhật thông tin cá nhân, chỉnh sửa trạng thái học tập.
- **Quản lý thông tin người thân:** Thêm, cập nhật, xóa thông tin người thân/người giám hộ gắn với mã sinh viên.

Chức năng tra cứu và thống kê dữ liệu:

- **Tìm kiếm nâng cao:** Cho phép tìm kiếm sinh viên theo nhiều tiêu chí khác nhau.
- **Thống kê - Báo cáo:** Thống kê tổng số lượng sinh viên theo từng khoa, từng lớp, giới tính

Đảm bảo an toàn và toàn vẹn dữ liệu:

- Thiết lập các ràng buộc khóa ngoại (Foreign Key) chặt chẽ giữa các bảng để tránh tình trạng mồ côi dữ liệu (ví dụ: Không thể xóa Khoa khi vẫn còn Ngành trực thuộc, hoặc không thể xóa Lớp khi vẫn còn sinh viên đang theo học).
- Sử dụng các thủ tục lưu trữ (Stored Procedure), hàm (Function) và bẫy lỗi (Trigger) nhằm tối ưu hóa tốc độ truy vấn và đảm bảo quy tắc nghiệp vụ toàn vẹn hệ thống.

CHƯƠNG 2: QUẢN LÝ GIAO TÁC

Trong thực tế, hệ quản trị cơ sở dữ liệu (HQTCSDL) phải đáp ứng hai yêu cầu cơ bản: cho phép nhiều người dùng cùng khai thác cơ sở dữ liệu (CSDL) một cách đồng thời nhưng vẫn phải giải quyết được các tranh chấp phát sinh, đồng thời phải đảm bảo tính nhất quán của dữ liệu ngay cả khi có sự cố kỹ thuật xảy ra trong quá trình xử lý.

Có thể thấy rõ vấn đề này qua một số tình huống thực tế:

- **Hệ thống đặt vé máy bay**: khi hai hay nhiều hành khách cùng đặt một ghế trống, có thể xảy ra trường hợp nhiều hành khách đều "đặt được" dù thực tế chỉ còn một ghế. Hệ thống cần giải quyết tranh chấp để đảm bảo nhất quán dữ liệu.
- **Hệ thống ngân hàng**: khi chuyển tiền từ tài khoản A sang tài khoản B, có thể xảy ra sự cố khiến tiền đã bị trừ ở A nhưng chưa được cộng vào B; hoặc khi nhiều người cùng rút tiền từ một tài khoản, số tiền rút ra có thể vượt quá số dư thực có.
- **Hệ thống quản lý học sinh**: khi thêm một học sinh mới vào lớp, có thể xảy ra trường hợp học sinh đã được thêm vào bảng Học sinh nhưng sĩ số của Lớp học lại không được cập nhật tương ứng.

Những tình huống trên cho thấy sự cần thiết của khái niệm **giao tác (transaction)** và cơ chế **quản lý giao tác** trong HQTCSDL, nhằm đảm bảo dữ liệu luôn ở trạng thái nhất quán dù có nhiều người dùng thao tác đồng thời hay có sự cố xảy ra.

1. GIAO TÁC (TRANSACTION)

1.1. Khái niệm

Một **giao tác** là một đơn vị xử lý nguyên tố gồm một chuỗi các hành động đọc/ghi trên các đối tượng của CSDL, thường được ký hiệu là **T**. Giao tác được xử lý như một đơn vị logic duy nhất, và việc thực hiện một giao tác phải đảm bảo tính nhất quán của CSDL. Trong kiến trúc của một HQTCSDL, bộ phận điều khiển đồng thời đóng vai trò quản lý giao tác.

1.2. Tính chất ACID của giao tác

Một giao tác cần thỏa mãn bốn tính chất, thường được gọi tắt là **ACID**:

- **Tính nguyên tố (Atomicity)**: Hoặc toàn bộ hoạt động của giao tác được phản ánh đúng đắn vào CSDL, hoặc không có hoạt động nào được thực hiện cả. Ví dụ, trong giao tác chuyển khoản gồm hai bước (trừ tiền tài khoản A, cộng tiền tài khoản B), nếu có sự cố xảy ra ở bước 2 thì HQTCSDL phải có cơ chế khôi phục lại dữ liệu như ban đầu.
- **Tính nhất quán (Consistency)**: Khi một giao tác kết thúc (dù thành công hay thất bại), CSDL phải ở trạng thái nhất quán. Một giao tác đưa CSDL từ trạng thái nhất quán này sang một trạng thái nhất quán khác. Ví dụ: với giao tác chuyển tiền, tổng số dư của tài khoản A và B trước và sau giao tác luôn không đổi; hoặc sĩ số của một lớp học phải luôn bằng đúng số học sinh thực sự thuộc lớp đó.
- **Tính cô lập (Isolation)**: Một giao tác khi thực hiện sẽ không bị ảnh hưởng bởi các giao tác khác đang được thực hiện đồng thời với nó. Nói cách khác, dù các giao tác có thể đan xen nhau về mặt thời gian, kết quả cuối cùng phải tương đương với kết quả của một dãy thực hiện tuần tự nào đó.
- **Tính bền vững (Durability)**: Mọi thay đổi trên CSDL sau khi giao tác đã hoàn tất phải được ghi nhận bền vững vào thiết bị lưu trữ, dù sau đó có sự cố xảy ra (mất điện, hỏng hệ thống...) thì dữ liệu vẫn không bị mất.

1.3. Đơn vị dữ liệu

Đơn vị dữ liệu là khối lượng dữ liệu nhỏ nhất mà CSDL có thể thao tác (đọc/ghi). Đơn vị dữ liệu có thể là:

- Cả CSDL (database)
- Bảng (table)
- Mẫu tin (record)
- Trường (field)

Kích thước của đơn vị dữ liệu ảnh hưởng trực tiếp đến hiệu năng xử lý đồng thời: đơn vị càng nhỏ thì mức độ đồng thời càng cao nhưng chi phí quản lý càng lớn, và ngược lại.

1.4. Các thao tác cơ bản của giao tác

Giao tác thao tác với dữ liệu thông qua vùng đệm (buffer) trung gian trước khi ghi xuống đĩa (database), với hai lệnh cơ bản:

- **Read(X, t)**: đọc đơn vị dữ liệu X từ CSDL vào biến trung gian t (trong buffer).
- **Write(X, t)**: ghi giá trị t vào đơn vị dữ liệu X.

Ví dụ minh họa: giả sử có hai đơn vị dữ liệu A và B với ràng buộc A = B. Giao tác T thực hiện việc nhân đôi cả A và B (A = A*2; B = B*2) có thể được biểu diễn như sau:

T: Read(A, t); t := t*2; Write(A, t); Read(B, t); t := t*2; Write(B, t)

Quá trình xử lý thực tế còn liên quan đến việc trao đổi dữ liệu giữa bộ nhớ (Mem) và đĩa (Disk) thông qua các thao tác Input(X) và Output(X).

1.5. Các trạng thái của giao tác

Một giao tác trong quá trình thực hiện sẽ lần lượt trải qua các trạng thái sau:

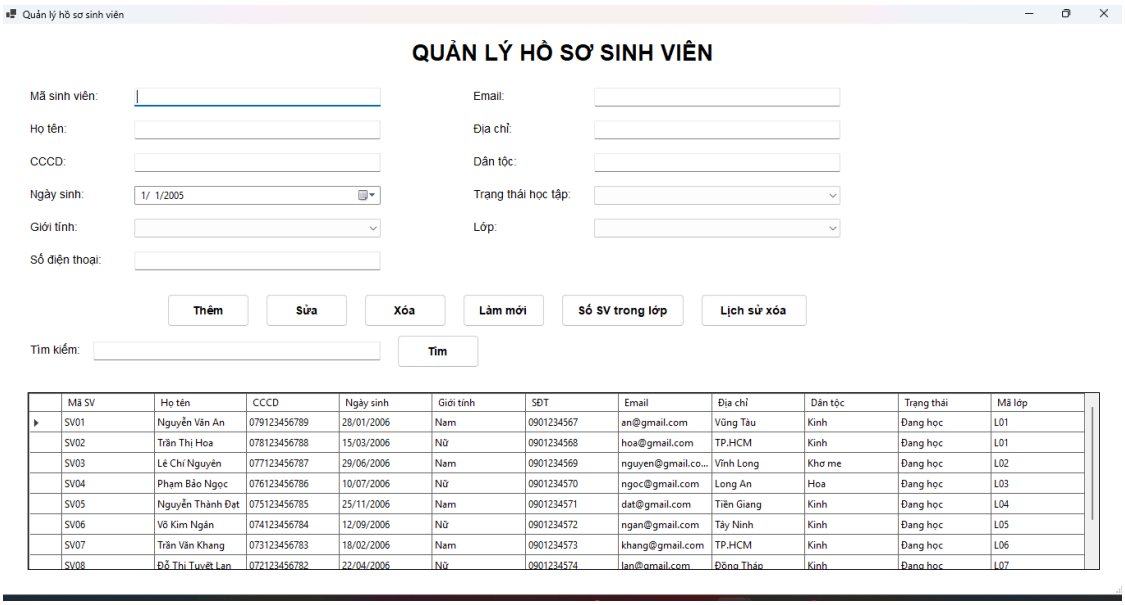
1. **Active**: trạng thái ngay sau khi giao tác bắt đầu thực hiện các hành động đọc/ghi.
2. **Partially Committed**: trạng thái sau khi lệnh cuối cùng của giao tác được thực hiện.
3. **Committed**: trạng thái sau khi giao tác hoàn thành thành công.
4. **Failed**: trạng thái xảy ra khi phát hiện rằng không thể tiếp tục thực hiện các hành động của giao tác được nữa.
5. **Aborted**: trạng thái sau khi giao tác đã bị cuộn lại (rollback) và CSDL đã được phục hồi về trạng thái trước khi giao tác bắt đầu.
6. **Terminated**: trạng thái chấm dứt giao tác (sau khi Committed hoặc Aborted).

1.6. Thực thi chức năng trên giao diện và code

Luồng hoạt động : GUI-> DTO->BUS->DAL-> SQL

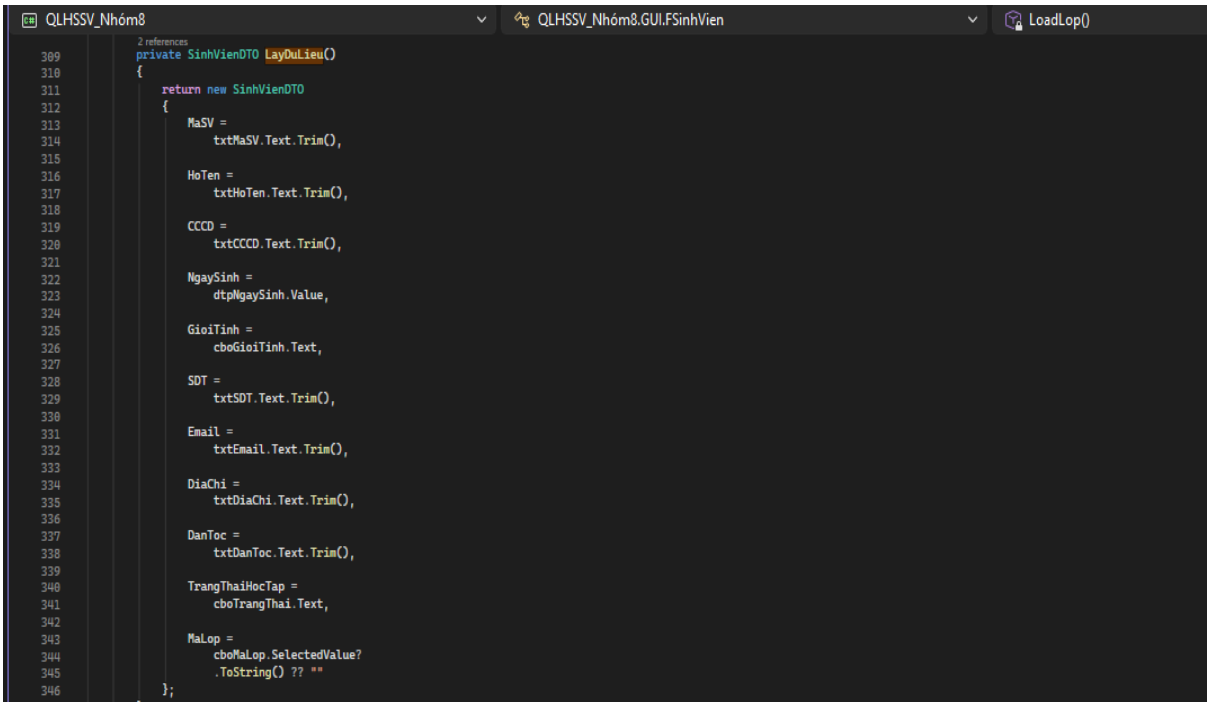
1.Thực hiện giao tác THÊM sinh viên

a) Giao diện



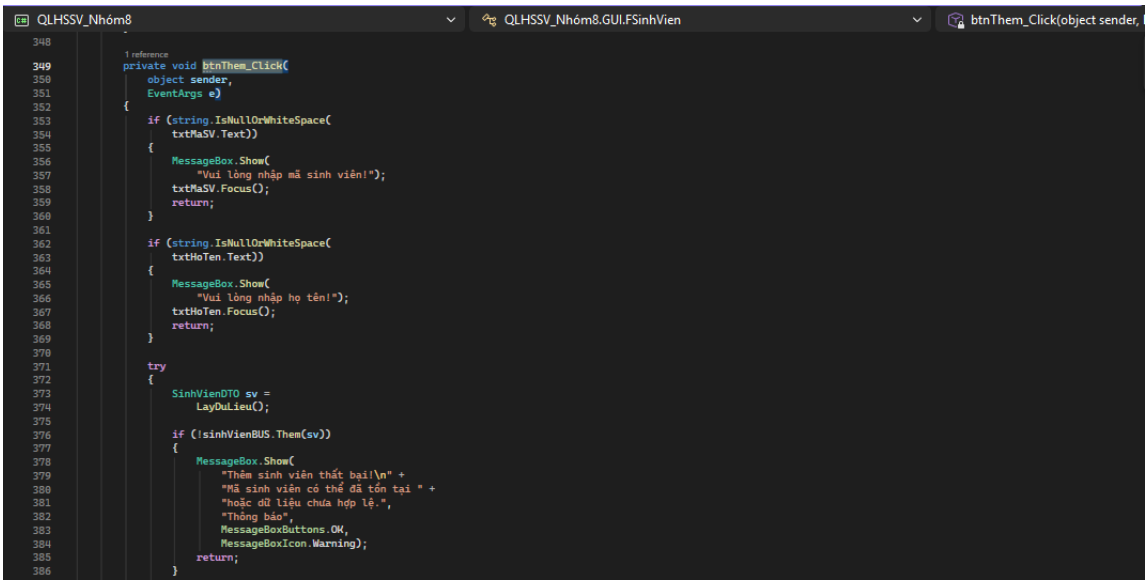
Để thêm một sinh viên mới, người dùng nhập các thông tin cần thiết vào các trường trên giao diện như Mã sinh viên, Họ tên, CCCD, Ngày sinh, Giới tính, Số điện thoại, Email, Địa chỉ, Dân tộc, Trạng thái học tập và Lớp. Sau khi nhập đầy đủ thông tin, người dùng nhấn nút Thêm để thực hiện thao tác.

b) Code xử lý



Phương thức lấy dữ liệu sinh viên tại tầng GUI

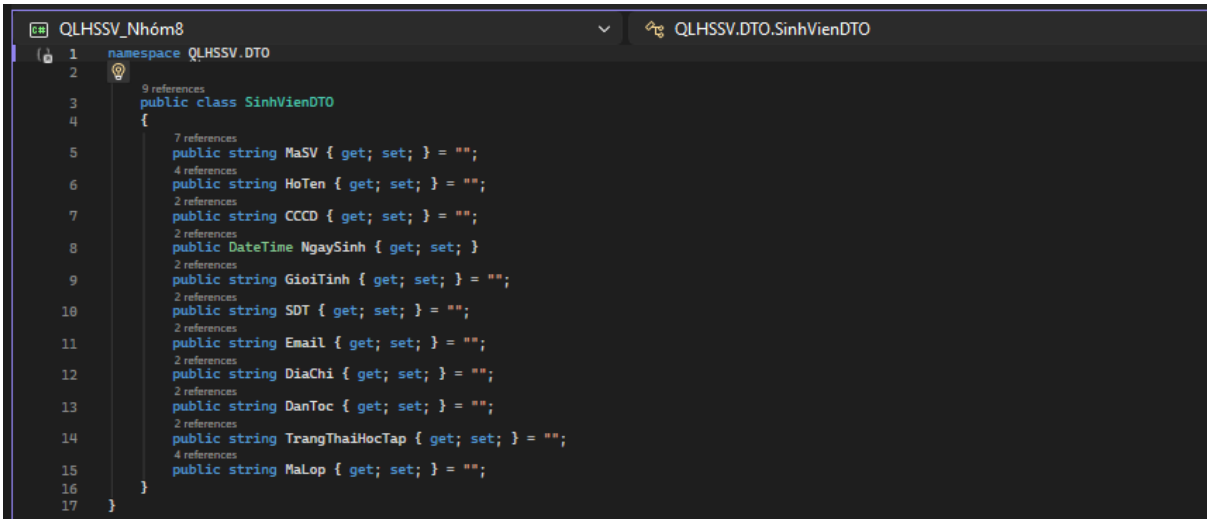
Phương thức LayDuLieu() có nhiệm vụ lấy dữ liệu từ các thành phần trên giao diện và đóng gói thành đối tượng SinhVienDTO. Các thông tin như mã sinh viên, họ tên, CCCD, ngày sinh, giới tính, số điện thoại, email, địa chỉ, dân tộc, trạng thái học tập và mã lớp được lấy từ các TextBox, DateTimePicker và ComboBox tương ứng.



Xử lý chức năng thêm sinh viên tại tầng GUI

Khi người dùng nhấn nút **Thêm**, hệ thống kiểm tra các thông tin bắt buộc như mã sinh viên và họ tên. Sau đó GUI gọi phương thức LayDuLieu() để tạo đối tượng SinhVienDTO và truyền đối tượng này cho phương thức Them() của SinhVienBUS. Nếu thao tác thành công, hệ thống thông báo kết quả và tải lại danh sách sinh viên.

Tầng DTO:



Mục đích: Khai báo lớp SinhVienDTO để đóng gói và vận chuyển dữ liệu sinh viên giữa các tầng trong kiến trúc ứng dụng.

Nội dung thực hiện: Lớp SinhVienDTO chứa các thuộc tính tương ứng với thông tin sinh viên như MaSV, HoTen, CCCD, NgaySinh, GioiTinh, SDT, Email, DiaChi, DanToc, TrangThaiHocTap và MaLop. Đối tượng này được tạo tại tầng GUI và truyền xuống tầng BUS để xử lý.

Tầng BUS:

```
public bool Them(SinhVienDTO sv)
{
    if (sv == null)
        return false;

    if (string.IsNullOrWhiteSpace(sv.MaSV))
        return false;

    if (string.IsNullOrWhiteSpace(sv.HoTen))
        return false;

    if (string.IsNullOrWhiteSpace(sv.MaLop))
        return false;

    if (dal.KiemTraTonTai(sv.MaSV))
        return false;

    return dal.Insert(sv);
}
```

Nhiệm vụ: Tiếp nhận đối tượng SinhVienDTO từ tầng GUI và kiểm tra các điều kiện nghiệp vụ cơ bản.

Nội dung thực hiện: BUS kiểm tra đối tượng DTO có hợp lệ hay không, kiểm tra mã sinh viên, họ tên và mã lớp có bị bỏ trống hay không. Đồng thời, hệ thống kiểm tra mã sinh viên đã tồn tại trong cơ sở dữ liệu hay chưa. Nếu dữ liệu hợp lệ, BUS gọi phương thức Insert() của tầng DAL để thực hiện thêm sinh viên vào cơ sở dữ liệu.

Tầng DAL:

```
public bool Insert(SinhVienDTO sv)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_ThemSinhVien", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            GanThamSoSinhVien(cmd, sv);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

Nhiệm vụ: Tầng DAL thiết lập kết nối với cơ sở dữ liệu và gọi Stored Procedure để thực hiện thao tác thêm sinh viên.

Nội dung thực hiện: DAL tạo kết nối thông qua Database.GetConnection(), mở kết nối và khởi tạo SqlCommand với Stored Procedure dbo.sp_ThemSinhVien. Các thông tin của sinh viên được truyền vào thông qua phương thức GanThamSoSinhVien(). Sau đó ExecuteNonQuery() được sử dụng để thực thi thao tác thêm dữ liệu.

SQL:

```
226  -- Tạo Procedure thêm Sinh Viên
227  create procedure sp_ThemSinhVien
228      @MaSV varchar(10),
229      @HoTen nvarchar(50),
230      @CCCD varchar(20),
231      @NgaySinh date,
232      @GioiTinh nvarchar(10),
233      @SDT varchar(20),
234      @Email varchar(50),
235      @DiaChi nvarchar(100),
236      @DanToc nvarchar(20),
237      @TrangThaiHocTap nvarchar(50),
238      @MaLop varchar(10)
239  as
240  begin
241      insert into SinhVien
242      (MaSV,HoTen,CCCD,
243      NgaySinh,GioiTinh,SDT,
244      Email,DiaChi,DanToc,
245      TrangThaiHocTap,MaLop)
246      valu
247      (@Ma                                    Tinh,@SDT,
248      @Email,@DiaChi,@DanToc,@TrangThaiHocTap,@MaLop);
249  end;
250  go
```

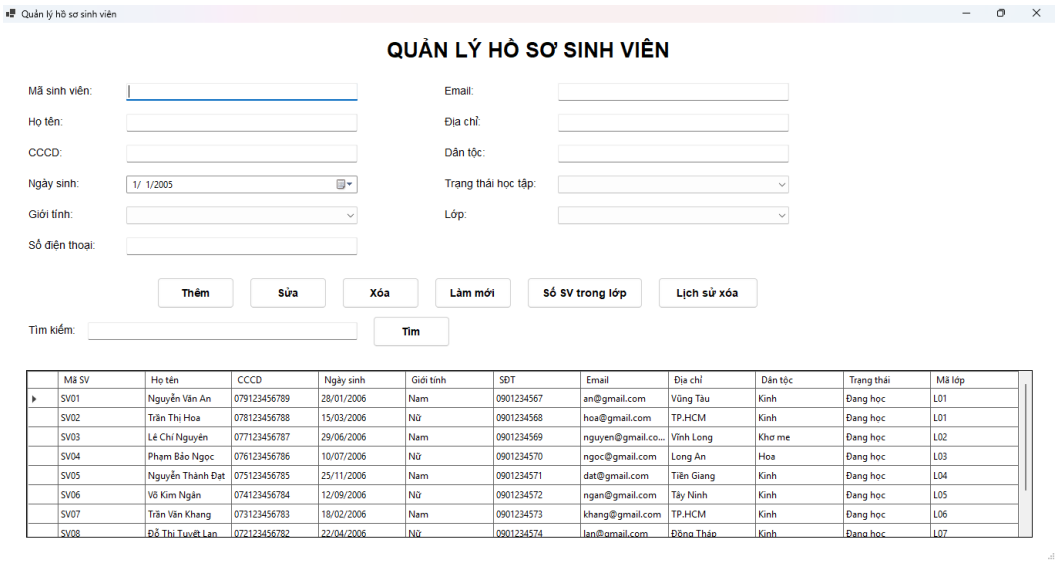
column TrangThaiHocTap(nvarchar, null)

Nhiệm vụ: Xây dựng Stored Procedure sp_ThemSinhVien trên SQL Server để thực hiện thêm thông tin sinh viên vào bảng SinhVien.

Nội dung thực hiện: Procedure nhận các tham số thông tin sinh viên như mã sinh viên, họ tên, CCCD, ngày sinh, giới tính, số điện thoại, email, địa chỉ, dân tộc, trạng thái học tập và mã lớp. Sau đó thực hiện câu lệnh INSERT để thêm dữ liệu vào bảng SinhVien.

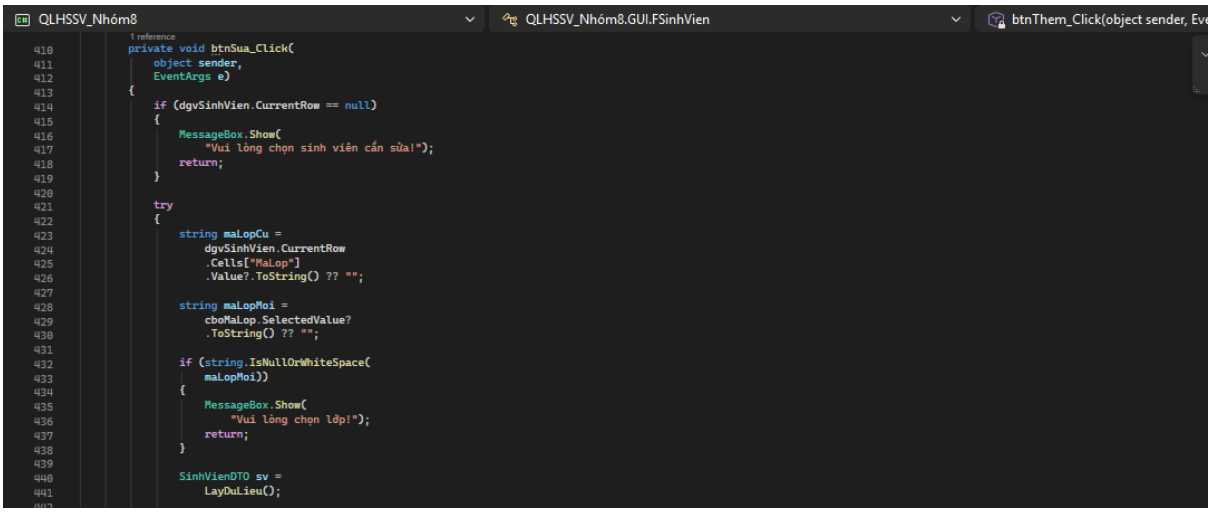
2. Thực hiện thao tác SỬA sinh viên

a) Giao diện



Để cập nhật thông tin sinh viên, người dùng chọn sinh viên cần sửa trên bảng dữ liệu, thay đổi các thông tin cần thiết trên các trường nhập liệu và nhấn nút Sửa.

2.Code xử lý



```
// Sửa thông tin, giữ lớp cũ
sv.MaLop = maLopCu;

if (!sinhVienBUS.Sua(sv))
{
    MessageBox.Show(
        "Cập nhật sinh viên thất bại!");
    return;
}
```

```
            // Đổi lớp bằng sp_ChuyenLop
            if (maLopCu != maLopMoi)
            {
                if (!sinhVienBUS.ChuyenLop(
                    sv.MaSV,
                    maLopMoi))
                {
                    MessageBox.Show(
                        "Thông tin đã cập nhật nhưng chuyển lớp thất bại!");
                    LoadData();
                    return;
                }

                MessageBox.Show(
                    $"Cập nhật thành công!\n" +
                    $"Sinh viên {sv.MaSV} đã chuyển " +
                    $"từ lớp {maLopCu} sang {maLopMoi}.");
            }
            else
            {
                MessageBox.Show(
                    "Cập nhật sinh viên thành công!");
            }

            LoadData();
            XoaTrang();
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                "Không thể cập nhật sinh viên!\n" +
                ex.Message);
        }
    }
```

Khi người dùng chọn sinh viên cần sửa, GUI lấy dữ liệu từ giao diện thông qua phương thức LayDuLieu() và tạo đối tượng SinhVienDTO. Đối tượng này được truyền xuống phương thức Sua() của SinhVienBUS. Trường hợp người dùng thay đổi lớp, hệ thống tiếp tục thực hiện chức năng chuyển lớp tương ứng.

Tầng DTO:

Chức năng sửa sử dụng lớp SinhVienDTO đã được trình bày ở phần trên để đóng gói thông tin sinh viên trước khi truyền xuống tầng BUS.

Tầng BUS:

```
1 reference
public bool Sua(SinhVienDTO sv)
{
    if (sv == null)
        return false;

    if (string.IsNullOrWhiteSpace(sv.MaSV))
        return false;

    if (string.IsNullOrWhiteSpace(sv.HoTen))
        return false;

    return dal.Update(sv);
}
```

BUS tiếp nhận đối tượng SinhVienDTO từ GUI và kiểm tra tính hợp lệ của dữ liệu. Hệ thống kiểm tra đối tượng, mã sinh viên và họ tên. Nếu dữ liệu hợp lệ, BUS gọi phương thức Update() tại tầng DAL để cập nhật thông tin sinh viên.

Tầng DAL:

```
// Sửa sinh viên bằng Stored Procedure
1 reference
public bool Update(SinhVienDTO sv)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_SuaSinhVien", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            GanThamSoSinhVien(cmd, sv);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
```

DAL tạo kết nối đến cơ sở dữ liệu và gọi Stored Procedure dbo.sp_SuaSinhVien. Các thông tin của sinh viên được truyền vào Stored Procedure và thực hiện cập nhật bằng ExecuteNonQuery().

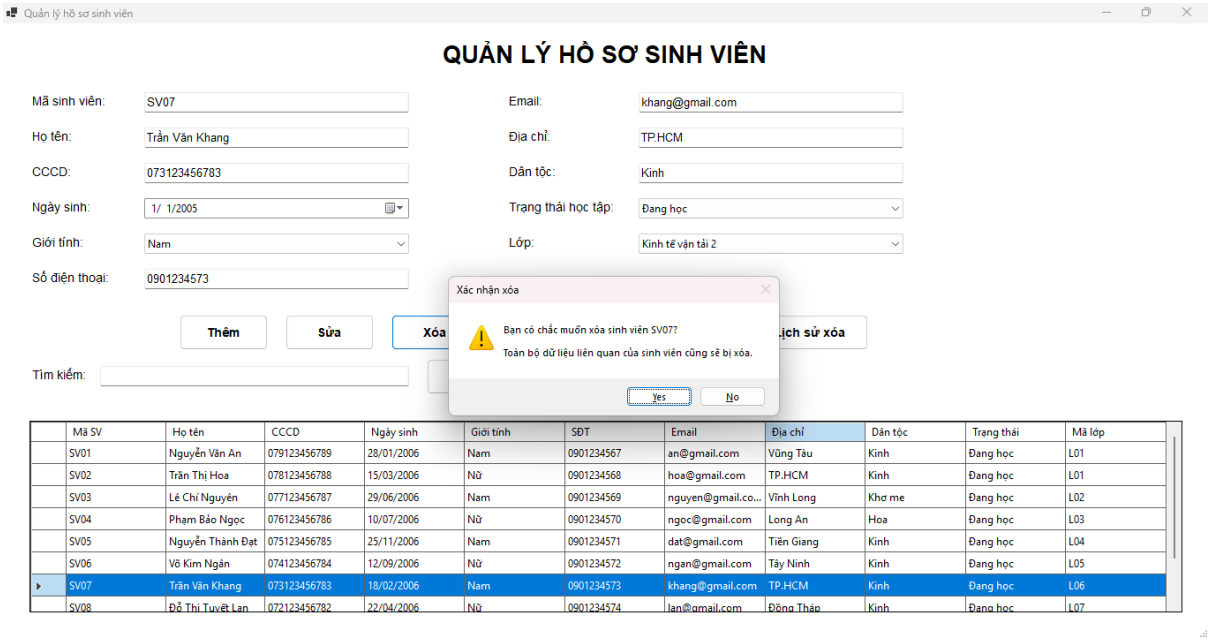
SQL:

```sql
--  Tạo Procedure sửa Sinh Viên
create procedure sp_SuaSinhVien
    @MaSV varchar(10),
    @HoTen nvarchar(50),
    @CCCD varchar(20),
    @NgaySinh date,
    @GioiTinh nvarchar(10),
    @SDT varchar(20),
    @Email varchar(50),
    @DiaChi nvarchar(100),
    @DanToc nvarchar(20),
    @TrangThaiHocTap nvarchar(50),
    @MaLop varchar(10)
as
begin
    update SinhVien
    set
        HoTen = @HoTen,
        CCCD = @CCCD,
        NgaySinh = @NgaySinh,
        GioiTinh = @GioiTinh,
        SDT = @SDT,
        Email = @Email,
        DiaChi = @DiaChi,
        DanToc = @DanToc,
        TrangThaiHocTap = @TrangThaiHocTap,
        MaLop = @MaLop
    where MaSV = @MaSV;
end;
go
```

Stored Procedure sp_SuaSinhVien nhận các thông tin của sinh viên và thực hiện câu lệnh UPDATE trên bảng SinhVien. Điều kiện WHERE MaSV = @MaSV đảm bảo chỉ cập nhật đúng sinh viên được lựa chọn.

3. Thực hiện giao tác XÓA sinh viên

a) Giao diện



b) Code xử lý

```
1 reference
private void btnXoa_Click(
    object sender,
    EventArgs e)
{
    if (dgvSinhVien.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn sinh viên cần xóa!",
            "Thông báo",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        return;
    }

    string maSV =
        dgvSinhVien.CurrentRow
        .Cells["MaSV"]
        .Value?.ToString() ?? "";

    if (string.IsNullOrWhiteSpace(maSV))
    {
        MessageBox.Show(
            "Không xác định được mã sinh viên!");
        return;
    }
```

```
        DialogResult result =
            MessageBox.Show(
                $"Bạn có chắc muốn xóa sinh viên {maSV}?\n\n" +
                "Toàn bộ dữ liệu liên quan của sinh viên cũng sẽ bị xóa.",
                "Xác nhận xóa",
                MessageBoxButtons.YesNo,
                MessageBoxIcon.Warning);

        if (result != DialogResult.Yes)
            return;

        try
        {
            if (!sinhVienBUS.Xoa(maSV))
            {
                LoadData();
                XoaTrang();

                MessageBox.Show(
                    "Không thể xóa sinh viên hoặc sinh viên không còn tồn tại!");
                return;
            }

            LoadData();
            XoaTrang();

            MessageBox.Show(
                "Xóa sinh viên và toàn bộ dữ liệu liên quan thành công!");
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                "Xóa sinh viên thất bại!\n" +
                ex.Message);
        }
    }
```

Khi người dùng chọn một sinh viên trên bảng dữ liệu và nhấn nút **Xóa**, GUI lấy mã sinh viên từ dòng đang được chọn. Hệ thống yêu cầu người dùng xác nhận trước khi xóa. Nếu người dùng đồng ý, mã sinh viên được truyền đến phương thức Xoa() của tầng BUS để tiếp tục xử lý.

Tầng DTO:

Đối với chức năng xóa, hệ thống sử dụng mã sinh viên maSV làm tham số định danh để xác định bản ghi cần xóa nên không cần tạo đối tượng SinhVienDTO.

Tầng BUS:

```
1 reference
public bool Xoa(string maSV)
{
    if (string.IsNullOrWhiteSpace(maSV))
        return false;

    return dal.Delete(maSV);
}
```

BUS tiếp nhận mã sinh viên từ tầng GUI và kiểm tra mã sinh viên có bị trống hay không. Nếu mã sinh viên hợp lệ, BUS gọi phương thức Delete() của tầng DAL để thực hiện thao tác xóa.

Tầng DAL:

```
// Xóa sinh viên bằng Stored Procedure
1 reference
public bool Delete(string maSV)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_XoaSinhVien", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaSV",
                SqlDbType.VarChar,
                10
            ).Value = maSV;

            return Convert.ToBoolean(
                cmd.ExecuteScalar()
            );
        }
    }
}
```

DAL thiết lập kết nối với cơ sở dữ liệu và gọi Stored Procedure dbo.sp_XoaSinhVien. Mã sinh viên được truyền vào tham số @MaSV. Sau khi thực thi Stored Procedure, DAL nhận kết quả trả về để xác định thao tác xóa thành công hay thất bại.

SQL:

```
281  -- Tạo Procedure xóa Sinh Viên
282  create procedure sp_XoaSinhVien
283      @MaSV VARCHAR(10)
284  AS
285  BEGIN
286      SET NOCOUNT ON;
287      SET XACT_ABORT ON;
288
289      -- Không tìm thấy sinh viên
290      IF NOT EXISTS
291      (
292          SELECT 1
293          FROM dbo.SinhVien
294          WHERE MaSV = @MaSV
295      )
296      BEGIN
297          SELECT CAST(0 AS BIT);
298          RETURN;
299      END;
```

```
301         BEGIN TRY
302             BEGIN TRANSACTION;
303
304             -- Xóa người thân trước
305             DELETE FROM dbo.NguoiThan
306             WHERE MaSV = @MaSV;
307
308             -- Xóa sinh viên
309             DELETE FROM dbo.SinhVien
310             WHERE MaSV = @MaSV;
311
312             COMMIT TRANSACTION;
313
314             -- Báo C# xóa thành công
315             SELECT CAST(1 AS BIT);
316         END TRY
317         BEGIN CATCH
318             IF @@TRANCOUNT > 0
319                 ROLLBACK TRANSACTION;
320
321             THROW;
322         END CATCH
323     END;
324     GO
```

Nhiệm vụ: Xây dựng Stored Procedure sp_XoaSinhVien để thực hiện xóa sinh viên và đảm bảo tính toàn vẹn dữ liệu trong quá trình thực hiện giao tác.

Nội dung thực hiện: Trước tiên, hệ thống kiểm tra sinh viên có tồn tại hay không. Nếu tồn tại, Stored Procedure bắt đầu giao tác bằng BEGIN TRANSACTION. Dữ liệu liên quan của sinh viên trong bảng NguoiThan được xóa trước, sau đó bản ghi sinh viên trong bảng SinhVien được xóa. Nếu các thao tác thực hiện thành công, hệ thống sử dụng COMMIT TRANSACTION để xác nhận giao tác. Trong trường hợp xảy ra lỗi, ROLLBACK TRANSACTION được sử dụng để hoàn tác các thay đổi đã thực hiện.

2. CÁC ĐỐI TƯỢNG TRONG SQL CỦA MỘT HỆ QUẢN TRỊ

2.1. VIEW

2.1.1. Khái niệm

View (khung nhìn) là một đối tượng cơ sở dữ liệu ảo, được định nghĩa bằng một câu lệnh `SELECT` lưu trữ sẵn trong hệ quản trị cơ sở dữ liệu. Về bản chất, View không lưu trữ dữ liệu vật lý riêng (trừ trường hợp Indexed View), mà chỉ lưu trữ **định nghĩa truy vấn**. Mỗi khi View được gọi đến, SQL Server sẽ thực thi lại câu lệnh `SELECT` gốc để lấy dữ liệu hiện hành từ các bảng cơ sở (base table) liên quan.

Có thể hình dung View như một "cửa sổ" cho phép người dùng nhìn vào dữ liệu của một hoặc nhiều bảng theo một góc nhìn đã được định hình sẵn, mà không cần quan tâm đến cấu trúc vật lý phức tạp phía sau

2.1.2. Đặc điểm

Không lưu trữ dữ liệu thực :

- View chủ yếu lưu định nghĩa truy vấn, không lưu bản sao dữ liệu.
- Dữ liệu hiển thị luôn được lấy trực tiếp và cập nhật theo thời gian thực từ bảng gốc.

Bảo mật và kiểm soát truy cập:

- View có thể giới hạn người dùng chỉ được xem một số dòng hoặc cột.
- Đây là cơ chế bảo mật ở mức logic giúp bảo vệ dữ liệu trong bảng gốc.

Đơn giản hóa truy vấn phức tạp :

- View đóng gói các truy vấn có JOIN, GROUP BY,...lại thành một đối tượng duy nhất.
- Người dùng chỉ cần gọi View thay vì lặp lại câu lệnh phức tạp nhiều lần.

‘ SELECT * FROM TenView ’

Tính độc lập dữ liệu :

- View tạo lớp trung gian giữa ứng dụng và cấu trúc CSDL.
- Có thể điều chỉnh View khi cấu trúc bảng gốc thay đổi.

Tính động (Dynamic):

- View không lưu bản sao dữ liệu.
- Khi dữ liệu bảng gốc thêm, sửa, xóa, View sẽ phản ánh thay đổi khi được truy vấn.

Giới hạn khi cập nhật dữ liệu :

- Không phải View nào cũng hỗ trợ INSERT, UPDATE, DELETE.
- Khả năng cập nhật phụ thuộc vào cấu trúc của View.

Ràng buộc với WITH CHECK OPTION :

- Đảm bảo dữ liệu INSERT/UPDATE qua View vẫn thỏa điều kiện WHERE.
- Dữ liệu sau khi ghi vẫn phải hiển thị được qua View.

Mã hóa định nghĩa với WITH ENCRYPTION:

- Cho phép mã hóa câu lệnh SELECT.
- Hạn chế người khác xem nội dung truy vấn gốc.

Indexed View (View được đánh chỉ mục) :

- SQL Server hỗ trợ tạo chỉ mục Clustered Index trực tiếp trên View.
- Dữ liệu được vật lý hóa, lưu trữ trên đĩa → tăng tốc truy vấn phức tạp.
- Thường dùng cho báo cáo, tổng hợp dữ liệu lớn, nhưng có nhiều ràng buộc.

2.1.3. Cách sử dụng

Cú pháp cơ bản:

```
CREATE VIEW TenView
```

AS

SELECT cot1, cot2, ...

FROM TenBang

WHERE dieu_kien ;

SELECT * FROM TenView;

- **Note:**
  + CREATE VIEW : Lệnh dùng để khởi tạo View.
  + TenView : Tên mà mình muốn đặt cho View (không được trùng với tên bảng đã có).
  + AS : Từ khóa phân cách giữa tên view và câu lệnh truy vấn.
  + SELECT cot1, cot2,...: Truy vấn lấy dữ liệu từ 1 bảng hay nhiều bảng.
  + FROM TenBang: lấy từ bảng đó tên gì.
  + WHERE: dieu_kien : Điều kiện để lấy dữ liệu.
  + SELECT * FROM TenView; : Truy vấn, hiển thị toàn bộ dữ liệu View vừa tạo.

2.1.4. Ưu điểm và nhược điểm

Ưu điểm:

- Đơn giản hóa việc viết truy vấn.
- Tăng cường bảo mật dữ liệu.
- Hỗ trợ tính độc lập logic giữa ứng dụng và CSDL.
- Cho phép tái sử dụng logic truy vấn, giảm trùng lặp mã SQL.

Nhược điểm:

- View phụ thuộc vào các bảng và cột bên dưới. Nếu thay đổi cấu trúc bảng thì View có thể bị ảnh hưởng.
- Không phải View nào cũng hỗ trợ cập nhật dữ liệu INSERT/UPDATE/DELETE.

2.1.5. Thực thi chức năng trên giao diện và code

Mô tả:

Tạo các View bằng cách kết hợp dữ liệu từ nhiều bảng, nhằm hiển thị thông tin sinh viên, lớp, ngành, người thân và chương trình đào tạo, đồng thời thống kê số lượng sinh viên theo lớp và ngành.”

1. Tạo View hiển thị danh sách sinh viên

a) Giao diện hiển thị danh sách sinh viên

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Nguyễn Văn An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Đang học	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

b) Code xử lý

1. Tầng GUI:

Khi mở form Sinh viên, chương trình sẽ gọi hàm LoadData() để tải danh sách sinh viên lên giao diện. Trong hàm này, GUI gọi sinhVienBUS.GetAll() để yêu cầu BUS lấy dữ liệu sinh viên. Sau khi nhận được dữ liệu, hàm HienThiDuLieu() sẽ đưa dữ liệu lên bảng DataGridView.

```
8 references
private void LoadData()
{
    HienThiDuLieu(
        sinhVienBUS.GetAll());
}
```

2. Tầng BUS:

BUS là tầng trung gian giữa GUI và DAL. Khi nhận yêu cầu lấy danh sách sinh viên từ GUI, BUS không trực tiếp truy vấn SQL mà gọi hàm dal.GetAll(). Sau khi DAL lấy được dữ liệu, BUS nhận kết quả và trả ngược về cho GUI để hiển thị.

```
8 references
public class SinhVienBUS
{
    private SinhVienDAL dal =
        new SinhVienDAL();

    3 references
    public DataTable GetAll()
    {
        return dal.GetAll();
    }
```

3. Tầng DAL:

DAL chịu trách nhiệm kết nối và làm việc trực tiếp với SQL Server. Trong hàm GetAll(), DAL tạo câu lệnh SELECT để lấy dữ liệu từ View vw_SinhVienDayDu. Kết quả truy vấn được lưu vào DataTable thông qua SqlDataAdapter, sau đó trả về cho BUS. Nhờ sử dụng View, DAL không cần viết lại nhiều câu lệnh JOIN trong mỗi lần truy vấn.

```
public class SinhVienDAL
{
    // Lấy danh sách sinh viên từ View
    1 reference
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT
                    MaSV,
                    HoTen,
                    CCCD,
                    NgaySinh,
                    GioiTinh,
                    SDT,
                    Email,
                    DiaChi,
                    DanToc,
                    TrangThaiHocTap,
                    MaLop
                FROM dbo.vw_SinhVienDayDu
                ORDER BY MaSV";

            using (SqlDataAdapter da = new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }

        return dt;
    }
```

4. SQL

```sql
-- Tạo View hiển thị danh sách sinh viên
CREATE VIEW dbo.vw_SinhVienDayDu
AS
SELECT
    sv.MaSV,
    sv.HoTen,
    sv.CCCD,
    sv.NgaySinh,
    sv.GioiTinh,
    sv.SDT,
    sv.Email,
    sv.DiaChi,
    sv.DanToc,
    sv.TrangThaiHocTap,
    sv.MaLop,
    l.TenLop,
    l.NienKhoa,
    n.MaNganh,
    n.TenNganh,
    k.MaKhoa,
    k.TenKhoa
FROM dbo.SinhVien sv
JOIN dbo.Lop l
    ON sv.MaLop = l.MaLop
JOIN dbo.Nganh n
    ON l.MaNganh = n.MaNganh
JOIN dbo.Khoa k
    ON n.MaKhoa = k.MaKhoa;
GO
```

+ View vw_SinhVienDayDu được tạo trong SQL Server để kết hợp dữ liệu từ bốn bảng SinhVien, Lop, Nganh và Khoa.

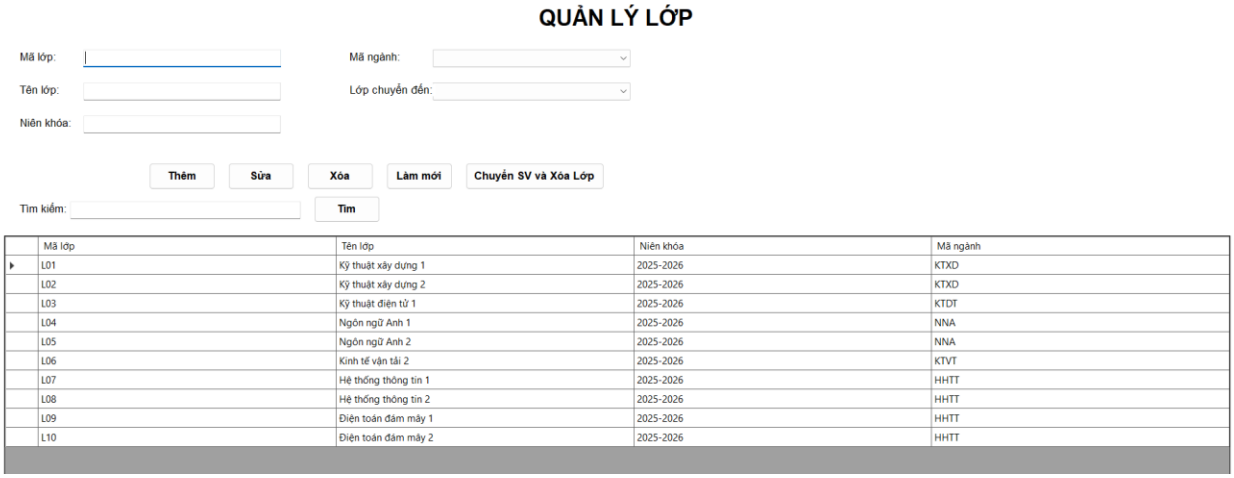
+ Bảng SinhVien chứa thông tin chính của sinh viên, bảng Lop cung cấp tên lớp, bảng Nganh cung cấp tên ngành và bảng Khoa cung cấp tên khoa.

+ Các bảng được liên kết với nhau thông qua các khóa MaLop, MaNganh và MaKhoa.

=> Nhờ View này, chương trình có thể lấy thông tin sinh viên cùng tên lớp, tên ngành và tên khoa bằng một câu lệnh SELECT đơn giản.

2. View lấy đầy đủ thông tin lớp

a) Giao diện đầy đủ thông tin lớp



Mã lớp	Tên lớp	Niên khóa	Mã ngành
L01	Kỹ thuật xây dựng 1	2025-2026	KTXD
L02	Kỹ thuật xây dựng 2	2025-2026	KTXD
L03	Kỹ thuật điện tử 1	2025-2026	KTDT
L04	Ngôn ngữ Anh 1	2025-2026	NNA
L05	Ngôn ngữ Anh 2	2025-2026	NNA
L06	Kinh tế vận tải 2	2025-2026	KTVT
L07	Hệ thống thông tin 1	2025-2026	HHTT
L08	Hệ thống thông tin 2	2025-2026	HHTT
L09	Điện toán đám mây 1	2025-2026	HHTT
L10	Điện toán đám mây 2	2025-2026	HHTT

b) Code xử lý

1. Tầng GUI:

Khi mở form Lớp, GUI gọi hàm LoadData() để lấy danh sách lớp. Hàm này gọi lopBUS.GetAll() để yêu cầu BUS lấy dữ liệu lớp. Sau khi nhận được dữ liệu, hàm HienThiDuLieu() sẽ hiển thị thông tin lên bảng DataGridView.

```
// Load dữ liệu
8 references
private void LoadData()
{
    HienThiDuLieu(lopBUS.GetAll());
}
```

2. Tầng BUS:

BUS nhận yêu cầu lấy danh sách lớp từ GUI và chuyển tiếp yêu cầu xuống DAL thông qua hàm dal.GetAll(). BUS đóng vai trò trung gian, không trực tiếp thực hiện câu lệnh SQL.

```
// Lấy danh sách lớp
4 references
public DataTable GetAll()
{
    return dal.GetAll();
}
```

3. Tầng DAL:

DAL kết nối với SQL Server và thực hiện câu lệnh SELECT từ View vw_LopDayDu. Kết quả truy vấn được lưu vào DataTable, sau đó trả về cho BUS. Nhờ sử dụng View, DAL có thể lấy thông tin lớp, ngành và khoa mà không cần viết lại nhiều câu lệnh JOIN.

```
2 references
public class LopDAL
{
    // Lấy danh sách lớp từ View
    2 references
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT
                    MaLop,
                    TenLop,
                    NienKhoa,
                    MaNganh
                FROM dbo.vw_LopDayDu
                ORDER BY MaLop";

            using (SqlDataAdapter da =
                   new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }

        return dt;
    }
```

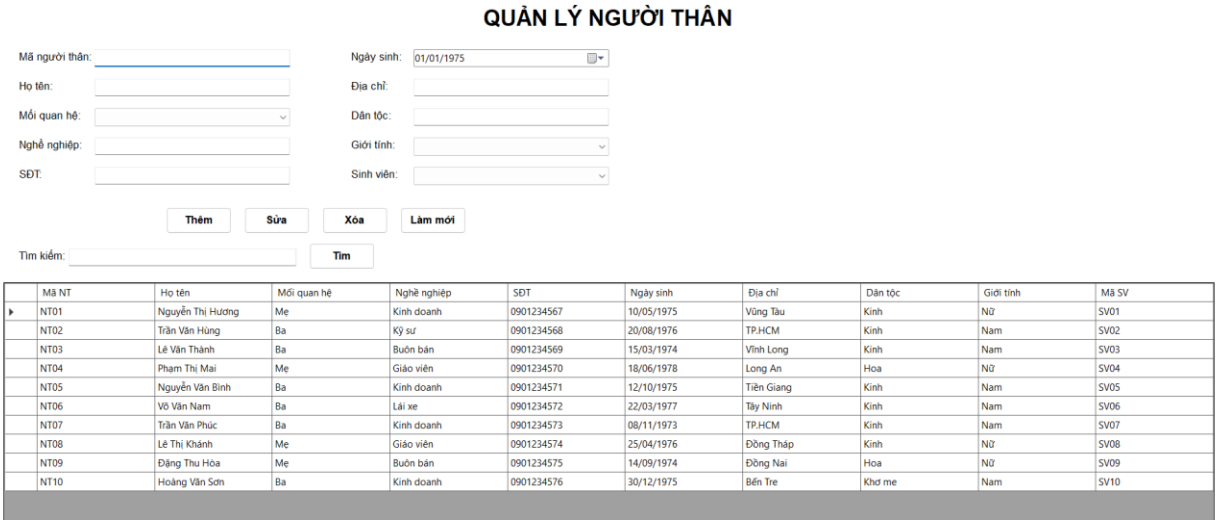
4. SQL:

View vw_LopDayDu dùng để lấy đầy đủ thông tin của lớp, bao gồm mã lớp, tên lớp, niên khóa, mã ngành, tên ngành, mã khoa và tên khoa. View kết hợp dữ liệu từ ba bảng Lop, Nganh thông qua khóa liên kết MaNganh.

```
34
35  -- View lấy đầy đủ thông tin lớp
36  CREATE OR ALTER VIEW dbo.vw_LopDayDu
37  AS
38  SELECT
39      l.MaLop,
40      l.TenLop,
41      l.NienKhoa,
42      l.MaNganh,
43      n.TenNganh
44  FROM dbo.Lop l
45  JOIN dbo.Nganh n
46      ON l.MaNganh = n.MaNganh;
47  GO
```

3. View lấy đầy đủ thông tin người thân

a) Giao diện thông tin người thân



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09
NT10	Hoàng Văn Sơn	Ba	Kinh doanh	0901234576	30/12/1975	Bến Tre	Khơ me	Nam	SV10

b) Code xử lý

1. Tầng GUI:

Khi mở form Người thân, GUI gọi hàm LoadData() để lấy danh sách người thân. Hàm này gọi nguoiThanBUS.GetAll() để yêu cầu BUS lấy dữ liệu. Sau khi nhận được dữ liệu, hàm HienThiDuLieu() sẽ hiển thị thông tin người thân lên bảng DataGridView.

```
5 references
private void LoadData()
{
    HienThiDuLieu(nguoiThanBUS.GetAll());
}
```

2. Tầng BUS:

BUS nhận yêu cầu lấy DS người thân từ GUI và chuyển yêu cầu xuống DAL thông qua hàm dal.GetAll(). BUS đóng vai trò trung gian giữa giao diện và tầng truy cập dữ liệu.

```
2 references
public class NguoiThanBUS
{
    private readonly NguoiThanDAL dal =
        new NguoiThanDAL();

    // Lấy danh sách người thân
    1 reference
    public DataTable GetAll()
    {
        return dal.GetAll();
    }
```

3. Tầng DAL:

DAL kết nối với SQL Server và thực hiện câu lệnh SELECT từ View vw_NguoiThanDayDu. Kết quả truy vấn được lưu vào DataTable, sau đó trả về cho BUS. Nhờ sử dụng View, DAL có thể lấy thông tin người thân cùng thông tin sinh viên liên quan.

```
2 references
public class NguoiThanDAL
{
    // Lấy danh sách từ View
    2 references
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT *
                FROM dbo.vw_NguoiThanDayDu
                ORDER BY MaNT";

            using (SqlDataAdapter da = new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }

        return dt;
    }
```

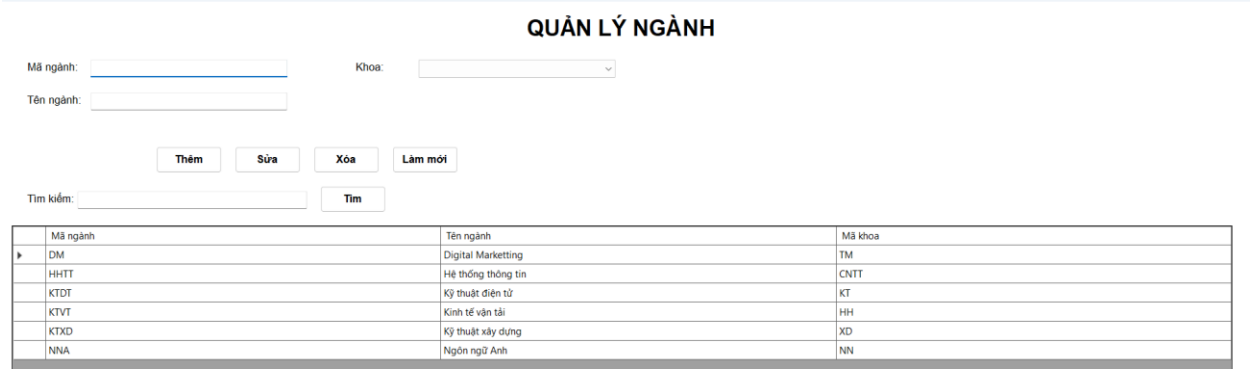
4. SQL:

View vw_NguoiThanDayDu dùng để lấy đầy đủ thông tin người thân và sinh viên mà người thân liên quan. Ngoài các thông tin của người thân như mã người thân, họ tên, mối quan hệ, nghề nghiệp và số điện thoại.

```
49  -- View lấy đầy đủ thông tin người thân
50  CREATE VIEW dbo.vw_NguoiThanDayDu
51  AS
52  SELECT
53      nt.MaNT,
54      nt.HoTen,
55      nt.MoiQuanHe,
56      nt.NgheNghiep,
57      nt.SDT,
58      nt.NgaySinh,
59      nt.DiaChi,
60      nt.DanToc,
61      nt.GioiTinh,
62      nt.MaSV
63  FROM dbo.NguoiThan nt;
64  GO
```

4. View đầy đủ thông tin ngành

a) Giao diện thông tin ngành



b) Code xử lý

1. Tầng GUI:

Khi mở form Ngành, GUI gọi hàm LoadData() để lấy danh sách ngành. Hàm này gọi nganhBUS.GetAll() để yêu cầu BUS lấy dữ liệu ngành. Sau khi nhận được dữ liệu, hàm HienThiDuLieu() sẽ hiển thị thông tin ngành lên bảng DataGridView.

```
6 references
private void LoadData()
{
    HienThiDuLieu(nganhBUS.GetAll());
}
```

2. Tầng BUS:

BUS nhận yêu cầu lấy danh sách ngành từ GUI và chuyển yêu cầu xuống DAL thông qua hàm dal.GetAll(). BUS đóng vai trò trung gian giữa giao diện và tầng truy cập dữ liệu.

```
public class NganhBUS
{
    private readonly NganhDAL dal =
        new NganhDAL();

    // Lấy danh sách ngành
    4 references
    public DataTable GetAll()
    {
        return dal.GetAll();
    }
```

3. Tầng DAL:

DAL kết nối với SQL Server và thực hiện câu lệnh SELECT từ View vw_NganhDayDu. Kết quả truy vấn được lưu vào DataTable, sau đó trả về cho BUS. Nhờ sử dụng View, DAL có thể lấy thông tin ngành cùng thông tin khoa mà không cần viết lại câu lệnh JOIN.

```
2 references
public class NganhDAL
{
    // Lấy danh sách ngành từ View
    2 references
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT
                    MaNganh,
                    TenNganh,
                    MaKhoa
                FROM dbo.vw_NganhDayDu
                ORDER BY MaNganh";

            using (SqlDataAdapter da =
                   new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }

        return dt;
    }
```

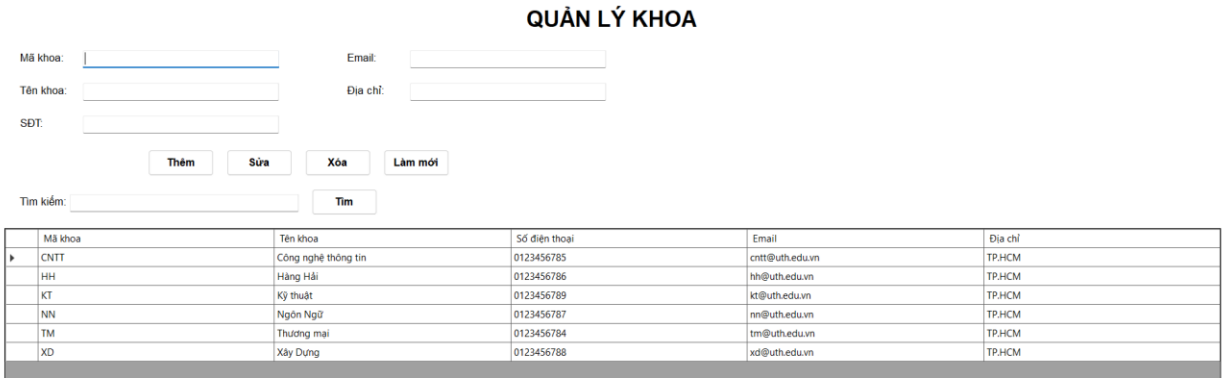
4. SQL:

View vw_NganhDayDu dùng để lấy đầy đủ thông tin của ngành, bao gồm mã ngành, tên ngành, mã khoa và tên khoa. View kết hợp hai bảng Nganh và Khoa thông qua khóa MaKhoa.

```
66  -- View đầy đủ thông tin ngành
67  CREATE VIEW dbo.vw_NganhDayDu
68  AS
69  SELECT
70      n.MaNganh,
71      n.TenNganh,
72      n.MaKhoa,
73      k.TenKhoa
74  FROM dbo.Nganh n
75  JOIN dbo.Khoa k
76      ON n.MaKhoa = k.MaKhoa;
77  GO
```

5. View thông tin khoa

a) Giao diện thông tin khoa



b) Code xử lý

1. GUI:

Khi mở form Khoa, GUI gọi hàm LoadData() để lấy danh sách khoa. Hàm này gọi khoaBUS.GetAll() để yêu cầu BUS lấy dữ liệu khoa. Sau khi nhận được dữ liệu, hàm HienThiDuLieu() sẽ hiển thị thông tin lên bảng DataGridView.

```
5 references
private void LoadData()
{
    HienThiDuLieu(khoaBUS.GetAll());
}
```

2. BUS:

BUS nhận yêu cầu lấy danh sách khoa từ GUI và chuyển tiếp yêu cầu xuống DAL thông qua hàm dal.GetAll(). BUS đóng vai trò trung gian giữa giao diện và tầng truy cập dữ liệu.

```
// Lấy danh sách khoa
4 references
public DataTable GetAll()
{
    return dal.GetAll();
}
```

3. DAL:

DAL kết nối với SQL Server và thực hiện câu lệnh SELECT từ View vw_KhoaDayDu. Kết quả truy vấn được lưu vào DataTable, sau đó trả về cho BUS. View giúp chương trình lấy thông tin khoa và số lượng ngành thuộc khoa trong cùng một lần truy vấn.

```
2 references
public class KhoaDAL
{
    // Lấy danh sách khoa từ View
    2 references
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT
                    MaKhoa,
                    TenKhoa,
                    SDT,
                    Email,
                    DiaChi
                FROM dbo.vw_KhoaDayDu
                ORDER BY MaKhoa";

            using (SqlDataAdapter da =
                   new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }

        return dt;
    }
```

4. SQL:

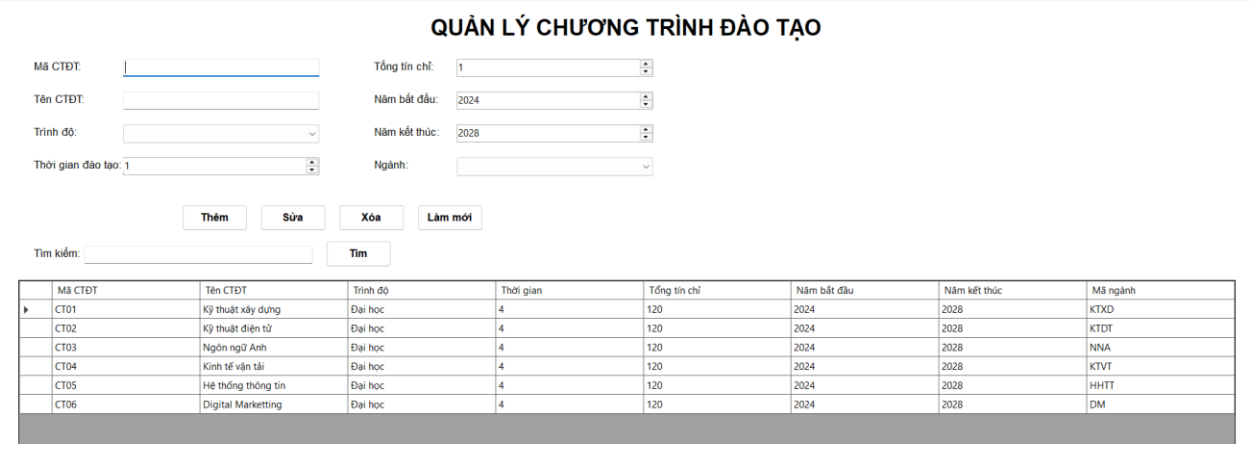
+ View vw_KhoaDayDu dùng để lấy đầy đủ các thông tin có trong bảng Khoa, bao gồm mã khoa, tên khoa, số điện thoại, email và địa chỉ.

+ Câu lệnh SELECT lấy dữ liệu trực tiếp từ bảng dbo.Khoa, sau đó View lưu lại câu truy vấn để chương trình có thể sử dụng khi cần lấy danh sách khoa.

```
78
79  -- View thông tin khoa
80  CREATE VIEW dbo.vw_KhoaDayDu
81  AS
82  SELECT
83      MaKhoa,
84      TenKhoa,
85      SDT,
86      Email,
87      DiaChi
88  FROM dbo.Khoa;
89  GO
```

6. View đầy đủ CTĐT

a) Giao diện thông tin CTĐT



b) Code xử lý

1. GUI:

Khi mở form Chương trình đào tạo, GUI gọi hàm LoadData() để lấy danh sách chương trình đào tạo. Hàm này gọi ctdtBUS.GetAll() yêu cầu BUS lấy dữ liệu. Sau khi nhận được dữ liệu, HienThiDuLieu() sẽ hiển thị thông tin CTĐT lên bảng DataGridView.

```
5 references
private void LoadData()
{
    HienThiDuLieu(ctdtBUS.GetAll());
}
```

2. BUS:

BUS nhận yêu cầu lấy danh sách CTĐT từ GUI và chuyển yêu cầu xuống DAL thông qua hàm dal.GetAll(). BUS đóng vai trò trung gian giữa giao diện và tầng truy cập dữ liệu, không trực tiếp thực hiện câu lệnh SQL.

```
// Lấy danh sách CTĐT
2 references
public DataTable GetAll()
{
    return dal.GetAll();
}
```

3. DAL:

DAL kết nối với SQL Server và thực hiện câu lệnh SELECT từ View vw_ChuongTrinhDaoTaoDayDu. Kết quả truy vấn được lưu vào DataTable, sau đó trả về

cho BUS. Nhờ sử dụng View, DAL có thể lấy thông tin chương trình đào tạo cùng thông tin ngành quản lý.

```
public class ChuongTrinhDaoTaoDAL
{
    // Lấy danh sách CTĐT từ View
    2 references
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT
                    MaCTDT,
                    TenCTDT,
                    TrinhDo,
                    ThoiGianDaoTao,
                    TongTinChi,
                    NamBatDau,
                    NamKetThuc,
                    MaNganh
                FROM dbo.vw_CTDTDayDu
                ORDER BY MaCTDT";

            using (SqlDataAdapter da =
                   new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }
```

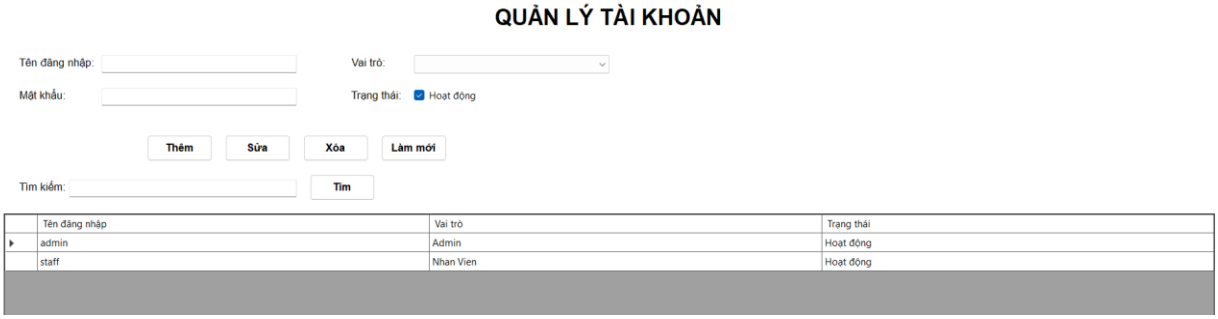
4. SQL:

View vw_ChuongTrinhDaoTaoDayDu dùng để lấy đầy đủ thông tin của chương trình đào tạo, bao gồm mã CTĐT, tên CTĐT, trình độ, thời gian đào tạo, tổng tín chỉ, năm bắt đầu, năm kết thúc và mã ngành. Ngoài ra, View còn lấy thêm tên ngành tương ứng bằng cách kết hợp bảng ChuongTrinhDaoTao với bảng Nganh thông qua khóa MaNganh.

```
91  -- View đầy đủ CTĐT
92  CREATE VIEW dbo.vw_CTDTDayDu
93  AS
94  SELECT
95      ct.MaCTDT,
96      ct.TenCTDT,
97      ct.TrinhDo,
98      ct.ThoiGianDaoTao,
99      ct.TongTinChi,
100     ct.NamBatDau,
101     ct.NamKetThuc,
102     ct.MaNganh,
103     n.TenNganh
104 FROM dbo.ChuongTrinhDaoTao ct
105 JOIN dbo.Nganh n
106     ON ct.MaNganh = n.MaNganh;
107 GO
```

7. View tài khoản

a) Giao diện tài khoản



b) Code xử lý

1. GUI:

Khi mở form Quản lý tài khoản, GUI gọi hàm LoadData() để lấy danh sách tài khoản. Hàm này gọi taiKhoanBUS.GetAll() để yêu cầu BUS lấy dữ liệu. Sau khi nhận được dữ liệu, HienThiDuLieu() sẽ hiển thị thông tin tài khoản lên bảng DataGridView

```
5 references
private void LoadData()
{
    HienThiDuLieu(
        taiKhoanBUS.GetAll());
}
```

2.BUS:

BUS nhận yêu cầu lấy danh sách tài khoản từ GUI và chuyển yêu cầu xuống DAL thông qua hàm dal.GetAll(). BUS đóng vai trò trung gian giữa giao diện và tầng truy cập dữ liệu, không trực tiếp thực hiện câu lệnh SQL

```
    new TaiKhoanDAL();

// Lấy danh sách tài khoản
1 reference
public DataTable GetAll()
{
    return dal.GetAll();
}
```

3.DAL:

DAL kết nối với SQL Server và thực hiện câu lệnh SELECT từ View vw_TaiKhoan. Kết quả truy vấn được lưu vào DataTable bằng SqlDataAdapter, sau đó trả về cho BUS. Dữ liệu được sắp xếp theo tên đăng nhập thông qua câu lệnh ORDER BY TenDangNhap.

```
public class TaiKhoanDAL
{
    // Lấy danh sách tài khoản
    2 references
    public DataTable GetAll()
    {
        DataTable dt = new DataTable();

        using (SqlConnection conn = Database.GetConnection())
        {
            string sql = @"
                SELECT
                    TenDangNhap,
                    MatKhau,
                    VaiTro,
                    TrangThai
                FROM dbo.vw_TaiKhoan
                ORDER BY TenDangNhap";

            using (SqlDataAdapter da =
                   new SqlDataAdapter(sql, conn))
            {
                da.Fill(dt);
            }
        }

        return dt;
    }
```

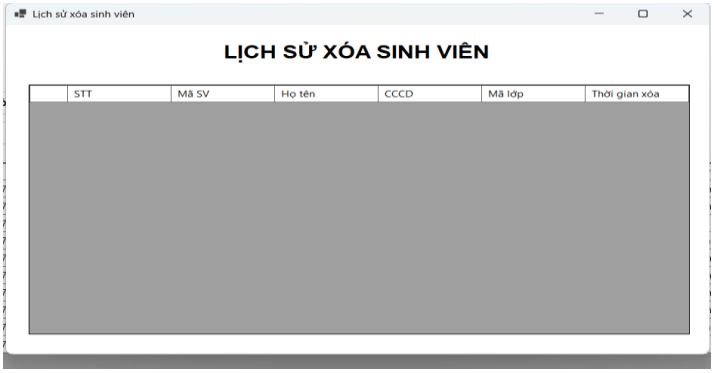
4.SQL:

View vw_TaiKhoan: lấy thông tin có trong bảng TaiKhoan, bao gồm tên đăng nhập, mật khẩu, vai trò và trạng thái tài khoản. View lấy dữ liệu trực tiếp từ bảng dbo.TaiKhoan, giúp tầng DAL truy vấn thông tin tài khoản thông qua một câu lệnh SELECT đơn giản.

```
109   -- View tài khoản
110   CREATE VIEW dbo.vw_TaiKhoan
111   AS
112   SELECT
113       TenDangNhap,
114       MatKhau,
115       VaiTro,
116       TrangThai
117   FROM dbo.TaiKhoan;
118   GO
```

8. View hiển thị lịch sử xóa

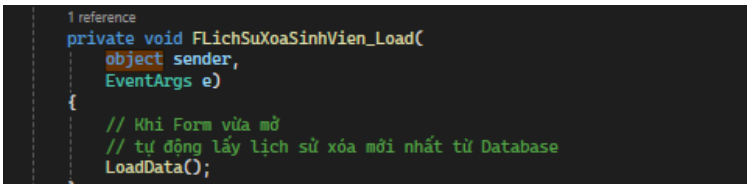
a) Giao diện hiển thị lịch xử xóa



b) Code xử lý

1. GUI:

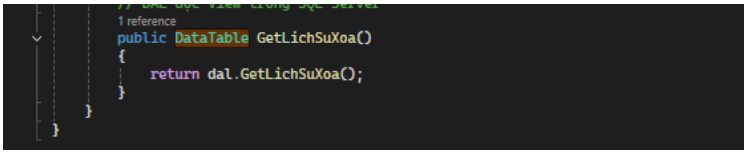
Khi mở form Lịch sử xóa sinh viên, GUI sẽ gọi hàm LoadData() để tải danh sách những sinh viên đã bị xóa. Hàm này gọi sinhVienBUS.GetLichSuXoa() để lấy dữ liệu từ BUS, sau đó gán dữ liệu vào DataGridView để hiển thị trên giao diện.



```
1 reference
private void FLichSuXoaSinhVien_Load(
    object sender,
    EventArgs e)
{
    // Khi Form vừa mở
    // tự động lấy lịch sử xóa mới nhất từ Database
    LoadData();
```

2.BUS:

BUS nhận yêu cầu lấy lịch sử xóa từ GUI và chuyển yêu cầu xuống DAL thông qua hàm dal.GetLichSuXoa(). BUS đóng vai trò trung gian giữa giao diện và tầng truy cập dữ liệu.



```
1 reference
public DataTable GetLichSuXoa()
{
    return dal.GetLichSuXoa();
}
}
}
```

3.DAL:

DAL kết nối SQL và thực hiện câu lệnh SELECT từ View vw_LichSuXoaSinhVien. Kết quả được lưu vào DataTable, sau đó trả về cho BUS. Dữ liệu được sắp xếp theo thời gian xóa giảm dần nên những sinh viên bị xóa gần đây sẽ được hiển thị trước.

```
// Lấy lịch sử xóa từ View
1 reference
public DataTable GetLichSuXoa()
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        string sql = @"
            SELECT
                Id,
                MaSV,
                HoTen,
                CCCD,
                MaLop,
                ThoiGianXoa
            FROM dbo.vw_LichSuXoaSinhVien
            ORDER BY ThoiGianXoa DESC";

        using (SqlDataAdapter da = new SqlDataAdapter(sql, conn))
        {
            da.Fill(dt);
        }
    }

    return dt;
}
```

4. SQL:

View vw_LichSuXoaSinhVien dùng để lấy thông tin những sinh viên đã bị xóa và được lưu trong bảng LichSuXoaSinhVien. View hiển thị các thông tin gồm mã lịch sử, mã sinh viên, họ tên, CCCD, mã lớp và thời gian xóa. Nhờ View này, chương trình có thể truy vấn lại lịch sử xóa mà không cần lấy trực tiếp từ bảng lưu trữ.

```
-- View hiển thị lịch sử xóa
CREATE view dbo.vw_LichSuXoaSinhVien
AS
SELECT
    Id,
    MaSV,
    HoTen,
    CCCD,
    MaLop,
    ThoiGianXoa
FROM dbo.LichSuXoaSinhVien;
GO
```

9. Tạo View hiển thị số lượng sinh viên theo lớp

a) Giao diện hiển thị số lượng sinh viên theo lớp

b)

Code xử lý

1. GUI:

Khi người dùng nhấn nút “Theo lớp”, sự kiện btnTheoLop_Click được thực hiện. Sự kiện này gọi hàm LoadTheoLop() để lấy dữ liệu thống kê theo lớp. Sau đó, dữ liệu được đưa lên bảng dgvThongKe để hiển thị cho người dùng..

```
private void btnTheoLop_Click(
    object sender,
    EventArgs e)
{
    LoadTheoLop();
}
```

```
3 references
private void LoadTheoLop()
{
    DataTable dt =
        sinhVienBUS
            .ThongKeTheoLop();

    dgvThongKe.DataSource = dt;
```

2. BUS:

GUI gọi hàm ThongKeTheoLop() trong BUS. BUS không trực tiếp truy vấn SQL mà chuyển yêu cầu xuống DAL bằng cách gọi dal.ThongKeTheoLop(). BUS có nhiệm vụ trung gian giữa GUI và DAL.

```
public DataTable ThongKeTheoLop()
{
    return dal.ThongKeTheoLop();
}
```

3.DAL:

Hàm ThongKeTheoLop() tạo một DataTable để lưu dữ liệu. Sau đó, hàm mở kết nối đến SQL Server bằng Database.GetConnection(). DAL thực hiện câu lệnh SQL để lấy toàn bộ dữ liệu từ View dbo.vw_SLSV_Lop và sắp xếp theo MaLop. Tiếp theo, SqlDataAdapter thực hiện câu truy vấn và dùng da.Fill(dt) để đưa kết quả vào DataTable. Cuối cùng, hàm trả DataTable về cho tầng BUS.

```
// Thống kê theo lớp từ View
1 reference
public DataTable ThongKeTheoLop()
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        string sql = @"
            SELECT *
            FROM dbo.vw_SLSV_Lop
            ORDER BY MaLop";

        using (SqlDataAdapter da = new SqlDataAdapter(sql, conn))
        {
            da.Fill(dt);
        }
    }

    return dt;
}
```

4.SQL:

View vw_SoLuongSinhVienTheoLop :thống kê số lượng sinh viên trong từng lớp. Câu lệnh LEFT JOIN kết hợp bảng Lop với bảng SinhVien thông qua MaLop. Hàm COUNT(sv.MaSV) dùng để đếm số sinh viên thuộc mỗi lớp. GROUP BY gom các sinh viên theo từng lớp để cho ra một dòng kết quả cho mỗi lớp.

```
133   -- Tạo View hiển thị số lượng sinh viên theo lớp
134   create view vw_SLSV_Lop
135   AS
136   SELECT
137       l.MaLop,
138       l.TenLop,
139       COUNT(s.MaSV) AS SoLuongSinhVien
140   FROM dbo.Lop l
141   LEFT JOIN dbo.SinhVien s
142       ON l.MaLop = s.MaLop
143   GROUP BY
144       l.MaLop,
145       l.TenLop;
146   GO
147
```

10. Thống kê số sinh viên theo giới tính

a) Giao diện thống kê số sinh viên theo giới tính

b) Code xử lý

1. GUI:

Khi người dùng nhấn nút “Sinh viên theo giới tính”, sự kiện btnTheoGioiTinh_Click() được thực hiện. GUI gọi hàm ThongKeTheoGioiTinh() của BUS, sau đó nhận kết quả và hiển thị lên dgvThongKe

```
1 reference
private void btnTheoGioiTinh_Click(
    object sender,
    EventArgs e)
{
    DataTable dt =
        sinhVienBUS
            .ThongKeTheoGioiTinh();

    dgvThongKe.DataSource = dt;
```

2.BUS:

BUS nhận yêu cầu GUI và gọi DAL để lấy dữ liệu thống kê theo giới tính. BUS làm nhiệm vụ trung gian giữa GUI và DAL. Khi GUI yêu cầu thống kê theo giới tính, BUS gọi hàm ThongKeTheoGioiTinh() trong DAL rồi trả kết quả về cho GUI.

```
1 reference
public DataTable ThongKeTheoGioiTinh()
{
    return dal.ThongKeTheoGioiTinh();
}
```

3.DAL:

DAL tạo DataTable, kết nối đến SQL Server, thực hiện SQL để lấy giới tính và đếm số sv theo giới tính. Sau đó, SqlDataAdapter đưa kết quả vào DataTable và trả bảng dữ liệu.

```
// Thống kê theo giới tính từ View
1 reference
public DataTable ThongKeTheoGioiTinh()
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        string sql = @"
            SELECT *
            FROM dbo.vw_SLSV_GioiTinh";

        using (SqlDataAdapter da = new SqlDataAdapter(sql, conn))
        {
            da.Fill(dt);
        }
    }

    return dt;
}
```

4.SQL:

SQL lấy dữ liệu từ bảng SinhVien, sau đó nhóm các sinh viên theo cột GioiTinh. Hàm COUNT(*) dùng để đếm số lượng sinh viên trong từng nhóm giới tính.

```
148  -- Thống kê số sinh viên theo giới tính
149  CREATE OR ALTER VIEW dbo.vw_SLSV_GioiTinh
150  AS
151  SELECT
152      GioiTinh,
153      COUNT(*) AS SoLuongSinhVien
154  FROM dbo.SinhVien
155  GROUP BY GioiTinh;
156  GO
```

2.2. STORED PROCEDURE

2.2.1 Khái niệm

- Stored Procedure là tập hợp 1 hoặc nhiều câu lệnh T-SQL thành 1 nhóm đơn vị xử lí logic, đc lưu trên Database Server
- Khi câu lệnh gọi chạy stored procedure lầu đầu thì SQL Server chạy, lưu trữ vào bộ nhớ đệm (plan cache), những lần tiếp theo sẽ sử dụng lại plan cache => tốc độ xử lí sẽ tối ưu.
- Tiện lợi cho người quản trị database, giúp tạo ra những nhóm câu lệnh và gửi đến 1 bộ phận khác mà họ kh cần quan tâm đến nd ở trong stored procedure, chỉ quan tâm tham số đầu vào/ra.

2.2.2. Đặc điểm

- Lưu trữ trực tiếp trong CSDL, có thể gọi lại nhiều lần khi cần.
- Nhận tham số đầu vào (input) truyền dữ liệu, trả về tham số đầu ra (output) đưa kết quả.
- Chứa nhiều câu lệnh SQL trong cùng một thủ tục.
- Có thể sử dụng các câu lệnh điều kiện if-else, vòng lặp,các câu lệnh xử lí khác.
- Thực hiện các thao tác: insert, update,delete,select
- Có thể gọi lại và tái sử dụng để giảm viết lại SQL

2.2.3. Cách sử dụng

Cú pháp cơ bản:

```
CREATE PROCEDURE sp_TenProcedure
    @ThamSo KieuDuLieu
AS
BEGIN
SET NOCOUNT ON;
END;
```

EXEC sp_TenProcedure;

- **Note:**

  + CREATE PROCEDURE : Lệnh khởi tạo stored procedure.

  + sp_TenProcedure : Tên của stored procedure.

  + @ThamSo KieuDuLieu : Tham số và kiểu dữ liệu.

  + AS....BEGIN : các câu lệnh SQL xử lí logic trong procedure.

  + SET NOCOUNT ON; : Không hiển thị thông báo số dòng bị thay đổi.

  + EXEC sp_TenProcedure; : Thực thi stored procedure.

2.2.4. Ưu điểm và nhược điểm

Ưu điểm:

- Tái sử dụng nhiều lần, giảm viết lại các câu lệnh SQL.
- Xử lí nhiều câu lệnh SQL trong cùng 1 thủ tục.
- Truyền tham số giúp stored procedure linh hoạt hơn.
- Tăng tính bảo mật do người dùng đc cấp quyền thực thi stored procedure thay vì truy cập trực tiếp vào bảng.
- Giúp chương trình dễ tổ chức, quản lí các nghiệp vụ về CSDL.

Nhược điểm:

- Nếu stored procedure quá dài , chứa nhiều các lệnh xử lí phức tạp thì khi lỗi khó bảo trì.
- Nó phụ thuộc vào hệ quản trị CSDL.
- Có quá nhiều stored procedure thì quản lí trở nên khó và phức tạp hơn.
- Khi thay đổi CSDL hay cấu trúc thì stored procedure nào liên quan thì phải thay đổi theo.

2.2.5. Thực thi chức năng trên giao diện và code

Mô tả: Stored Procedure dùng để thực hiện các thao tác thêm, tìm kiếm, sửa, xóa và cập nhật dữ liệu trên các bảng Khoa, Ngành, CTDT, Lớp, Sinh Viên, Người Thân.

SINH VIÊN:

1.Thêm sinh viên

- Giao diện màn hình trước khi thêm sinh viên.

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

- Code C# GUI thêm sinh viên: Kiểm tra dữ liệu đầu vào trước khi thực hiện thêm sinh viên. Nếu mã sinh viên hoặc họ tên chưa được nhập thì chương trình thông báo người dùng nhập lại.

```
// ================================
// NÚT THÊM
// ================================

private void btnThem_Click(
    object sender,
    EventArgs e)
{
    // Kiểm tra mã sinh viên
    if (string.IsNullOrWhiteSpace(txtMaSV.Text))
    {
        MessageBox.Show(
            "Vui lòng nhập mã sinh viên!");

        txtMaSV.Focus();
        return;
    }

    // Kiểm tra họ tên
    if (string.IsNullOrWhiteSpace(txtHoTen.Text))
    {
        MessageBox.Show(
            "Vui lòng nhập họ tên!");

        txtHoTen.Focus();
        return;
    }
```

- Code C# DTO thêm sinh viên: Sau khi dữ liệu hợp lệ thì chương trình đóng gói lại rồi tạo đối tượng SinhVienDTO và lấy dữ liệu từ các ô nhập trên giao diện để lưu vào đối tượng sinh viên.

```
// Tạo đối tượng SinhVienDTO
SinhVienDTO sv = new SinhVienDTO();

sv.MaSV = txtMaSV.Text.Trim();
sv.HoTen = txtHoTen.Text.Trim();
sv.CCCD = txtCCCD.Text.Trim();

sv.NgaySinh =
    dtpNgaySinh.Value;

sv.GioiTinh =
    cboGioiTinh.Text;

sv.SDT =
    txtSDT.Text.Trim();

sv.Email =
    txtEmail.Text.Trim();

sv.DiaChi =
    txtDiaChi.Text.Trim();

sv.DanToc =
    txtDanToc.Text.Trim();

sv.TrangThaiHocTap =
    cboTrangThai.Text;

sv.MaLop =
    cboMaLop.SelectedValue?
    .ToString() ?? "";
```

- C# đóng gói DTO truyền xuống BUS ->DAL->SQL:

```
// Gọi BUS để thêm
bool ketQua =
    sinhVienBUS.Them(sv);
```

```
public bool Them(SinhVienDTO sv)
{
    if (sv == null)
        return false;

    if (string.IsNullOrWhiteSpace(sv.MaSV))
        return false;

    if (string.IsNullOrWhiteSpace(sv.HoTen))
        return false;

    if (string.IsNullOrWhiteSpace(sv.MaLop))
        return false;

    if (dal.KiemTraTonTai(sv.MaSV))
        return false;

    return dal.Insert(sv);
}
```

```
// Thêm sinh viên
public bool Insert(SinhVienDTO sv)
{
    using (SqlConnection conn =
           Database.GetConnection())
    {
        conn.Open();

        string sql = @"
            INSERT INTO SinhVien
            (
                MaSV,
                HoTen,
                CCCD,
                NgaySinh,
                GioiTinh,
                SDT,
                Email,
                DiaChi,
                DanToc,
                TrangThaiHocTap,
                MaLop
            )
            VALUES
            (
                @MaSV,
                @HoTen,
                @CCCD,
                @NgaySinh,
                @GioiTinh,
                @SDT,
                @Email,
                @DiaChi,
                @DanToc,
                @TrangThaiHocTap,
                @MaLop
            )";

        SqlCommand cmd =
            new SqlCommand(sql, conn);
```

-Code SQL

```
-- Tạo Procedure thêm Sinh Viên
create procedure sp_ThemSinhVien
    @MaSV varchar(10),
    @HoTen nvarchar(50),
    @CCCD varchar(20),
    @NgaySinh date,
    @GioiTinh nvarchar(10),
    @SDT varchar(20),
    @Email varchar(50),
    @DiaChi nvarchar(100),
    @DanToc nvarchar(20),
    @TrangThaiHocTap nvarchar(50),
    @MaLop varchar(10)
as
begin
    insert into SinhVien
    (MaSV,HoTen,CCCD,
    NgaySinh,GioiTinh,SDT,
    Email,DiaChi,DanToc,
    TrangThaiHocTap,MaLop)
    values
    (@MaSV,@HoTen,@CCCD,@NgaySinh,@GioiTinh,@SDT,
    @Email,@DiaChi,@DanToc,@TrangThaiHocTap,@MaLop);
end;
go
```

Nhiệm vụ:

+ Dùng để thêm một sinh viên mới.

+ SP nhận đầy đủ thông tin sinh viên như: MaSV, Họ Tên, CCCD, Ngày sinh, Giới tính, SĐT, Email, Địa chỉ, Dân tộc, Trạng thái học tập, Mã lớp.

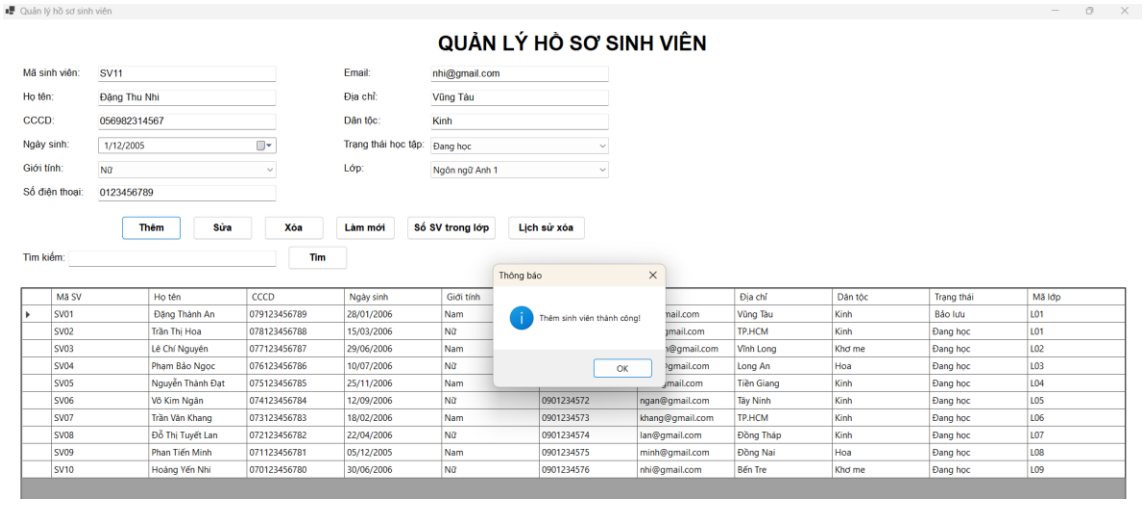
+ Sau đó INSERT vào bảng SinhVien.

- Code C# DAL Stored Procedure thêm sinh viên: Sau khi Stored Procedure thực hiện, KQ được trả về chương trình. Nếu thêm thành công, hệ thống thông báo thêm thành công. Nếu thất bại, hệ thống hiển thị thông báo lỗi.

```
        if (ketQua)
        {
            MessageBox.Show(
                "Thêm sinh viên thành công!");

            LoadData();
            XoaTrang();
        }
        else
        {
            MessageBox.Show(
                "Thêm sinh viên thất bại!\n" +
                "Mã sinh viên có thể đã tồn tại " +
                "hoặc dữ liệu chưa hợp lệ.");
        }
    }
```

- Giao diện khi thêm sinh viên vào:

- Giao diện thêm sinh viên thành công:



2. Tìm sinh viên

- Giao diện ban đầu trước khi tìm

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

- Code C# GUI tìm sinh viên: Khi nhấn nút Tìm kiếm, chương trình lấy nội dung người dùng nhập trong txtTimKiem, sau đó gọi TimKiem() của SinhVienBUS. Kết quả tìm kiếm được lưu vào DataTable và hiển thị lên giao diện.

```
1 reference
private void btnTimKiem_Click(
    object sender,
    EventArgs e)
{
    try
    {
        DataTable danhSach =
            sinhVienBUS.TimKiem(
                txtTimKiem.Text.Trim());

        HienThiDuLieu(danhSach);
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể tìm kiếm!\n" +
            ex.Message);
    }
```

- Code C# gọi BUS Tìm sinh viên: SinhVienBUS nhận từ khóa tìm kiếm keyword từ giao diện và gọi phương thức Search() của DAL để thực hiện tìm kiếm. Kết quả được trả về dưới dạng DataTable.

```
1 reference
public DataTable TimKiem(string keyword)
{
    return dal.Search(keyword);
}
```

- Code C# DAL gọi Stored Procedure: DAL gọi Stored Procedure dbo.sp_TimSinhVien để tìm kiếm sinh viên. Từ khóa tìm kiếm được truyền vào tham số @TuKhoa. Kết quả truy vấn được SqlDataAdapter lấy và lưu vào DataTable, sau đó trả kết quả về BUS.

```
// Tìm sinh viên bằng Stored Procedure
1 reference
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_TimSinhVien", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da = new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
```

- Code SQL

```
    -- Tìm sinh viên có dữ liệu chứa từ khóa
    SELECT
        MaSV,
        HoTen,
        CCCD,
        NgaySinh,
        GioiTinh,
        SDT,
        Email,
        DiaChi,
        DanToc,
        TrangThaiHocTap,
        MaLop
    FROM dbo.vw_SinhVienDayDu
    WHERE
        MaSV LIKE '%' + @TuKhoa + '%'
        OR HoTen LIKE N'%' + @TuKhoa + N'%'
        OR CCCD LIKE '%' + @TuKhoa + '%'
    ORDER BY MaSV;
END;
GO

-- Kiểm tra tính tồn tại của sinh viên
CREATE procedure dbo.sp_KiemTraSinhVienTonTai
    @MaSV VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS
    (
        SELECT 1
        FROM dbo.SinhVien
        WHERE MaSV = @MaSV
    )
        SELECT CAST(1 AS BIT);
    ELSE
        SELECT CAST(0 AS BIT);
END;
GO
```

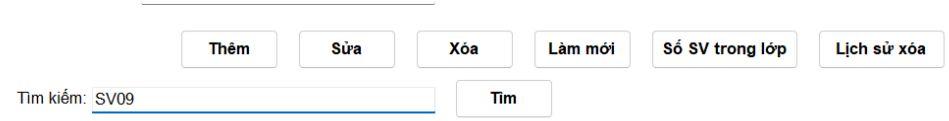
Nhiệm vụ:

+ Dùng để tìm kiếm sinh viên

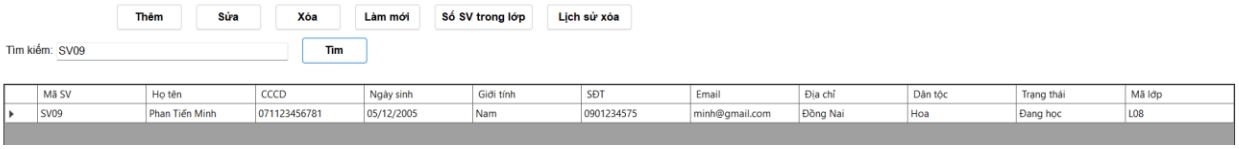
+ SP nhận @TuKhoa rồi tìm theo: MaSV, HoTen,CCCD

+ SP lấy dữ liệu từ vw_SinhVienDayDu

- Giao diện tìm sinh viên



- Giao diện hiển thị kết quả đã tìm kiếm



3. Sửa sinh viên

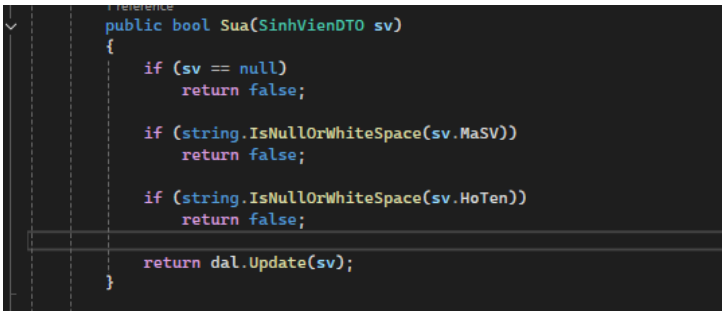
- Giao diện màn hình trước khi sửa sinh viên

- Code C# GUI sửa sinh viên : Khi nhấn nút Sửa, chương trình kiểm tra người dùng đã chọn sinh viên cần sửa hay chưa. Sau đó lấy mã lớp hiện tại của sinh viên, lấy các thông

tin cần sửa thông qua LayDuLieu() và tạo đối tượng SinhVienDTO, chương trình gọi sinhVienBUS.Sua(sv) để thực hiện cập nhật.

```
1 reference
private void btnSua_Click(
    object sender,
    EventArgs e)
{
    if (dgvSinhVien.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn sinh viên cần sửa!");
        return;
    }

    try
    {
        string maLopCu =
            dgvSinhVien.CurrentRow
            .Cells["MaLop"]
            .Value?.ToString() ?? "";

        SinhVienDTO sv =
            LayDuLieu();

        // Sửa thông tin, giữ lớp cũ
        sv.MaLop = maLopCu;

        if (!sinhVienBUS.Sua(sv))
        {
            MessageBox.Show(
                "Cập nhật sinh viên thất bại!");
            return;
        }
```

- Code C# BUS sửa sinh viên: SinhVienBUS kiểm tra dữ liệu trước khi sửa. Nếu sinh viên, mã sinh viên và họ tên hợp lệ thì gọi dal.Update(sv) để chuyển yêu cầu xuống DAL.



```
public bool Sua(SinhVienDTO sv)
{
    if (sv == null)
        return false;

    if (string.IsNullOrWhiteSpace(sv.MaSV))
        return false;

    if (string.IsNullOrWhiteSpace(sv.HoTen))
        return false;

    return dal.Update(sv);
}
```

- Code C# DAL gọi Stored Procedure tìm sinh viên: SinhVienDAL kết nối đến SQL Server và gọi Stored Procedure dbo.sp_SuaSinhVien. Dữ liệu trong SinhVienDTO được truyền vào các tham số của Stored Procedure, sau đó ExecuteNonQuery() thực hiện cập nhật thông tin sinh viên.

```
// Sửa sinh viên bằng Stored Procedure
1 reference
public bool Update(SinhVienDTO sv)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_SuaSinhVien", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            GanThamSoSinhVien(cmd, sv);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

- Code SQL

```
251
252  --  Tạo Procedure sửa Sinh Viên
253  create procedure sp_SuaSinhVien
254      @MaSV varchar(10),
255      @HoTen nvarchar(50),
256      @CCCD varchar(20),
257      @NgaySinh date,
258      @GioiTinh nvarchar(10),
259      @SDT varchar(20),
260      @Email varchar(50),
261      @DiaChi nvarchar(100),
262      @DanToc nvarchar(20),
263      @TrangThaiHocTap nvarchar(50),
264      @MaLop varchar(10)
265  as
266  begin
267      update SinhVien
268      set
269          HoTen = @HoTen,
270          CCCD = @CCCD,
271          NgaySinh = @NgaySinh,
272          GioiTinh = @GioiTinh,
273          SDT = @SDT,
274          Email = @Email,
275          DiaChi = @DiaChi,
276          DanToc = @DanToc,
277          TrangThaiHocTap = @TrangThaiHocTap,
278          MaLop = @MaLop
279      where MaSV = @MaSV;
280  end;
281  go
282
```

Nhiệm vụ:

+ Dùng để sửa thông tin sinh viên.

+ SP sd @MaSV để xác định sinh viên cần sửa, sau đó cập nhật các thông tin còn lại

4. Xóa sinh viên

- Giao diện màn hình trước khi xóa sinh viên

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

- Code C# GUI xóa sinh viên : Khi nhấn nút Xóa, chương trình kiểm tra người dùng đã chọn sinh viên hay chưa, sau đó lấy MaSV từ dòng được chọn. Xác nhận yêu cầu xóa trước khi thực hiện xóa. Nếu đồng ý, chương trình gọi sinhVienBUS.Xoa(maSV) để chuyển yêu cầu xóa xuống tầng BUS.

```
1 reference
private void btnXoa_Click(
    object sender,
    EventArgs e)
{
    if (dgvSinhVien.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn sinh viên cần xóa!",
            "Thông báo",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        return;
    }

    string maSV =
        dgvSinhVien.CurrentRow
        .Cells["MaSV"]
        .Value?.ToString() ?? "";

    if (string.IsNullOrWhiteSpace(maSV))
    {
        MessageBox.Show(
            "Không xác định được mã sinh viên!");
        return;
    }

    DialogResult result =
        MessageBox.Show(
            $"Bạn có chắc muốn xóa sinh viên {maSV}?\n\n" +
            "Toàn bộ dữ liệu liên quan của sinh viên cũng sẽ bị xóa.",
            "Xác nhận xóa",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);
```

```
        if (result != DialogResult.Yes)
            return;

        try
        {
            if (!sinhVienBUS.Xoa(maSV))
            {
                LoadData();
                XoaTrang();

                MessageBox.Show(
                    "Không thể xóa sinh viên hoặc sinh viên không còn tồn tại!");
                return;
            }

            LoadData();
            XoaTrang();

            MessageBox.Show(
                "Xóa sinh viên và toàn bộ dữ liệu liên quan thành công!");
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                "Xóa sinh viên thất bại!\n" +
                ex.Message);
        }
    }
```

- Code C# Bus xóa sinh viên: Tạo phương thức Xoa trong lớp BUS. Sau đó kiểm tra mã sinh siên có rỗng hay chứa khoảng trắng không, không hợp lệ thì trả về false (xóa thất bại), nếu hợp lệ thì BUS gọi xuống DAL để thực hiện thao tác xóa.

```
    1 reference
    public bool Xoa(string maSV)
    {
        if (string.IsNullOrWhiteSpace(maSV))
            return false;

        return dal.Delete(maSV);
    }
```

- Code C# gọi DAL Stored Procedure xóa sinh viên: DAL kết nối đến SQL Server và gọi Stored Procedure sp_XoaSinhVien để xóa sinh viên dựa trên mã sinh viên.

```
    // Xóa sinh viên bằng Stored Procedure
    1 reference
    public bool Delete(string maSV)
    {
        using (SqlConnection conn = Database.GetConnection())
        {
            conn.Open();

            using (SqlCommand cmd =
                   new SqlCommand("dbo.sp_XoaSinhVien", conn))
            {
                cmd.CommandType = CommandType.StoredProcedure;

                cmd.Parameters.Add(
                    "@MaSV",
                    SqlDbType.VarChar,
                    10
                ).Value = maSV;

                return Convert.ToBoolean(
                    cmd.ExecuteScalar()
```

- Code SQL

```sql
-- Tạo Procedure xóa Sinh Viên
create procedure sp_XoaSinhVien
    @MaSV VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    -- Không tìm thấy sinh viên
    IF NOT EXISTS
    (
        SELECT 1
        FROM dbo.SinhVien
        WHERE MaSV = @MaSV
    )
    BEGIN
        SELECT CAST(0 AS BIT);
        RETURN;
    END;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Xóa người thân trước
        DELETE FROM dbo.NguoiThan
        WHERE MaSV = @MaSV;

        -- Xóa sinh viên
        DELETE FROM dbo.SinhVien
        WHERE MaSV = @MaSV;

        COMMIT TRANSACTION;

        -- Báo C# xóa thành công
        SELECT CAST(1 AS BIT);
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH
END;
GO
```

Nhiệm vụ:

+Đây là SP xóa sinh viên, nhưng có xử lý thêm dữ liệu liên quan.

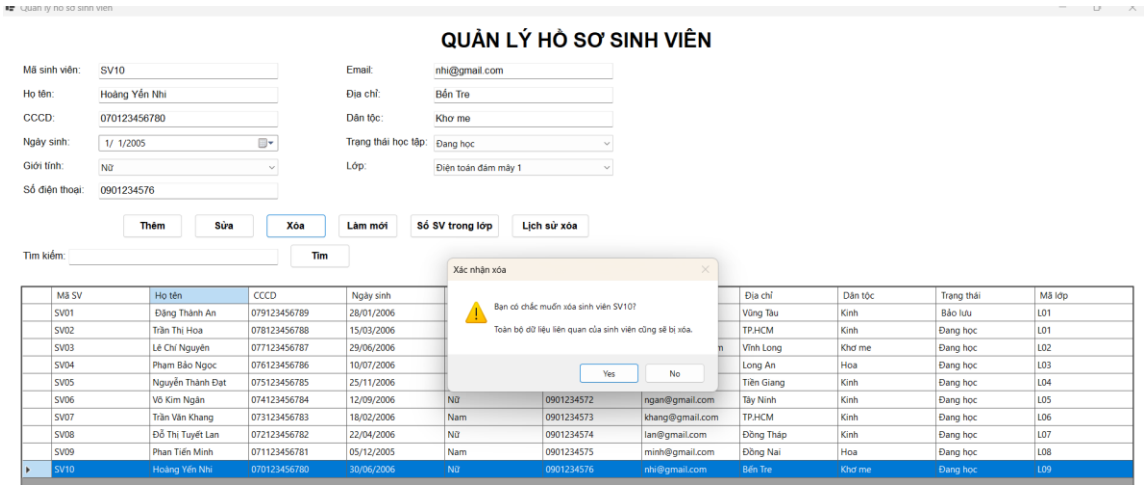
+ Khi xóa:

- Kiểm tra sinh viên có tồn tại không.
- Bắt đầu transaction.
- Xóa người thân của sinh viên trước.
- Sau đó xóa sinh viên.
- Nếu thành công thì COMMIT.
- Nếu xảy ra lỗi thì ROLLBACK.

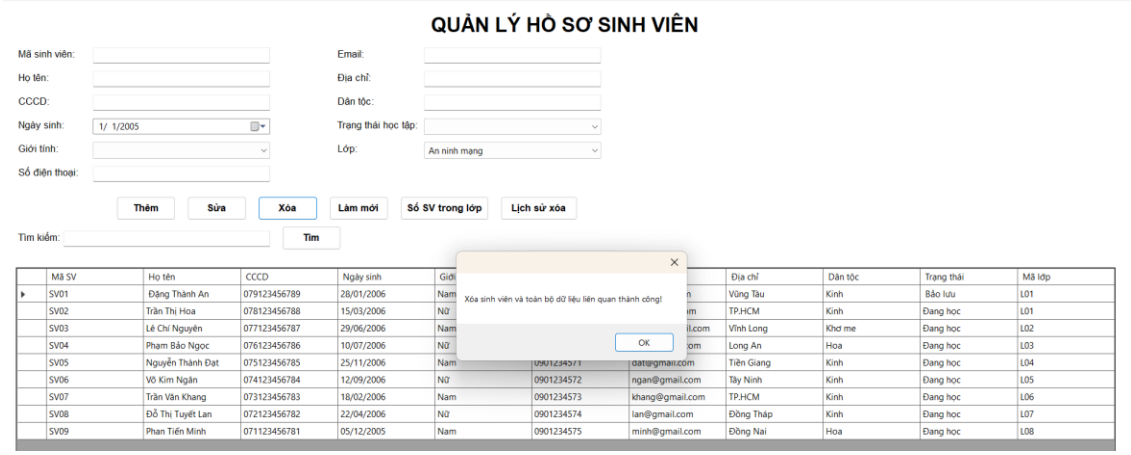
- Giao diện chọn sinh viên để chuẩn bị xóa: Click chuột SV10 để chọn SinhVien cần xóa

Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ	0901234568	hoa@gmail.com	TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam	0901234569	nguyen@gmail.com	Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ	0901234570	ngoc@gmail.com	Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam	0901234571	dat@gmail.com	Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ	0901234572	ngan@gmail.com	Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam	0901234573	khang@gmail.com	TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L09

- Giao diện hiển thị điều kiện chắc chắn mình muốn xóa tất cả dữ liệu liên quan tới SV10



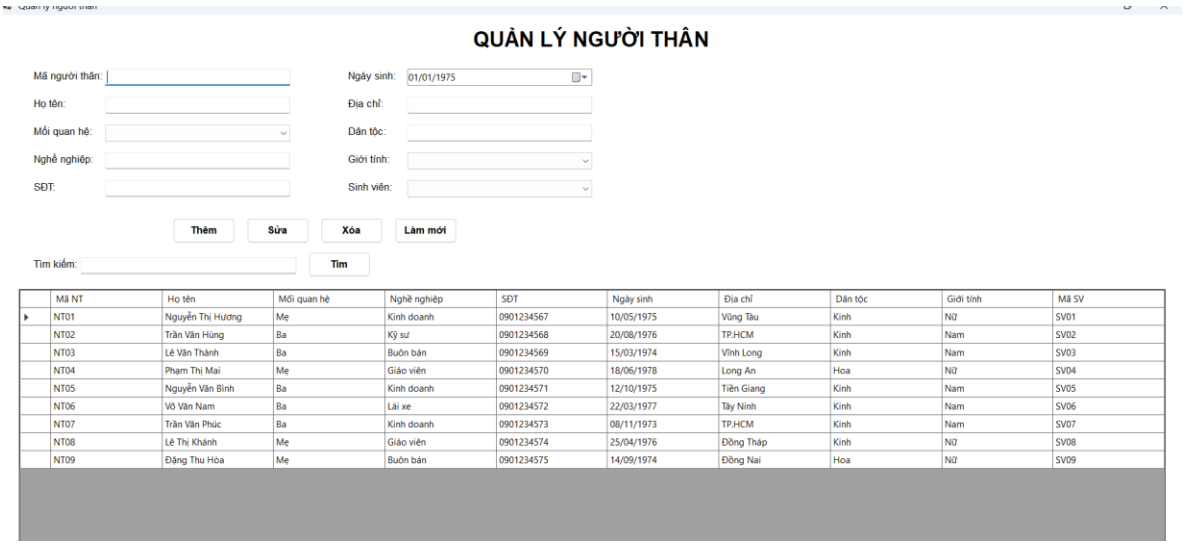
- Giao diên đã xóa thành công SV10: Chọn yes -> mình đã xóa SV10



NGƯỜI THÂN:

1. Thêm người thân

- Giao diện màn hình trước khi thêm người thân



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09

- Code C# GUI thêm người thân: Lấy dữ liệu người thân mà người dùng đã nhập trên giao diện và tạo thành một đối tượng NguoiThanDTO. Sau đó dùng DTO để đóng gói các thông tin người thân trước khi truyền xuống BUS.

```
1 reference
private void btnThem_Click(object sender, EventArgs e)
{
    try
    {
        NguoiThanDTO nt = LayDuLieu();

        if (!nguoiThanBUS.Them(nt))
        {
            MessageBox.Show(
                "Thêm thất bại! Kiểm tra dữ liệu hoặc mã đã tồn tại.");
            return;
        }

        MessageBox.Show("Thêm người thân thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể thêm người thân!\n" + ex.Message);
    }
}
```

- Code C# BUS thêm người thân: BUS thực hiện kiểm tra dữ liệu người thân có hợp lệ và kiểm tra mã người thân đã tồn tại hay chưa. Nếu dữ liệu hợp lệ và mã chưa tồn tại, BUS gọi DAL để thực hiện thêm người thân vào cơ sở dữ liệu.

```
// Thêm người thân
1 reference
public bool Them(NguoiThanDTO nt)
{
    if (!DuLieuHopLe(nt))
        return false;

    if (dal.KiemTraTonTai(nt.MaNT))
        return false;

    return dal.Insert(nt);
}
```

- Code C# gọi DAL Stored Procedure thêm người thân: DAL kết nối đến SQL Server, gọi Stored Procedure sp_ThemNT, truyền các thông tin người thân vào các tham số và thực thi Stored Procedure để thêm dữ liệu vào bảng NguoiThan.

```
// Thêm bằng Stored Procedure
1 reference
public bool Insert(NguoiThanDTO nt)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_ThemNT", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, nt);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

Code SQL

```
-- Tạo Procedure thêm Người Thân
create procedure sp_ThemNT
    @MaNT varchar(10),
    @HoTen nvarchar(50),
    @MoiQuanHe nvarchar(20),
    @NgheNghiep nvarchar(20),
    @SDT varchar(20),
    @NgaySinh date,
    @DiaChi nvarchar(100),
    @DanToc nvarchar(20),
    @GioiTinh nvarchar(10),
    @MaSV varchar(10)
as
begin
    insert into NguoiThan
    (MaNT,HoTen,MoiQuanHe,NgheNghiep,SDT,
    NgaySinh,DiaChi,DanToc,GioiTinh,MaSV)
    values
    (@MaNT,@HoTen,@MoiQuanHe,@NgheNghiep,@SDT,
     @NgaySinh,@DiaChi,@DanToc,@GioiTinh,@MaSV);
end;
go
```

Nhiệm vụ:

+ Dùng để thêm người thân của sinh viên.

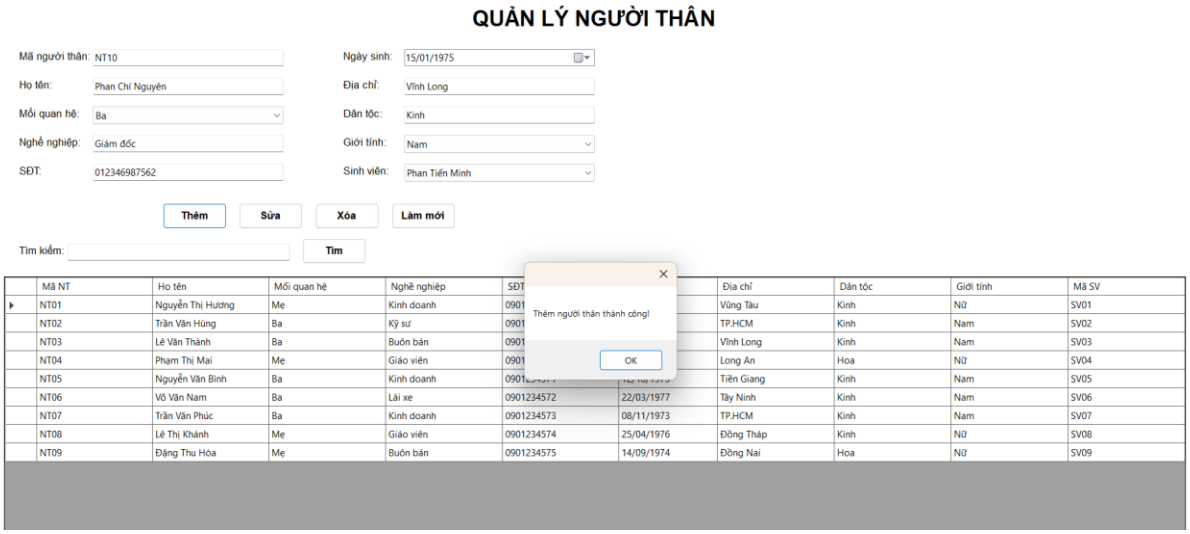
+Nhận thông tin người thân như: MaNT, Họ tên, SĐT, Địa chỉ, Mã sinh viên

+ Sau đó thêm vào bảng NguoiThan

- Giao diện màn hình khi thêm thông tin người thân:

Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09

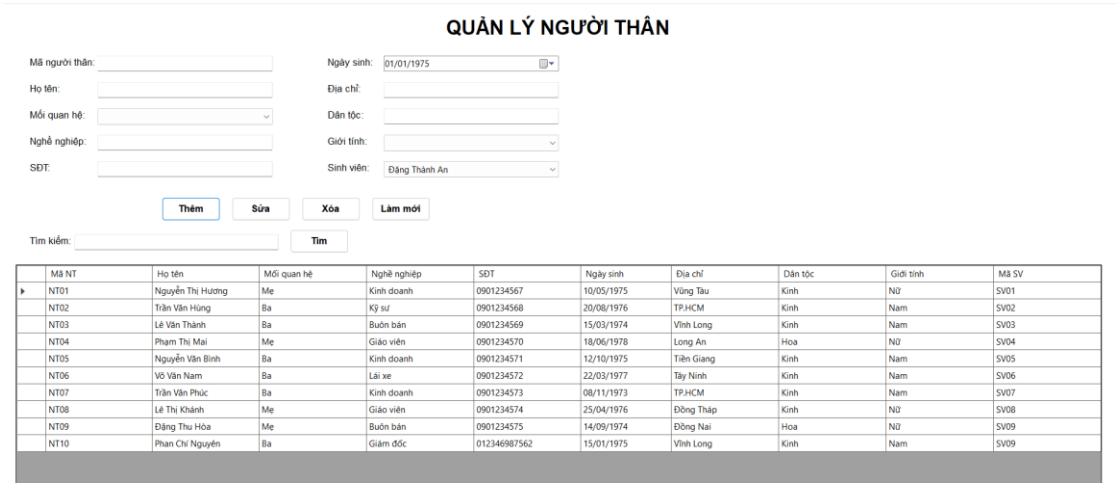
- Giao diện màn hình sau khi thêm người thân: hiện bản thêm NT10 thành công.



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901		Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901		TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901		Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901		Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901		Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09

2. Sửa người thân

- Giao diện màn hình trước khi sửa người thân:



- Code C# GUI xử lí nút sửa: GUI kiểm tra người dùng đã chọn người thân cần sửa, sau đó lấy dữ liệu từ giao diện tạo thành NguoiThanDTO và gửi đến BUS để cập nhật. Nếu cập nhật thành công, chương trình thông báo và tải lại danh sách dữ liệu.

```
1 reference
private void btnSua_Click(object sender, EventArgs e)
{
    if (dgvNguoiThan.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn người thân cần sửa!");
        return;
    }

    try
    {
        NguoiThanDTO nt = LayDuLieu();

        if (!nguoiThanBUS.Sua(nt))
        {
            MessageBox.Show(
                "Cập nhật thất bại! Kiểm tra dữ liệu.");
            return;
        }

        MessageBox.Show(
            "Cập nhật người thân thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể cập nhật người thân!\n" + ex.Message);
    }
}
```

- Code C# BUS sửa người thân: BUS kiểm tra dữ liệu người thân có hợp lệ hay không. Nếu hợp lệ, BUS gọi DAL để thực hiện cập nhật thông tin người thân trong cơ sở dữ liệu.

```
// Sửa người thân
1 reference
public bool Sua(NguoiThanDTO nt)
{
    if (!DuLieuHopLe(nt))
        return false;

    return dal.Update(nt);
}
```

- Code C# DAL Stored Procedure sửa người thân: DAL kết nối đến SQL Server, gọi Stored Procedure sp_SuaNT, truyền thông tin người thân thông qua GanThamSo() và thực thi Stored Procedure để cập nhật dữ liệu trong bảng NguoiThan.

```
// Sửa bằng Stored Procedure
1 reference
public bool Update(NguoiThanDTO nt)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_SuaNT", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, nt);
            cmd.ExecuteNonQuery();

            return true;
        }
```

- Code SQL

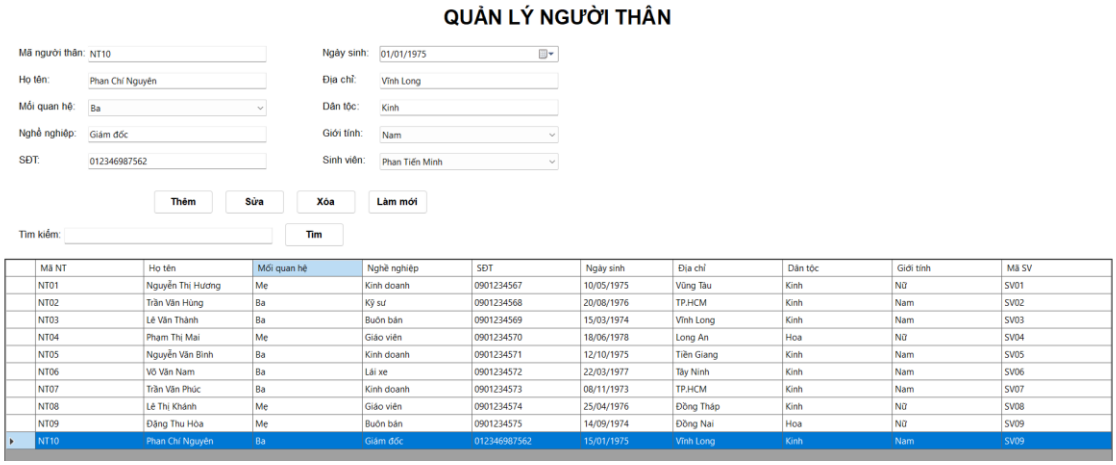
```
-- Tạo Procedure sửa Người Thân
create procedure sp_SuaNT
    @MaNT varchar(10),
    @HoTen nvarchar(50),
    @MoiQuanHe nvarchar(20),
    @NgheNghiep nvarchar(20),
    @SDT varchar(20),
    @NgaySinh date,
    @DiaChi nvarchar(100),
    @DanToc nvarchar(20),
    @GioiTinh nvarchar(10),
    @MaSV varchar(10)
as
begin
    update NguoiThan
    set
        HoTen = @HoTen,
        MoiQuanHe = @MoiQuanHe,
        NgheNghiep = @NgheNghiep,
        SDT = @SDT,
        NgaySinh = @NgaySinh,
        DiaChi = @DiaChi,
        DanToc = @DanToc,
        GioiTinh = @GioiTinh,
        MaSV = @MaSV
    where MaNT = @MaNT;
end;
go
```

Nhiệm vụ:

+ Dùng để sửa thông tin người thân.

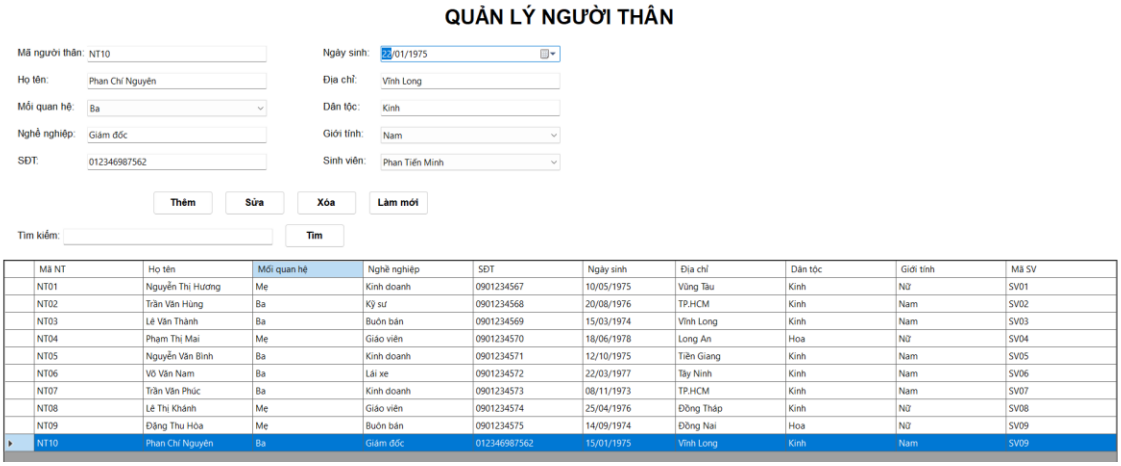
+Dựa vào MaNT để xác định người thân cần sửa, cập nhật các thông tin còn lại.

- Giao diện màn hình chọn người thân để sửa: Click chuột vào ô NT10 để sửa



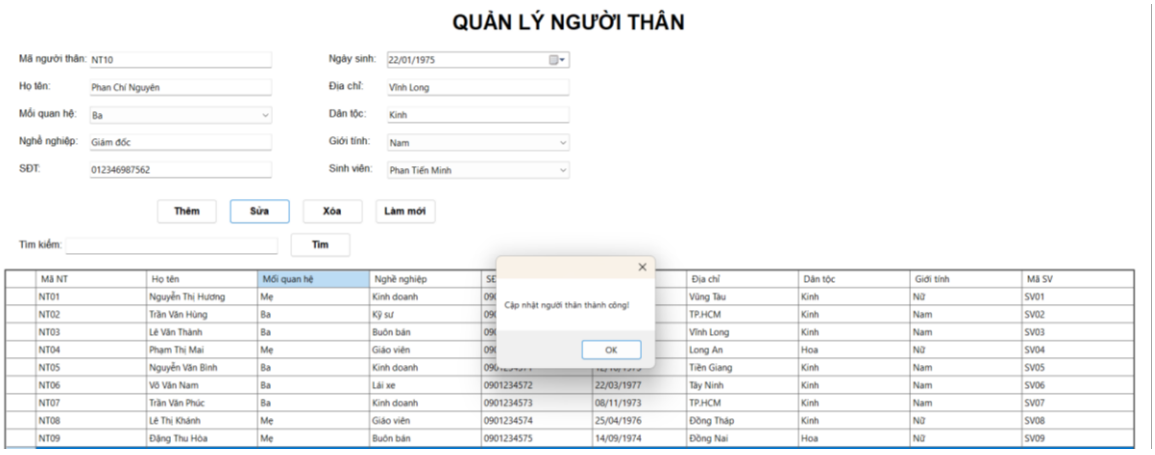
Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09
NT10	Phan Chí Nguyên	Ba	Giám đốc	012346987562	15/01/1975	Vĩnh Long	Kinh	Nam	SV09

- Giao diện màn hình khi sửa người thân: sửa ngày sinh 15/01/1975 thành 22/01/1975



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09
NT10	Phan Chí Nguyên	Ba	Giám đốc	012346987562	15/01/1975	Vĩnh Long	Kinh	Nam	SV09

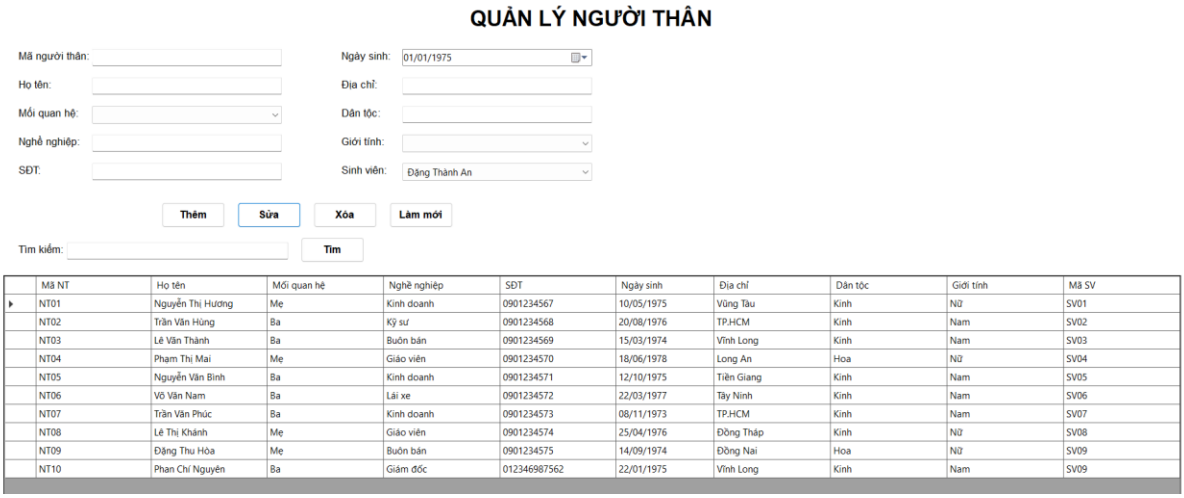
- Giao diện màn hình khi bấm sửa người thân : hiện “ cập nhật người thân thành công!”.



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09

3. Xóa người thân

- Giao diện màn hình trước khi xóa người thân

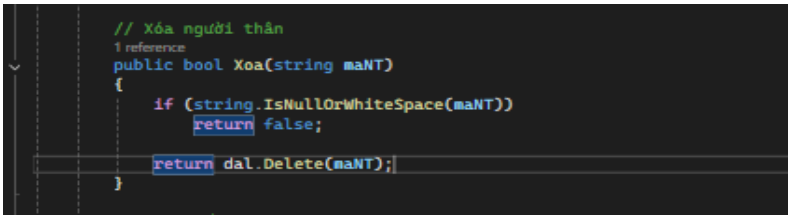


Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09
NT10	Phan Chí Nguyên	Ba	Giám đốc	012346987562	22/01/1975	Vĩnh Long	Kinh	Nam	SV09

- Code C# GUI xóa người thân: GUI kiểm tra người dùng đã chọn người thân cần xóa, lấy mã người thân và gửi mã này đến BUS để thực hiện xóa. Sau khi xóa thành công, danh sách được tải lại và giao diện được làm mới.

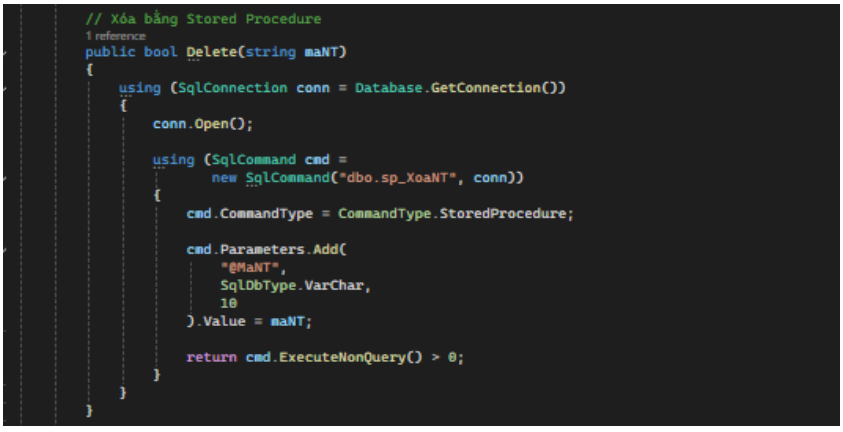
```
1 reference
private void btnXoa_Click(object sender, EventArgs e)
{
    if (dgvNguoiThan.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn người thân cần xóa!");
        return;
    }

    string ma = dgvNguoiThan.CurrentRow
        .Cells["MaNT"]
        .Value?.ToString() ?? "";

    if (MessageBox.Show(
            "Bạn có chắc muốn xóa " + ma + "?",
            "Xác nhận",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question)
        != DialogResult.Yes)
    {
        return;
    }

    try
    {
        if (!nguoiThanBUS.Xoa(ma))
        {
            MessageBox.Show("Xóa người thân thất bại!");
            return;
        }

        MessageBox.Show("Xóa người thân thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể xóa người thân!\n" + ex.Message);
    }
}
```

- Code C# gọi BUS xóa người thân: BUS kiểm tra mã người thân có hợp lệ hay không. Nếu mã hợp lệ, BUS gọi DAL để thực hiện xóa người thân trong cơ sở dữ liệu.



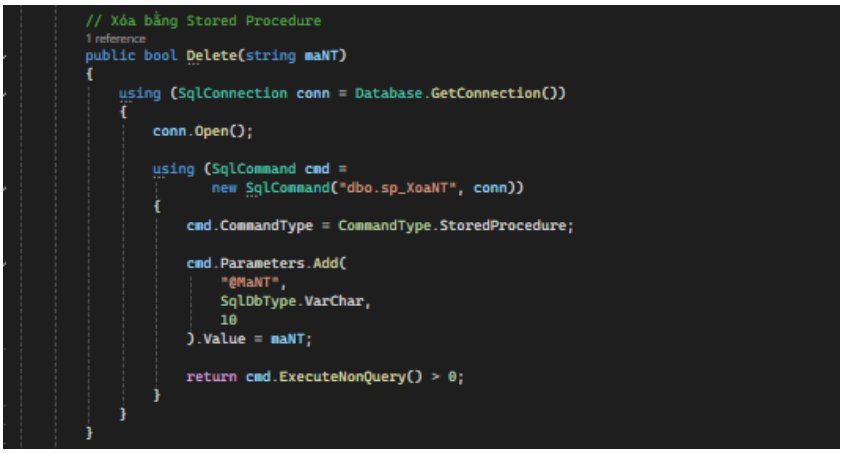
```
// Xóa người thân
1 reference
public bool Xoa(string maNT)
{
    if (string.IsNullOrWhiteSpace(maNT))
        return false;

    return dal.Delete(maNT);
}
```

- Code C# DAL gọi Stored Procedure xóa người thân: DAL kết nối SQL Server, gọi sp_XoaNT, truyền mã người thân MaNT và thực thi xóa người thân khỏi CSDL.



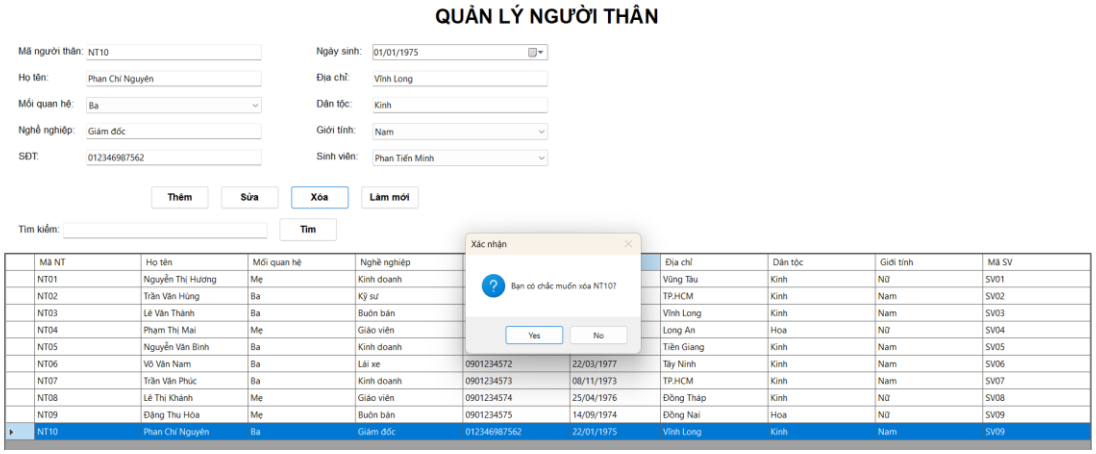
```
// Xóa bằng Stored Procedure
1 reference
public bool Delete(string maNT)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_XoaNT", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaNT",
                SqlDbType.VarChar,
                10
            ).Value = maNT;

            return cmd.ExecuteNonQuery() > 0;
        }
    }
}
```

- Giao diện trước khi xóa người thân: Click vô NT10 cho hiện thông tin người thân



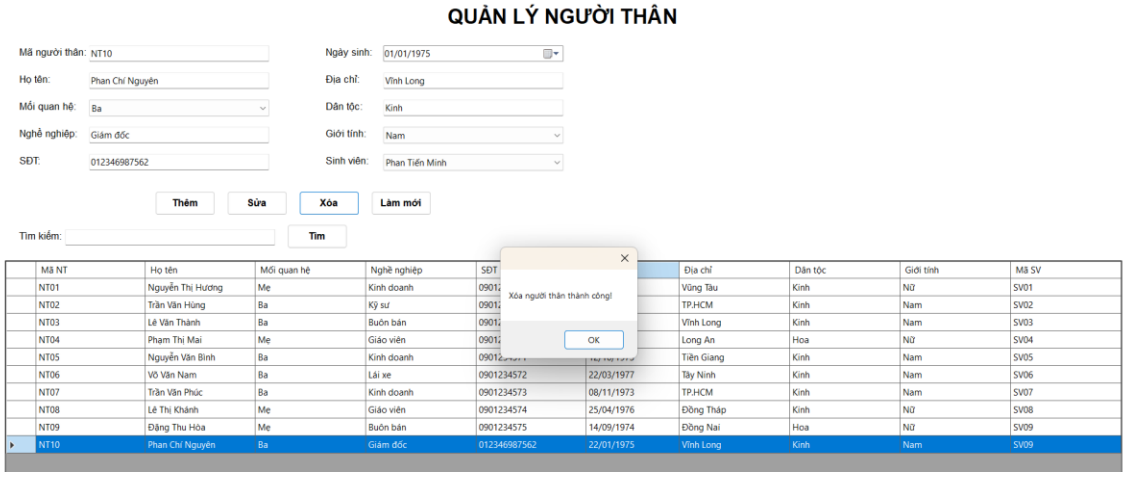
```
// Xóa bằng Stored Procedure
1 reference
public bool Delete(string maNT)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_XoaNT", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaNT",
                SqlDbType.VarChar,
                10
            ).Value = maNT;

            return cmd.ExecuteNonQuery() > 0;
        }
    }
}
```

- Giao diện khi xóa người thân: bấm nút xóa thì hiện “bạn có chắc chắn xóa”



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09
NT10	Phan Chí Nguyên	Ba	Giám đốc	012346987562	22/01/1975	Vĩnh Long	Kinh	Nam	SV09

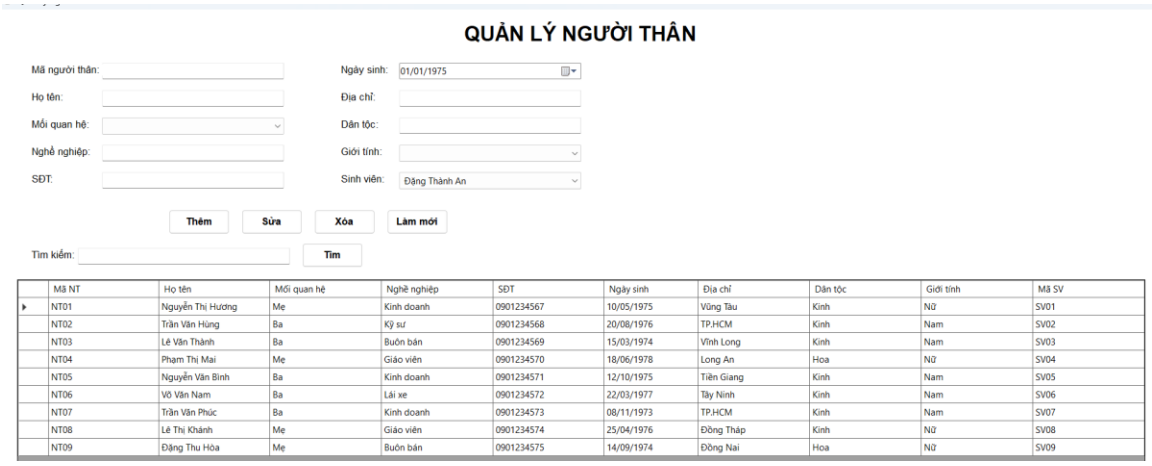
- Giao diện đã xóa người thân thành công: Bấm yes->hiện " xóa người thân thành công!"



Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09
NT10	Phan Chí Nguyên	Ba	Giám đốc	012346987562	22/01/1975	Vĩnh Long	Kinh	Nam	SV09

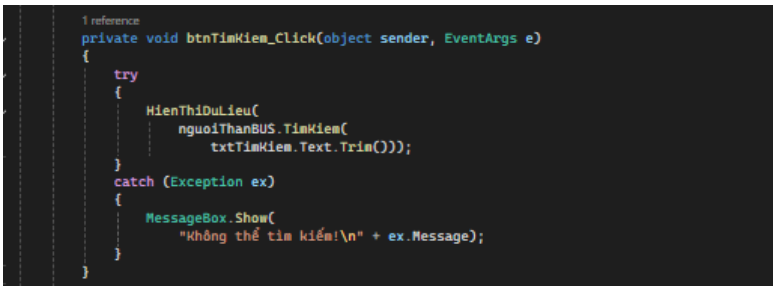
4. Tìm người thân

- Giao diện màn hình trước khi tìm người thân



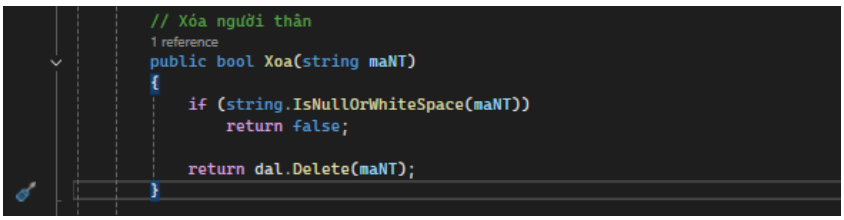
Mã NT	Họ tên	Mối quan hệ	Nghề nghiệp	SĐT	Ngày sinh	Địa chỉ	Dân tộc	Giới tính	Mã SV
NT01	Nguyễn Thị Hương	Mẹ	Kinh doanh	0901234567	10/05/1975	Vũng Tàu	Kinh	Nữ	SV01
NT02	Trần Văn Hùng	Ba	Kỹ sư	0901234568	20/08/1976	TP.HCM	Kinh	Nam	SV02
NT03	Lê Văn Thành	Ba	Buôn bán	0901234569	15/03/1974	Vĩnh Long	Kinh	Nam	SV03
NT04	Phạm Thị Mai	Mẹ	Giáo viên	0901234570	18/06/1978	Long An	Hoa	Nữ	SV04
NT05	Nguyễn Văn Bình	Ba	Kinh doanh	0901234571	12/10/1975	Tiền Giang	Kinh	Nam	SV05
NT06	Võ Văn Nam	Ba	Lái xe	0901234572	22/03/1977	Tây Ninh	Kinh	Nam	SV06
NT07	Trần Văn Phúc	Ba	Kinh doanh	0901234573	08/11/1973	TP.HCM	Kinh	Nam	SV07
NT08	Lê Thị Khánh	Mẹ	Giáo viên	0901234574	25/04/1976	Đồng Tháp	Kinh	Nữ	SV08
NT09	Đặng Thu Hòa	Mẹ	Buôn bán	0901234575	14/09/1974	Đồng Nai	Hoa	Nữ	SV09

- Code C# GUI tìm người thân: GUI nhận từ khóa tìm kiếm từ ô nhập liệu, gọi BUS để thực hiện tìm kiếm và hiển thị kết quả lên giao diện. Nếu xảy ra lỗi, chương trình sẽ thông báo cho người dùng.



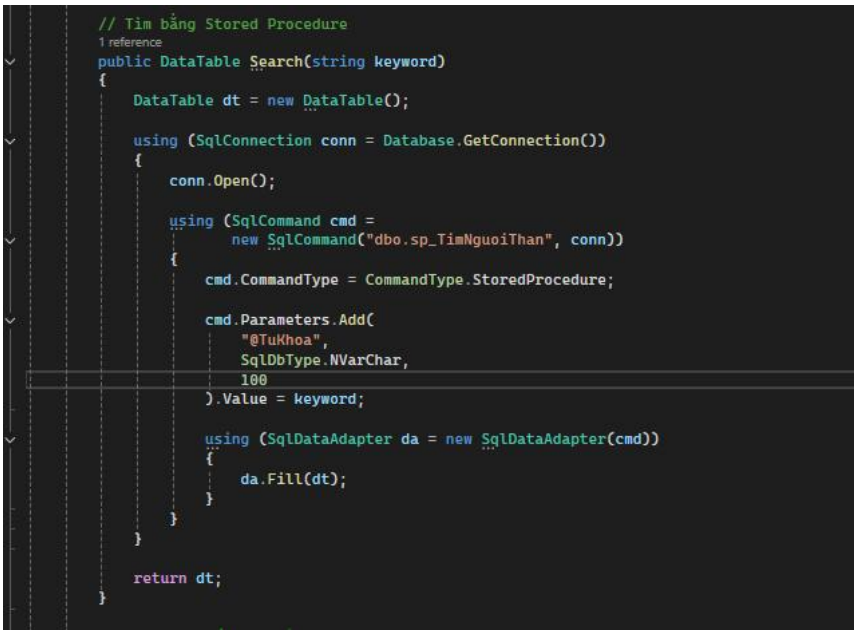
```
1 reference
private void btnTimKiem_Click(object sender, EventArgs e)
{
    try
    {
        HienThiDuLieu(
            nguoiThanBUS.TimKiem(
                txtTimKiem.Text.Trim()));
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể tìm kiếm!\n" + ex.Message);
    }
}
```

- Code C# gọi BUS tìm người thân: BUS kiểm tra từ khóa tìm kiếm. Nếu từ khóa rỗng thì lấy toàn bộ danh sách người thân, nếu có từ khóa thì gọi DAL để tìm kiếm theo từ khóa.



```
// Xóa người thân
1 reference
public bool Xoa(string maNT)
{
    if (string.IsNullOrWhiteSpace(maNT))
        return false;

    return dal.Delete(maNT);
}
```

- Code C# gọi DAL Stored Procedure tìm người thân: DAL kết nối đến SQL Server, gọi sp_TimNguoiThan và truyền từ khóa tìm kiếm vào tham số @TuKhoa. Kết quả được lấy vào DataTable và trả về cho BUS để hiển thị trên giao diện.



```
// Tìm bằng Stored Procedure
1 reference
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand("dbo.sp_TimNguoiThan", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da = new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
        }
    }

    return dt;
}
```

- Code SQL

```sql
-- Tìm người thân
CREATE PROCEDURE dbo.sp_TimNguoiThan
    @TuKhoa NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT *
    FROM dbo.vw_NguoiThanDayDu
    WHERE
        MaNT LIKE '%' + @TuKhoa + '%'
        OR HoTen LIKE N'%' + @TuKhoa + N'%'
        OR MaSV LIKE '%' + @TuKhoa + '%'
    ORDER BY MaNT;
END;
GO

-- Kiểm tra người thân tồn tại
CREATE OR ALTER PROCEDURE dbo.sp_KiemTraNguoiThanTonTai
    @MaNT VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS
    (
        SELECT 1
        FROM dbo.NguoiThan
        WHERE MaNT = @MaNT
    )
        SELECT CAST(1 AS BIT);
    ELSE
        SELECT CAST(0 AS BIT);
END;
GO

```

Nhiệm vụ:

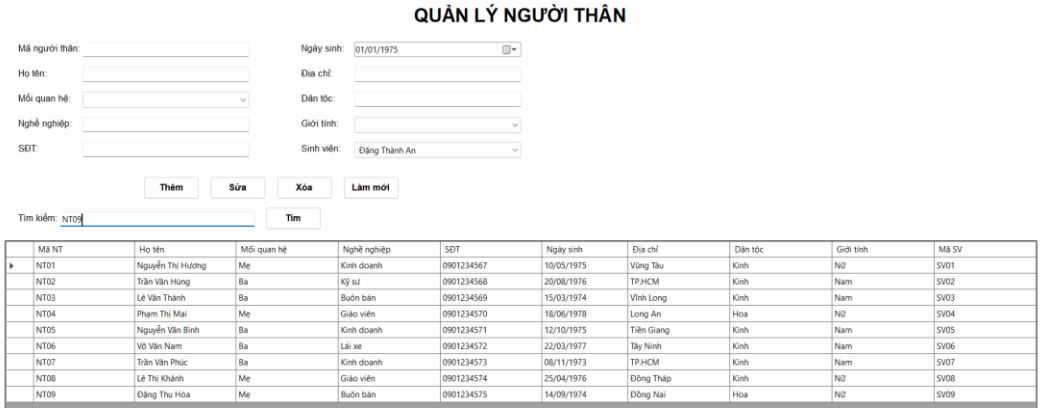
+Dùng để xóa người thân.

+Chỉ cần truyền @MaNT, sau đó:

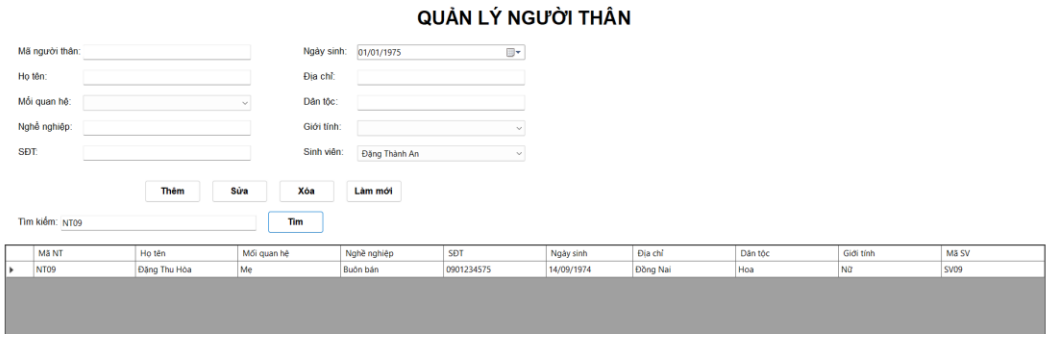
delete from NguoiThan

where MaNT = @MaNT

- Giao diện khi tìm kiếm: nhập mã người thân NT09



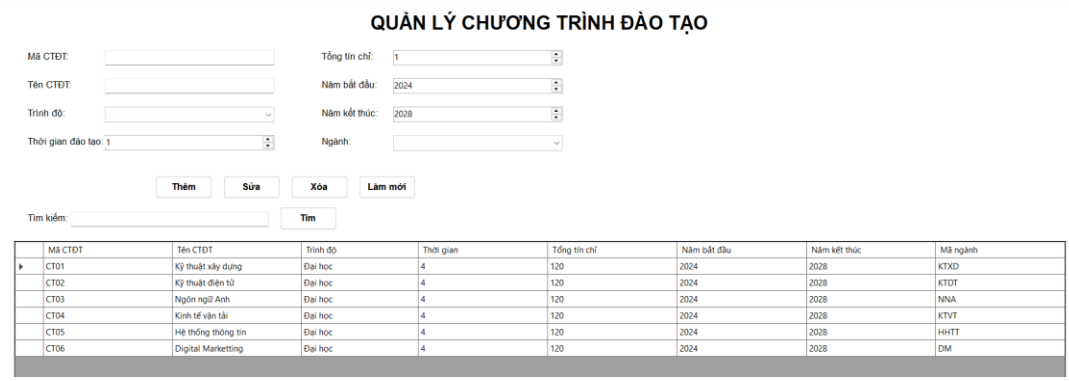
- Giao diện tìm kiếm thành công NT09



CHƯƠNG TRÌNH ĐÀO TẠO

1. Thêm CTDT

- Giao diện màn hình trước khi thêm CTDT:



- Code C# GUI thêm CTDT: GUI lấy thông tin chương trình đào tạo từ giao diện thông qua LayDuLieu(), tạo ChuongTrinhDaoTaoDTO và gửi đến BUS để thực hiện thêm. Nếu thêm thành công, chương trình thông báo, tải lại danh sách và xóa trắng các ô nhập.

```
private void btnThem_Click(
    object sender,
    EventArgs e)
{
    try
    {
        ChuongTrinhDaoTaoDTO ct =
            LayDuLieu();

        if (!ctdtBUS.Them(ct))
        {
            MessageBox.Show(
                "Thêm thất bại! Kiểm tra dữ liệu hoặc mã đã tồn tại.");
            return;
        }

        MessageBox.Show(
            "Thêm chương trình đào tạo thành công!");

        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể thêm chương trình đào tạo!\n" +
            ex.Message);
    }
```

- Code C# DTO thêm CTDT: DTO dùng để đóng gói các thông tin của chương trình đào tạo và truyền dữ liệu từ GUI xuống BUS.

```
        {
            ChuongTrinhDaoTaoDTO ct =
                LayDuLieu();

            if (!ctdtBUS.Sua(ct))
```

```
10 references
public class ChuongTrinhDaoTaoDTO
{
    5 references
    public string MaCTDT { get; set; } = "";
    3 references
    public string TenCTDT { get; set; } = "";
    3 references
    public string TrinhDo { get; set; } = "";

    3 references
    public int ThoiGianDaoTao { get; set; }
    3 references
    public int TongTinChi { get; set; }
    4 references
    public int NamBatDau { get; set; }
    4 references
    public int NamKetThuc { get; set; }

    3 references
    public string MaNganh { get; set; } = "";
}
}
```

- Code C# BUS thêm CTDT: BUS kiểm tra dữ liệu chương trình đào tạo hợp lệ và thực hiện các kiểm tra cần thiết trước khi gọi DAL để thêm dữ liệu.

```
// Thêm CTĐT
1 reference
public bool Them(ChuongTrinhDaoTaoDTO ct)
{
    if (!DuLieuHopLe(ct))
        return false;

    if (dal.KiemTraTonTai(ct.MaCTDT))
        return false;

    return dal.Insert(ct);
}
```

- Code C# DAL gọi Stored Procedure thêm CTDT: DAL kết nối với SQL Server, truyền dữ liệu chương trình đào tạo và gọi Stored Procedure để thêm dữ liệu vào cơ sở dữ liệu.

```
// Thêm CTĐT
1 reference
public bool Insert(ChuongTrinhDaoTaoDTO ct)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_ThemCTDT",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, ct);
            cmd.ExecuteNonQuery();

            return true;
        }
```

- Code SQL: Stored Procedure nhận các tham số từ DAL và thực hiện câu lệnh thêm chương trình đào tạo vào bảng ChuongTrinhDaoTao.

```sql
-- Tạo Procedure thêm Chương Trình Đào Tạo
create procedure sp_ThemCTDT
    @MaCTDT varchar(10),
    @TenCTDT nvarchar(50),
    @TrinhDo nvarchar(20),
    @ThoiGianDaoTao int,
    @TongTinChi int,
    @NamBatDau int,
    @NamKetThuc int,
    @MaNganh varchar(10)
as
begin
    insert into ChuongTrinhDaoTao
    (MaCTDT,TenCTDT,TrinhDo,ThoiGianDaoTao,
    TongTinChi,NamBatDau,NamKetThuc,MaNganh)
    values
    (@MaCTDT,@TenCTDT,@TrinhDo,@ThoiGianDaoTao,
    @TongTinChi,@NamBatDau,@NamKetThuc,@MaNganh);
end;
go
--Tìm CTDT
CREATE OR ALTER PROCEDURE dbo.sp_TimCTDT
    @TuKhoa NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        MaCTDT,
        TenCTDT,
        TrinhDo,
        ThoiGianDaoTao,
        TongTinChi,
        NamBatDau,
        NamKetThuc,
        MaNganh,
        TenNganh
    FROM dbo.vw_CTDTDayDu
    WHERE
        MaCTDT LIKE '%' + @TuKhoa + '%'
        OR TenCTDT LIKE N'%' + @TuKhoa + '%'
        OR TrinhDo LIKE N'%' + @TuKhoa + '%'
        OR MaNganh LIKE '%' + @TuKhoa + '%'
        OR TenNganh LIKE N'%' + @TuKhoa + '%'
    ORDER BY MaCTDT;
END;
GO
```

Nhiệm vụ:

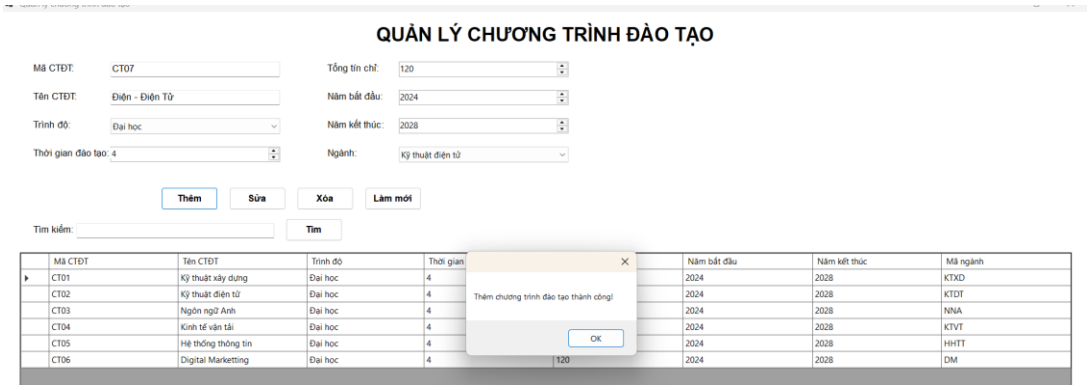
+ Dùng để thêm chương trình đào tạo.

+ Nhận các thông tin của chương trình: Mã CTĐT ,Tên CTĐT ,Trình độ ,Mã ngành

+ Rồi thêm vào bảng ChuongTrinhDaoTao.

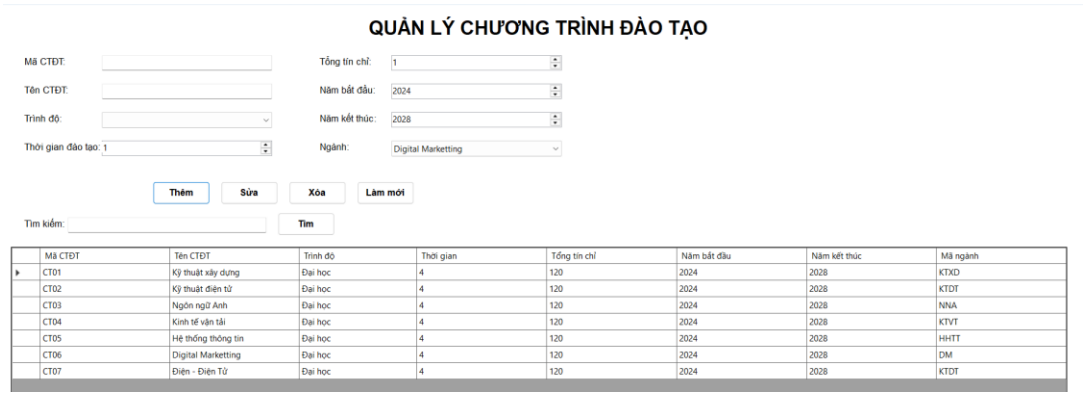
- Giao diện thêm CTDT : thêm CT07 , nhập hết dữ diệu của chương trình

- Giao diện đã thêm CTDT CT07 xong hiển thị " thêm CT thành công!"



Mã CTĐT	Tên CTĐT	Trình độ	Thời gian	Tổng tín chỉ	Năm bắt đầu	Năm kết thúc	Mã ngành
CT01	Kỹ thuật xây dựng	Đại học	4	120	2024	2028	KTXD
CT02	Kỹ thuật điện tử	Đại học	4	120	2024	2028	KTDT
CT03	Ngôn ngữ Anh	Đại học	4	120	2024	2028	NNA
CT04	Kinh tế vận tải	Đại học	4	120	2024	2028	KTVT
CT05	Hệ thống thông tin	Đại học	4	120	2024	2028	HHTT
CT06	Digital Marketting	Đại học	4	120	2024	2028	DM

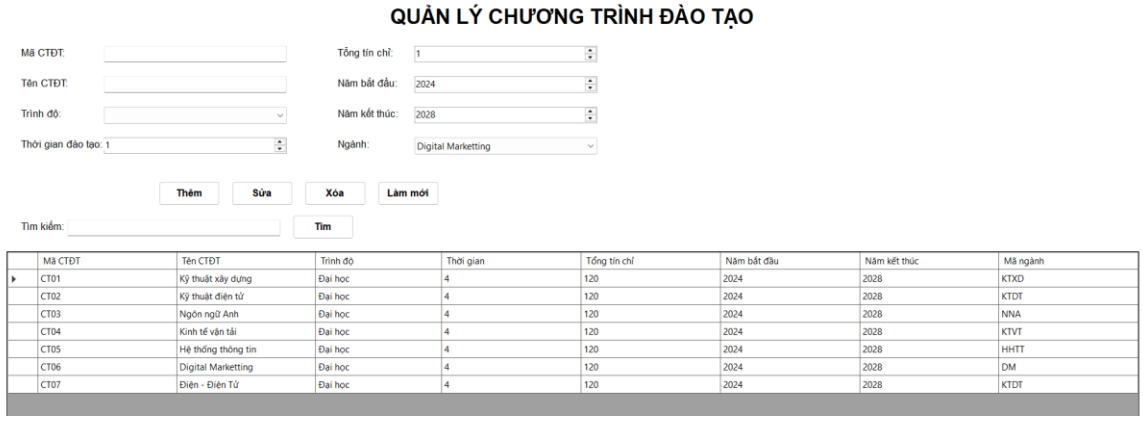
- Giao diện CTDT đã thêm



Mã CTĐT	Tên CTĐT	Trình độ	Thời gian	Tổng tín chỉ	Năm bắt đầu	Năm kết thúc	Mã ngành
CT01	Kỹ thuật xây dựng	Đại học	4	120	2024	2028	KTXD
CT02	Kỹ thuật điện tử	Đại học	4	120	2024	2028	KTDT
CT03	Ngôn ngữ Anh	Đại học	4	120	2024	2028	NNA
CT04	Kinh tế vận tải	Đại học	4	120	2024	2028	KTVT
CT05	Hệ thống thông tin	Đại học	4	120	2024	2028	HHTT
CT06	Digital Marketting	Đại học	4	120	2024	2028	DM
CT07	Điện - Điện Tử	Đại học	4	120	2024	2028	KTDT

2. Sửa CTDT

- Giao diện màn hình trước khi sửa CTDT:



Mã CTĐT	Tên CTĐT	Trình độ	Thời gian	Tổng tín chỉ	Năm bắt đầu	Năm kết thúc	Mã ngành
CT01	Kỹ thuật xây dựng	Đại học	4	120	2024	2028	KTXD
CT02	Kỹ thuật điện tử	Đại học	4	120	2024	2028	KTDT
CT03	Ngôn ngữ Anh	Đại học	4	120	2024	2028	NNA
CT04	Kinh tế vận tải	Đại học	4	120	2024	2028	KTVT
CT05	Hệ thống thông tin	Đại học	4	120	2024	2028	HHTT
CT06	Digital Marketting	Đại học	4	120	2024	2028	DM
CT07	Điện - Điện Tử	Đại học	4	120	2024	2028	KTDT

- C# Code GUI sửa CTDT: GUI lấy thông tin CTDT từ giao diện thông qua LayDuLieu(), đóng gói vào ChuongTrinhDaoTaoDTO và gọi ctdtBUS.Sua(ct) để sửa.

```
private void btnSua_Click(
    object sender,
    EventArgs e)
{
    if (dgvCTDT.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn chương trình cần sửa!");
        return;
    }

    try
    {
        ChuongTrinhDaoTaoDTO ct =
            LayDuLieu();

        if (!ctdtBUS.Sua(ct))
        {
            MessageBox.Show(
                "Cập nhật thất bại! Kiểm tra dữ liệu.");
            return;
        }

        MessageBox.Show(
            "Cập nhật chương trình đào tạo thành công!");

        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể cập nhật chương trình đào tạo!\n" +
            ex.Message);
    }
}
```

- C# Code DTO sửa CTDT: Dữ liệu CTDT được lấy từ giao diện và đóng gói vào ChuongTrinhDaoTaoDTO ct, sau đó truyền cho BUS thông qua ctdtBUS.Sua(ct).

```
        {
            ChuongTrinhDaoTaoDTO ct =
                LayDuLieu();

            if (!ctdtBUS.Sua(ct))
```

- C# Code gọi BUS sửa CTDT: BUS kiểm tra dữ liệu CTDT có hợp lệ hay không. Nếu hợp lệ thì gọi dal.Update(ct) để thực hiện sửa.

```
// Sửa CTĐT
1 reference
public bool Sua(ChuongTrinhDaoTaoDTO ct)
{
    if (!DuLieuHopLe(ct))
        return false;

    if (!dal.KiemTraTonTai(ct.MaCTDT))
        return false;

    return dal.Update(ct);
}
```

- C# Code gọi DAL Stored Procedure sửa CTDT: DAL mở kết nối SQL Server, gọi Stored Procedure sp_SuaCTDT, truyền dữ liệu từ DTO qua GanThamSo() và thực hiện cập nhật bằng ExecuteNonQuery().

```
// Sửa CTĐT
1 reference
public bool Update(ChuongTrinhDaoTaoDTO ct)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_SuaCTDT",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, ct);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

- Code SQL: Stored Procedure sp_SuaCTDT nhận các tham số được truyền từ DAL và thực hiện cập nhật thông tin chương trình đào tạo trong cơ sở dữ liệu.

```
779
780   -- Tạo Procedure sửa Chương Trình Đào Tạo
781   create procedure sp_SuaCTDT
782       @MaCTDT varchar(10),
783       @TenCTDT nvarchar(50),
784       @TrinhDo nvarchar(20),
785       @ThoiGianDaoTao int,
786       @TongTinChi int,
787       @NamBatDau int,
788       @NamKetThuc int,
789       @MaNganh varchar(10)
790   as
791   begin
792       update ChuongTrinhDaoTao
793       set
794           TenCTDT = @TenCTDT,
795           TrinhDo = @TrinhDo,
796           ThoiGianDaoTao = @ThoiGianDaoTao,
797           TongTinChi = @TongTinChi,
798           NamBatDau = @NamBatDau,
799           NamKetThuc = @NamKetThuc,
800           MaNganh = @MaNganh
801       where MaCTDT = @MaCTDT;
802   end;
803   go
```

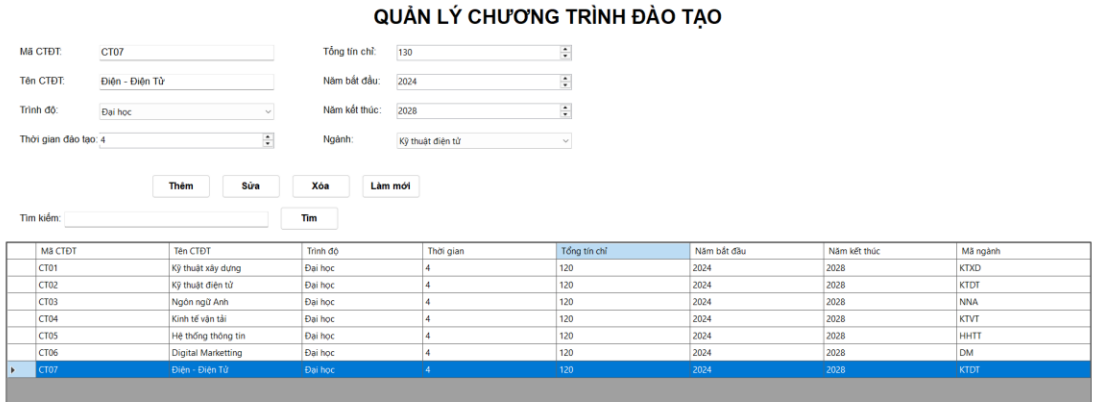
Nhiệm vụ:

+Dùng để sửa chương trình đào tạo.

+Dựa vào MaCTDT để xác định chương trình cần sửa, sau đó cập nhật các thông tin: Tên CTĐT, Trình độ ,Mã ngành

- Giao diện trước khi sửa CTDT: Click vô CTDT cho hiển thị thông tin để sửa

- Giao diện sửa CTDT : sửa số tín chỉ 120 thành 130



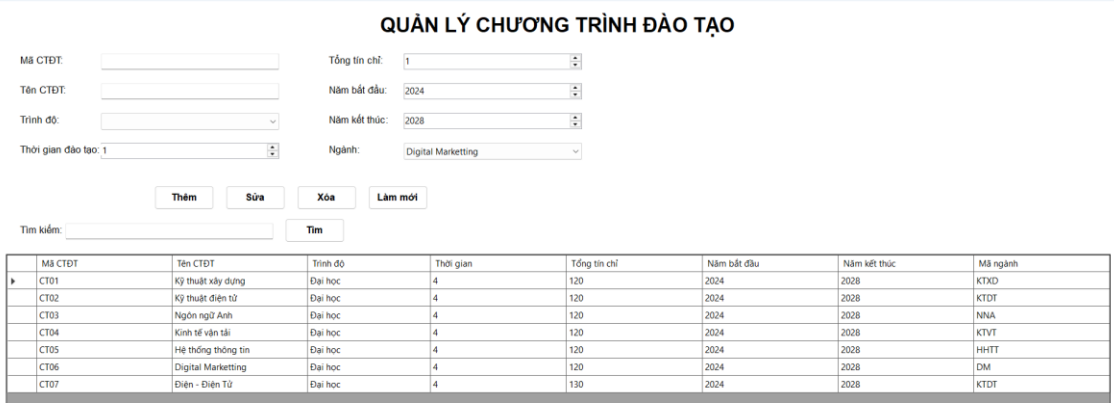
- Giao diện sửa CTDT : sửa xong hiển thị “ cập nhật CT thành công!”

- Giao diện hiển thị đã sửa xong CTDT

Mã CTĐT	Tên CTĐT	Trình độ	Thời gian	Tổng tín chỉ	Năm bắt đầu	Năm kết thúc	Mã ngành
CT01	Kỹ thuật xây dựng	Đại học	4	120	2024	2028	KTXD
CT02	Kỹ thuật điện tử	Đại học	4	120	2024	2028	KTDT
CT03	Ngôn ngữ Anh	Đại học	4	120	2024	2028	NNA
CT04	Kinh tế vận tải	Đại học	4	120	2024	2028	KTVT
CT05	Hệ thống thông tin	Đại học	4	120	2024	2028	HHTT
CT06	Digital Marketting	Đại học	4	120	2024	2028	DM
CT07	Điện - Điện Tử	Đại học	4	130	2024	2028	KTDT

3. Tìm CTĐT

- Giao diện màn hình trước khi tìm CTĐT



Mã CTĐT	Tên CTĐT	Trình độ	Thời gian	Tổng tín chỉ	Năm bắt đầu	Năm kết thúc	Mã ngành
CT01	Kỹ thuật xây dựng	Đại học	4	120	2024	2028	KTXD
CT02	Kỹ thuật điện tử	Đại học	4	120	2024	2028	KTDT
CT03	Ngôn ngữ Anh	Đại học	4	120	2024	2028	NNA
CT04	Kinh tế vận tải	Đại học	4	120	2024	2028	KTVT
CT05	Hệ thống thông tin	Đại học	4	120	2024	2028	HHTT
CT06	Digital Marketting	Đại học	4	120	2024	2028	DM
CT07	Điện - Điện Tử	Đại học	4	130	2024	2028	KTDT

- Code C# GUI tìm CTĐT: GUI lấy từ khóa người dùng nhập trong txtTimKiem, gọi hàm TimKiem() của BUS và hiển thị kết quả lên giao diện thông qua HienThiDuLieu().

```
// Tìm CTĐT
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_TimCTDT",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da =
                   new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
```

- Code C# gọi BUS tìm CTDT: BUS nhận từ khóa từ GUI. Nếu rỗng thì lấy toàn bộ danh sách CTĐT thông qua GetAll(). Nếu có thì gọi dal.Search() để tìm kiếm CTĐT.

```
// Tìm kiếm CTĐT
1 reference
public DataTable TimKiem(string keyword)
{
    if (string.IsNullOrWhiteSpace(keyword))
        return dal.GetAll();

    return dal.Search(keyword.Trim());
}
```

- Code C# gọi DAL Stored Procedure tìm CTDT: DAL kết nối đến SQL Server, gọi sp_TimCTDT và truyền từ khóa tìm kiếm vào tham số @TuKhoa. Sau đó SqlDataAdapter lấy kết quả từ SQL Server và lưu vào DataTable để trả về cho BUS.

```
// Tìm CTĐT
1 reference
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_TimCTDT",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da =
                   new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
        }
```

- Code SQL

```
--Tìm CTDT
CREATE OR ALTER PROCEDURE dbo.sp_TimCTDT
    @TuKhoa NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        MaCTDT,
        TenCTDT,
        TrinhDo,
        ThoiGianDaoTao,
        TongTinChi,
        NamBatDau,
        NamKetThuc,
        MaNganh,
        TenNganh
    FROM dbo.vw_CTDTDayDu
    WHERE
        MaCTDT LIKE '%' + @TuKhoa + '%'
        OR TenCTDT LIKE N'%' + @TuKhoa + '%'
        OR TrinhDo LIKE N'%' + @TuKhoa + '%'
        OR MaNganh LIKE '%' + @TuKhoa + '%'
        OR TenNganh LIKE N'%' + @TuKhoa + '%'
    ORDER BY MaCTDT;
END;
GO
```

Nhiệm vụ:

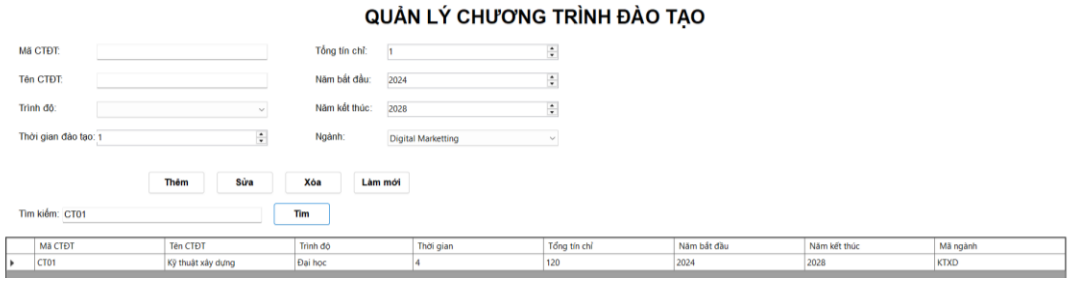
+ Dùng để tìm kiếm chương trình đào tạo.

+ Có thể tìm theo: Mã CTĐT, Tên CTĐT,Trình độ, Mã ngành,Tên ngành

+ SP sử dụng vw_CTDTDayDu để lấy cả thông tin ngành.

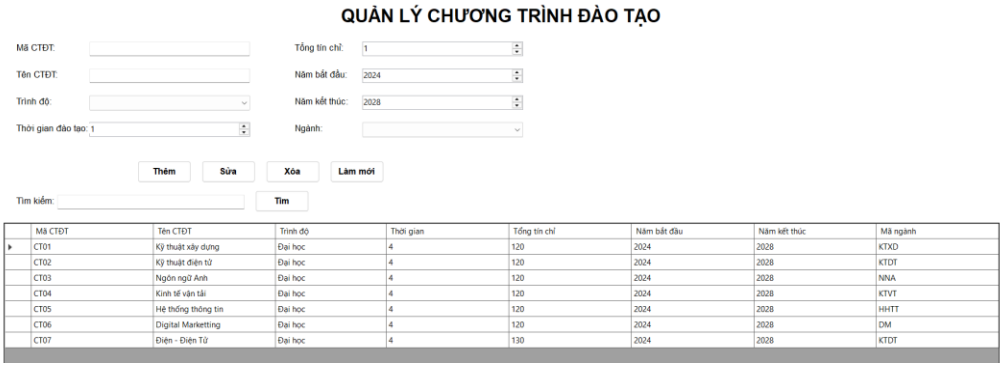
- Giao diện Tìm kiếm CTDT: nhập mã CTDT CT01

- Giao diện đã tìm kiếm xong CTDT

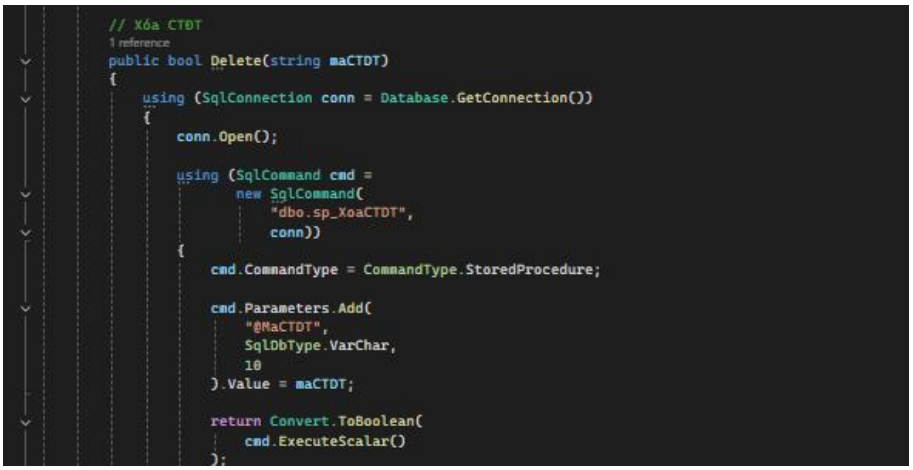


4. Xóa CTDT

- Giao diện màn hình trước khi xóa CTDT

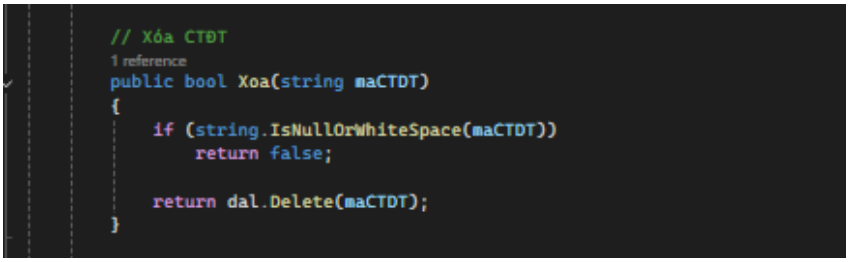


- Code C# GUI xóa CTDT: GUI kiểm tra người dùng đã chọn CTĐT chưa, sau đó lấy MaCTDT của dòng được chọn. Hệ thông hỏi xác nhận xóa. Nếu có thì GUI gọi ctdtBUS.Xoa(maCTDT). Khi xóa thành công, dữ liệu được tải lại và from trắng.



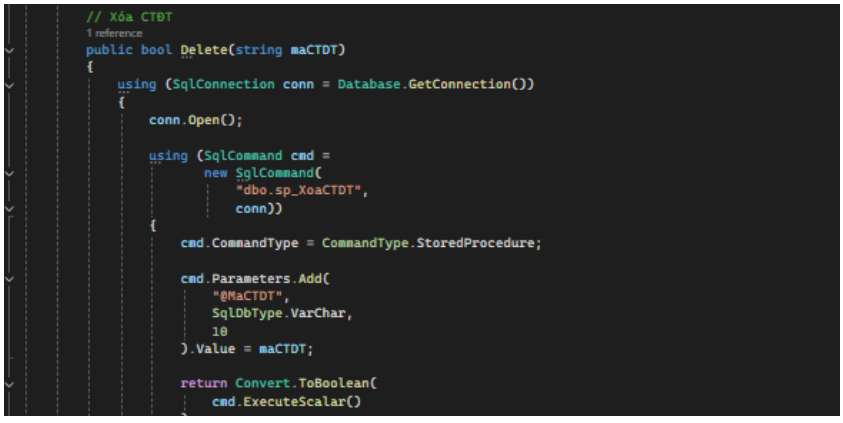
```
// Xóa CTĐT
1 reference
public bool Delete(string maCTDT)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_XoaCTDT",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaCTDT",
                SqlDbType.VarChar,
                10
            ).Value = maCTDT;

            return Convert.ToBoolean(
                cmd.ExecuteScalar()
            );
```

- Code C# gọi BUS xóa CTDT: BUS nhận MaCTDT từ GUI, kiểm tra mã có rỗng hay không. Nếu hợp lệ, BUS gọi dal.Delete(maCTDT) để thực hiện xóa dữ liệu.



```
// Xóa CTĐT
1 reference
public bool Xoa(string maCTDT)
{
    if (string.IsNullOrWhiteSpace(maCTDT))
        return false;

    return dal.Delete(maCTDT);
}
```

- Code C# gọi DAL Stored Procedure xóa CTDT: DAL mở kết nối đến SQL Server, gọi sp_XoaCTDT, truyền mã CTĐT vào tham số @MaCTDT và thực thi SP bằng ExecuteScalar(). Kết quả trả về được chuyển thành kiểu bool.



```
// Xóa CTĐT
1 reference
public bool Delete(string maCTDT)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_XoaCTDT",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaCTDT",
                SqlDbType.VarChar,
                10
            ).Value = maCTDT;

            return Convert.ToBoolean(
                cmd.ExecuteScalar()
```

- Code SQL

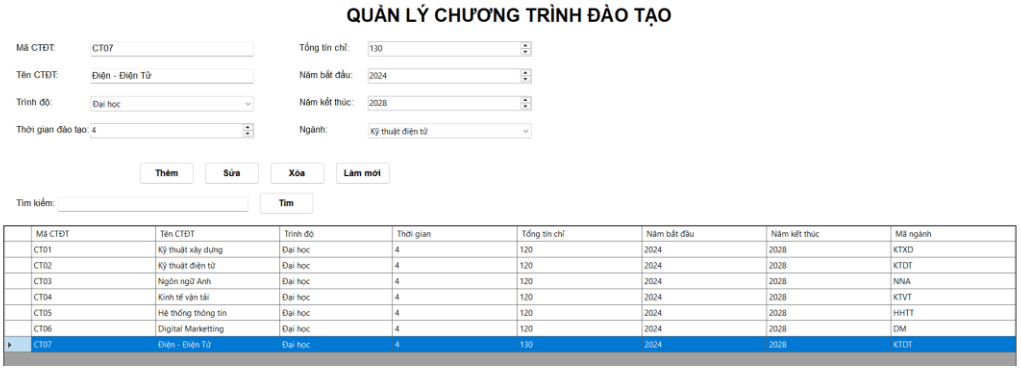
```sql
-- Xóa CTDT
CREATE OR ALTER PROCEDURE dbo.sp_XoaCTDT
    @MaCTDT VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    -- Không tìm thấy CTĐT
    IF NOT EXISTS
    (
        SELECT 1
        FROM dbo.ChuongTrinhDaoTao
        WHERE MaCTDT = @MaCTDT
    )
    BEGIN
        SELECT CAST(0 AS BIT);
        RETURN;
    END;

    -- Xóa CTĐT
    DELETE FROM dbo.ChuongTrinhDaoTao
    WHERE MaCTDT = @MaCTDT;

    SELECT CAST(1 AS BIT);
END;
GO

-- Kiểm tra mã CTĐT tồn tại
CREATE PROCEDURE dbo.sp_KiemTraCTDTTonTai
    @MaCTDT VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM dbo.ChuongTrinhDaoTao
        WHERE MaCTDT = @MaCTDT
    )
        SELECT CAST(1 AS BIT);
    ELSE
        SELECT CAST(0 AS BIT);
END;
GO

```

Nhiệm vụ:

Dùng để xóa chương trình đào tạo. SP sẽ:

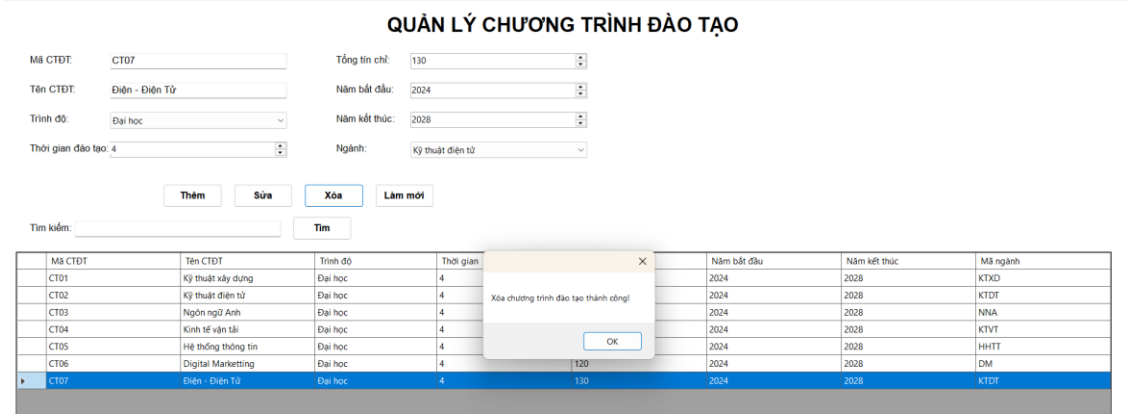
- Kiểm tra CTĐT có tồn tại không.
- Nếu có thì thực hiện xóa.
- Trả về kết quả 1 nếu xóa thành công.

- Giao diện hiển thị xóa CTDT : Click vô mã CT07 để hiển thị dữ liệu cần xóa

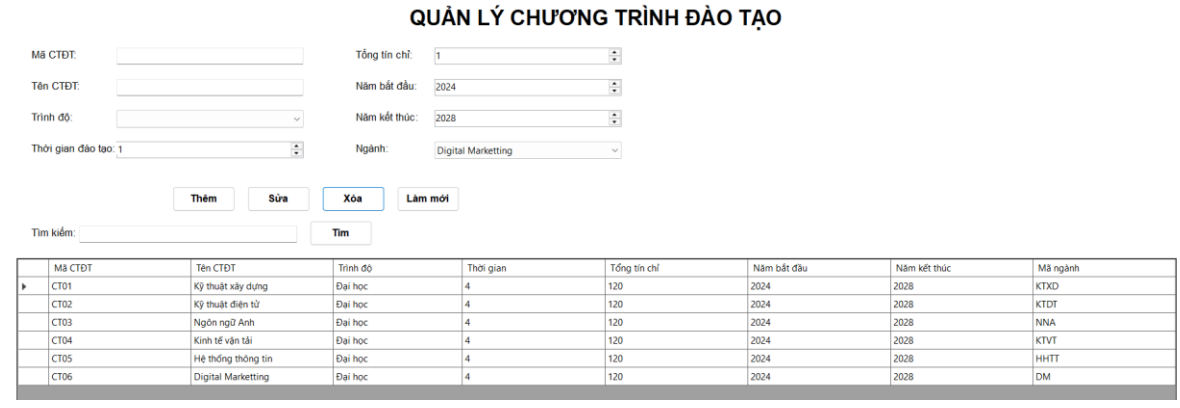


- Giao diện hiển thị có chắc chắn xóa CTDT

- Giao diện đã xóa thành công CTDT CT07: Bấm yes xóa thành công



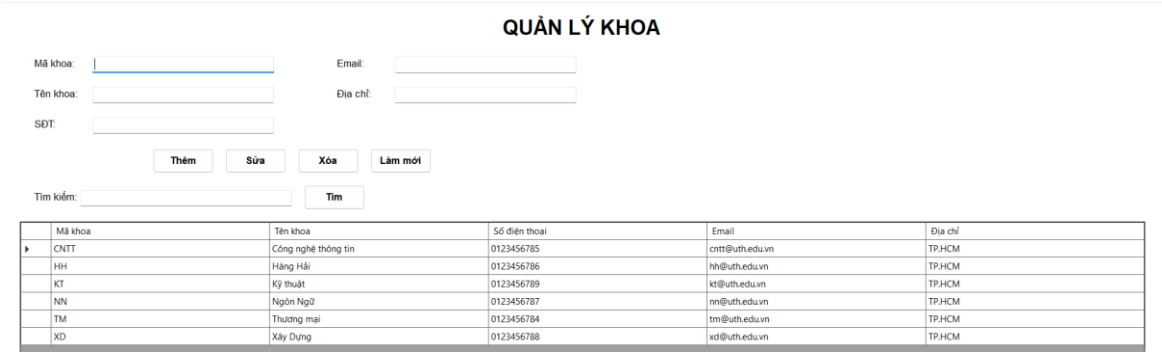
- Giao diện đã xóa CT07



KHOA

1. Thêm khoa

- Giao diện màn hình trước khi thêm khoa



- Code C# GUI thêm khoa: GUI lấy thông tin Khoa từ giao diện thông qua LayDuLieu() và đóng gói vào KhoaDTO. Sau đó gọi khoaBUS.Them(khoa) để chuyển dữ liệu xuống BUS xử lý. Nếu thêm thành công, hệ thống thông báo, tải lại danh sách.

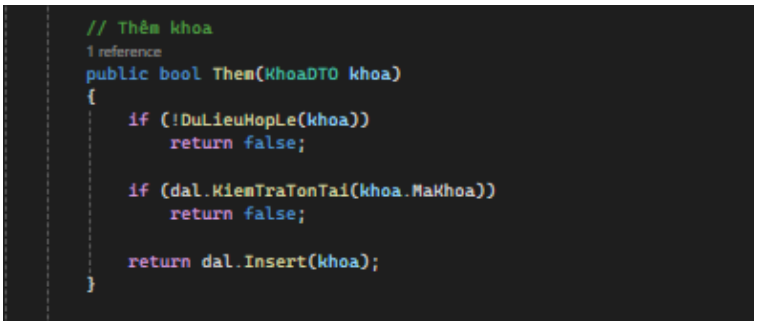
```
1 reference
private void btnThem_Click(object sender, EventArgs e)
{
    try
    {
        KhoaDTO khoa = LayDuLieu();

        if (!khoaBUS.Them(khoa))
        {
            MessageBox.Show(
                "Thêm khoa thất bại! Kiểm tra dữ liệu hoặc mã khoa bị trùng.");
            return;
        }

        MessageBox.Show("Thêm khoa thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể thêm khoa!\n" + ex.Message);
    }
}
```

- Code C# DTO thêm khoa: KhoaDTO dùng để đóng gói các thông tin của Khoa thành một đối tượng, giúp truyền dữ liệu từ GUI xuống BUS.

```
public class KhoaDTO
{
    5 references
    public string MaKhoa { get; set; } = "";
    3 references
    public string TenKhoa { get; set; } = "";
    2 references
    public string SDT { get; set; } = "";
    2 references
    public string Email { get; set; } = "";
    2 references
    public string DiaChi { get; set; } = "";
}
}
```

```
{
    KhoaDTO khoa = LayDuLieu();

    if (!khoaBUS.Them(khoa))
```

- Code C# gọi BUS thêm khoa: BUS nhận đối tượng KhoaDTO từ GUI, kiểm tra dữ liệu Khoa có hợp lệ hay không và kiểm tra mã Khoa đã tồn tại trong cơ sở dữ liệu chưa. Nếu dữ liệu hợp lệ và mã Khoa chưa tồn tại thì BUS gọi DAL để thực hiện thêm Khoa.



```
// Thêm khoa
1 reference
public bool Them(KhoaDTO khoa)
{
    if (!DuLieuHopLe(khoa))
        return false;

    if (dal.KiemTraTonTai(khoa.MaKhoa))
        return false;

    return dal.Insert(khoa);
}
```

- Code C# gọi DAL Stored Procedure thêm khoa: “DAL kết nối SQL Server, gọi Stored Procedure sp_ThemKhoa, dùng GanThamSo để truyền dữ liệu từ DTO vào các tham số của Stored Procedure, sau đó dùng ExecuteNonQuery() để thực hiện thêm Khoa.”

```
// Thêm khoa
1 reference
public bool Insert(KhoaDTO khoa)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_ThemKhoa",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, khoa);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

- Code SQL

```
618   -- Tạo Procedure thêm Khoa
619   create procedure sp_ThemKhoa
620       @MaKhoa varchar (10),
621       @TenKhoa nvarchar(50),
622       @SDT varchar(20),
623       @Email varchar(50),
624       @DiaChi nvarchar(100)
625   as
626   begin
627       insert into Khoa
628       (MaKhoa,TenKhoa,SDT,Email,DiaChi)
629       values
630       (@MaKhoa,@TenKhoa,@SDT,@Email,@DiaChi);
631   end;
632   go
```

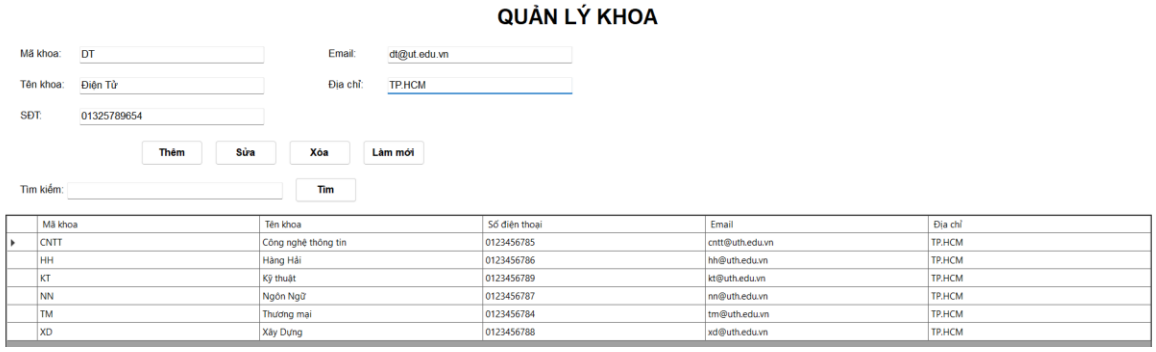
Nhiệm vụ:

+Dùng để thêm khoa mới vào bảng Khoa.

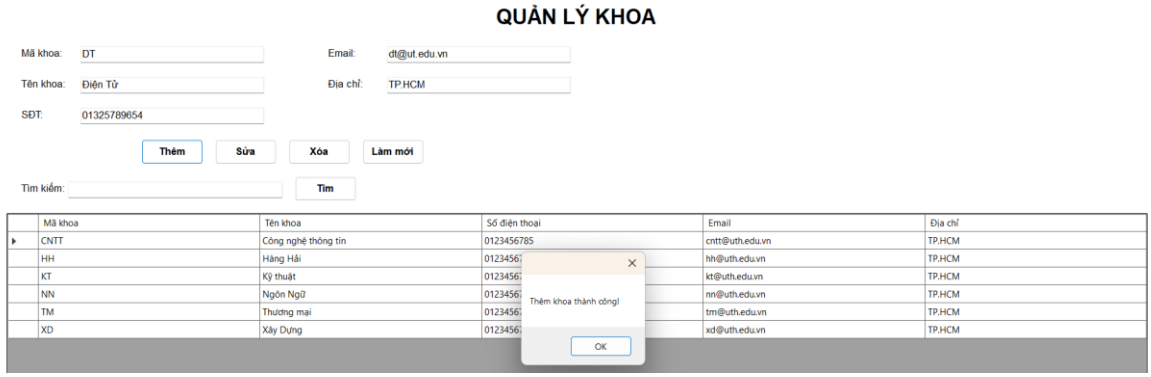
+Nhận các thông tin: Mã khoa, Tên khoa,SĐT,Email , Địa chỉ

+ Sau đó INSERT.

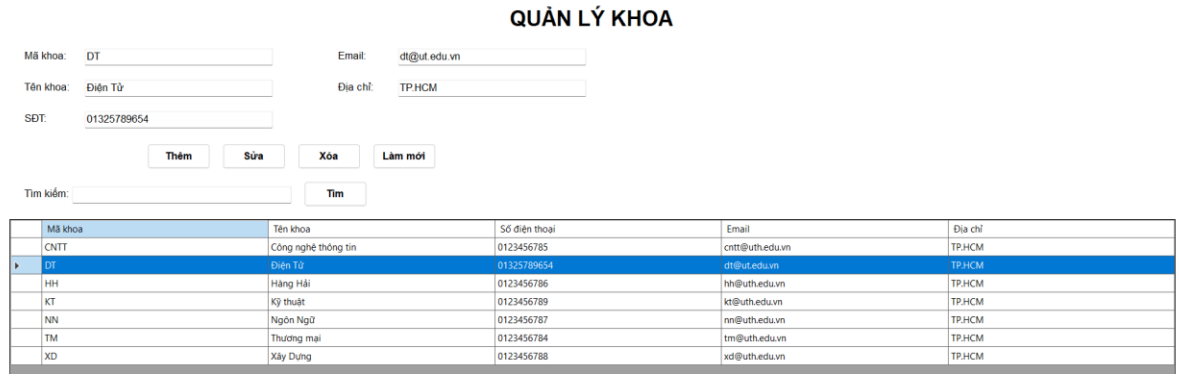
- Hiển thị giao diện thêm khoa: nhập các dữ liệu của khoa



- Hiển thị giao diện thêm khoa thành công " thêm khoa thành công!"

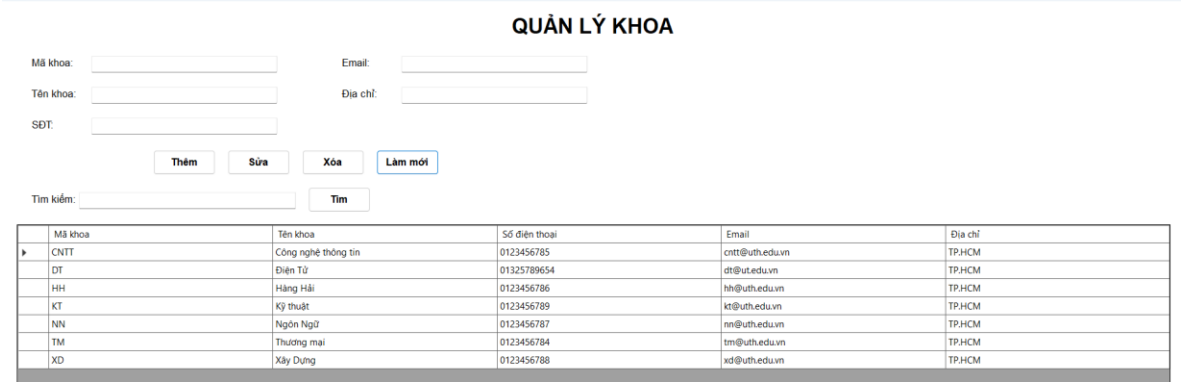


- Giao diện đã thêm khoa xong



2. Sửa khoa

- Giao diện màn hình trước khi sửa khoa



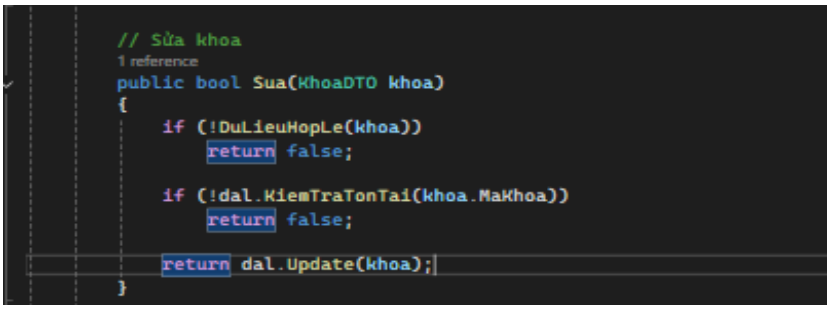
- Code C# GUI sửa khoa: GUI lấy dữ liệu Khoa từ giao diện thông qua LayDuLieu(), đóng gói dữ liệu vào KhoaDTO và gọi khoaBUS.Sua(khoa) để thực hiện cập nhật.

```
1 reference
private void btnSua_Click(object sender, EventArgs e)
{
    if (dgvKhoa.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn khoa cần sửa!");
        return;
    }

    try
    {
        KhoaDTO khoa = LayDuLieu();

        if (!khoaBUS.Sua(khoa))
        {
            MessageBox.Show(
                "Cập nhật khoa thất bại!");
            return;
        }

        MessageBox.Show(
            "Cập nhật khoa thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể cập nhật khoa!\n" + ex.Message);
    }
}
```

- Code C# DTO sửa khoa: KhoaDTO dùng để đóng gói thông tin của Khoa cần sửa thành một đối tượng, sau đó truyền đối tượng này xuống BUS.

```
{
    KhoaDTO khoa = LayDuLieu();

    if (!khoaBUS.Them(khoa))
    {
```

- Code C# gọi BUS sửa khoa: BUS nhận đối tượng KhoaDTO từ GUI, kiểm tra dữ liệu hợp lệ và kiểm tra mã khoa có tồn tại hay không. Nếu tất cả đều hợp lệ thì BUS gọi dal.Update(khoa) để DAL thực hiện sửa dữ liệu trong SQL Server.



```
// Sửa khoa
1 reference
public bool Sua(KhoaDTO khoa)
{
    if (!DuLieuHopLe(khoa))
        return false;

    if (!dal.KiemTraTonTai(khoa.MaKhoa))
        return false;

    return dal.Update(khoa);
}
```

- Code C# gọi DAL Stored Procedure sửa khoa: DAL kết nối với SQL Server và thực hiện SP sp_SuaKhoa. DAL nhận dữ liệu từ KhoaDTO, truyền dữ liệu vào các tham số bằng GanThamSo, sau đó dùng ExecuteNonQuery() để thực hiện cập nhật

```
// Sửa khoa
1 reference
public bool Update(KhoaDTO khoa)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_SuaKhoa",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, khoa);
            cmd.ExecuteNonQuery();

            return true;
        }
```

- Code SQL

```
-- Tạo Procedure sửa Khoa
create procedure sp_SuaKhoa
    @MaKhoa varchar(10),
    @TenKhoa nvarchar(50),
    @SDT varchar(20),
    @Email varchar(50),
    @DiaChi nvarchar(100)
as
begin
    update Khoa
    set
        TenKhoa = @TenKhoa,
        SDT = @SDT,
        Email = @Email,
        DiaChi = @DiaChi
    where MaKhoa = @MaKhoa;
end;
go
```

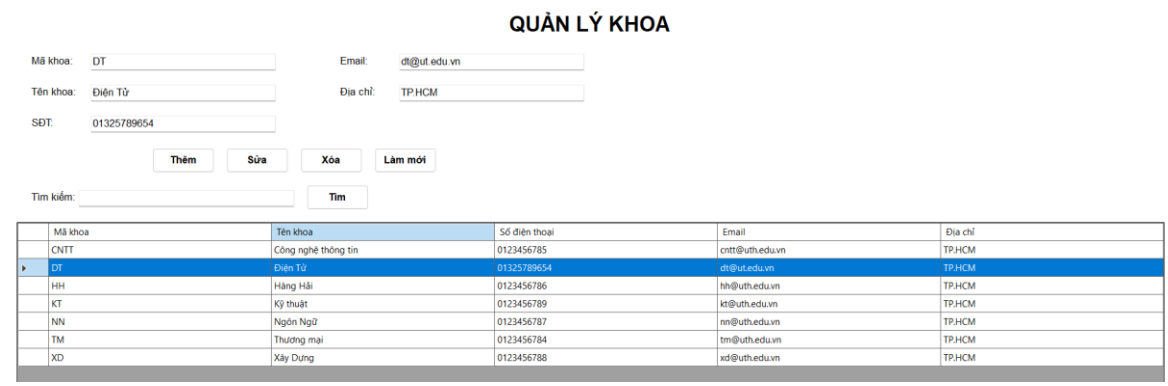
Nhiệm vụ:

+Dùng để sửa thông tin khoa.

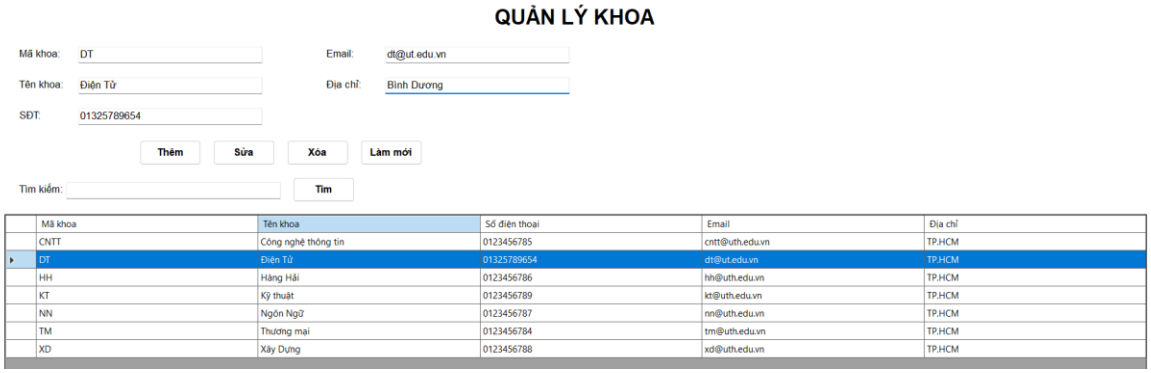
+Dựa vào MaKhoa để xd khoa cần sửa, cập nhật: Tên khoa, SDT Email, Địa chỉ

+ Note: MaKhoa = điều kiện xác định khoa cần sửa.

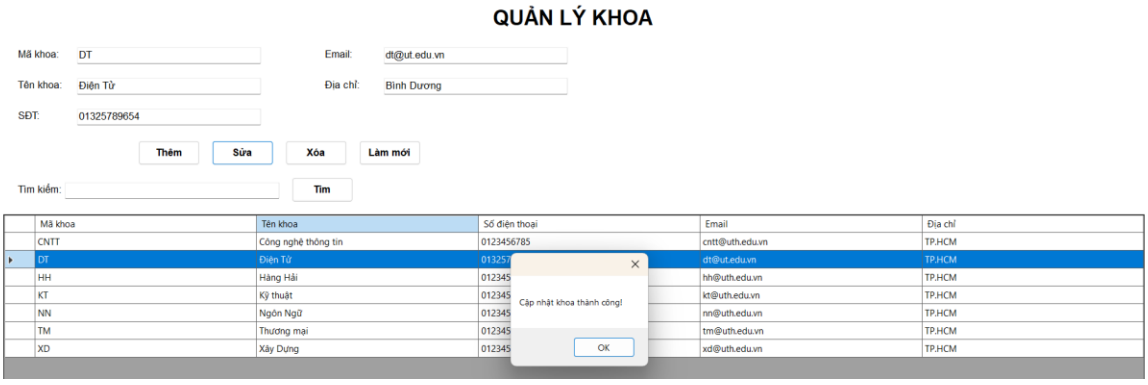
- Giao diện hiển thị sửa khoa: Click chọn mã khoa để sửa, chọn mã khoa DT để sửa



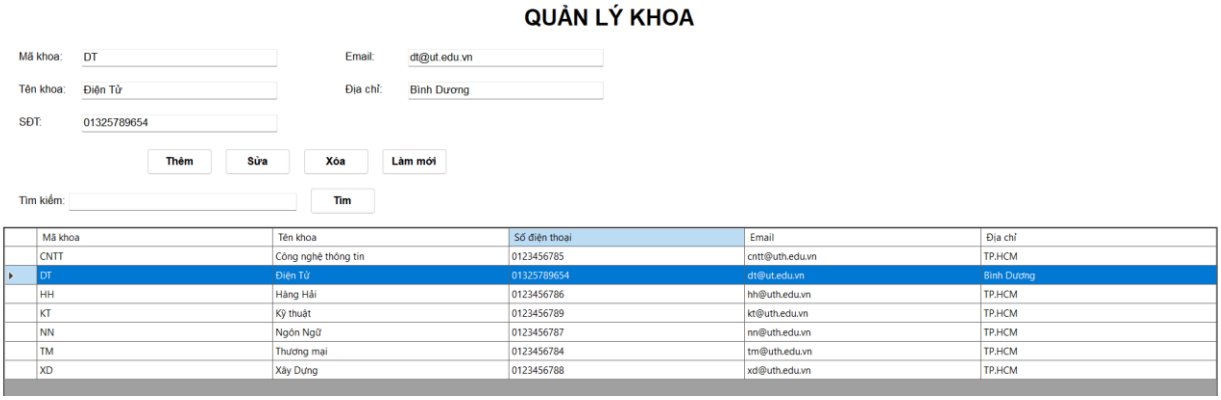
- Giao diện hiển thị sửa khoa: sửa địa chỉ khoa từ TP.HCM thành Bình Dương



- Giao diện hiển thị đã sửa khoa : hiển thị “cập nhật khoa thành công!”

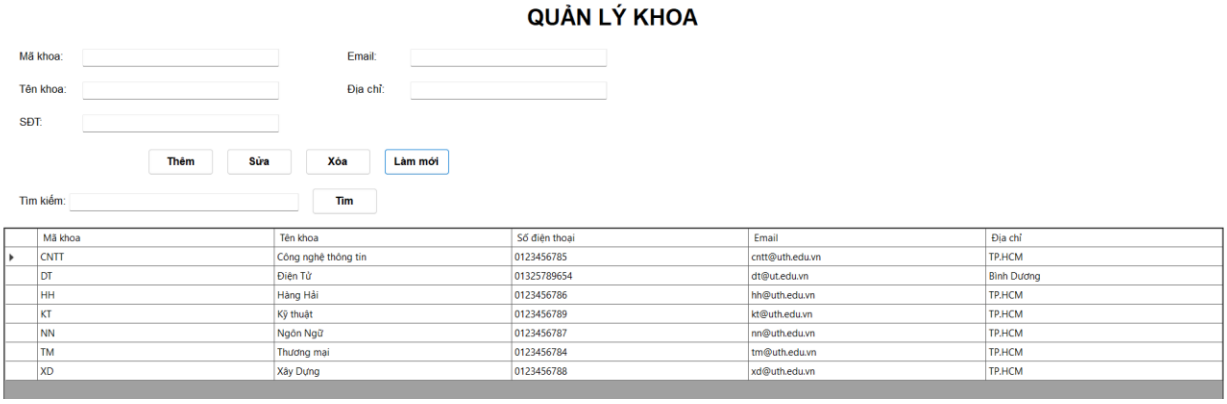


- Giao diện đã sửa xong địa chỉ khoa DT



3. Tìm khoa

- Giao diện màn hình trước khi tìm khoa



- Code C# GUI tìm khoa: Khi nhấn Tìm kiếm, GUI lấy từ khóa từ ô tìm kiếm, gửi đến khoaBUS.TimKiem() và hiển thị kết quả lên giao diện.

```
1 reference
private void btnTimKiem_Click(object sender, EventArgs e)
{
    try
    {
        HienThiDuLieu(
            khoaBUS.TimKiem(
                txtTimKiem.Text.Trim()));
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể tìm kiếm!\n" + ex.Message);
    }
}
```

- Code C# gọi BUS tìm khoa: BUS nhận từ khóa tìm kiếm từ GUI. Nếu người dùng không nhập từ khóa thì BUS lấy toàn bộ danh sách khoa, còn nếu có từ khóa thì gọi DAL để thực hiện tìm kiếm và trả về kết quả.

```
// Tìm kiếm khoa
1 reference
public DataTable TimKiem(string keyword)
{
    if (string.IsNullOrWhiteSpace(keyword))
        return dal.GetAll();

    return dal.Search(keyword.Trim());
}
```

- Code C# gọi DAL Stored Procedure tìm khoa: DAL kết nối đến SQL Server, gọi SP sp_TimKhoa và truyền từ khóa tìm kiếm vào tham số @TuKhoa. Kết quả được lưu vào DataTable và trả về cho BUS.

```
// Tìm khoa
1 reference
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_TimKhoa",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da =
                   new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
```

- Code SQL

```
--Tìm Khoa
CREATE OR ALTER PROCEDURE dbo.sp_TimKhoa
    @TuKhoa NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        MaKhoa,
        TenKhoa,
        SDT,
        Email,
        DiaChi
    FROM dbo.vw_KhoaDayDu
    WHERE
        MaKhoa LIKE '%' + @TuKhoa + '%'
        OR TenKhoa LIKE N'%' + @TuKhoa + '%'
        OR SDT LIKE '%' + @TuKhoa + '%'
        OR Email LIKE '%' + @TuKhoa + '%'
        OR DiaChi LIKE N'%' + @TuKhoa + '%'
    ORDER BY MaKhoa;
END;
GO
```

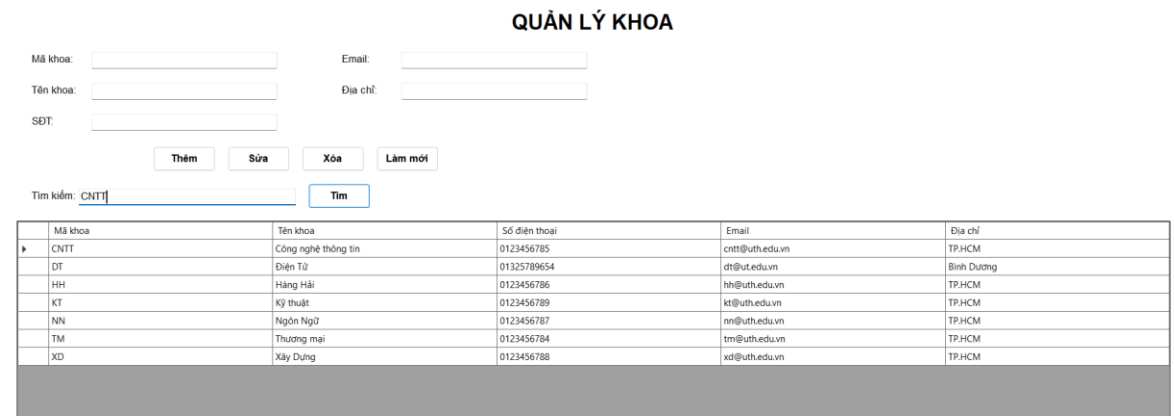
Nhiệm vụ:

+ Dùng để tìm kiếm khoa.

+ Tìm theo: Mã khoa, Tên khoa, SDT, Email,Địa chỉ

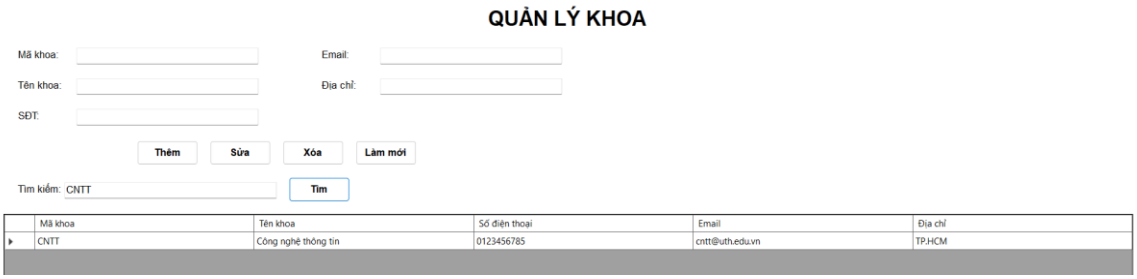
+ SP lấy dữ liệu từ vw_KhoaDayDu.

- Giao diện hiển thị tìm kiếm khoa: Click mã khoa vô, click mã CNTT



Mã khoa	Tên khoa	Số điện thoại	Email	Địa chỉ
CNTT	Công nghệ thông tin	0123456785	cntt@uth.edu.vn	TP.HCM
DT	Điện Tử	01325789654	dt@ut.edu.vn	Bình Dương
HH	Hàng Hải	0123456786	hh@uth.edu.vn	TP.HCM
KT	Kỹ thuật	0123456789	kt@uth.edu.vn	TP.HCM
NN	Ngôn Ngữ	0123456787	nn@uth.edu.vn	TP.HCM
TM	Thương mại	0123456784	tm@uth.edu.vn	TP.HCM
XD	Xây Dựng	0123456788	xd@uth.edu.vn	TP.HCM

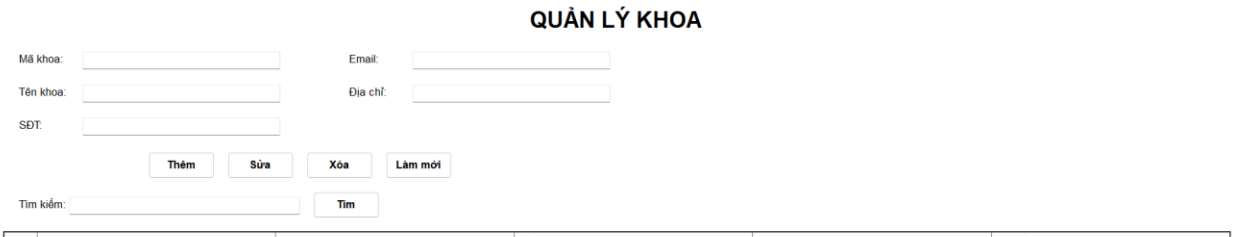
- Giao diện tìm khoa xong rồi



Mã khoa	Tên khoa	Số điện thoại	Email	Địa chỉ
CNTT	Công nghệ thông tin	0123456785	cntt@uth.edu.vn	TP.HCM

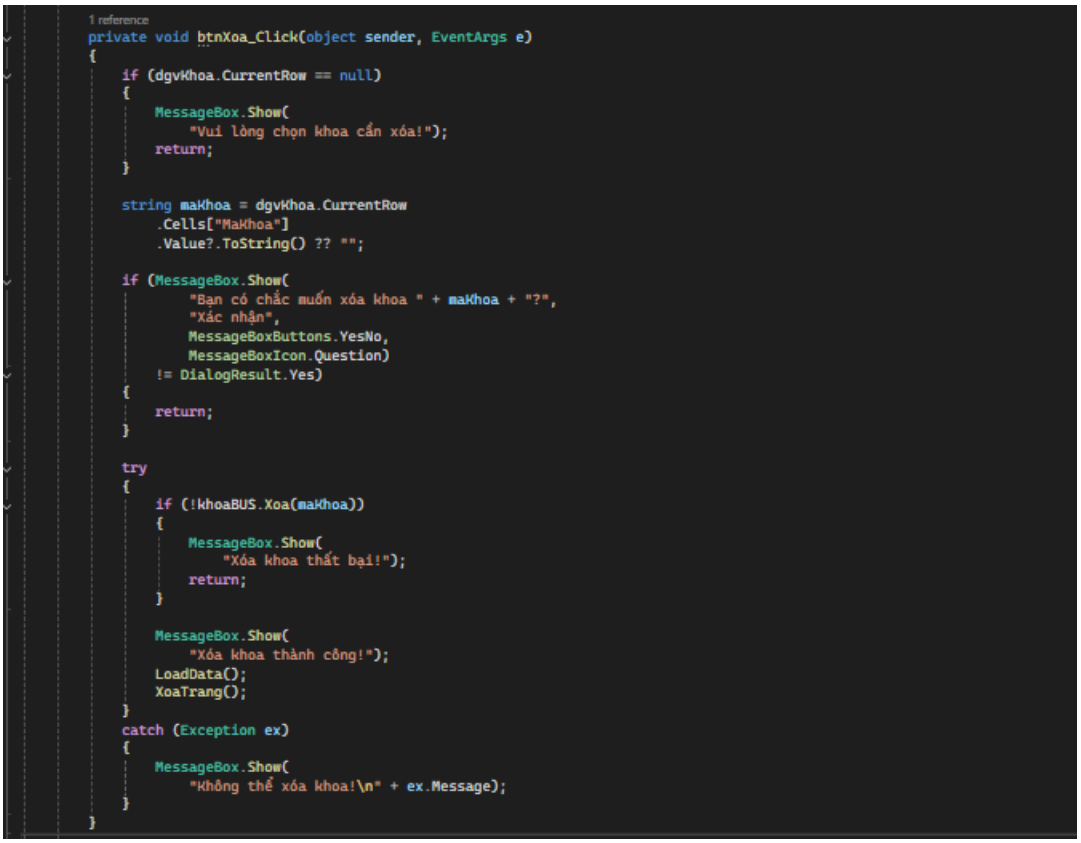
4. Xóa khoa

- Giao diện màn hình trước khi xóa khoa



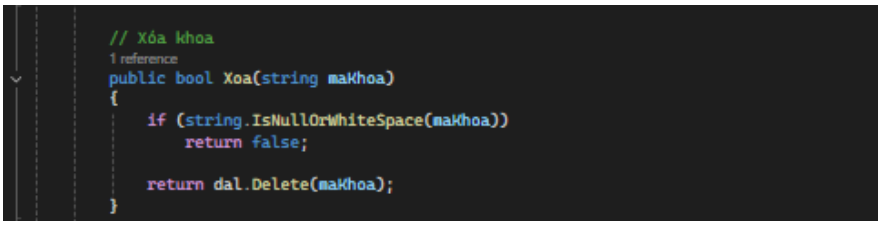
Mã khoa	Tên khoa	Số điện thoại	Email	Địa chỉ
CNTT	Công nghệ thông tin	0123456785	cntt@uth.edu.vn	TP.HCM
DT	Điện Tử	01325789654	dt@ut.edu.vn	Bình Dương
HH	Hàng Hải	0123456786	hh@uth.edu.vn	TP.HCM
KT	Kỹ thuật	0123456789	kt@uth.edu.vn	TP.HCM
NN	Ngôn Ngữ	0123456787	nn@uth.edu.vn	TP.HCM
TM	Thương mại	0123456784	tm@uth.edu.vn	TP.HCM
XD	Xây Dựng	0123456788	xd@uth.edu.vn	TP.HCM

- Code C# GUI xóa khoa: GUI kiểm tra người dùng đã chọn khoa chưa, lấy MaKhoa của dòng được chọn và hỏi xác nhận trước khi xóa. Sau đó gọi khoaBUS.Xoa(maKhoa). Nếu xóa thành công thì tải lại danh sách và xóa trắng giao diện.



```
1 reference
private void btnXoa_Click(object sender, EventArgs e)
{
    if (dgvKhoa.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn khoa cần xóa!");
        return;
    }

    string maKhoa = dgvKhoa.CurrentRow
        .Cells["MaKhoa"]
        .Value?.ToString() ?? "";

    if (MessageBox.Show(
            "Bạn có chắc muốn xóa khoa " + maKhoa + "?",
            "Xác nhận",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question)
        != DialogResult.Yes)
    {
        return;
    }

    try
    {
        if (!khoaBUS.Xoa(maKhoa))
        {
            MessageBox.Show(
                "Xóa khoa thất bại!");
            return;
        }

        MessageBox.Show(
            "Xóa khoa thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể xóa khoa!\n" + ex.Message);
    }
}
```

- Code C# gọi BUS xóa khoa: BUS nhận MaKhoa, kiểm tra mã khoa có hợp lệ hay không. Nếu hợp lệ thì gọi dal.Delete(maKhoa) để thực hiện xóa.



```
// Xóa khoa
1 reference
public bool Xoa(string maKhoa)
{
    if (string.IsNullOrWhiteSpace(maKhoa))
        return false;

    return dal.Delete(maKhoa);
}
```

- Code C# gọi DAL Stored Procedure xóa khoa: DAL kết nối SQL Server, gọi SP sp_XoaKhoa và truyền MaKhoa vào @MaKhoa. Sau đó thực thi và trả kết quả về BUS.

```
// Xóa khoa
1 reference
public bool Delete(string maKhoa)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_XoaKhoa",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaKhoa",
                SqlDbType.VarChar,
                10
            ).Value = maKhoa;

            return Convert.ToBoolean(
                cmd.ExecuteScalar()
            );
```

- Code SQL

```
576
577   -- Tạo Procedure xóa Khoa
578   CREATE OR ALTER PROCEDURE dbo.sp_XoaKhoa
579       @MaKhoa VARCHAR(10)
580   AS
581   BEGIN
582       SET NOCOUNT ON;
583
584       IF NOT EXISTS
585       (
586           SELECT 1
587           FROM dbo.Khoa
588           WHERE MaKhoa = @MaKhoa
589       )
590       BEGIN
591           SELECT CAST(0 AS BIT);
592           RETURN;
593       END;
594
595       IF EXISTS
596       (
597           SELECT 1
598           FROM dbo.Nganh
599           WHERE MaKhoa = @MaKhoa
700       )
701       BEGIN
702           THROW 50030,
703                 N'Không thể xóa khoa vì đang có ngành thuộc khoa này!',
704                 1;
705       END;
706
707       DELETE FROM dbo.Khoa
708       WHERE MaKhoa = @MaKhoa;
709
710       SELECT CAST(1 AS BIT);
711   END;
712   GO
713
```

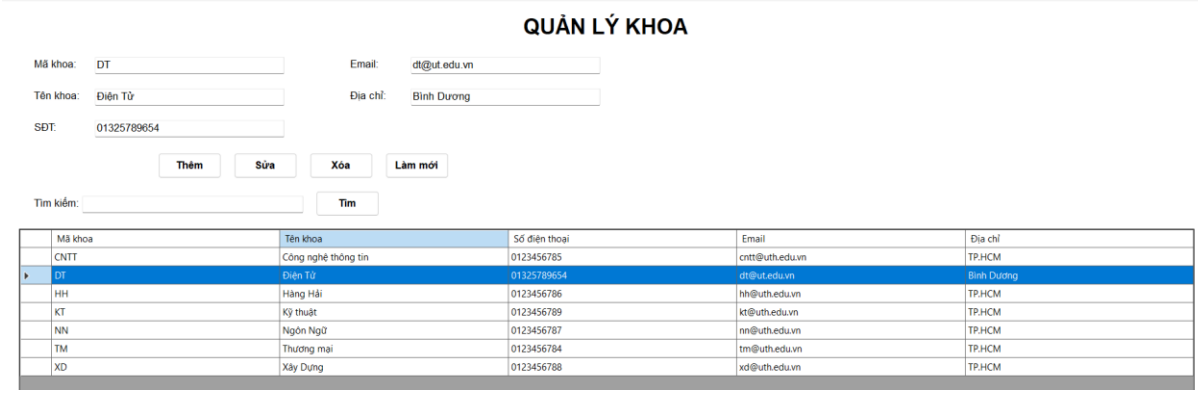
Nhiệm vụ:

+ Dùng để xóa khoa.

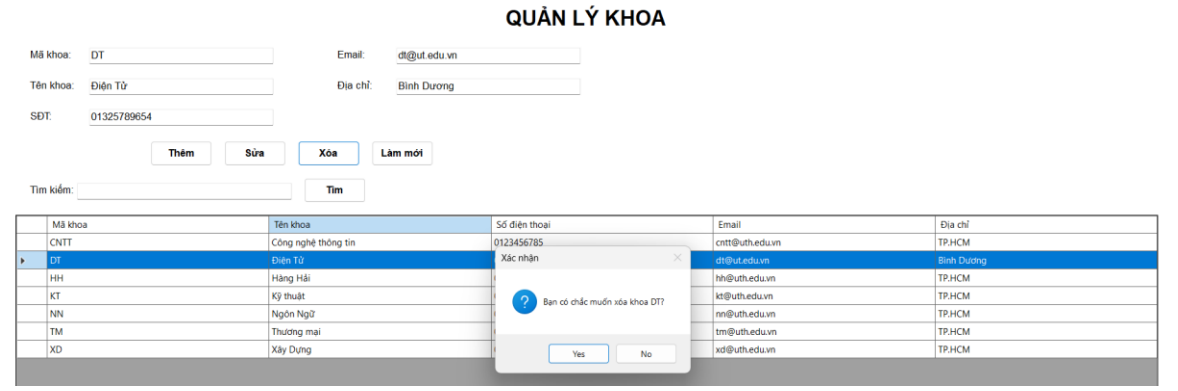
+ Nhưng trước khi xóa, SP kiểm tra xem khoa đó có ngành nào đang sd hay không.

- Có ngành → không cho xóa.
- Không có ngành → cho xóa.

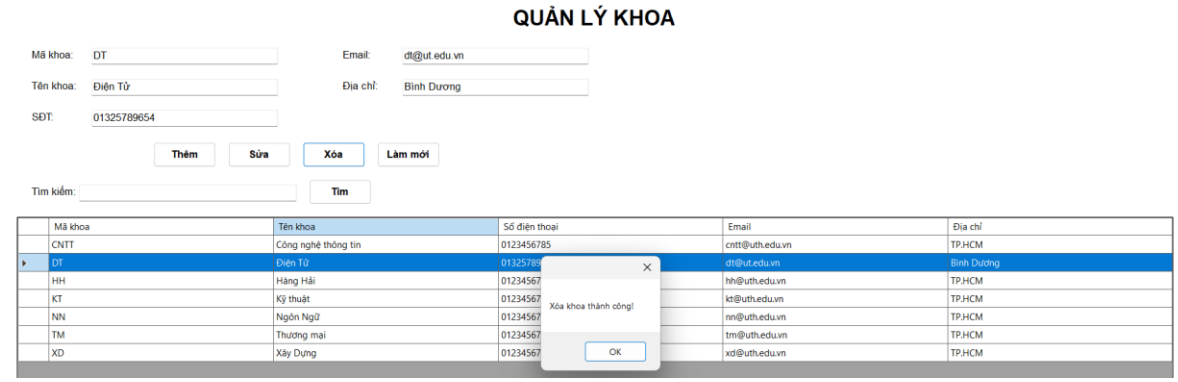
- Giao diện hiển thị xóa khoa: Click vô Khoa cần xóa để hiện dữ liệu, Click DT để xóa



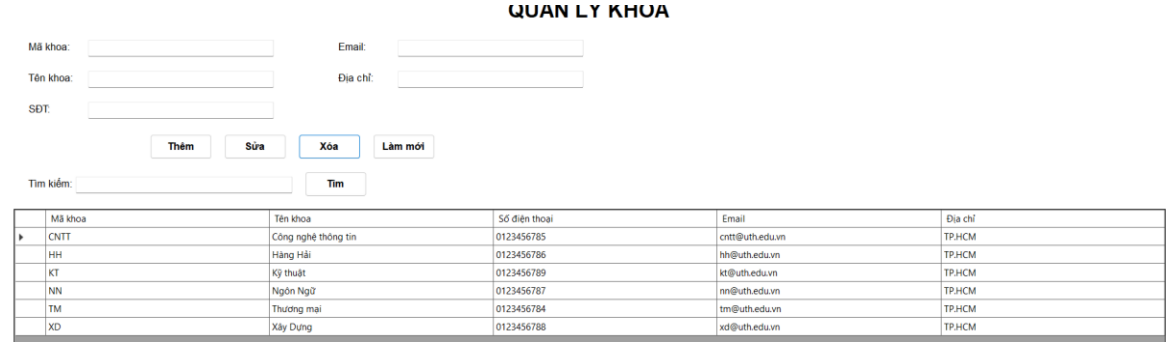
- Giao diện xác nhận bạn có muốn xóa khoa không, có -> xóa



- Giao diện hiển thị “đã xóa khoa thành công”



- Giao diện đã xóa khoa



NGÀNH

1. Thêm ngành

- Giao diện màn hình trước khi thêm ngành

-

Code C# GUI thêm ngành: GUI lấy thông tin ngành thông qua LayDuLieu(), tạo NganhDTO gửi BUS để them ngành. Nếu thành công thì tải lại dữ liệu và xóa trắng form.

```
1 reference
private void btnThem_Click(object sender, EventArgs e)
{
    try
    {
        NganhDTO nganh = LayDuLieu();

        if (!nganhBUS.Them(nganh))
        {
            MessageBox.Show(
                "Thêm ngành thất bại! Kiểm tra dữ liệu hoặc mã ngành bị trùng.");
            return;
        }

        MessageBox.Show("Thêm ngành thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể thêm ngành!\n" + ex.Message);
    }
}
```

- Code C# DTO thêm ngành: NganhDTO dùng để đóng gói thông tin ngành gồm mã ngành, tên ngành và mã khoa, sau đó truyền dữ liệu sang BUS.

```
10 references
public class NganhDTO
{
    5 references
    public string MaNganh { get; set; } = "";
    3 references
    public string TenNganh { get; set; } = "";
    3 references
    public string MaKhoa { get; set; } = "";
}
```

```
{
    NganhDTO nganh = LayDuLieu();

    if (!nganhBUS.Them(nganh))
```

```
// Lấy dữ liệu từ Form
2 references
private NganhDTO LayDuLieu()
{
    return new NganhDTO
    {
        MaNganh = txtMaNganh.Text.Trim(),
        TenNganh = txtTenNganh.Text.Trim(),
        MaKhoa = cboMaKhoa.SelectedValue?.ToString() ?? ""
    };
```

- Code C# gọi BUS thêm ngành: BUS nhận NganhDTO, kiểm tra dữ liệu hợp lệ và kiểm tra mã ngành đã tồn tại hay chưa. Nếu hợp lệ và chưa bị trùng thì gọi dal.Insert(nganh) để thêm ngành vào cơ sở dữ liệu.

```
// Thêm ngành
1 reference
public bool Them(NganhDTO nganh)
{
    if (!DuLieuHopLe(nganh))
        return false;

    if (dal.KiemTraTonTai(nganh.MaNganh))
        return false;

    return dal.Insert(nganh);
}
```

- Code C# gọi DAL Stored Procedure thêm ngành: DAL kết nối đến SQL Server, gọi Stored Procedure sp_ThemNganh và truyền thông tin ngành từ NganhDTO thông qua GanThamSo(). Sau đó ExecuteNonQuery() thực hiện thêm ngành vào cơ sở dữ liệu.

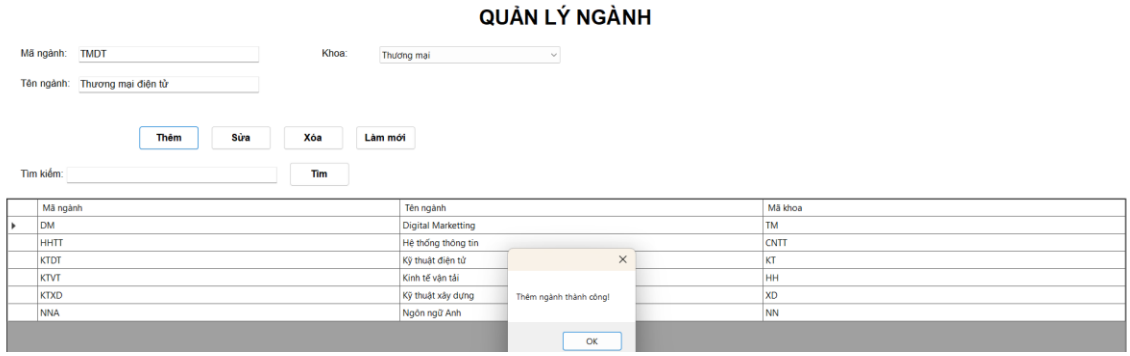
```
// Thêm ngành bằng Stored Procedure
1 reference
public bool Insert(NganhDTO nganh)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_ThemNganh",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, nganh);
            cmd.ExecuteNonQuery();

            return true;
```

- Code SQL

```
498  -- Tạo Procedure thêm Ngành
499  create procedure sp_ThemNganh
500      @MaNganh varchar(10),
501      @TenNganh nvarchar(50),
502      @MaKhoa varchar(10)
503  as
504  begin
505      insert into Nganh
506      (MaNganh,TenNganh,MaKhoa)
507      values
508      (@MaNganh,@TenNganh,@MaKhoa);
509  end;
510  go
511
```

- Giao diện thêm ngành: thêm thông tin dữ liệu của ngành

- Giao diện thêm ngành hiển thị " thêm ngành thành công"

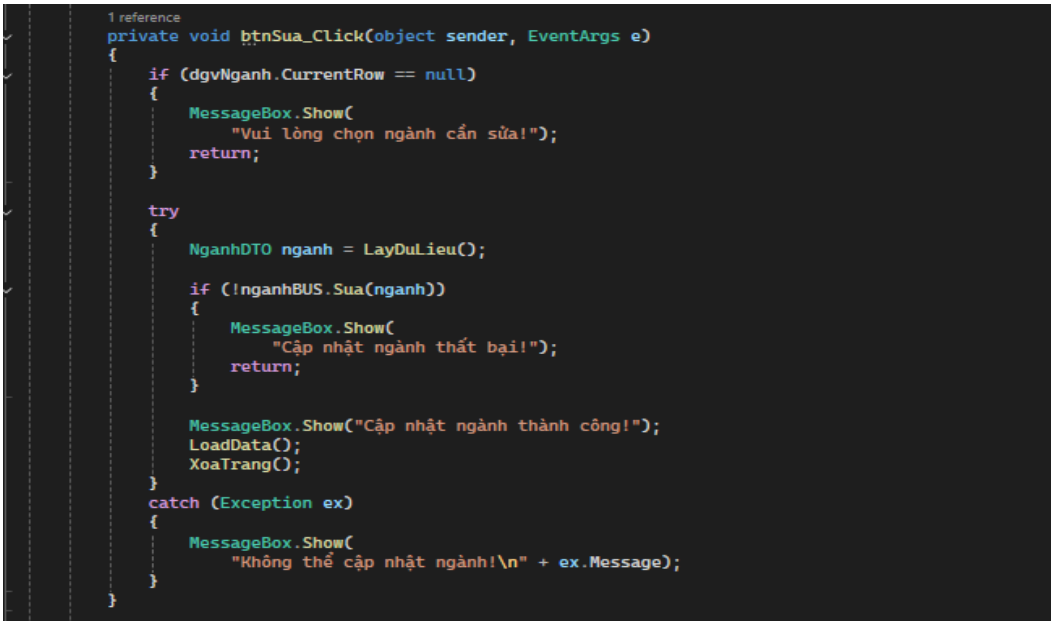


- Giao diện đã thêm ngành xong

2. Sửa ngành

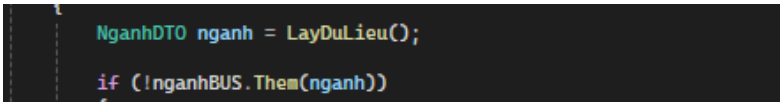
- Giao diện màn hình trước khi sửa ngành

- Code C# GUI sửa ngành: GUI kiểm tra người dùng chọn ngành, lấy thông tin ngành từ form và gọi BUS để thực hiện sửa. Nếu thành công thì tải lại dữ liệu và xóa trắng form.

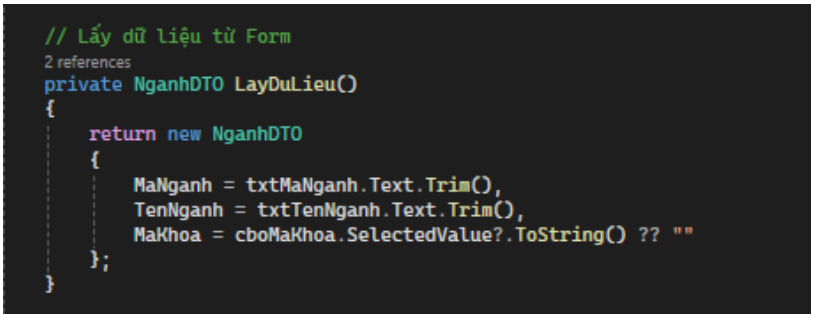


```
1 reference
private void btnSua_Click(object sender, EventArgs e)
{
    if (dgvNganh.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn ngành cần sửa!");
        return;
    }

    try
    {
        NganhDTO nganh = LayDuLieu();

        if (!nganhBUS.Sua(nganh))
        {
            MessageBox.Show(
                "Cập nhật ngành thất bại!");
            return;
        }

        MessageBox.Show("Cập nhật ngành thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể cập nhật ngành!\n" + ex.Message);
    }
}
```

- Code C# DTO sửa ngành: NganhDTO đóng gói các thông tin của ngành gồm MaNganh, TenNganh và MaKhoa để truyền xuống BUS.

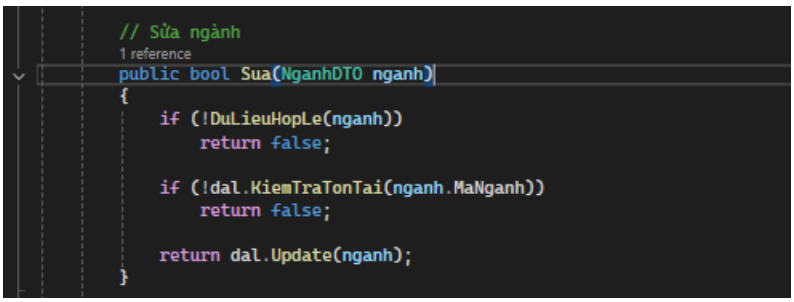


```
{
    NganhDTO nganh = LayDuLieu();

    if (!nganhBUS.Them(nganh))
```



```
// Lấy dữ liệu từ Form
2 references
private NganhDTO LayDuLieu()
{
    return new NganhDTO
    {
        MaNganh = txtMaNganh.Text.Trim(),
        TenNganh = txtTenNganh.Text.Trim(),
        MaKhoa = cboMaKhoa.SelectedValue?.ToString() ?? ""
    };
}
```

- Code C# gọi BUS sửa ngành: BUS nhận NganhDTO, kiểm tra dữ liệu hợp lệ và kiểm tra MaNganh có tồn tại hay không. Nếu hợp lệ thì gọi dal.Update(nganh) đểthực hiện sửa.



```
// Sửa ngành
1 reference
public bool Sua(NganhDTO nganh)
{
    if (!DuLieuHopLe(nganh))
        return false;

    if (!dal.KiemTraTonTai(nganh.MaNganh))
        return false;

    return dal.Update(nganh);
}
```

- Code C# gọi DAL Stored Procedure sửa ngành: DAL kết nối đến SQL Server, gọi Stored Procedure sp_SuaNganh và truyền thông tin ngành từ NganhDTO qua GanThamSo(). Sau đó ExecuteNonQuery() thực hiện cập nhật ngành trong cơ sở dữ liệu.

```
// Sửa ngành bằng Stored Procedure
1 reference
public bool Update(NganhDTO nganh)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_SuaNganh",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, nganh);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

- Code SQL

```
512  -- Tạo Procedure sửa Ngành
513  create procedure sp_SuaNganh
514      @MaNganh varchar(10),
515      @TenNganh nvarchar(50),
516      @MaKhoa varchar(10)
517  as
518  begin
519      update Nganh
520      set
521          TenNganh = @TenNganh,
522          MaKhoa = @MaKhoa
523      where MaNganh = @MaNganh;
524  end;
525  go
526
```

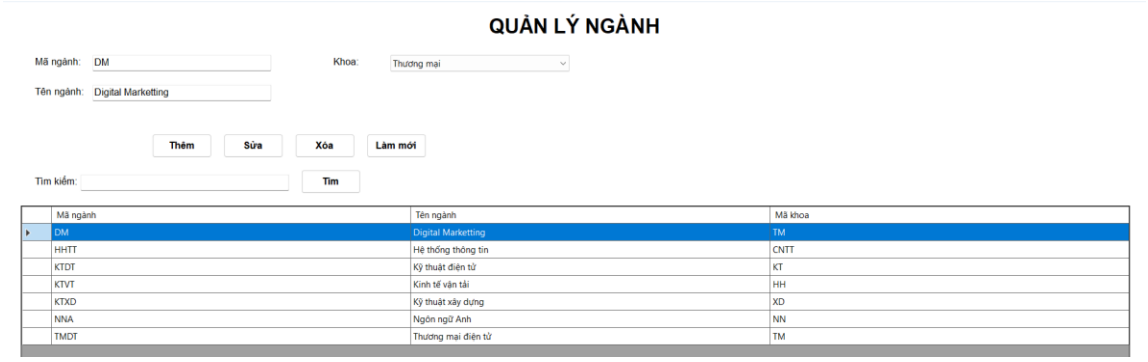
Nhiệm vụ:

+ Dùng để thêm ngành mới.

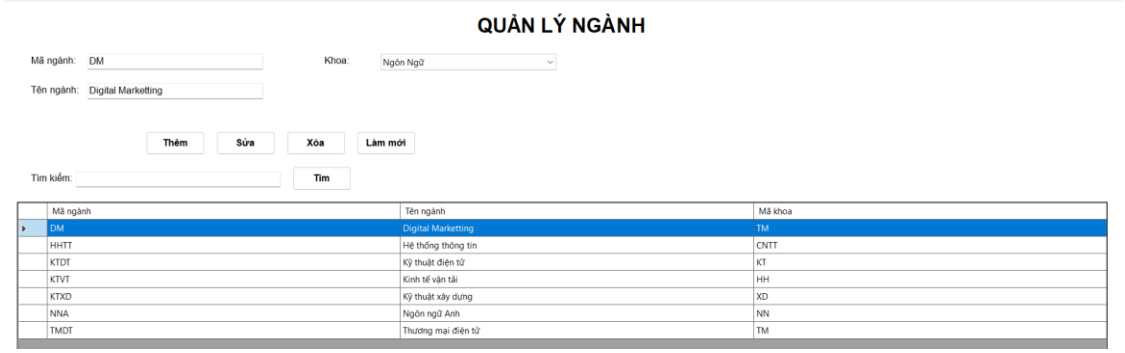
+ Nhận:Mã ngành ,Tên ngành Mã khoa

+ Rồi thêm vào bảng Nganh.

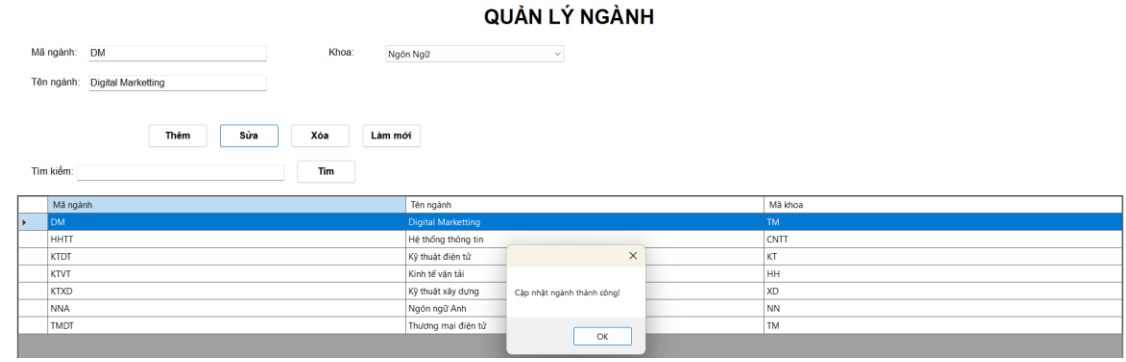
- Giao diện hiển thị sửa ngành: Click vô ngành muốn sửa để hiện dữ liệu



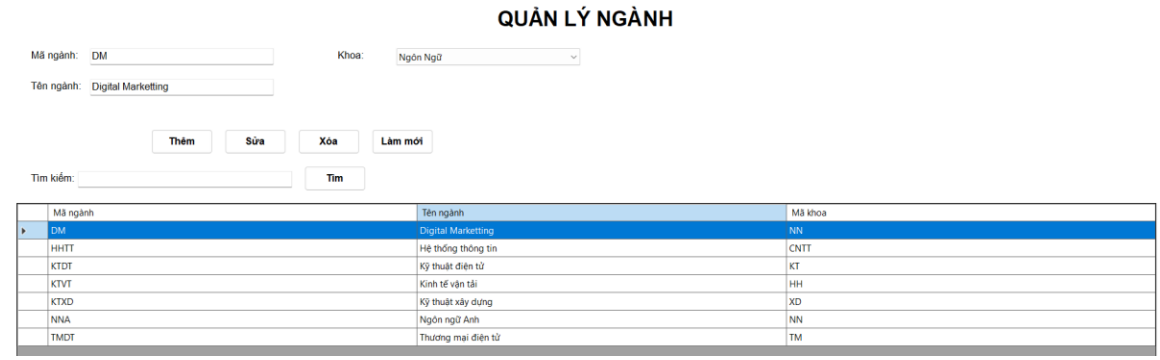
- Giao diện sửa ngành: sửa ngành DM từ khoa “thương mại” sang khoa “ngôn ngữ”



- Giao diện hiển thị sửa ngành: “ cập nhật thành công!”



- Giao diện đã cập nhật xong sửa ngành



3. Tìm ngành

- Giao diện màn hình trước khi tìm ngành

- Code C# GUI tìm ngành: GUI lấy từ khóa từ ô tìm kiếm. Nếu ô tìm kiếm trống thì hiển thị toàn bộ dữ liệu, nếu có từ khóa thì gửi đến BUS để tìm kiếm và hiển thị kết quả.

```
1 reference
private void btnTimKiem_Click(object sender, EventArgs e)
{
    try
    {
        string keyword = txtTimKiem.Text.Trim();

        if (string.IsNullOrWhiteSpace(keyword))
        {
            LoadData();
            return;
        }

        HienThiDuLieu(
            nganhBUS.TimKiem(keyword));
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể tìm kiếm!\n" + ex.Message);
    }
}
```

- Code C# gọi BUS tìm ngành: BUS nhận từ khóa tìm kiếm. Nếu trống thì lấy toàn bộ danh sách ngành, nếu có thì gọi dal.Search() để tìm kiếm và trả kết quả về GUI.

```
// Tìm kiếm ngành
1 reference
public DataTable TimKiem(string keyword)
{
    if (string.IsNullOrWhiteSpace(keyword))
        return dal.GetAll();

    return dal.Search(keyword.Trim());
}
```

- Code C# gọi DAL Stored Procedure tìm ngành: DAL kết nối đến SQL Server, gọi Stored Procedure sp_TimNganh và truyền từ khóa vào tham số @TuKhoa. Kết quả tìm kiếm được lưu vào DataTable rồi trả về cho BUS.

```
// Tìm ngành bằng Stored Procedure
1 reference
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_TimNganh",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da =
                   new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
        }
    }

    return dt;
}
```

- Code SQL

```
576
577 -- Tìm ngành
578 CREATE OR ALTER PROCEDURE dbo.sp_TimNganh
579     @TuKhoa NVARCHAR(100)
580 AS
581 BEGIN
582     SET NOCOUNT ON;
583
584     SELECT
585         n.MaNganh,
586         n.TenNganh,
587         n.MaKhoa
588     FROM dbo.Nganh AS n
589     LEFT JOIN dbo.Khoa AS k
590         ON n.MaKhoa = k.MaKhoa
591     WHERE
592         n.MaNganh LIKE '%' + @TuKhoa + '%'
593         OR n.TenNganh LIKE N'%' + @TuKhoa + N'%'
594         OR n.MaKhoa LIKE '%' + @TuKhoa + '%'
595         OR k.TenKhoa LIKE N'%' + @TuKhoa + N'%'
596     ORDER BY n.MaNganh;
597 END;
598 GO
599
```

Nhiệm vụ:

+ Dùng để tìm kiếm ngành.

+ Có thể tìm theo: Mã ngành, Tên ngành, Mã khoa, Tên Khoa

+ Vì có cả thông tin khoa nên SP có JOIN với bảng Khoa.

- Giao diện hiển thị tìm ngành: nhập mã ngành cần tìm , nhập TMDT

- Giao diện đã tìm kiếm ngành xong

4. Xóa ngành

- Giao diện màn hình trước khi xóa ngành

- Code C# GUI xóa ngành : GUI kiểm tra ngành được chọn, lấy MaNganh và xác nhận trước khi xóa. Sau đó gọi nganhBUS.Xoa(maNganh), nếu thành công thì tải lại dữ liệu.

```
1 reference
private void btnXoa_Click(object sender, EventArgs e)
{
    if (dgvNganh.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn ngành cần xóa!");
        return;
    }

    string maNganh = dgvNganh.CurrentRow
        .Cells["MaNganh"]
        .Value?.ToString() ?? "";

    if (MessageBox.Show(
            "Bạn có chắc muốn xóa ngành " + maNganh + "?",
            "Xác nhận",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question)
        != DialogResult.Yes)
    {
        return;
    }

    try
    {
        if (!nganhBUS.Xoa(maNganh))
        {
            MessageBox.Show("Xóa ngành thất bại!");
            return;
        }

        MessageBox.Show("Xóa ngành thành công!");
        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể xóa ngành!\n" + ex.Message);
    }
}
```

- Code C# gọi BUS xóa ngành : BUS nhận MaNganh, kiểm tra mã ngành có hợp lệ hay không. Nếu hợp lệ thì gọi DAL để thực hiện xóa.

```
// Xóa ngành
1 reference
public bool Xoa(string maNganh)
{
    if (string.IsNullOrWhiteSpace(maNganh))
        return false;

    return dal.Delete(maNganh);
}
```

- Code C# gọi DAL xóa ngành : DAL kết nối đến SQL Server, gọi SP sp_XoaNganh và truyền MaNganh vào @MaNganh. Sau đó thực thi lệnh và trả kết quả xóa về BUS.

```
// Xóa ngành bằng Stored Procedure
1 reference
public bool Delete(string maNganh)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_XoaNganh",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaNganh",
                SqlDbType.VarChar,
                10
            ).Value = maNganh;

            return Convert.ToBoolean(
                cmd.ExecuteScalar()
            );
        }
    }
}
```

- Code SQL

```sql

-- Tạo Procedure xóa Ngành
CREATE OR ALTER PROCEDURE dbo.sp_XoaNganh
    @MaNganh VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS
    (
        SELECT 1
        FROM dbo.Nganh
        WHERE MaNganh = @MaNganh
    )
    BEGIN
        SELECT CAST(0 AS BIT);
        RETURN;
    END;

    IF EXISTS
    (
        SELECT 1
        FROM dbo.Lop
        WHERE MaNganh = @MaNganh
    )
    BEGIN
        THROW 50020,
              N'Không thể xóa ngành vì đang có lớp thuộc ngành này!',
              1;
    END;

    IF EXISTS
    (
        SELECT 1
        FROM dbo.ChuongTrinhDaoTao
        WHERE MaNganh = @MaNganh
    )
    BEGIN
        THROW 50021,
              N'Không thể xóa ngành vì đang có chương trình đào tạo thuộc ngành này!',
              1;
    END;

    DELETE FROM dbo.Nganh
    WHERE MaNganh = @MaNganh;

    SELECT CAST(1 AS BIT);
END;
GO

```

Nhiệm vụ:

+Dùng để xóa ngành, nhưng có kiểm tra dữ liệu liên quan.

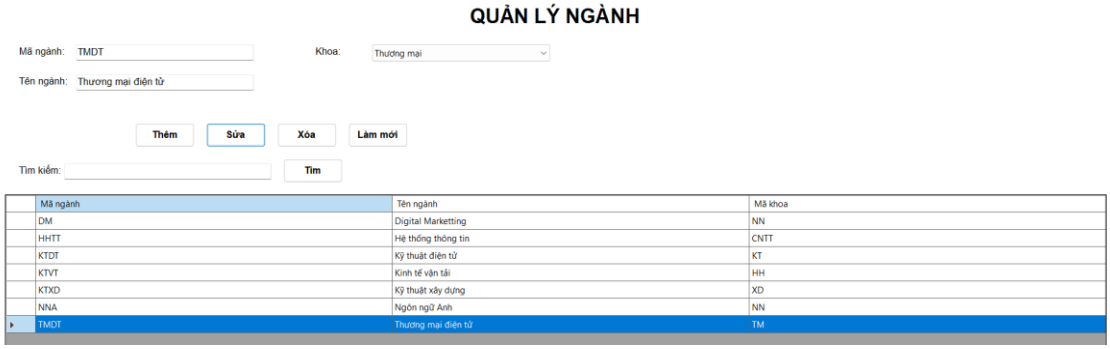
+Trước khi xóa, SP kiểm tra:

- Ngành có tồn tại không?
- Có lớp nào đang thuộc ngành đó không?
- Có chương trình đào tạo nào đang thuộc ngành đó không?

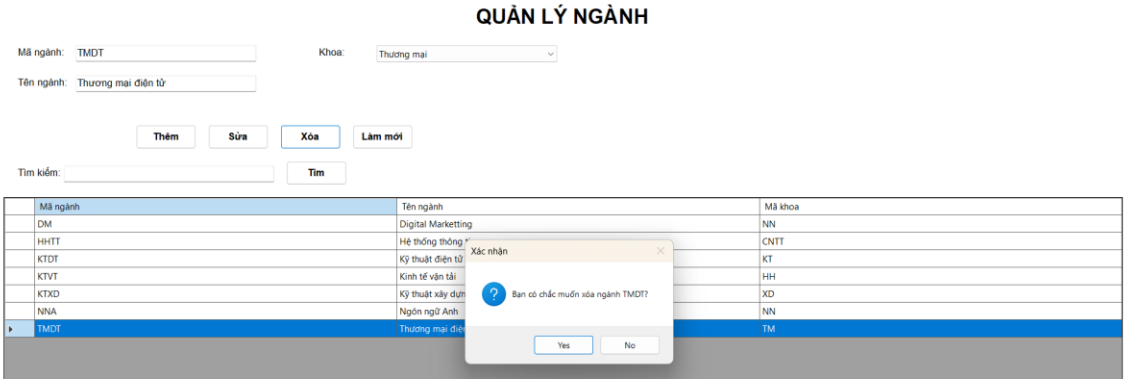
+ Nếu còn dữ liệu liên quan thì không cho xóa.

+ Nếu không có thì mới xóa.

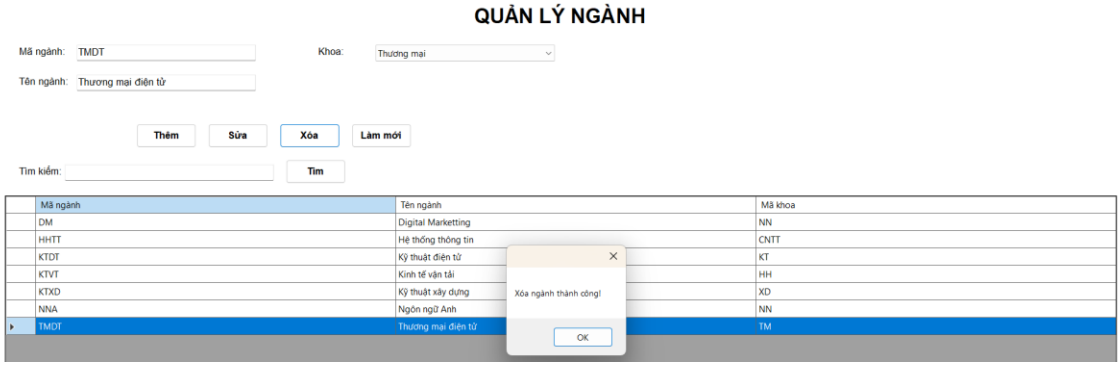
- Giao diện hiển thị xóa ngành: Click vô ngành muốn xóa cho nó hiển thị dữ liệu , xóa TMDT



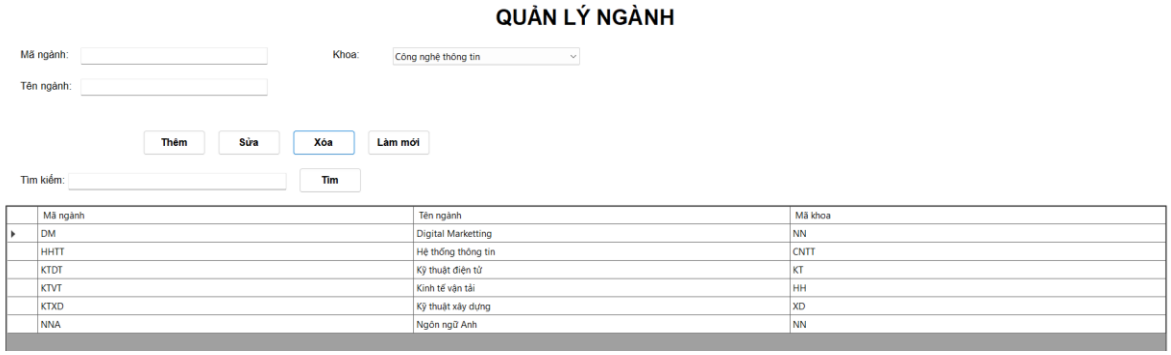
- Giao diện xóa ngành: hiển thị bạn có chắc chắn muốn xóa ngành TMDT không



- Giao diện đã xóa ngành: hiển thị " xóa ngành thành công !"



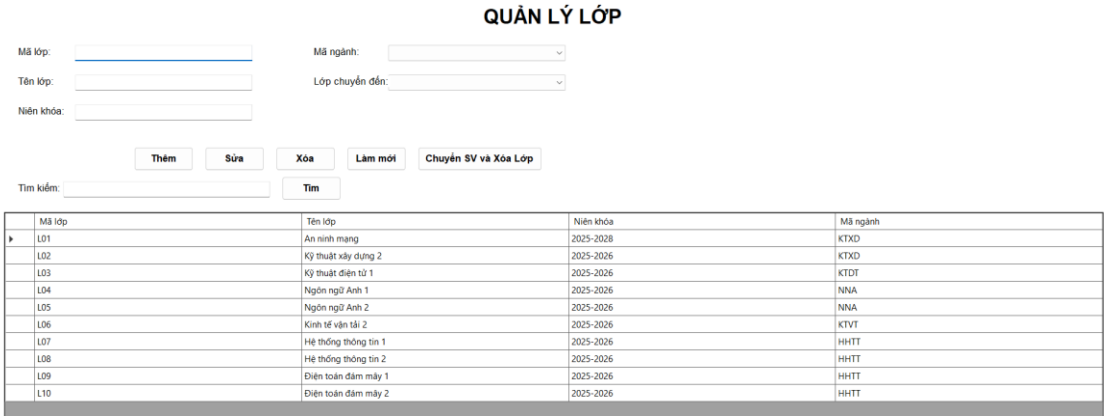
- Giao diện cập nhật lại số ngành: đã mất ngành TMDT



LỚP:

1. Thêm lớp

- Giao diện màn hình trước khi thêm lớp



Mã lớp	Tên lớp	Niên khóa	Mã ngành
L01	An ninh mạng	2025-2028	KTXD
L02	Kỹ thuật xây dựng 2	2025-2026	KTXD
L03	Kỹ thuật điện tử 1	2025-2026	KTDT
L04	Ngôn ngữ Anh 1	2025-2026	NNA
L05	Ngôn ngữ Anh 2	2025-2026	NNA
L06	Kinh tế vận tải 2	2025-2026	KTVT
L07	Hệ thống thông tin 1	2025-2026	HHTT
L08	Hệ thống thông tin 2	2025-2026	HHTT
L09	Điện toán đám mây 1	2025-2026	HHTT
L10	Điện toán đám mây 2	2025-2026	HHTT

- Code C# GUI thêm lớp: Lấy dữ liệu từ các ô Mã lớp, Tên lớp, Niên khóa, Mã ngành, tạo thành một đối tượng LopDTO, sau đó gọi lopBUS.Them(lop) để xử lý thêm lớp. Nếu thêm thành công thì tải lại dữ liệu và xóa trắng các ô nhập.

```
// Thêm lớp

private void btnThem_Click(object sender, EventArgs e)
{
    try
    {
        LopDTO lop = LayDuLieu();

        if (!lopBUS.Them(lop))
        {
            MessageBox.Show(
                "Thêm thất bại! Kiểm tra dữ liệu hoặc mã lớp bị trùng."
            );

            return;
        }

        MessageBox.Show("Thêm lớp thành công!");

        LoadData();
        LoadLopMoi();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể thêm lớp!\n" +
            ex.Message
        );
    }
}
```

- Code C# DTO thêm lớp: LopDTO dùng để đóng gói thông tin của một lớp thành một đối tượng để truyền giữa GUI, BUS và DAL. DTO của lớp gồm 4 thuộc tính: MaLop, TenLop, NienKhoa và MaNganh

```
public class LopDTO
{
    7 references
    public string MaLop { get; set; } = "";
    5 references
    public string TenLop { get; set; } = "";
    5 references
    public string NienKhoa { get; set; } = "";
    5 references
    public string MaNganh { get; set; } = "";
}
```

- Code C# gọi BUS thêm lớp: BUS nhận đối tượng LopDTO từ GUI. Trước tiên, BUS kiểm tra dữ liệu có hợp lệ hay không; sau đó kiểm tra mã lớp đã tồn tại chưa. Nếu dữ liệu hợp lệ và mã lớp chưa bị trùng thì BUS gọi dal.Insert(lop) để chuyển sang DAL thực hiện thêm vào SQL Server.

```
// Thêm lớp
1 reference
public bool Them(LopDTO lop)
{
    if (!DuLieuHopLe(lop))
        return false;

    if (dal.KiemTraTonTai(lop.MaLop))
        return false;

    return dal.Insert(lop);
}
```

- Code C# gọi DAL Stored Procedure thêm lớp: DAL mở kết nối đến SQL Server và tạo SqlCommand để gọi Stored Procedure dbo.sp_ThemLop. Sau đó GanThamSo() truyền 4 thông tin của lớp vào các tham số @MaLop, @TenLop, @NienKhoa, @MaNganh, rồi ExecuteNonQuery() thực hiện Stored Procedure.

```
// Thêm lớp bằng Stored Procedure
1 reference
public bool Insert(LopDTO lop)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_ThemLop",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, lop);

            cmd.ExecuteNonQuery();
            return true;
        }
    }
}
```

```
// Gán tham số Thêm/Sửa
2 references
private void GanThamSo(
    SqlCommand cmd,
    LopDTO lop)
{
    cmd.Parameters.Add(
        "@MaLop",
        SqlDbType.VarChar,
        10
    ).Value = lop.MaLop;

    cmd.Parameters.Add(
        "@TenLop",
        SqlDbType.NVarChar,
        50
    ).Value = lop.TenLop;

    cmd.Parameters.Add(
        "@NienKhoa",
        SqlDbType.NVarChar,
        10
    ).Value = lop.NienKhoa;

    cmd.Parameters.Add(
        "@MaNganh",
        SqlDbType.VarChar,
        10
    ).Value = lop.MaNganh;
}
```

- Code SQL

```
375
376   -- Tạo Procedure thêm Lớp
377   create procedure sp_ThemLop
378       @MaLop VARCHAR(10),
379       @TenLop NVARCHAR(50),
380       @NienKhoa NVARCHAR(10),
381       @MaNganh VARCHAR(10)
382   AS
383   BEGIN
384       SET NOCOUNT ON;
385
386       INSERT INTO dbo.Lop
387           (MaLop, TenLop, NienKhoa, MaNganh)
388       VALUES
389           (@MaLop, @TenLop, @NienKhoa, @MaNganh);
390   END;
391   GO
392
```

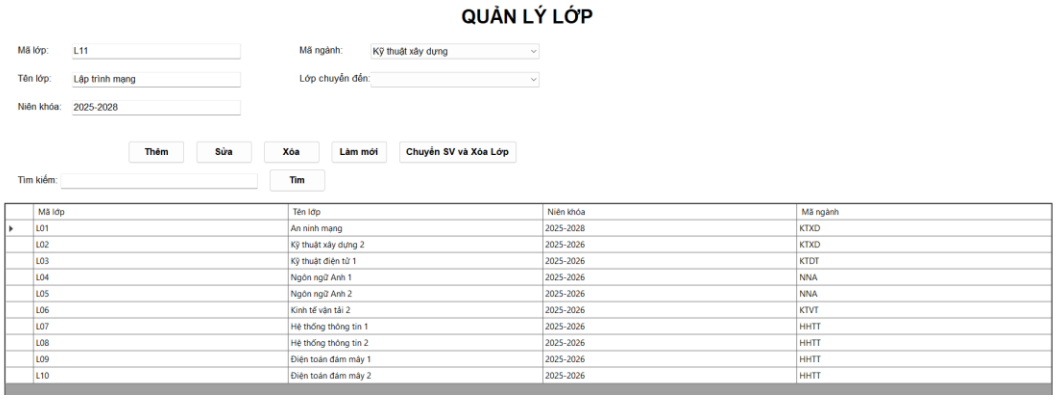
Nhiệm vụ:

+ Dùng để thêm lớp mới.

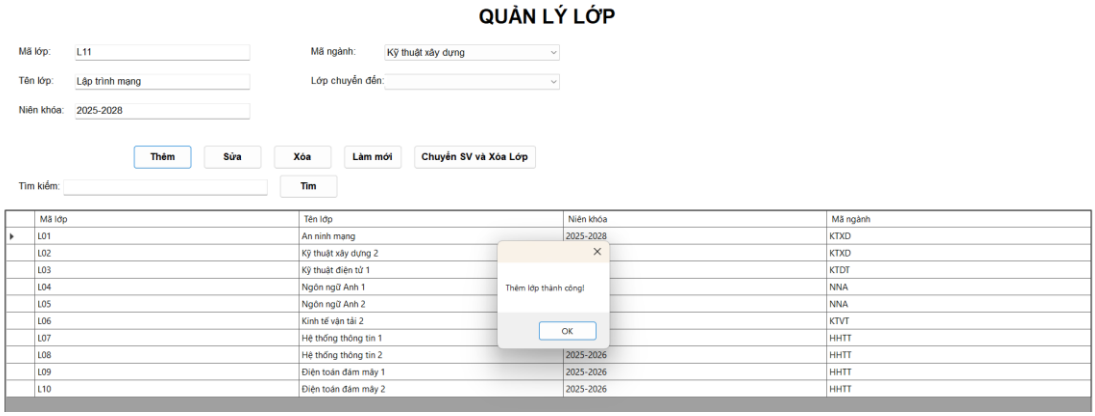
+ Nhận: Mã lớp, Tên lớp, Niên khóa, Mã ngành

+ Rồi INSERT vào bảng Lop

- Giao diện thêm lớp: thêm thông tin của lớp

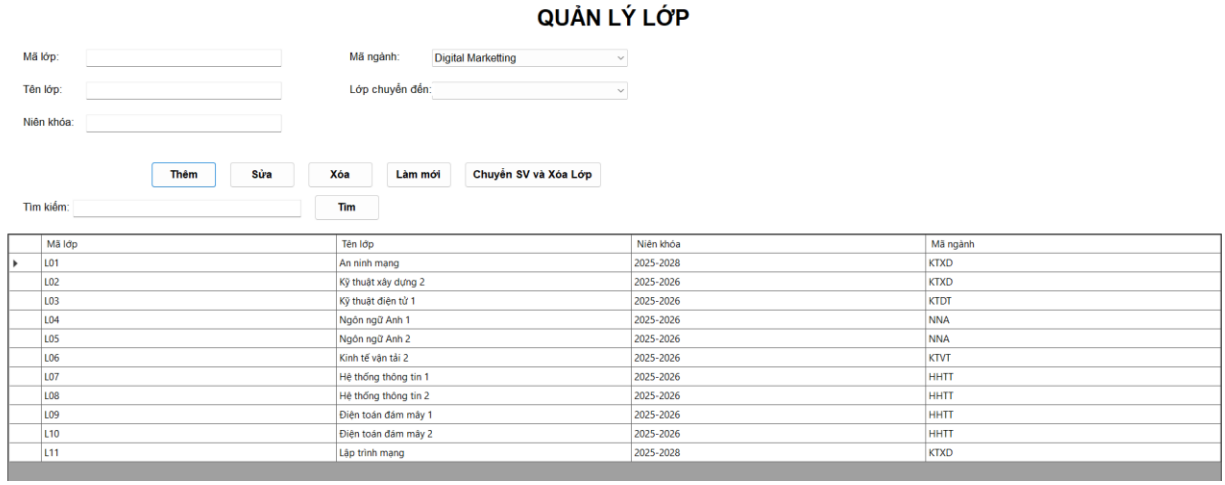


- Giao diện thêm lớp : hiển thị “thêm lớp thành công”



Mã lớp	Tên lớp	Niên khóa	Mã ngành
L01	An ninh mạng	2025-2028	KTXD
L02	Kỹ thuật xây dựng 2		KTXD
L03	Kỹ thuật điện tử 1		KTDT
L04	Ngôn ngữ Anh 1		NNA
L05	Ngôn ngữ Anh 2		NNA
L06	Kinh tế vận tải 2		KTVT
L07	Hệ thống thông tin 1		HHTT
L08	Hệ thống thông tin 2	2025-2026	HHTT
L09	Điện toán đám mây 1	2025-2026	HHTT
L10	Điện toán đám mây 2	2025-2026	HHTT

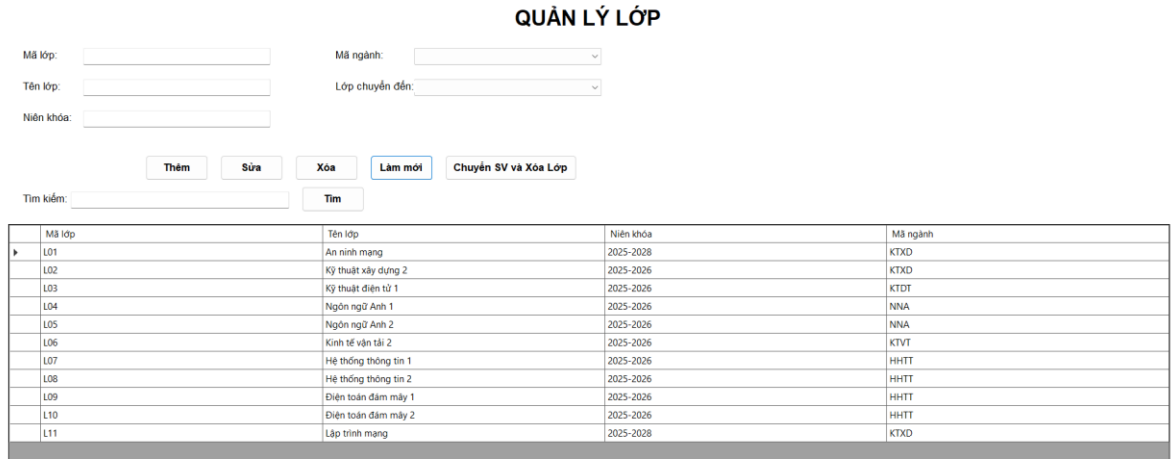
- Giao diện cập nhật thêm lớp xong



Mã lớp	Tên lớp	Niên khóa	Mã ngành
L01	An ninh mạng	2025-2028	KTXD
L02	Kỹ thuật xây dựng 2	2025-2026	KTXD
L03	Kỹ thuật điện tử 1	2025-2026	KTDT
L04	Ngôn ngữ Anh 1	2025-2026	NNA
L05	Ngôn ngữ Anh 2	2025-2026	NNA
L06	Kinh tế vận tải 2	2025-2026	KTVT
L07	Hệ thống thông tin 1	2025-2026	HHTT
L08	Hệ thống thông tin 2	2025-2026	HHTT
L09	Điện toán đám mây 1	2025-2026	HHTT
L10	Điện toán đám mây 2	2025-2026	HHTT
L11	Lập trình mạng	2025-2028	KTXD

2. Sửa lớp

- Giao diện màn hình trước khi sửa lớp



Mã lớp	Tên lớp	Niên khóa	Mã ngành
L01	An ninh mạng	2025-2028	KTXD
L02	Kỹ thuật xây dựng 2	2025-2026	KTXD
L03	Kỹ thuật điện tử 1	2025-2026	KTDT
L04	Ngôn ngữ Anh 1	2025-2026	NNA
L05	Ngôn ngữ Anh 2	2025-2026	NNA
L06	Kinh tế vận tải 2	2025-2026	KTVT
L07	Hệ thống thông tin 1	2025-2026	HHTT
L08	Hệ thống thông tin 2	2025-2026	HHTT
L09	Điện toán đám mây 1	2025-2026	HHTT
L10	Điện toán đám mây 2	2025-2026	HHTT
L11	Lập trình mạng	2025-2028	KTXD

- Code C# GUI sửa lớp: Khi sửa lớp, GUI trước tiên kiểm tra người dùng đã chọn lớp trong DataGridView hay chưa. Sau đó lấy thông tin lớp từ giao diện, tạo LopDTO và gọi lopBUS.Sua(lop); nếu sửa thành công thì tải lại danh sách lớp và xóa trắng form.

```
// Lấy dữ liệu
2 references
private LopDTO LayDuLieu()
{
    return new LopDTO
    {
        MaLop = txtMaLop.Text.Trim(),
        TenLop = txtTenLop.Text.Trim(),
        NienKhoa = txtNienKhoa.Text.Trim(),

        MaNganh =
            cboMaNganh.SelectedValue?
            .ToString()
            .Trim() ?? ""
    };
}
```

```
// Sửa lớp
1 reference
private void btnSua_Click(object sender, EventArgs e)
{
    if (dgvLop.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn lớp cần sửa!"
        );

        return;
    }

    try
    {
        LopDTO lop = LayDuLieu();

        if (!lopBUS.Sua(lop))
        {
            MessageBox.Show(
                "Cập nhật lớp thất bại!"
            );

            return;
        }

        MessageBox.Show(
            "Cập nhật lớp thành công!"
        );

        LoadData();
        LoadLopMoi();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể cập nhật lớp!\n" +
            ex.Message
        );
    }
}
```

- Code C# DTO sửa lớp: LopDTO chứa 4 thông tin của lớp: mã lớp, tên lớp, niên khóa và mã ngành. Khi sửa, GUI tạo một LopDTO chứa dữ liệu mới rồi truyền xuống BUS.

```
public class LopDTO
{
    7 references
    public string MaLop { get; set; } = "";
    5 references
    public string TenLop { get; set; } = "";
    5 references
    public string NienKhoa { get; set; } = "";
    5 references
    public string MaNganh { get; set; } = "";
}
```

- Code C# gọi BUS sửa lớp: BUS nhận LopDTO từ GUI, sau đó kiểm tra dữ liệu có hợp lệ hay không. Tiếp theo BUS kiểm tra mã lớp có tồn tại trong cơ sở dữ liệu hay không; nếu tồn tại thì gọi dal.Update(lop) để thực hiện cập nhật.

```
// Sửa lớp
1 reference
public bool Sua(LopDTO lop)
{
    if (!DuLieuHopLe(lop))
        return false;

    if (!dal.KiemTraTonTai(lop.MaLop))
        return false;

    return dal.Update(lop);
}
```

- Code C# gọi DAL Stored Procedure sửa lớp: DAL mở kết nối đến SQL Server, tạo SqlCommand để gọi Stored Procedure dbo.sp_SuaLop và đặt CommandType là StoredProcedure. Sau đó GanThamSo() truyền thông tin của LopDTO vào các tham số của Stored Procedure và ExecuteNonQuery() thực hiện cập nhật dữ liệu.

```
// Sửa lớp bằng Stored Procedure
1 reference
public bool Update(LopDTO lop)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_SuaLop",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, lop);

            cmd.ExecuteNonQuery();
            return true;
        }
    }
}
```

```
        // Gán tham số Thêm/Sửa
        2 references
        private void GanThamSo(
            SqlCommand cmd,
            LopDTO lop)
        {
            cmd.Parameters.Add(
                "@MaLop",
                SqlDbType.VarChar,
                10
            ).Value = lop.MaLop;

            cmd.Parameters.Add(
                "@TenLop",
                SqlDbType.NVarChar,
                50
            ).Value = lop.TenLop;

            cmd.Parameters.Add(
                "@NienKhoa",
                SqlDbType.NVarChar,
                10
            ).Value = lop.NienKhoa;

            cmd.Parameters.Add(
                "@MaNganh",
                SqlDbType.VarChar,
                10
            ).Value = lop.MaNganh;
        }
    }
}
```

- Code SQL

```
392
393   -- Sửa lớp
394   CREATE PROCEDURE dbo.sp_SuaLop
395       @MaLop VARCHAR(10),
396       @TenLop NVARCHAR(50),
397       @NienKhoa NVARCHAR(10),
398       @MaNganh VARCHAR(10)
399   AS
400   BEGIN
401       SET NOCOUNT ON;
402
403       UPDATE dbo.Lop
404       SET
405           TenLop = @TenLop,
406           NienKhoa = @NienKhoa,
407           MaNganh = @MaNganh
408       WHERE MaLop = @MaLop;
409   END;
410   GO
411
```

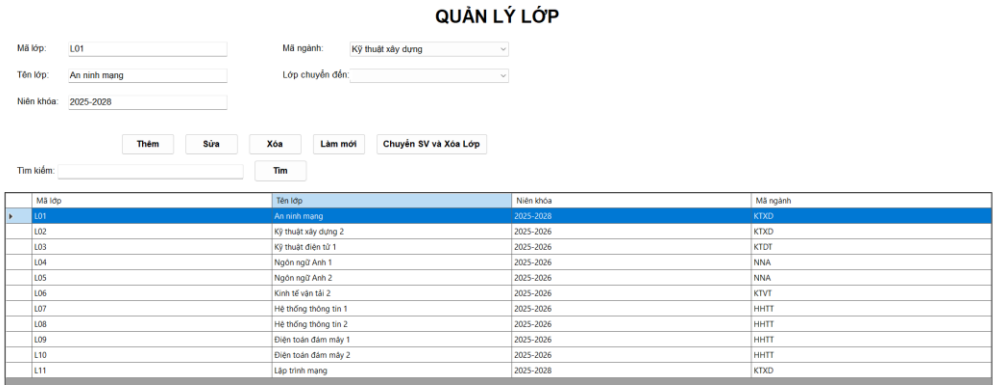
Nhiệm vụ:

+ Dùng để sửa thông tin lớp.

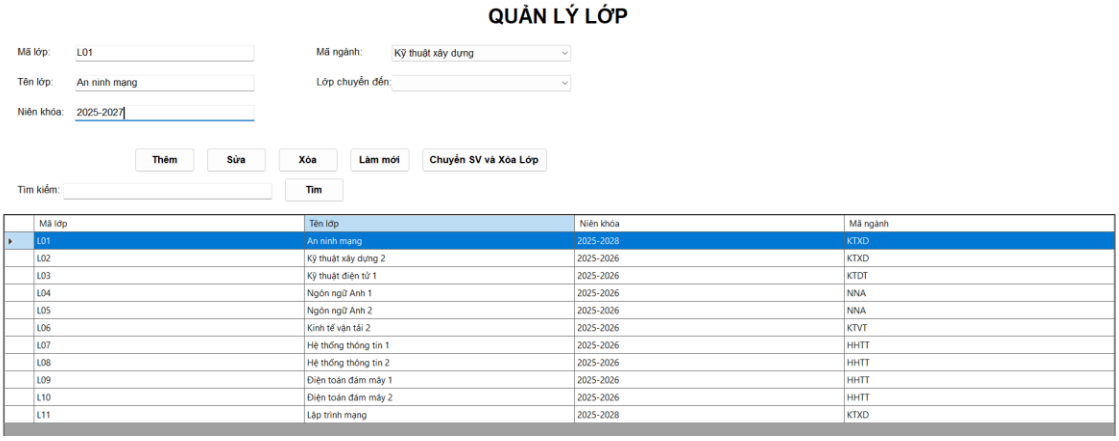
+ SP tìm lớp bằng @MaLop, sau đó cập nhật: Tên lớp, niên khóa, mã ngành

+ Note: MaLop là điều kiện để xác định lớp cần sửa.

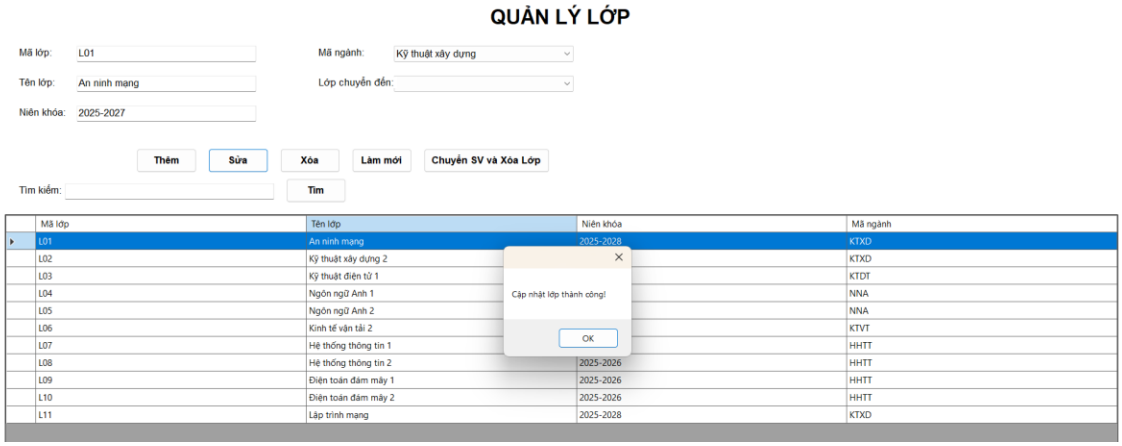
- Giao diện hiển thị sửa lớp: Click vô lớp muốn sửa để hiện thông tin lớp, Click mã L01



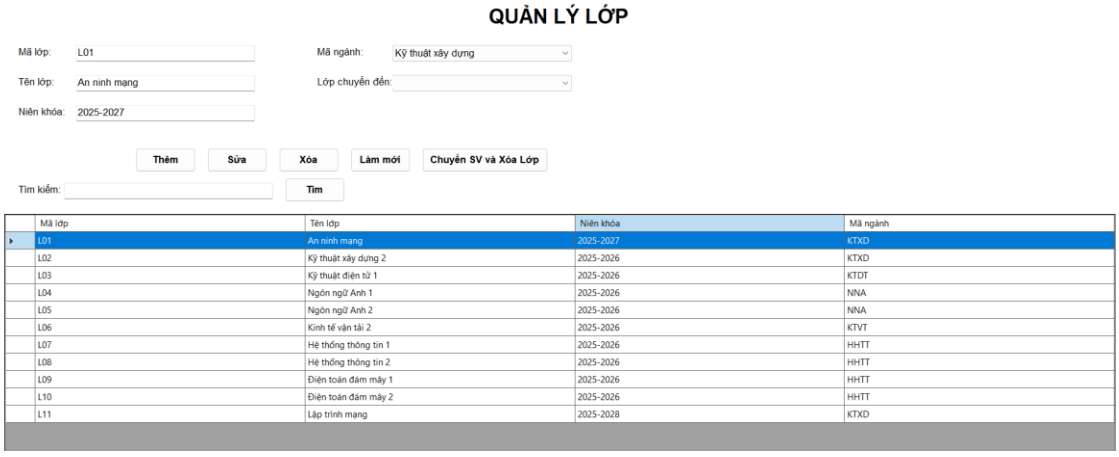
- Giao diện hiển thị sửa lớp: sửa lớp L01 từ năm 2025 -2028 thành 2025-2027



- Giao diện sửa lớp hiển thị "cập nhật thành công!"

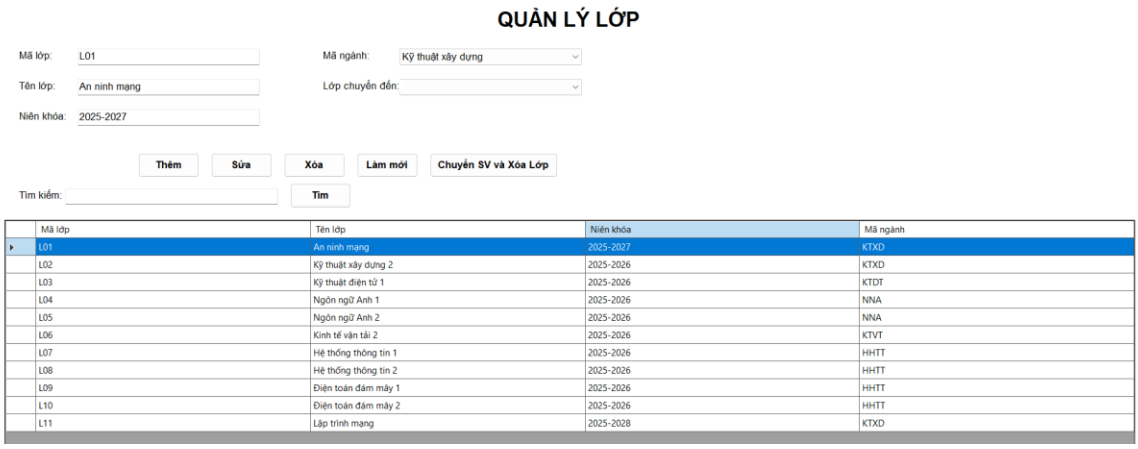


- Giao diện hiển thị sửa niêm khóa của lớp L01 đã xong



3. Chuyển lớp

- Giao diện màn hình trước khi chuyển lớp



- Code C# GUI chuyển lớp: GUI thực hiện các bước:Chọn lớp cũ → chọn lớp mới → kiểm tra 2 lớp có giống nhau không → hỏi xác nhận → gọi BUS → nhận kết quả → hiển thị thông báo.

```
// Chuyển SV và xóa lớp
private void btnChuyenVaXoaLop_Click(
    object sender,
    EventArgs e)
{
    if (dgvLop.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn lớp cũ cần xóa!"
        );

        return;
    }

    if (cboLopMoi.SelectedValue == null)
    {
        MessageBox.Show(
            "Vui lòng chọn lớp chuyển đến!"
        );

        return;
    }

    string maLopCu =
        dgvLop.CurrentRow
        .Cells["MaLop"]
        .Value?
        .ToString()
        .Trim() ?? "";

    string maLopMoi =
        cboLopMoi.SelectedValue
        .ToString()
        .Trim();

    if (maLopCu == maLopMoi)
    {
        MessageBox.Show(
            "Lớp cũ và lớp mới không được giống nhau!"
        );

        return;
    }
```

```
        DialogResult xacNhan =
            MessageBox.Show(
                $"Chuyển toàn bộ sinh viên từ lớp {maLopCu} " +
                $"sang lớp {maLopMoi} và xóa lớp {maLopCu}?",
                "Xác nhận transaction",
                MessageBoxButtons.YesNo,
                MessageBoxIcon.Warning
            );

        if (xacNhan != DialogResult.Yes)
            return;

        try
        {
            string ketQua =
                lopBUS.ChuyenSinhVienVaXoaLop(
                    maLopCu,
                    maLopMoi
                );

            MessageBox.Show(
                ketQua,
                "Transaction thành công",
                MessageBoxButtons.OK,
                MessageBoxIcon.Information
            );

            LoadData();
            LoadLopMoi();
            XoaTrang();
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                "Transaction thất bại!\n" +
                "Dữ liệu đã được ROLLBACK.\n\n" +
                ex.Message,
                "Rollback",
                MessageBoxButtons.OK,
                MessageBoxIcon.Error
            );

            LoadData();
            LoadLopMoi();
        }
    }
```

- Code C# gọi BUS chuyển lớp : BUS có nhiệm vụ kiểm tra dữ liệu trước khi gọi DAL: Có chọn lớp cũ chưa, có chọn lớp mới chưa, lớp cũ và lớp mới có trùng nhau không, nếu hợp lệ → gọi DAL.

```
    // Transaction chuyển SV và xóa lớp
    public string ChuyenSinhVienVaXoaLop(
        string maLopCu,
        string maLopMoi)
    {
        if (string.IsNullOrWhiteSpace(maLopCu))
            return "Vui lòng chọn lớp cũ!";

        if (string.IsNullOrWhiteSpace(maLopMoi))
            return "Vui lòng chọn lớp mới!";

        if (maLopCu.Trim() == maLopMoi.Trim())
            return "Lớp cũ và lớp mới không được giống nhau!";

        return dal.ChuyenSinhVienVaXoaLop(
            maLopCu.Trim(),
            maLopMoi.Trim());
    }
```

- Code C# gọi DAL Stored Procedure chuyển lớp: DAL nhận mã lớp cũ và mã lớp mới, kết nối đến SQL Server và thực hiện Transaction. DAL chuyển sinh viên sang lớp mới, xóa lớp cũ; nếu thành công thì Commit, nếu có lỗi thì Rollback.

```
        // Transaction: chuyển SV sang lớp mới rồi xóa lớp cũ
        public string ChuyenSinhVienVaXoaLop(
            string maLopCu,
            string maLopMoi)
        {
            using (SqlConnection conn = Database.GetConnection())
            {
                conn.Open();

                string sql = @"
SET XACT_ABORT ON;

BEGIN TRY
    BEGIN TRANSACTION;

    IF @MaLopCu = @MaLopMoi
        THROW 50001, N'Lớp cũ và lớp mới không được giống nhau!', 1;

    IF NOT EXISTS
    (
        SELECT 1
        FROM Lop
        WHERE MaLop = @MaLopCu
    )
        THROW 50002, N'Lớp cũ không tồn tại!', 1;

    IF NOT EXISTS
    (
        SELECT 1
        FROM Lop
        WHERE MaLop = @MaLopMoi
    )
        THROW 50003, N'Lớp mới không tồn tại!', 1;

    UPDATE SinhVien
    SET MaLop = @MaLopMoi
    WHERE MaLop = @MaLopCu;

    DECLARE @SoSinhVien INT = @@ROWCOUNT;

    DELETE FROM Lop
    WHERE MaLop = @MaLopCu;
```

```
    COMMIT TRANSACTION;

    SELECT
        N'Đã chuyển ' +
        CAST(@SoSinhVien AS NVARCHAR(10)) +
        N' sinh viên sang lớp ' +
        @MaLopMoi +
        N' và xóa lớp ' +
        @MaLopCu +
        N' thành công!';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;";

                using (SqlCommand cmd =
                       new SqlCommand(sql, conn))
                {
                    cmd.Parameters.Add(
                        "@MaLopCu",
                        SqlDbType.VarChar,
                        10
                    ).Value = maLopCu;

                    cmd.Parameters.Add(
                        "@MaLopMoi",
                        SqlDbType.VarChar,
                        10
                    ).Value = maLopMoi;

                    object? result =
                        cmd.ExecuteScalar();

                    return result?.ToString()
                           ?? "Giao tác hoàn tất.";
                }
            }
        }
```

```
        // Gán tham số Thêm/Sửa
        private void GanThamSo(
            SqlCommand cmd,
            LopDTO lop)
        {
            cmd.Parameters.Add(
                "@MaLop",
                SqlDbType.VarChar,
                10
            ).Value = lop.MaLop;

            cmd.Parameters.Add(
                "@TenLop",
                SqlDbType.NVarChar,
                50
            ).Value = lop.TenLop;

            cmd.Parameters.Add(
                "@NienKhoa",
                SqlDbType.NVarChar,
                10
            ).Value = lop.NienKhoa;

            cmd.Parameters.Add(
                "@MaNganh",
                SqlDbType.VarChar,
                10
            ).Value = lop.MaNganh;
        }
    }
}
```

- Code SQL

```
-- Tạo Procedure Chuyển Lớp
create procedure sp_ChuyenLop
    @MaSV varchar(10),
    @MaLopMoi varchar(10)
as
begin
    update SinhVien
    set MaLop = @MaLopMoi
    where MaSV = @MaSV;
end;
go
-- Tạo Procedure thêm Ngành
create procedure sp_ThemNganh
    @MaNganh varchar(10),
    @TenNganh nvarchar(50),
    @MaKhoa varchar(10)
as
begin
    insert into Nganh
    (MaNganh,TenNganh,MaKhoa)
    values
    (@MaNganh,@TenNganh,@MaKhoa);
end;
go
```

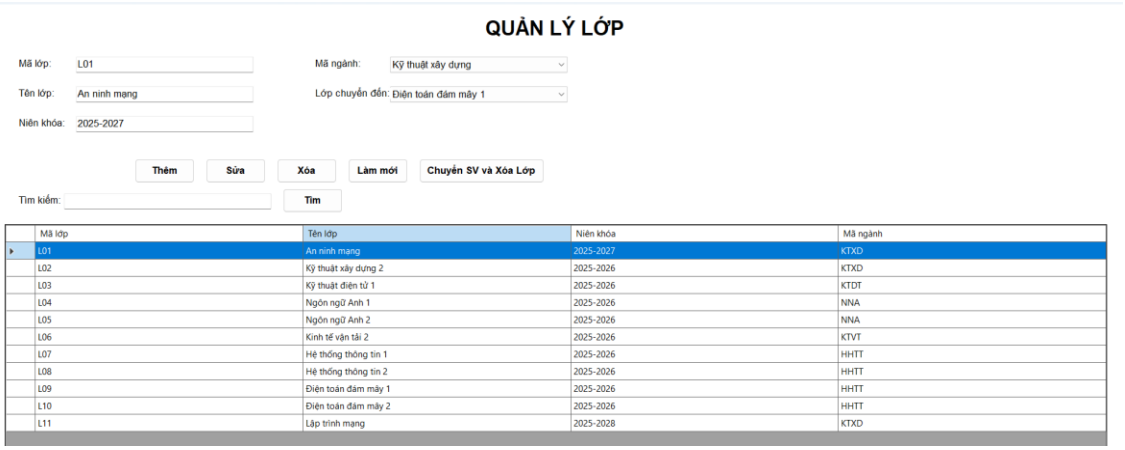
Nhiệm vụ:

+Dùng để chuyển một sinh viên từ lớp này sang lớp khác.

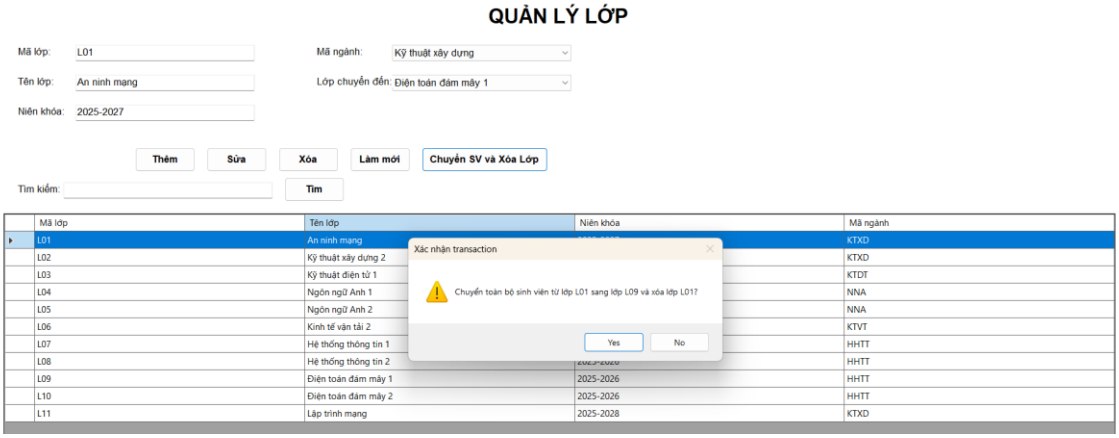
+Nó nhận: @MaSV ,@MaLop

+Sau đó cập nhật MaLop của sinh viên đó.

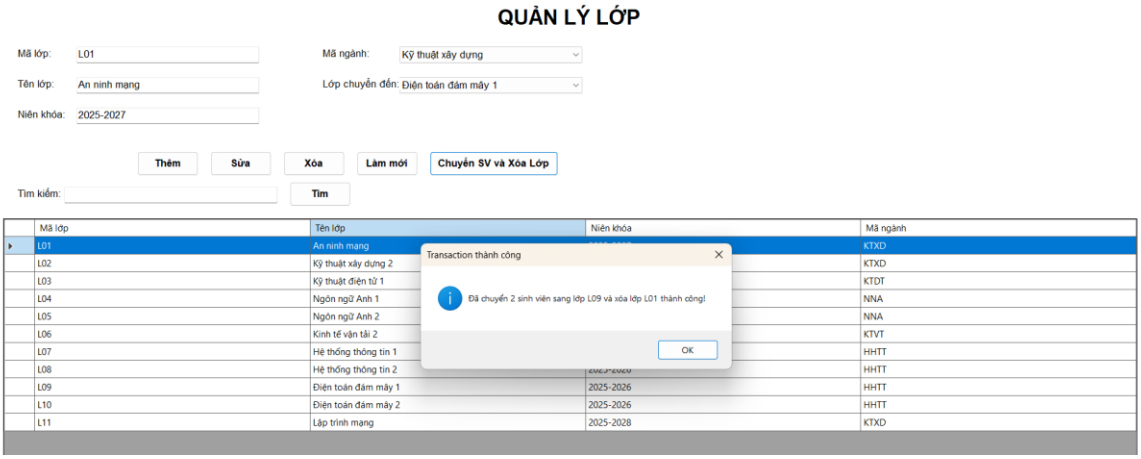
- Giao diện chuyển lớp: Click vô lớp muốn chuyển, sửa chỗ lớp chuyển đến . Chuyển lớp L01 sang L09



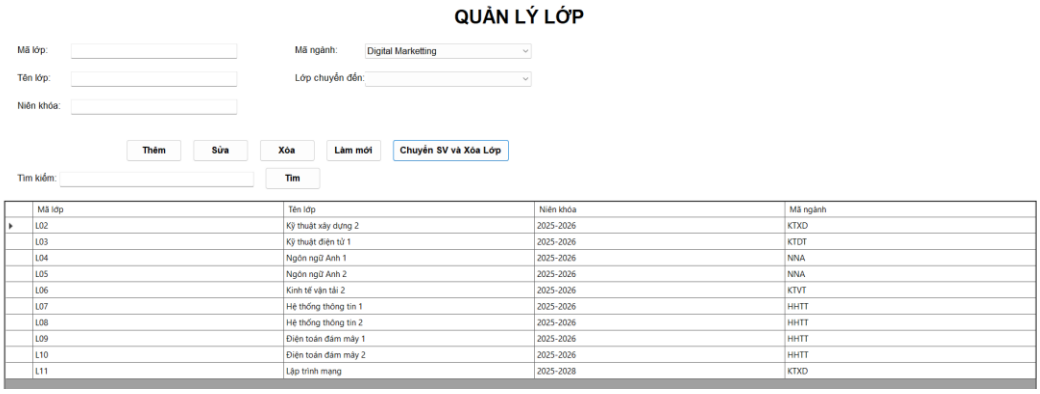
- Giao diện chuyển lớp “chuyển toàn bộ sinh viên lớp L01 sang lớp L09 và xóa L01”



- Giao diện cập nhật chuyển lớp: bấm yes -> hiển thị đã chuyển thành công 2 sinh viên



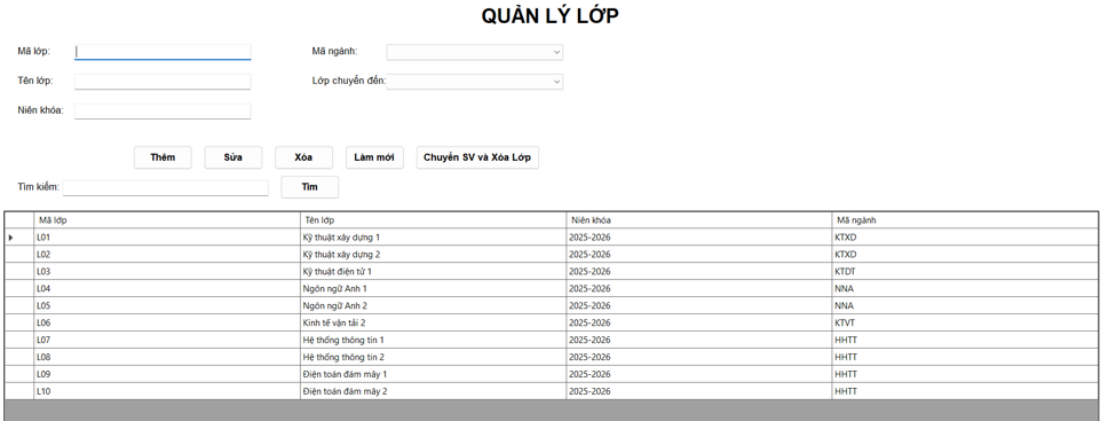
- Giao diện cập nhật đã chuyển lớp L01 sang L09 và xóa lớp L01



4. Tìm lớp

a. Tìm lớp

- Giao diện màn hình trước khi tìm lớp



- Code C# GUI tìm lớp: GUI lấy từ khóa người dùng nhập vào ô tìm kiếm và gọi lopBUS.TimKiem() để tìm lớp.

```
// Tìm kiếm
private void btnTimKiem_Click(object sender, EventArgs e)
{
    try
    {
        string tuKhoa =
            txtTimKiem.Text.Trim();

        if (string.IsNullOrWhiteSpace(tuKhoa))
        {
            LoadData();
            return;
        }

        HienThiDuLieu(
            lopBUS.TimKiem(tuKhoa)
        );
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể tìm kiếm!\n" +
            ex.Message
        );
    }
}
```

- Code C# DTO tìm lớp: LopDTO dùng để lưu thông tin của lớp gồm mã lớp, tên lớp, niên khóa và mã ngành. DTO giúp dữ liệu lớp được truyền giữa các tầng.

```
public class LopDTO
{
    public string MaLop { get; set; } = "";
    public string TenLop { get; set; } = "";
    public string NienKhoa { get; set; } = "";
    public string MaNganh { get; set; } = "";
}
```

- Code C# gọi BUS tìm lớp: BUS tiếp nhận từ khóa và gọi DAL để lấy dữ liệu lớp. Sau đó BUS chuyển dữ liệu từ DataTable thành danh sách LopDTO và trả kết quả về GUI

```
// Tìm kiếm lớp
public List<LopDTO> TimKiem(string keyword)
{
    DataTable dt;

    if (string.IsNullOrWhiteSpace(keyword))
        dt = dal.GetAll();
    else
        dt = dal.Search(keyword.Trim());

    List<LopDTO> ketQua =
        new List<LopDTO>();

    foreach (DataRow row in dt.Rows)
    {
        ketQua.Add(new LopDTO
        {
            MaLop =
                row["MaLop"].ToString() ?? "",

            TenLop =
                row["TenLop"].ToString() ?? "",

            NienKhoa =
                row["NienKhoa"].ToString() ?? "",

            MaNganh =
                row["MaNganh"].ToString() ?? ""
        });
    }

    return ketQua;
}
```

- Code C# gọi DAL Stored Procedure tìm lớp: DAL chịu trách nhiệm kết nối với cơ sở dữ liệu và thực hiện yêu cầu tìm kiếm lớp từ BUS, sau đó trả kết quả về BUS để xử lý.

```
// Tìm lớp bằng Stored Procedure
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_TimLop",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da =
                   new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
        }
    }

    return dt;
}
```

- Code SQL

```
411
412  -- Tạo Procedure Tìm Lớp
413  CREATE OR ALTER PROCEDURE dbo.sp_TimLop
414      @TuKhoa NVARCHAR(100)
415  AS
416  BEGIN
417      SET NOCOUNT ON;
418
419      SELECT
420          MaLop,
421          TenLop,
422          NienKhoa,
423          MaNganh
424      FROM dbo.vw_LopDayDu
425      WHERE
426          MaLop LIKE '%' + @TuKhoa + '%'
427          OR TenLop LIKE N'%' + @TuKhoa + N'%'
428          OR NienKhoa LIKE N'%' + @TuKhoa + N'%'
429          OR MaNganh LIKE '%' + @TuKhoa + '%'
430          OR TenNganh LIKE N'%' + @TuKhoa + N'%'
431      ORDER BY MaLop;
432  END;
433  GO
434
435
```

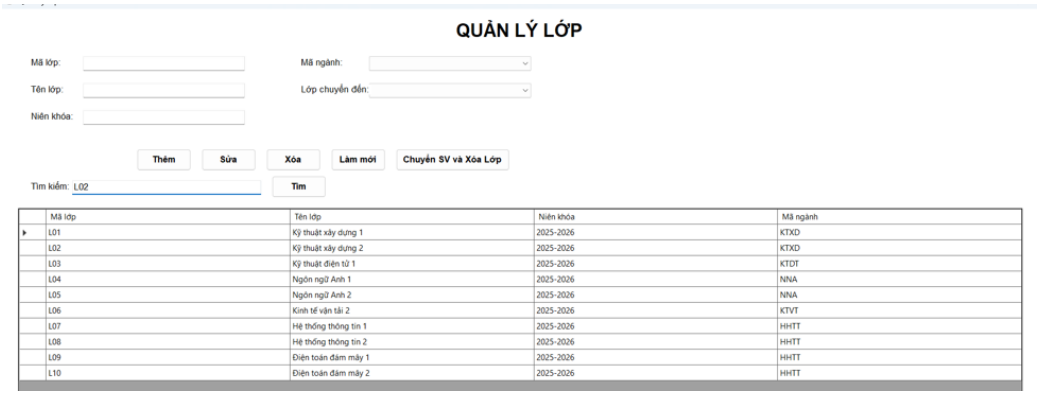
Nhiệm vụ:

+ Dùng để tìm kiếm lớp.

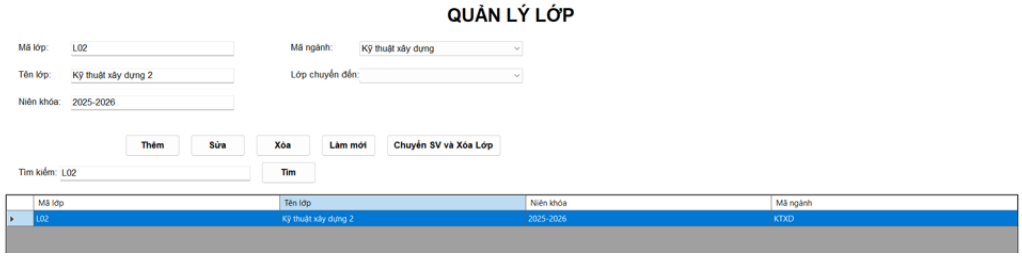
+SP cho phép tìm theo nhiều thông tin: Mã lớp, tên lớp, niên khóa, mã ngành, tên ngành

+ Dữ liệu được lấy từ vw_LopDayDu.

- Giao diện tìm lớp: Click ô tìm kiếm nhập mã lớp L02

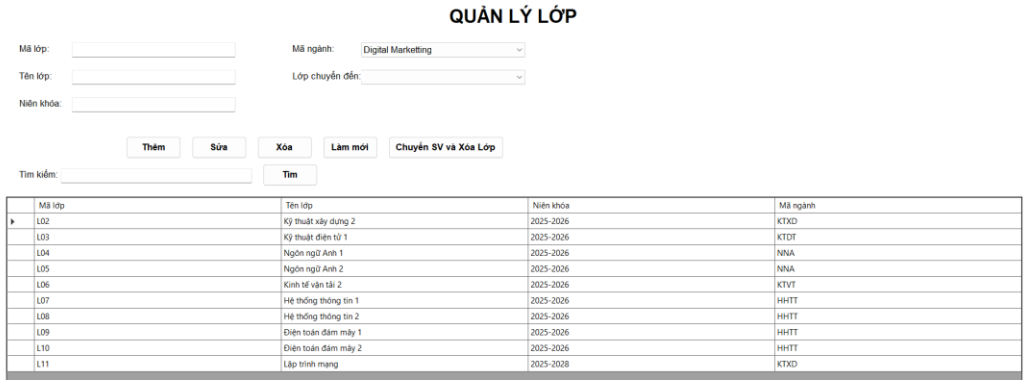


- Giao diện đã tìm lớp L02 xong



5. Xóa lớp

- Giao diện màn hình trước khi xóa lớp



- Code C# GUI xóa lớp: Khi bấm Xóa, GUI kiểm tra đã chọn lớp trong DataGridView chưa. Sau đó lấy MaLop của dòng đang chọn và hiển thị hộp thoại xác nhận. Nếu người dùng đồng ý, GUI gọi lopBUS.Xoa(maLop) để thực hiện xóa; nếu thành công thì tải lại danh sách lớp và xóa trắng giao diện.

```
// Xóa lớp
1 reference
private void btnXoa_Click(object sender, EventArgs e)
{
    if (dgvLop.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn lớp cần xóa!"
        );

        return;
    }

    string maLop =
        dgvLop.CurrentRow
        .Cells["MaLop"]
        .Value?
        .ToString()
        .Trim() ?? "";

    if (MessageBox.Show(
            "Bạn có chắc muốn xóa lớp " + maLop + "?",
            "Xác nhận",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question)
        != DialogResult.Yes)
    {
        return;
    }
```

```
    try
    {
        if (!lopBUS.Xoa(maLop))
        {
            MessageBox.Show(
                "Xóa lớp thất bại!"
            );

            return;
        }

        MessageBox.Show(
            "Xóa lớp thành công!"
        );

        LoadData();
        LoadLopMoi();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể xóa lớp!\n" +
            ex.Message
        );
    }
}
```

- Code C# DTO xóa lớp: Xóa chỉ cần mã lớp MaLop để xác định lớp cần xóa, không cần các thông tin như tên lớp, niên khóa hay mã ngành. Vì vậy em truyền trực tiếp MaLop kiểu string, không cần tạo LopDTO

```
public class LopDTO
{
    7 references
    public string MaLop { get; set; } = "";
    5 references
    public string TenLop { get; set; } = "";
    5 references
    public string NienKhoa { get; set; } = "";
    5 references
    public string MaNganh { get; set; } = "";
}
```

- Code C# gọi BUS xóa lớp: BUS nhận MaLop từ GUI và kiểm tra mã lớp có rỗng hay chỉ chứa khoảng trắng hay không. Nếu mã lớp hợp lệ, BUS gọi dal.Delete(maLop) để chuyển yêu cầu xuống DAL thực hiện xóa trong cơ sở dữ liệu.

```
// Xóa lớp
1 reference
public bool Xoa(string maLop)
{
    if (string.IsNullOrWhiteSpace(maLop))
        return false;

    return dal.Delete(maLop);
}
```

- Code C# gọi DAL Stored Procedure xóa lớp: DAL mở kết nối đến SQL Server thông qua Database.GetConnection(). Sau đó tạo SqlCommand để gọi Stored Procedure dbo.sp_XoaLop, đặt kiểu lệnh là StoredProcedure và truyền mã lớp vào tham số @MaLop. Cuối cùng, ExecuteScalar() thực hiện Stored Procedure và lấy kết quả trả về, sau đó Convert.ToBoolean() chuyển kết quả đó thành true hoặc false.

```
// Xóa lớp bằng Stored Procedure
1 reference
public bool Delete(string maLop)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_XoaLop",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@MaLop",
                SqlDbType.VarChar,
                10
            ).Value = maLop;

            return Convert.ToBoolean(
                cmd.ExecuteScalar()
            );
        }
    }
}
```

- Code SQL

```sql
-- Tạo Procedure Xóa Lớp
CREATE PROCEDURE dbo.sp_XoaLop
    @MaLop VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (
        SELECT 1
        FROM dbo.Lop
        WHERE MaLop = @MaLop
    )
    BEGIN
        SELECT CAST(0 AS BIT);
        RETURN;
    END;

    -- Không xóa lớp đang có sinh viên
    IF EXISTS (
        SELECT 1
        FROM dbo.SinhVien
        WHERE MaLop = @MaLop
    )
    BEGIN
        THROW 50010, N'Không thể xóa lớp đang có sinh viên!', 1;
    END;

    DELETE FROM dbo.Lop
    WHERE MaLop = @MaLop;

    SELECT CAST(1 AS BIT);
END;
GO
```

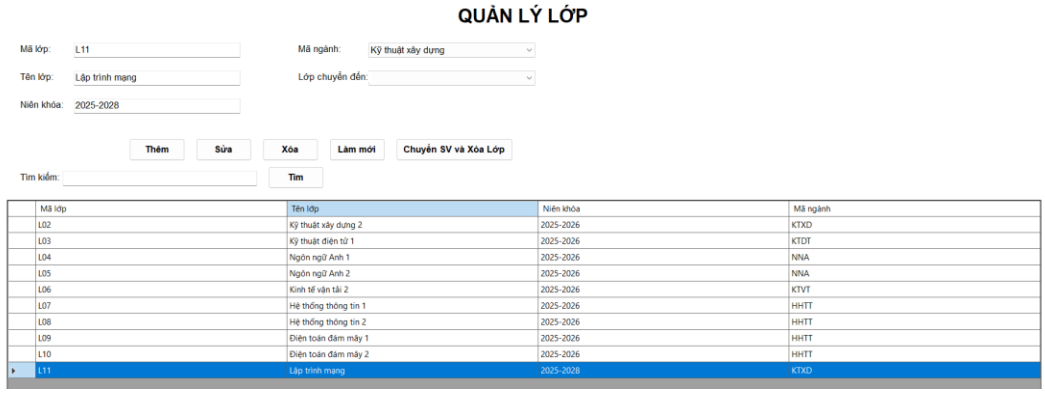
Nhiệm vụ:

+ Dùng để xóa lớp, nhưng trước khi xóa sẽ kiểm tra lớp có sinh viên hay không.

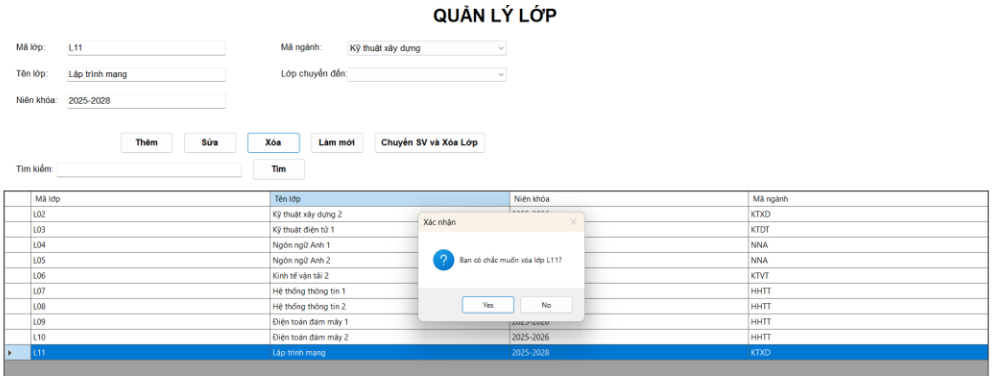
+ Quy trình:

Kiểm tra lớp tồn tại → kiểm tra có sinh viên → nếu có thì không cho xóa → nếu không có thì xóa.

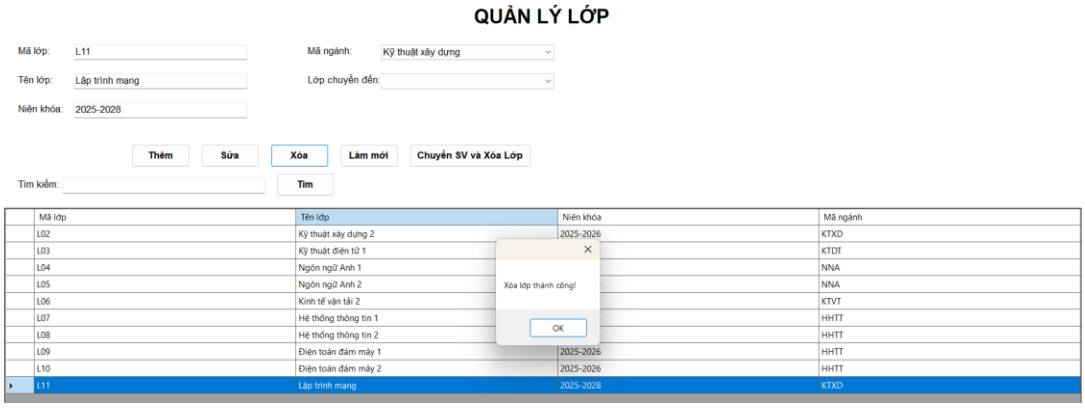
- Giao diện xóa lớp: Click vô lớp muốn xóa cho nó hiện dữ liệu. Xóa L11



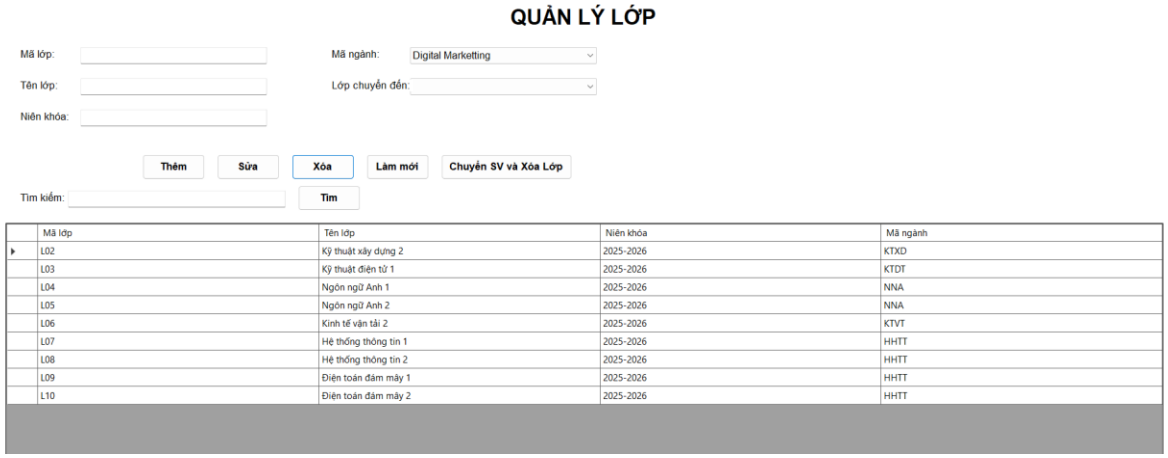
- Giao diện hiển thị bạn có chắc chắn xóa lớp " L11" không



- Giao diện xóa lớp: bấm yes -> xóa thành công



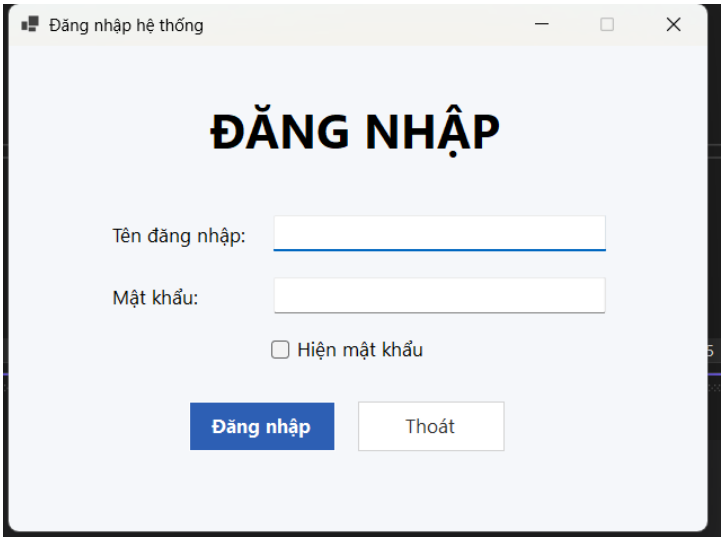
- Giao diện cập nhật lại: không còn lớp L11 nữa



TÀI KHOẢN:

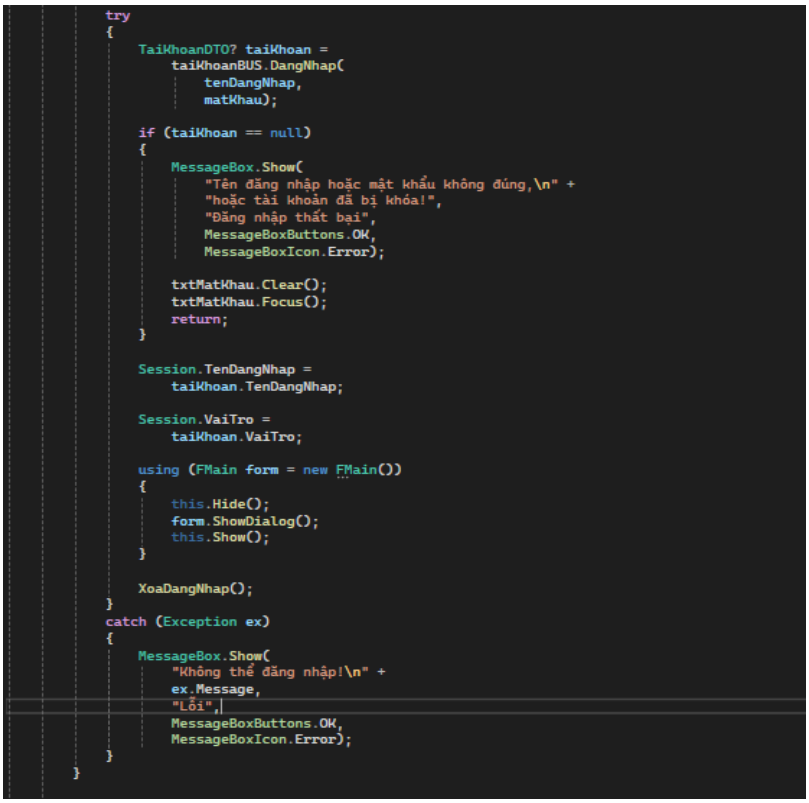
1. Đăng nhập tài khoản

- Giao diện màn hình trước khi đăng nhập



- Code C# GUI đăng nhập tài khoản: GUI lấy tên đăng nhập và mật khẩu từ hai ô nhập liệu. Sau đó kiểm tra dữ liệu có bị bỏ trống hay không rồi gọi taiKhoanBUS.DangNhap(). Nếu đăng nhập thành công, chương trình lưu tên đăng nhập và vai trò vào Session, ẩn form đăng nhập và mở FMain; nếu thất bại thì hiển thị thông báo lỗi.

```
// Đăng nhập
1 reference
private void DangNhap()
{
    string tenDangNhap =
        txtTenDangNhap.Text.Trim();

    string matKhau =
        txtMatKhau.Text;

    if (string.IsNullOrWhiteSpace(tenDangNhap))
    {
        MessageBox.Show(
            "Vui lòng nhập tên đăng nhập!",
            "Thông báo",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        txtTenDangNhap.Focus();
        return;
    }

    if (string.IsNullOrWhiteSpace(matKhau))
    {
        MessageBox.Show(
            "Vui lòng nhập mật khẩu!",
            "Thông báo",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        txtMatKhau.Focus();
        return;
    }
```



```
    try
    {
        TaiKhoanDTO? taiKhoan =
            taiKhoanBUS.DangNhap(
                tenDangNhap,
                matKhau);

        if (taiKhoan == null)
        {
            MessageBox.Show(
                "Tên đăng nhập hoặc mật khẩu không đúng,\n" +
                "hoặc tài khoản đã bị khóa!",
                "Đăng nhập thất bại",
                MessageBoxButtons.OK,
                MessageBoxIcon.Error);

            txtMatKhau.Clear();
            txtMatKhau.Focus();
            return;
        }

        Session.TenDangNhap =
            taiKhoan.TenDangNhap;

        Session.VaiTro =
            taiKhoan.VaiTro;

        using (FMain form = new FMain())
        {
            this.Hide();
            form.ShowDialog();
            this.Show();
        }

        XoaDangNhap();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể đăng nhập!\n" +
            ex.Message,
            "Lỗi",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

- Code C# DTO đăng nhập tài khoản: TaiKhoanDTO chứa thông tin của tài khoản gồm **tên** đăng nhập, mật khẩu, vai trò và trạng thái. Trong chức năng đăng nhập, DTO được dùng để nhận thông tin tài khoản mà DAL trả về sau khi kiểm tra thành công.

```
14 references
public class TaiKhoanDTO
{
    7 references
    public string TenDangNhap { get; set; } = "";
    4 references
    public string MatKhau { get; set; } = "";
    5 references
    public string VaiTro { get; set; } = "";
    3 references
    public bool TrangThai { get; set; }
}
```

- Code C# gọi BUS đăng nhập tài khoản: BUS nhận tên đăng nhập và mật khẩu từ GUI. BUS kiểm tra hai thông tin này có bị rỗng hay không; nếu rỗng thì trả về null. Nếu hợp lệ, BUS gọi dal.DangNhap() để kiểm tra tài khoản trong cơ sở dữ liệu..

```
// Đăng nhập
1 reference
public TaiKhoanDTO? DangNhap(
    string tenDangNhap,
    string matKhau)
{
    if (string.IsNullOrWhiteSpace(tenDangNhap) ||
        string.IsNullOrWhiteSpace(matKhau))
    {
        return null;
    }

    return dal.DangNhap(
        tenDangNhap.Trim(),
        matKhau);
}
```

- Code C# gọi DAL Stored Procedure đăng nhập tài khoản: DAL mở kết nối đến SQL Server và gọi Stored Procedure dbo.sp_DangNhap. Hai tham số @TenDangNhap và @MatKhau được truyền vào Stored Procedure. DAL dùng ExecuteReader() để đọc kết quả; nếu không có tài khoản phù hợp thì trả về null, còn nếu có thì tạo TaiKhoanDTO chứa thông tin tài khoản và trả về.

```
// Đăng nhập
1 reference
public TaiKhoanDTO? DangNhap(
    string tenDangNhap,
    string matKhau)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_DangNhap",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TenDangNhap",
                SqlDbType.NVarChar,
                50
            ).Value = tenDangNhap;

            cmd.Parameters.Add(
                "@MatKhau",
                SqlDbType.VarChar,
                100
            ).Value = matKhau;
```

```
            using (SqlDataReader reader =
                   cmd.ExecuteReader())
            {
                if (!reader.Read())
                    return null;

                return new TaiKhoanDTO
                {
                    TenDangNhap =
                        reader["TenDangNhap"]
                        .ToString() ?? "",

                    MatKhau =
                        reader["MatKhau"]
                        .ToString() ?? "",

                    VaiTro =
                        reader["VaiTro"]
                        .ToString() ?? "",

                    TrangThai =
                        Convert.ToBoolean(
                            reader["TrangThai"])
                };
```

- Code SQL

```
-- Đăng nhập tài khoản
CREATE PROCEDURE dbo.sp_DangNhap
    @TenDangNhap VARCHAR(50),
    @MatKhau VARCHAR(100)
AS
BEGIN
    SELECT TenDangNhap, MatKhau, VaiTro, TrangThai
    FROM dbo.TaiKhoan
    WHERE TenDangNhap = @TenDangNhap AND MatKhau = @MatKhau AND TrangThai = 1
END
GO
-- Thêm tài khoản

CREATE OR ALTER PROCEDURE dbo.sp_ThemTaiKhoan
    @TenDangNhap NVARCHAR(50),
    @MatKhau VARCHAR(100),
    @VaiTro NVARCHAR(20),
    @TrangThai BIT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.TaiKhoan
        (TenDangNhap, MatKhau, VaiTro, TrangThai)
    VALUES
        (@TenDangNhap, @MatKhau, @VaiTro, @TrangThai);
END;
GO
```

Nhiệm vụ:

+ Dùng để kiểm tra thông tin đăng nhập

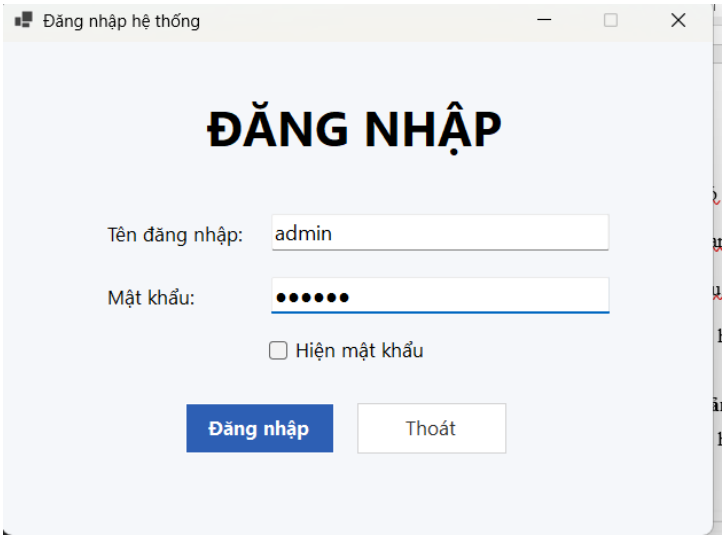
+ Nhận vào @TenDangNhap và @MatKhau

+ Tìm tài khoản có đúng tên đăng nhập, mật khẩu và TrangThai = 1.

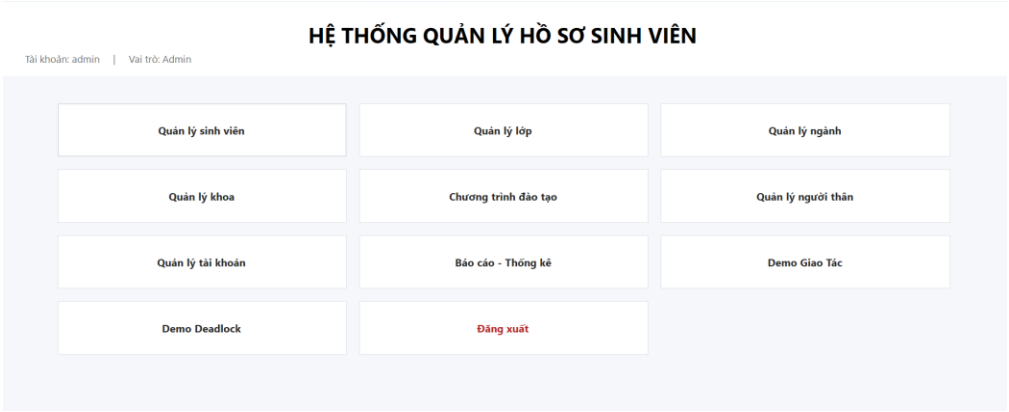
+ Nếu tài khoản đang hoạt động thì trả về thông tin tài khoản.

+ Nếu sai mật khẩu, sai tên đăng nhập hoặc tài khoản bị khóa thì không trả về kết quả.

- Giao diện màn hình khi đăng nhập

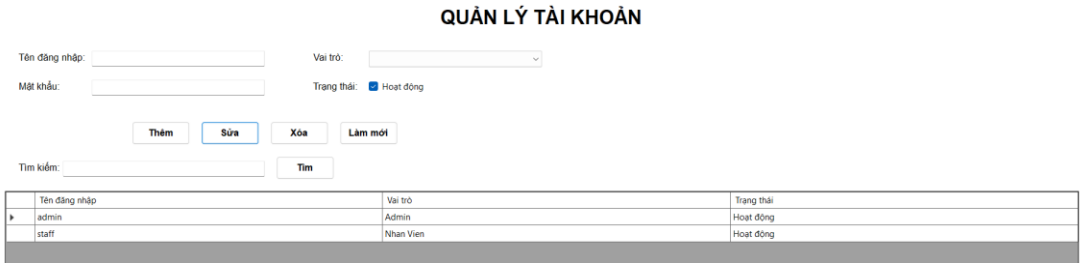


- Giao diện màn hình đăng nhập thành công



2. Thêm tài khoản

- Giao diện màn hình trước khi thêm tài khoản



- Code C# GUI thêm tài khoản: GUI lấy tên đăng nhập, mật khẩu, vai trò và trạng thái từ Form, tạo TaiKhoanDTO, sau đó gọi taiKhoanBUS.Them(tk). Nếu thêm thành công thì hiển thị thông báo, tải lại danh sách và xóa trắng Form.

```
1 reference
private void btnThem_Click(
    object sender,
    EventArgs e)
{
    try
    {
        TaiKhoanDTO tk =
            LayDuLieu();

        if (!taiKhoanBUS.Them(tk))
        {
            MessageBox.Show(
                "Thêm tài khoản thất bại! " +
                "Kiểm tra dữ liệu hoặc tên đăng nhập đã tồn tại.");
            return;
        }

        MessageBox.Show(
            "Thêm tài khoản thành công!");

        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể thêm tài khoản!\n" +
            ex.Message);
    }
}
```

```
// Lấy dữ liệu từ Form

private TaiKhoanDTO LayDuLieu()
{
    return new TaiKhoanDTO
    {
        TenDangNhap =
            txtTenDangNhap.Text.Trim(),

        MatKhau =
            txtMatKhau.Text,

        VaiTro =
            cboVaiTro.Text,

        TrangThai =
            chkTrangThai.Checked
    };
}
```

- Code C# DTO thêm tài khoản: DTO tài khoản gồm 4 thuộc tính là tên đăng nhập, mật khẩu, vai trò và trạng thái. DTO dùng để đóng gói các thông tin này thành một đối tượng TaiKhoanDTO để truyền xuống BUS và DAL.

```
public class TaiKhoanDTO
{
    7 references
    public string TenDangNhap { get; set; } = "";
    4 references
    public string MatKhau { get; set; } = "";
    5 references
    public string VaiTro { get; set; } = "";
    3 references
    public bool TrangThai { get; set; }
}
}
```

- Code C# gọi BUS thêm tài khoản: BUS nhận TaiKhoanDTO từ GUI, kiểm tra dữ liệu có hợp lệ không và kiểm tra tên đăng nhập có bị trùng không. Nếu dữ liệu hợp lệ và chưa tồn tại thì BUS gọi DAL để thêm tài khoản.

```
// Thêm tài khoản
1 reference
public bool Them(TaiKhoanDTO tk)
{
    if (!DuLieuHopLe(tk))
        return false;

    if (dal.KiemTraTonTai(tk.TenDangNhap))
        return false;

    return dal.Insert(tk);
}
```

- Code C# gọi DAL Stored Procedure thêm tài khoản: DAL mở kết nối SQL Server, tạo SqlCommand để gọi Stored Procedure dbo.sp_ThemTaiKhoan, sau đó truyền 4 thông tin của TaiKhoanDTO vào các tham số tương ứng. Cuối cùng ExecuteNonQuery() thực hiện Stored Procedure và trả về true nếu thao tác được thực hiện.

```
// Thêm tài khoản
1 reference
public bool Insert(TaiKhoanDTO tk)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_ThemTaiKhoan",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, tk);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

```
        // Gán tham số Thêm/Sửa
        2 references
        private void GanThamSo(
            SqlCommand cmd,
            TaiKhoanDTO tk)
        {
            cmd.Parameters.Add(
                "@TenDangNhap",
                SqlDbType.NVarChar,
                50
            ).Value = tk.TenDangNhap;

            cmd.Parameters.Add(
                "@MatKhau",
                SqlDbType.VarChar,
                100
            ).Value = tk.MatKhau;

            cmd.Parameters.Add(
                "@VaiTro",
                SqlDbType.NVarChar,
                20
            ).Value = tk.VaiTro;

            cmd.Parameters.Add(
                "@TrangThai",
                SqlDbType.Bit
            ).Value = tk.TrangThai;
        }
    }
}
```

- Code SQL

```
171   -- Thêm tài khoản
172
173   CREATE OR ALTER PROCEDURE dbo.sp_ThemTaiKhoan
174       @TenDangNhap NVARCHAR(50),
175       @MatKhau VARCHAR(100),
176       @VaiTro NVARCHAR(20),
177       @TrangThai BIT
178   AS
179   BEGIN
180       SET NOCOUNT ON;
181
182       INSERT INTO dbo.TaiKhoan
183           (TenDangNhap, MatKhau, VaiTro, TrangThai)
184       VALUES
185           (@TenDangNhap, @MatKhau, @VaiTro, @TrangThai);
186   END;
187   GO
188
```

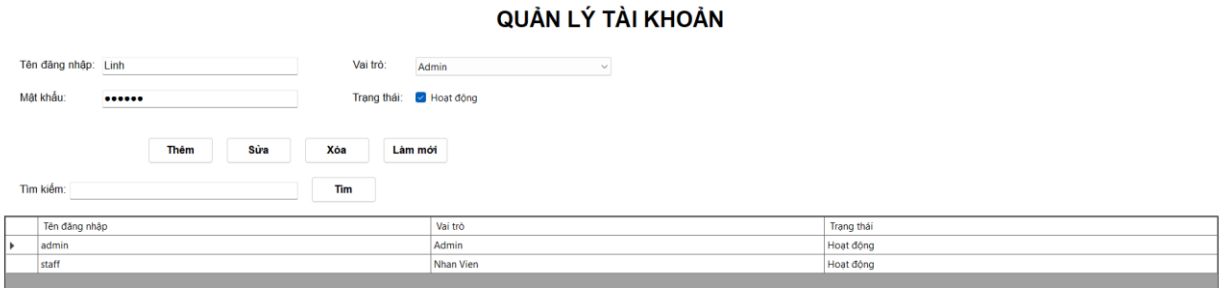
Nhiệm vụ:

+ Dùng để thêm tài khoản mới vào bảng TaiKhoan.

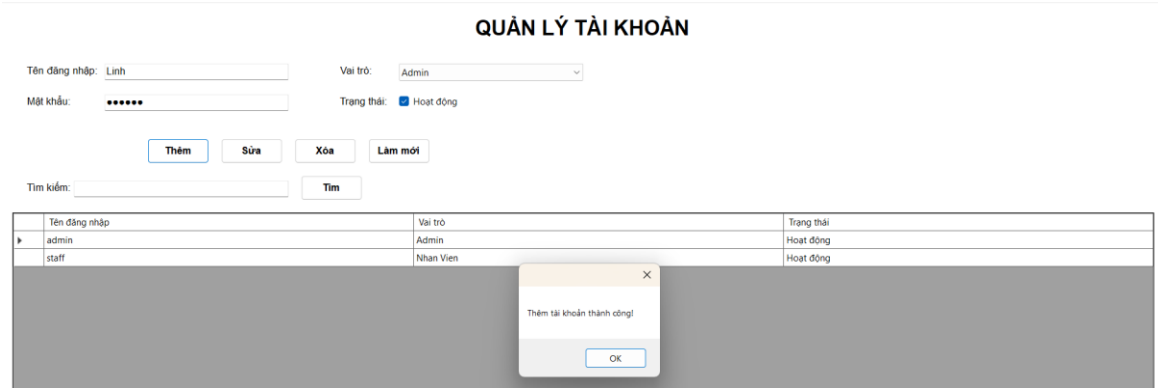
+ Nhận các thông tin: Tên đăng nhập , Mật khẩu,Vai trò ,Trạng thái

+ Sau đó thực hiện INSERT vào bảng.

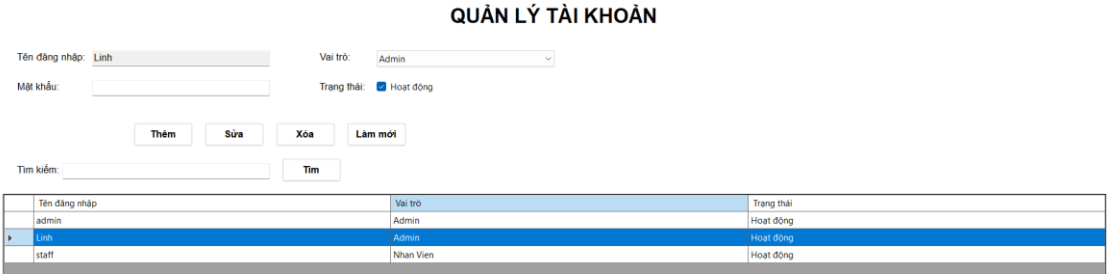
- Giao diện thêm tài khoản: nhập thông tin tài khoản



- Giao diện thêm tài khoản hiển thị thành công

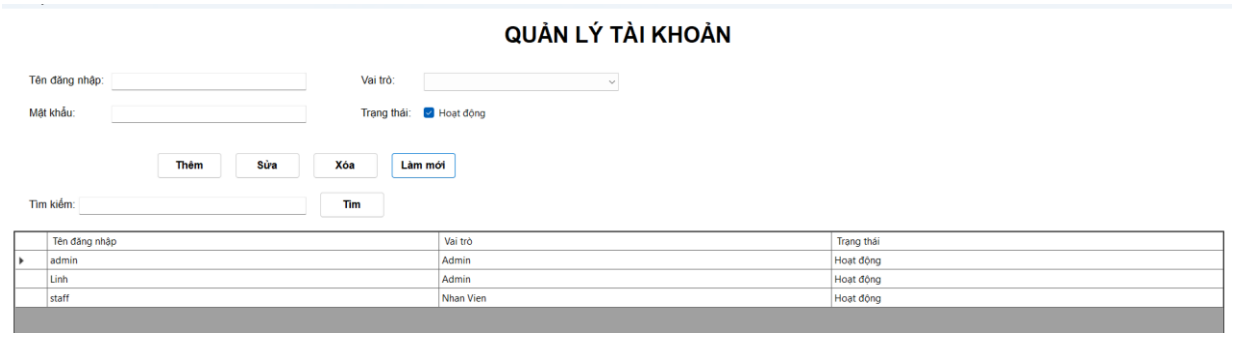


- Giao diện thêm tài khoản được cập nhật ở dưới



3. Sửa tài khoản

- Giao diện màn hình trước khi sửa tài khoản

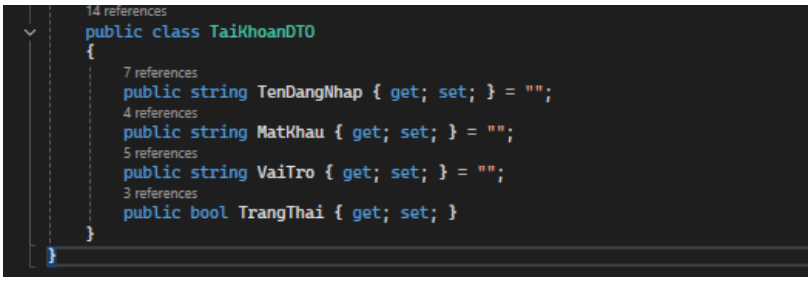


- Code C# GUI sửa tài khoản: Khi bấm Sửa, GUI kiểm tra người dùng đã chọn tài khoản chưa và kiểm tra mật khẩu mới có bị bỏ trống không. Sau đó GUI lấy dữ liệu từ Form, tạo TaiKhoanDTO rồi gọi taiKhoanBUS.Sua(tk). Nếu sửa thành công, danh sách tài khoản được tải lại và Form được xóa trắng.

```
private void btnSua_Click(
    object sender,
    EventArgs e)
{
    if (dgvTaiKhoan.CurrentRow == null)
    {
        MessageBox.Show(
            "Vui lòng chọn tài khoản cần sửa!");
        return;
    }

    if (string.IsNullOrWhiteSpace(
        txtMatKhau.Text))
    {
        MessageBox.Show(
            "Vui lòng nhập mật khẩu mới!");

        txtMatKhau.Focus();
        return;
    }

    try
    {
        TaiKhoanDTO tk =
            LayDuLieu();

        if (!taiKhoanBUS.Sua(tk))
        {
            MessageBox.Show(
                "Cập nhật tài khoản thất bại!");
            return;
        }

        MessageBox.Show(
            "Cập nhật tài khoản thành công!");

        LoadData();
        XoaTrang();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể cập nhật tài khoản!\n" +
            ex.Message);
    }
}
```

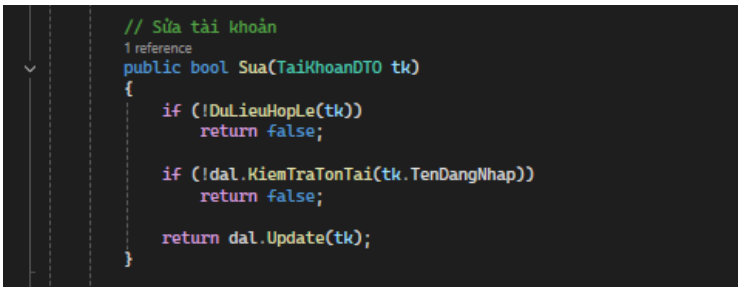
```
// Lấy dữ liệu từ Form
2 references
private TaiKhoanDTO LayDuLieu()
{
    return new TaiKhoanDTO
    {
        TenDangNhap =
            txtTenDangNhap.Text.Trim(),

        MatKhau =
            txtMatKhau.Text,

        VaiTro =
            cboVaiTro.Text,

        TrangThai =
            chkTrangThai.Checked
    };
}
```

- Code C# DTO sửa tài khoản: TaiKhoanDTO đóng gói thông tin tài khoản cần cập nhật gồm Tên đăng nhập, Mật khẩu, Vai trò và Trạng thái. GUI tạo đối tượng này rồi truyền xuống BUS để xử lý.



```
14 references
public class TaiKhoanDTO
{
    7 references
    public string TenDangNhap { get; set; } = "";
    4 references
    public string MatKhau { get; set; } = "";
    5 references
    public string VaiTro { get; set; } = "";
    3 references
    public bool TrangThai { get; set; }
}
```

- Code C# gọi BUS sửa tài khoản: BUS nhận TaiKhoanDTO từ GUI. Đầu tiên BUS kiểm tra dữ liệu có hợp lệ hay không, sau đó kiểm tra tên đăng nhập có tồn tại trong cơ sở dữ liệu hay không. Nếu tài khoản tồn tại thì BUS gọi dal.Update(tk) để thực hiện cập nhật.



```
// Sửa tài khoản
1 reference
public bool Sua(TaiKhoanDTO tk)
{
    if (!DuLieuHopLe(tk))
        return false;

    if (!dal.KiemTraTonTai(tk.TenDangNhap))
        return false;

    return dal.Update(tk);
}
```

- Code C# gọi DAL Stored Procedure sửa tài khoản: DAL mở kết nối SQL Server và tạo SqlCommand để gọi Stored Procedure dbo.sp_SuaTaiKhoan. Sau đó GanThamSo() truyền 4 thông tin của TaiKhoanDTO vào các tham số @TenDangNhap, @MatKhau, @VaiTro, @TrangThai. Cuối cùng ExecuteNonQuery() thực hiện cập nhật tài khoản.

```
// Sửa tài khoản
1 reference
public bool Update(TaiKhoanDTO tk)
{
    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_SuaTaiKhoan",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            GanThamSo(cmd, tk);
            cmd.ExecuteNonQuery();

            return true;
        }
    }
}
```

```
        // Gán tham số Thêm/Sửa
        2 references
        private void GanThamSo(
            SqlCommand cmd,
            TaiKhoanDTO tk)
        {
            cmd.Parameters.Add(
                "@TenDangNhap",
                SqlDbType.NVarChar,
                50
            ).Value = tk.TenDangNhap;

            cmd.Parameters.Add(
                "@MatKhau",
                SqlDbType.VarChar,
                100
            ).Value = tk.MatKhau;

            cmd.Parameters.Add(
                "@VaiTro",
                SqlDbType.NVarChar,
                20
            ).Value = tk.VaiTro;

            cmd.Parameters.Add(
                "@TrangThai",
                SqlDbType.Bit
            ).Value = tk.TrangThai;
        }
    }
}
```

- Code SQL

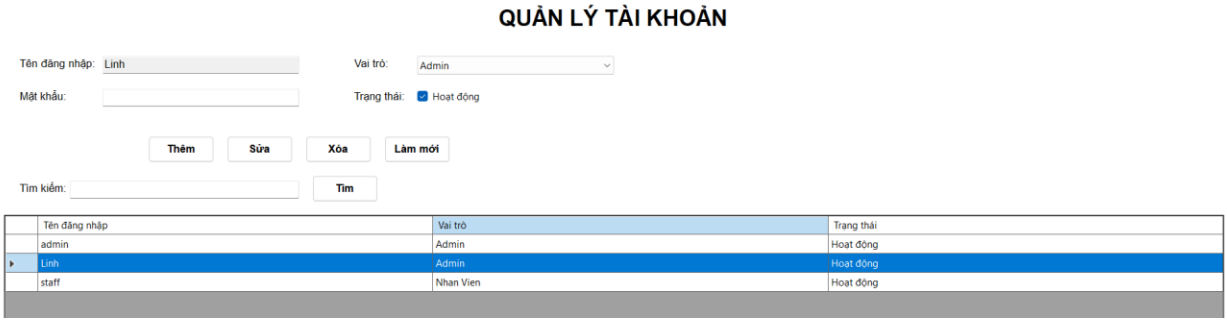
```
    -- Sửa tài khoản
    CREATE OR ALTER PROCEDURE dbo.sp_SuaTaiKhoan
        @TenDangNhap NVARCHAR(50),
        @MatKhau VARCHAR(100),
        @VaiTro NVARCHAR(20),
        @TrangThai BIT
    AS
    BEGIN
        SET NOCOUNT ON;

        UPDATE dbo.TaiKhoan
        SET
            MatKhau = @MatKhau,
            VaiTro = @VaiTro,
            TrangThai = @TrangThai
        WHERE TenDangNhap = @TenDangNhap;
    END;
    GO
```

Nhiệm vụ:

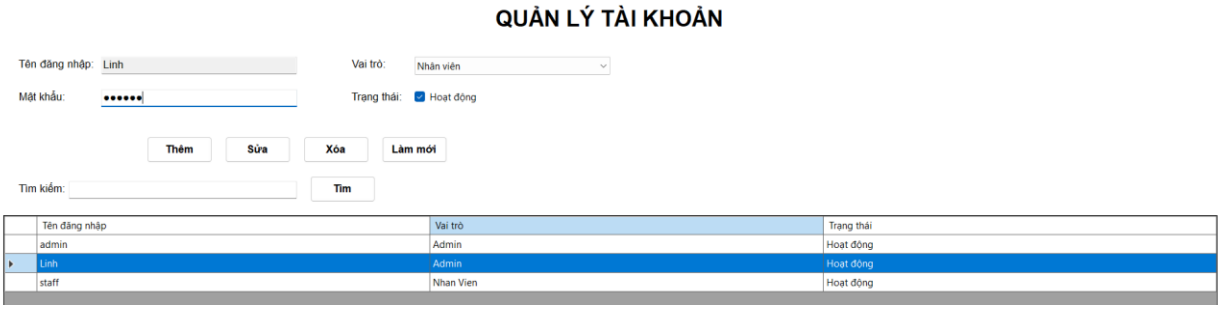
+ Dùng để cập nhật tài khoản đã có.

+ Tìm tài khoản dựa vào @TenDangNhap, sau đó cập nhật: Mật khẩu, Vai trò, Trạng thái

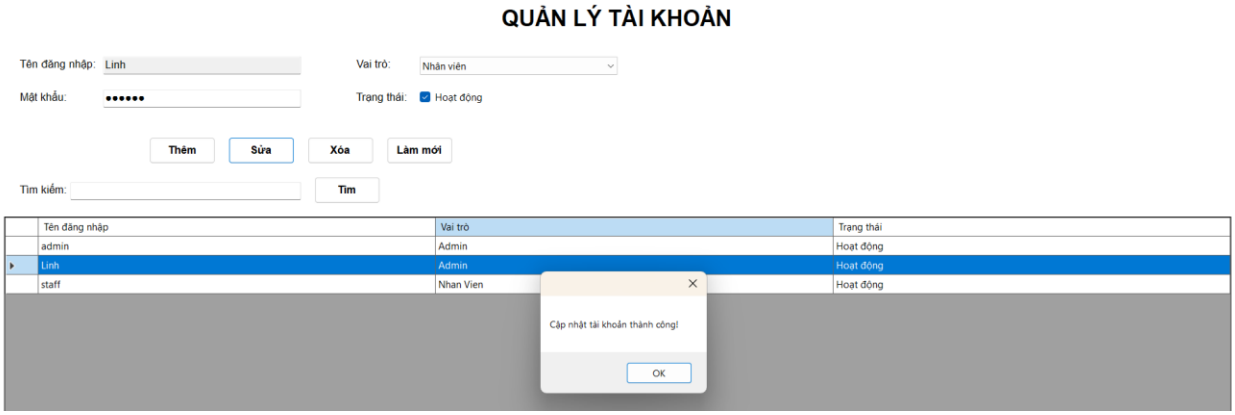
- Giao diện sửa tài khoản: click vô tài khoản muốn sửa chọn để nó hiển thị thông tin



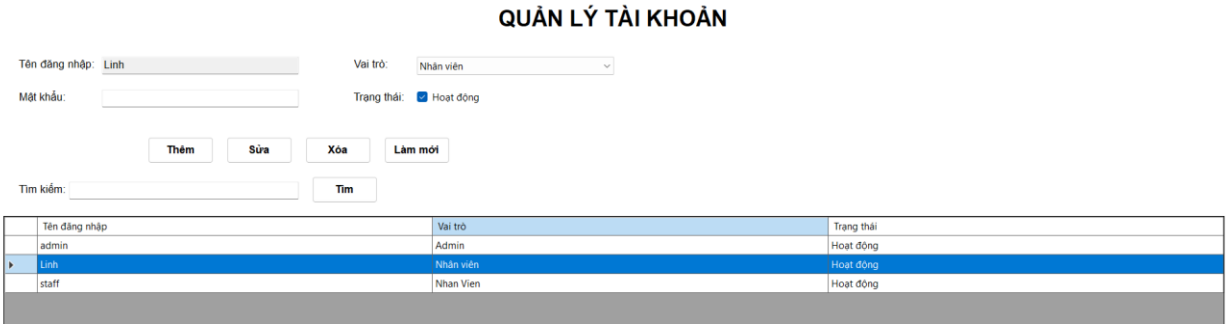
- Giao diện sửa tài khoản: vai trò admin thành nhân viên , sửa yêu cầu nhập pass mới



- Giao diện sửa tài khoản thành công

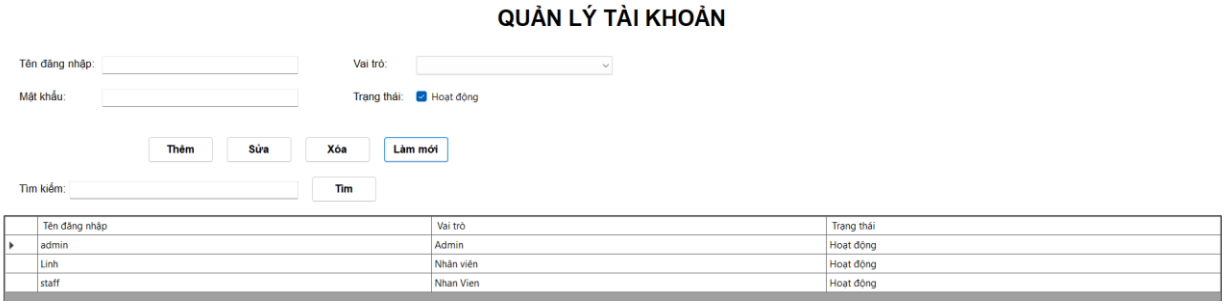


- Giao diện đã sửa từ admin thành nhân viên



4. Tìm tài khoản

- Giao diện màn hình trước tìm tài khoản



- Code C# GUI tìm tài khoản: Khi nhập từ khóa vào ô txtTimKiem và bấm Tìm, GUI lấy từ khóa, gọi taiKhoanBUS.TimKiem(). Kết quả trả về được truyền vào HienThiDuLieu() để hiển thị lên DataGridView.

```
1 reference
private void btnTimKiem_Click(
    object sender,
    EventArgs e)
{
    try
    {
        HienThiDuLieu(
            taiKhoanBUS.TimKiem(
                txtTimKiem.Text.Trim()));
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Không thể tìm kiếm!\n" +
            ex.Message);
    }
```

- Code C# gọi BUS tìm tài khoản: BUS nhận keyword từ GUI. Nếu từ khóa rỗng hoặc chỉ chứa khoảng trắng thì BUS gọi dal.GetAll() để lấy toàn bộ danh sách tài khoản. Nếu có từ khóa thì BUS bỏ khoảng trắng bằng.Trim() và gọi dal.Search(keyword) để tìm kiếm.

```
// Tìm kiếm tài khoản
1 reference
public DataTable TimKiem(string keyword)
{
    if (string.IsNullOrWhiteSpace(keyword))
        return dal.GetAll();

    return dal.Search(keyword.Trim());
}
```

- Code C# gọi DAL Stored Procedure tìm tài khoản: DAL mở kết nối SQL Server và tạo SqlCommand để gọi Stored Procedure sp_TimTaiKhoan. Từ khóa được truyền vào tham số @TuKhoa. Sau đó SqlDataAdapter thực hiện Stored Procedure và đổ kết quả vào DataTable, rồi trả DataTable về BUS.

```
// Tìm tài khoản
1 reference
public DataTable Search(string keyword)
{
    DataTable dt = new DataTable();

    using (SqlConnection conn = Database.GetConnection())
    {
        conn.Open();

        using (SqlCommand cmd =
               new SqlCommand(
                   "dbo.sp_TimTaiKhoan",
                   conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add(
                "@TuKhoa",
                SqlDbType.NVarChar,
                100
            ).Value = keyword;

            using (SqlDataAdapter da =
                   new SqlDataAdapter(cmd))
            {
                da.Fill(dt);
            }
```

- Code SQL

```sql
-- Tìm tài khoản
CREATE OR ALTER PROCEDURE dbo.sp_TimTaiKhoan
    @TuKhoa NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        TenDangNhap,
        MatKhau,
        VaiTro,
        TrangThai
    FROM dbo.vw_TaiKhoan
    WHERE
        TenDangNhap LIKE N'%' + @TuKhoa + N'%'
        OR VaiTro LIKE N'%' + @TuKhoa + N'%'
    ORDER BY TenDangNhap;
END;
GO
```

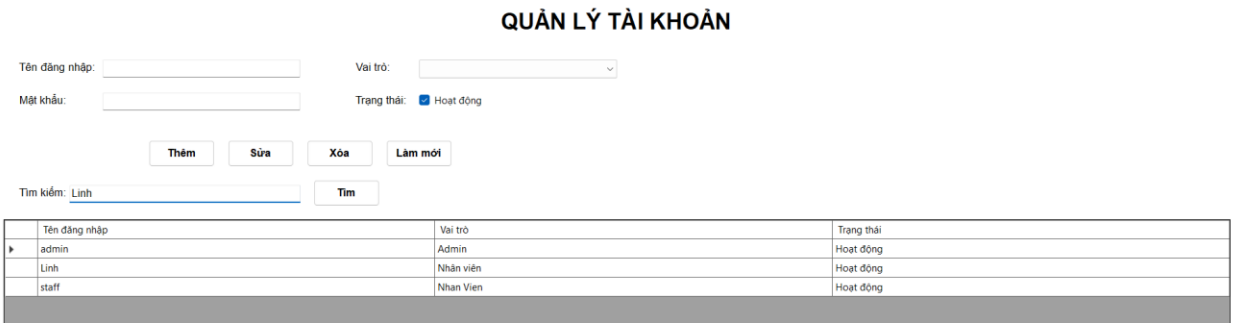
Nhiệm vụ:

+ Dùng để tìm kiếm tài khoản.

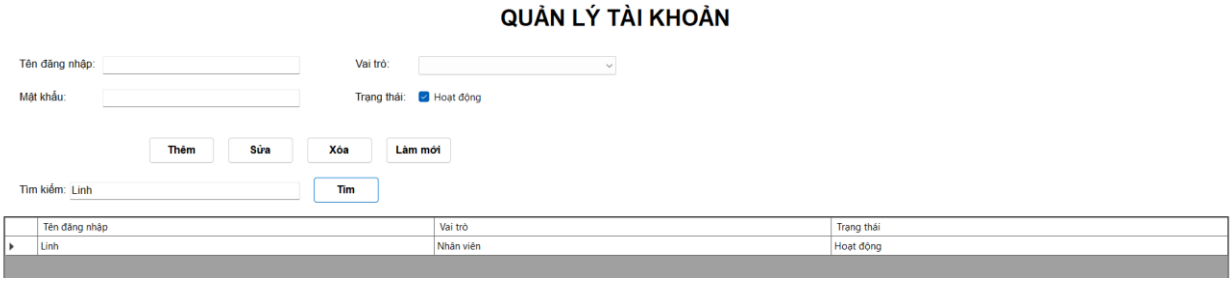
+ Người dùng nhập từ khóa, SP tìm trong: TenDangNhap, VaiTro

+ Sử dụng LIKE để tìm những dữ liệu có chứa từ khóa.

- Giao diện tìm tài khoản :click tìm kiếm tên tài khoản, Linh



- Giao diện đã tìm thấy tài khoản



2.3. FUNCTION

2.3.1. Khái niệm

Function (hàm) trong SQL là một đối tượng quan trọng thuộc hệ quản trị cơ sở dữ liệu, được sử dụng để thực hiện các phép tính toán hoặc xử lý dữ liệu và trả về một giá trị duy nhất hoặc một tập kết quả (dạng bảng). Khác với Stored Procedure, Function luôn bắt buộc phải có giá trị trả về thông qua câu lệnh **RETURN.**

2.3.2. Đặc điểm

Một trong những đặc điểm nổi bật của Function là khả năng tái sử dụng cao. Sau khi được định nghĩa, Function có thể được gọi lại nhiều lần trong các câu lệnh SQL khác nhau như **SELECT, WHERE, JOIN, GROUP BY**, giúp giảm thiểu việc lặp lại code và làm cho chương trình trở nên gọn gàng, dễ bảo trì hơn.

Function có thể nhận tham số đầu vào (input parameters) để xử lý linh hoạt theo từng yêu cầu cụ thể. Điều này cho phép cùng một Function có thể áp dụng cho nhiều trường hợp khác nhau mà không cần phải viết lại logic xử lý.

Ngoài ra, Function trong SQL thường không được sử dụng để thay đổi dữ liệu trong bảng (không thực hiện các thao tác như INSERT, UPDATE, DELETE). Điều này giúp đảm bảo tính an toàn và ổn định của dữ liệu khi sử dụng Function trong các truy vấn.

Function được chia thành hai loại chính:

- Scalar Function: trả về một giá trị đơn (ví dụ: số nguyên, chuỗi, ngày tháng,...).
- Table-valued Function: trả về một bảng dữ liệu, có thể sử dụng như một bảng trong câu lệnh **SELECT.**

Một đặc điểm quan trọng khác là Function có thể được lồng vào trong các biểu thức SQL, ví dụ như dùng trong điều kiện lọc hoặc tính toán cột, điều mà Stored Procedure không làm được. Tuy nhiên, do giới hạn về khả năng thao tác dữ liệu, Function thường chỉ phù hợp cho các tác vụ xử lý logic đơn giản hoặc tính toán, thay vì các nghiệp vụ phức tạp.

2.3.3. Cách sử dụng

- **Tạo Function:**

```
CREATE FUNCTION fn_Tong2So
(
   @a INT,
   @b INT
)
RETURNS INT
AS
BEGIN
   RETURN (@a + @b)
END
```

- **Gọi Function**

```
SELECT dbo.fn_Tong2So(5, 3) AS KetQua
```

Kết quả: 8

Ví dụ thực tế:

Tính tuổi từ năm sinh:

```
CREATE FUNCTION fn_TinhTuoi
(
   @NamSinh INT
)
RETURNS INT
AS
```

```
BEGIN
    RETURN (YEAR(GETDATE()) - @NamSinh)
END
```

Gọi:

```
SELECT dbo.fn_TinhTuoi(2006) AS Tuoi
```

2.3.4. Ưu điểm và nhược điểm

Ưu điểm:

- **Tái sử dụng cao:** Cho phép đóng gói một khối logic tính toán và gọi lại ở nhiều nơi (câu lệnh SQL, Stored Procedure, hoặc từ các tầng ứng dụng khác nhau như DAL) mà không cần viết lại mã nguồn.
- **Trừu tượng hóa dữ liệu:** Giúp đơn giản hóa các câu lệnh truy vấn phức tạp, giữ cho mã nguồn ở tầng DAL gọn gàng và dễ bảo trì hơn.
- **Hiệu suất tốt với dữ liệu đơn giá trị:** Đối với các scalar function thực hiện tính toán đơn giản, SQL Server tối ưu hóa thực thi khá tốt.

Nhược điểm:

- **Hạn chế về thao tác dữ liệu:** Function không được phép thực hiện các lệnh làm thay đổi trạng thái cơ sở dữ liệu (như INSERT, UPDATE, DELETE trực tiếp trên các bảng chính bên trong thân hàm).
- **Ảnh hưởng hiệu suất khi dùng dạng Inline/Multi-statement Table-valued:** Nếu viết hàm trả về bảng phức tạp không đúng cách, SQL Server có thể gặp khó khăn trong việc tối ưu hóa câu lệnh (execution plan), dẫn đến chậm khi truy vấn dữ liệu lớn.
- **Khó gỡ lỗi (Debugging):** Việc debug lỗi bên trong Function phức tạp và khó khăn hơn so với Stored Procedure.

2.3.5. Thực thi chức năng trên giao diện và code

Function là đối tượng cơ sở dữ liệu chuyên dùng để tính toán và trả về kết quả (giá trị đơn hoặc dạng bảng) mà không làm thay đổi dữ liệu gốc trong bảng.

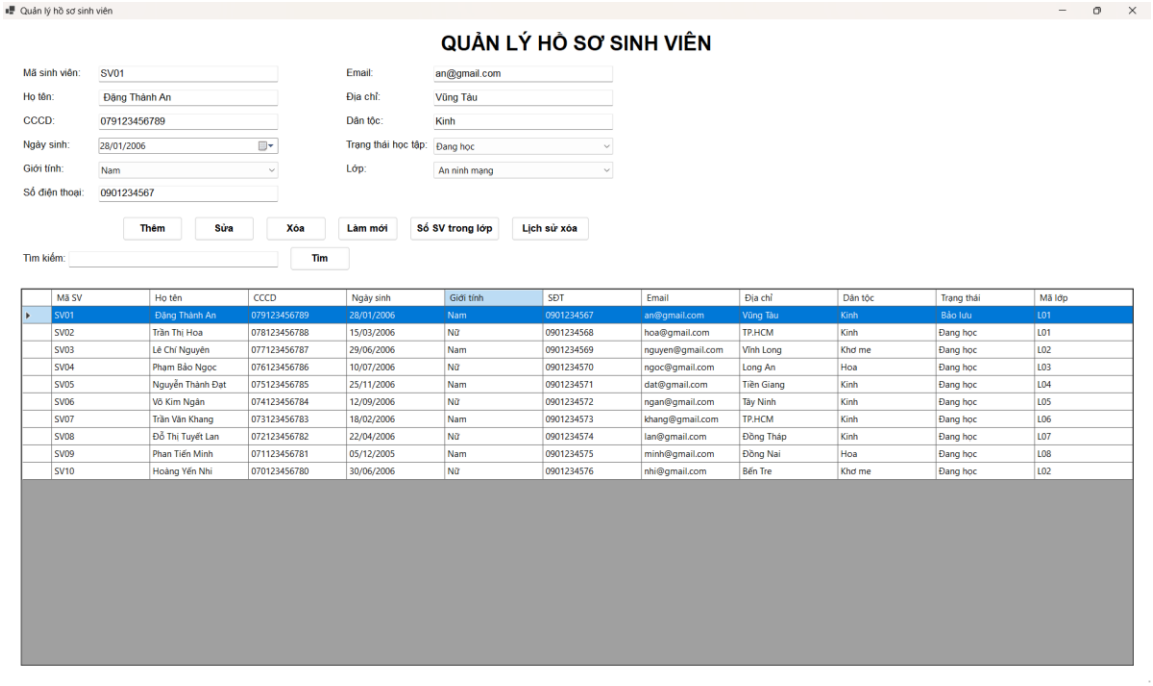
1. Tạo Function đếm số sinh viên trong 1 lớp

a) Giao diện:

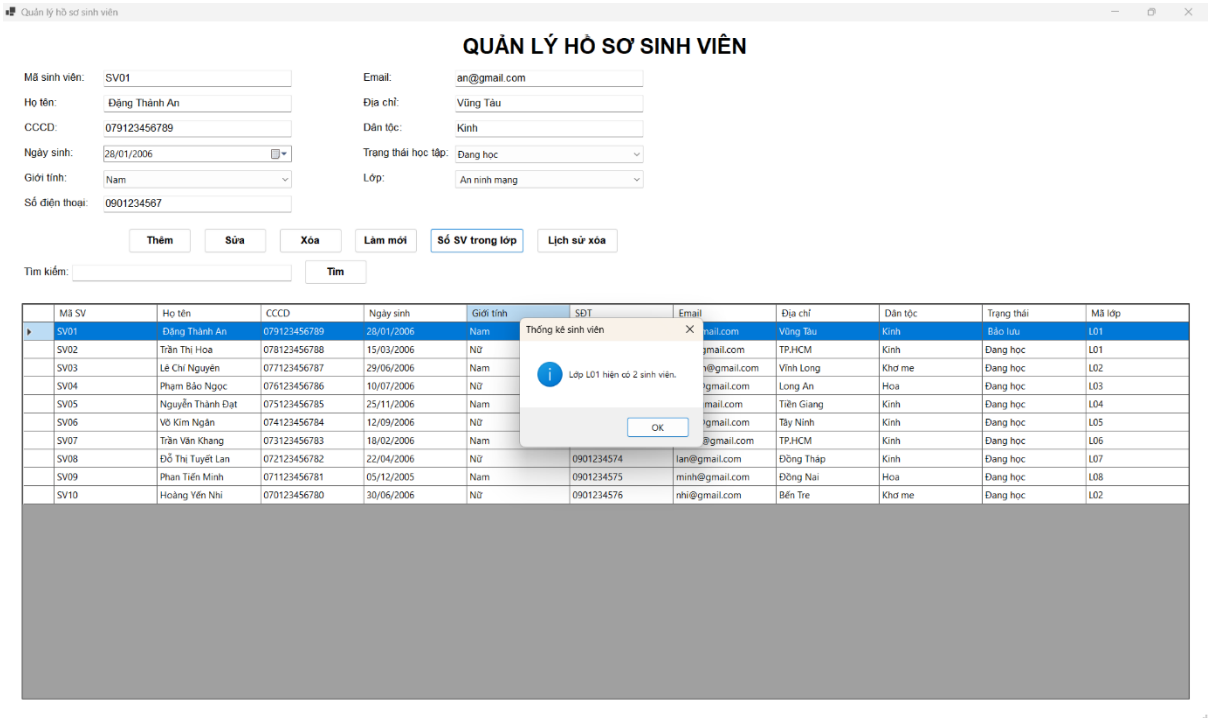
Để xem số lượng sinh viên của một lớp (ví dụ: lớp L01), thực hiện các thao tác:

Bước 1: Chọn một dòng sinh viên bất kỳ thuộc lớp cần xem (ví dụ: sinh viên có mã lớp L01) trên bảng dữ liệu.

Bước 2: Nhấn vào nút Số SV trong lớp.



Bước 3: Số sinh viên có trong lớp sẽ được thể hiện ở bảng.



Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam			Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ			TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam			Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ			Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam			Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ			Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam			TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L02

b) Code xử lý:

1. DTO

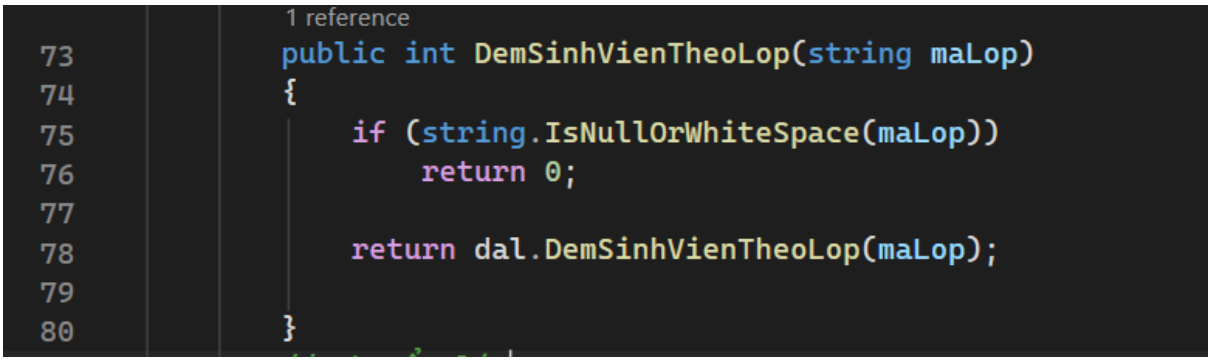
```
1   namespace QLHSSV.DTO
2   {
        9 references
3       public class SinhVienDTO
4       {
            7 references
5           public string MaSV { get; set; } = "";
            4 references
6           public string HoTen { get; set; } = "";
            2 references
7           public string CCCD { get; set; } = "";
            2 references
8           public DateTime NgaySinh { get; set; }
            2 references
9           public string GioiTinh { get; set; } = "";
            2 references
10          public string SDT { get; set; } = "";
            2 references
11          public string Email { get; set; } = "";
            2 references
12          public string DiaChi { get; set; } = "";
            2 references
13          public string DanToc { get; set; } = "";
            2 references
14          public string TrangThaiHocTap { get; set; } = "";
            4 references
15          public string MaLop { get; set; } = "";
16      }
17  }
```

Mục đích: Khai báo lớp SinhVienDTO để đóng gói và vận chuyển dữ liệu sinh viên giữa các tầng trong kiến trúc ứng dụng.

Nội dung thực hiện:

- Xây dựng lớp SinhVienDTO chứa các thuộc tính tương ứng với các cột trong cơ sở dữ liệu như MaSV, HoTen, MaLop, Email,... (được minh họa ở hình trên).
- Đối với chức năng đếm số sinh viên trong lớp, thuộc tính MaLop(string) đóng vai trò là tham số đầu vào quan trọng, giúp lấy ra mã lớp của sinh viên đang được chọn trên giao diện để chuyển tiếp lên tầng BUS và DAL xử lý.

2. BUS



Nhiệm vụ: Tiếp nhận mã lớp từ tầng giao diện (GUI), thực hiện kiểm tra nghiệp vụ cơ bản (kiểm tra maLopcó bị trống hay không).

Xử lý: Nếu dữ liệu hợp lệ, tầng BUS sẽ gọi tiếp phương thức DemSinhVienTheoLop(maLop) ở tầng DAL để thực thi truy vấn xuống cơ sở dữ liệu và nhận lại giá trị số nguyên (int) biểu thị tổng số sinh viên trong lớp đó.

3. DAL

```
222         // Đếm sinh viên trong lớp bằng Function
            1 reference
223         public int DemSinhVienTheoLop(string maLop)
224         {
225             using (SqlConnection conn = Database.GetConnection())
226             {
227                 conn.Open();
228
229                 using (SqlCommand cmd =
230                        new SqlCommand(
231                            "SELECT dbo.fn_DemSVLop(@MaLop)",
232                            conn))
233                 {
234                     cmd.Parameters.Add(
235                         "@MaLop",
236                         SqlDbType.VarChar,
237                         10
238                     ).Value = maLop;
239
240                     return Convert.ToInt32(
241                         cmd.ExecuteScalar()
242                     );
243                 }
244             }
245         }
246
```

Nhiệm vụ: Thiết lập kết nối cơ sở dữ liệu và trực tiếp gọi Function dbo.fn_DemSVLop trong SQL Server thông qua câu lệnh SQL dạng SELECT dbo.fn_DemSVLop(@MaLop).

Xử lý:

- Truyền tham số @MaLop (kiểu VarChar) xuống cơ sở dữ liệu.
- Sử dụng phương thức cmd.ExecuteScalar() để lấy về một giá trị duy nhất (kết quả đếm số sinh viên).
- Chuyển đổi kết quả sang kiểu số nguyên (int) thông qua Convert.ToInt32() và trả ngược về cho tầng BUS.

4. SQL

```
948    -- Tạo function đếm số sinh viên trong một lớp
949    create function fn_DemSVLop
950    (
951        @MaLop varchar(10)
952    )
953    returns int
954    as
955    begin
956        declare @SoSinhVien int;
957
958        select @SoSinhVien = count(*)
959        from SinhVien
960        where MaLop = @MaLop;
961
962        return @SoSinhVien;
963    end;
964    go
965
966    select dbo.fn_DemSVLop('L02') as SoSinhVien;
967    go
968
```

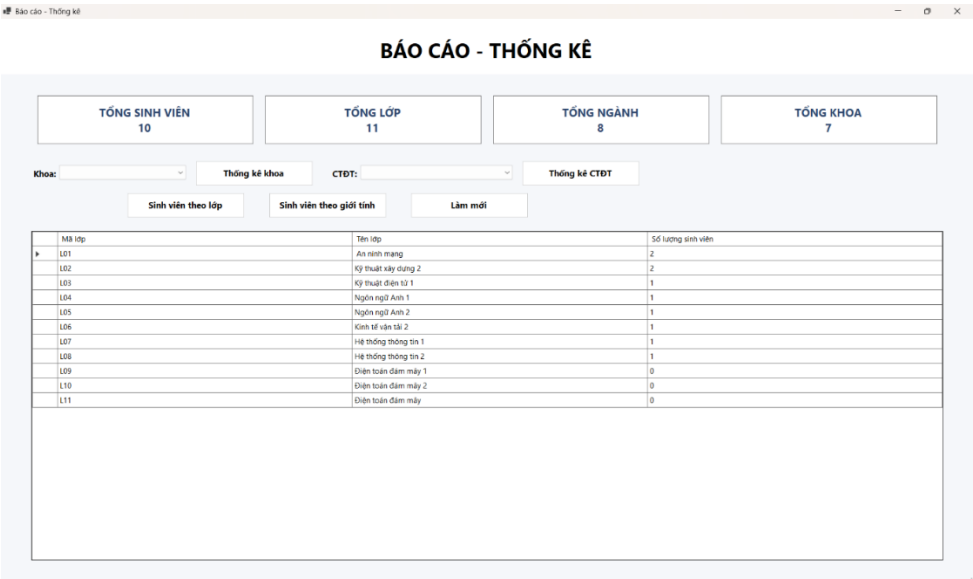
Nhiệm vụ:Xây dựng Scalar Function fn_DemSVLop trực tiếp trên hệ quản trị cơ sở dữ liệu SQL Server để thực hiện logic đếm số lượng sinh viên theo mã lớp được truyền vào.

Nội dung thực thi:

- Nhận tham số đầu vào @MaLop (kiểu dữ liệu varchar(10)).
- Khai báo biến @SoSinhVien kiểu int để lưu kết quả đếm.
- Thực thi câu lệnh SELECT kết hợp hàm COUNT(*) trên bảng SinhVien với điều kiện lọc theo MaLop.
- Trả về kết quả là một giá trị số nguyên duy nhất thông qua lệnh RETURN @SoSinhVien

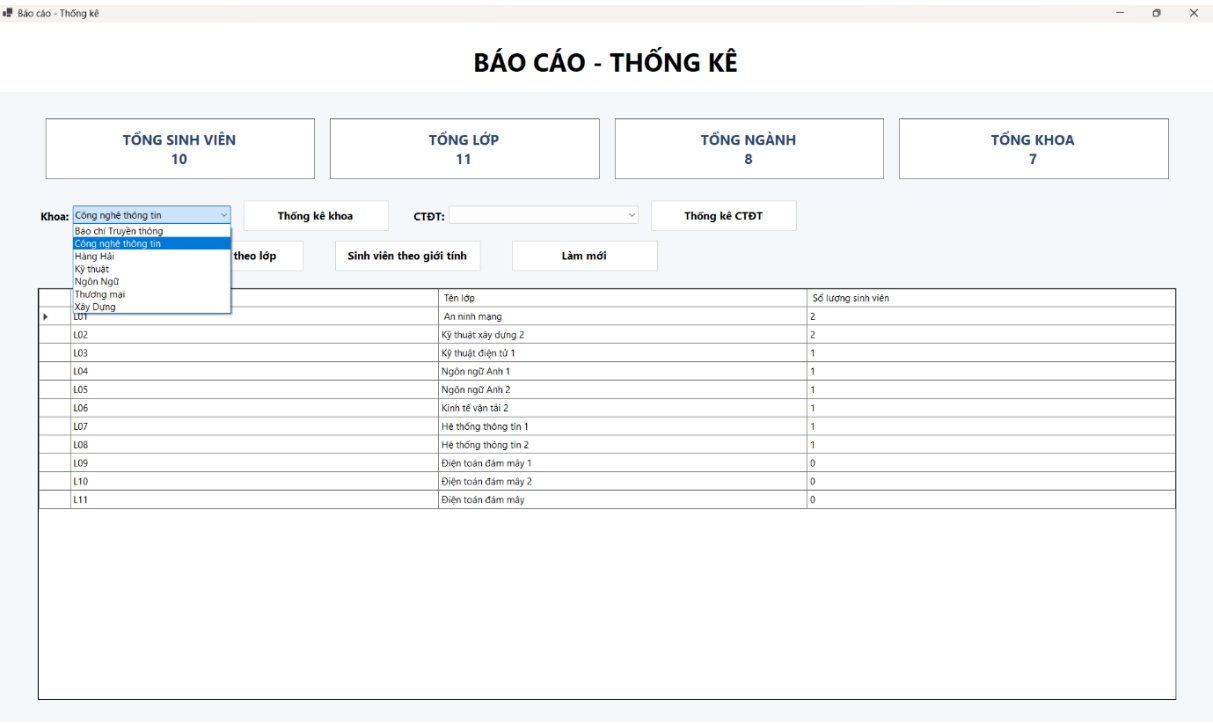
2. Tạo Function đếm số sinh viên trong khoa

a) Giao diện

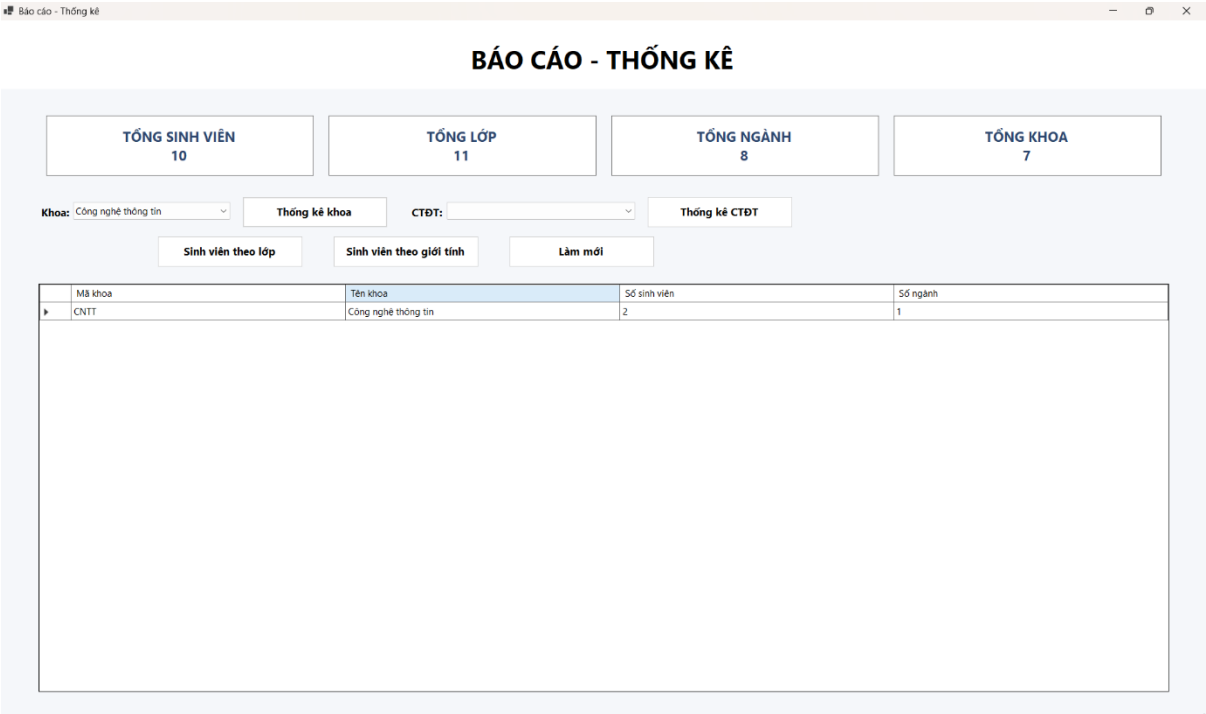


Bước 1: Chọn khoa cần xem số lượng sinh viên

Bước 2: Chọn “Thống kê khoa”



Bước 3: Hiển thị bảng kết quả thống kê tương ứng với khoa đã chọn, bao gồm các thông tin chi tiết như Mã khoa, Tên khoa, Số sinh viên và Số ngành.



b) Code xử lý

1. BUS

```
10        // Đếm sinh viên trong khoa
11        public int DemSinhVienKhoa(string maKhoa)
12        {
13            if (string.IsNullOrWhiteSpace(maKhoa))
14                return 0;
15
16            return dal.DemSinhVienKhoa(
17                maKhoa.Trim());
18        }
```

Nhiệm vụ: Tiếp nhận tham số maKhoa từ tầng giao diện, thực hiện kiểm tra tính hợp lệ dữ liệu (kiểm tra maKhoa có bị trống hay không bằng string.IsNullOrWhiteSpace).

Nội dung thực thi: Nếu dữ liệu hợp lệ, tầng BUS sẽ gọi tiếp phương thức tương ứng ở tầng DAL (cắt bỏ khoảng trắng thừa bằng Trim()) để lấy kết quả xử lý trả về cho giao diện.

2. DAL

```
8       {
9           // Đếm sinh viên trong khoa bằng Function
10          public int DemSinhVienKhoa(string maKhoa)
11          {
12              using (SqlConnection conn = Database.GetConnection())
13              {
14                  conn.Open();
15
16                  string sql =
17                      "SELECT dbo.fn_DemSSVKhoa(@MaKhoa)";
18
19                  using (SqlCommand cmd =
20                         new SqlCommand(sql, conn))
21                  {
22                      cmd.Parameters.Add(
23                          "@MaKhoa",
24                          SqlDbType.VarChar,
25                          10
26                      ).Value = maKhoa;
27
28                      return Convert.ToInt32(
29                          cmd.ExecuteScalar());
30                  }
31              }
32          }
33
```

Nhiệm vụ: Thiết lập kết nối cơ sở dữ liệu và gọi trực tiếp Function trong SQL Server để thực hiện truy vấn.

Nội dung thực thi:

- Khởi tạo kết nối thông qua Database.GetConnection() và mở kết nối bằng conn.Open().
- Chuẩn bị câu lệnh SQL gọi Function dạng chuỗi: "SELECT dbo.fn_DemSSSVKhoa(@MaKhoa)".
- Thêm tham số truyền vào @MaLop (ở đây là @MaKhoa) với kiểu dữ liệu VarChar(10).

- Sử dụng phương thức cmd.ExecuteScalar() để thực thi và lấy về một giá trị duy nhất, sau đó chuyển đổi kết quả sang kiểu số nguyên (int) bằng Convert.ToInt32() để trả về cho tầng BUS.

3. SQL

```
969   -- Tạo function đếm số sinh viên trong khoa
970   create function fn_DemSSVKhoa
971   (
972       @MaKhoa varchar(10)
973   )
974   returns int
975   as
976   begin
977       declare @SoSinhVien int;
978
979       select @SoSinhVien = count(*)
980       from SinhVien
981       join Lop on SinhVien.MaLop = Lop.MaLop
982       join Nganh on Lop.MaNganh = Nganh.MaNganh
983       where Nganh.MaKhoa = @MaKhoa;
984
985       return @SoSinhVien;
986   end;
987   go
988
```

Nhiệm vụ: Xây dựng Scalar Function fn_DemSSVKhoa trực tiếp trên SQL Server để thực hiện logic đếm tổng số sinh viên thuộc một khoa dựa vào mã khoa được truyền vào.

Nội dung thực thi:

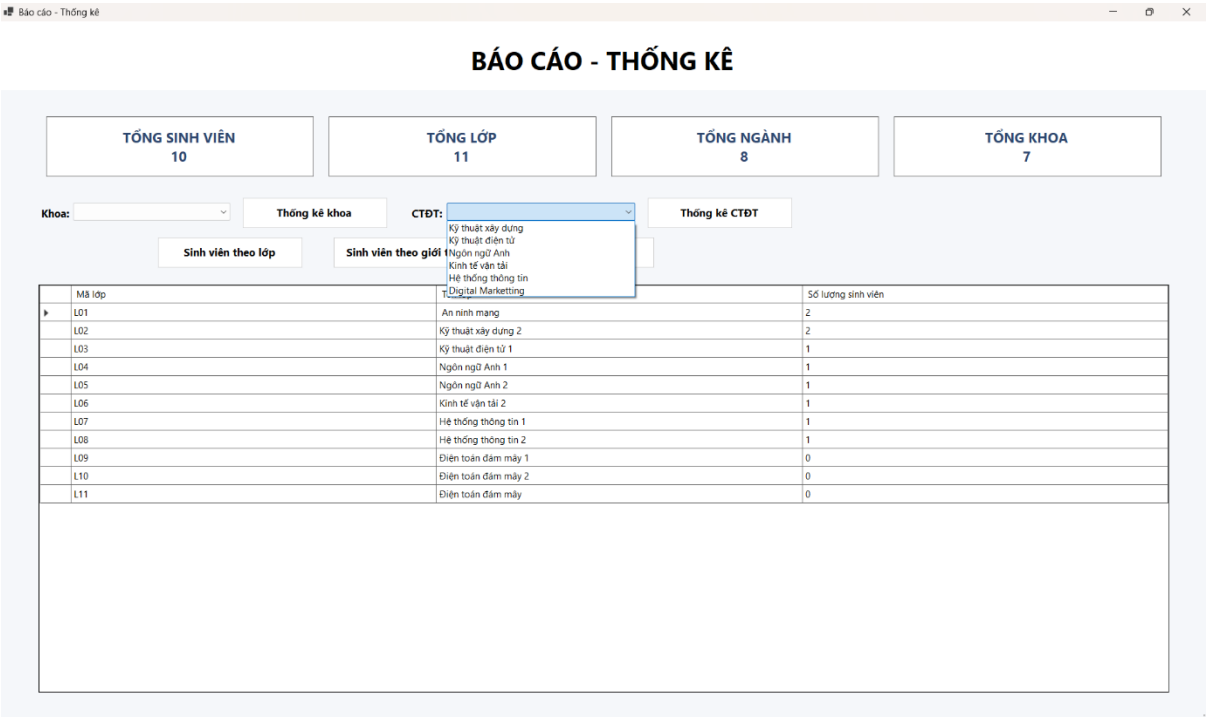
- Nhận tham số đầu vào @MaKhoa (kiểu dữ liệu varchar(10)).
- Khai báo biến @SoSinhVien kiểu int để lưu trữ kết quả.
- Thực hiện câu lệnh SELECT kết hợp hàm COUNT(*) từ bảng SinhVien, sau đó dùng lệnh JOIN qua bảng Lop và bảng Nganh với điều kiện lọc Nganh.MaKhoa = @MaKhoa.
- Trả về kết quả là một giá trị số nguyên duy nhất đại diện cho tổng số sinh viên của khoa thông qua lệnh RETURN @SoSinhVien.

3. Tạo Function đếm số sinh viên trong Chương trình đào tạo

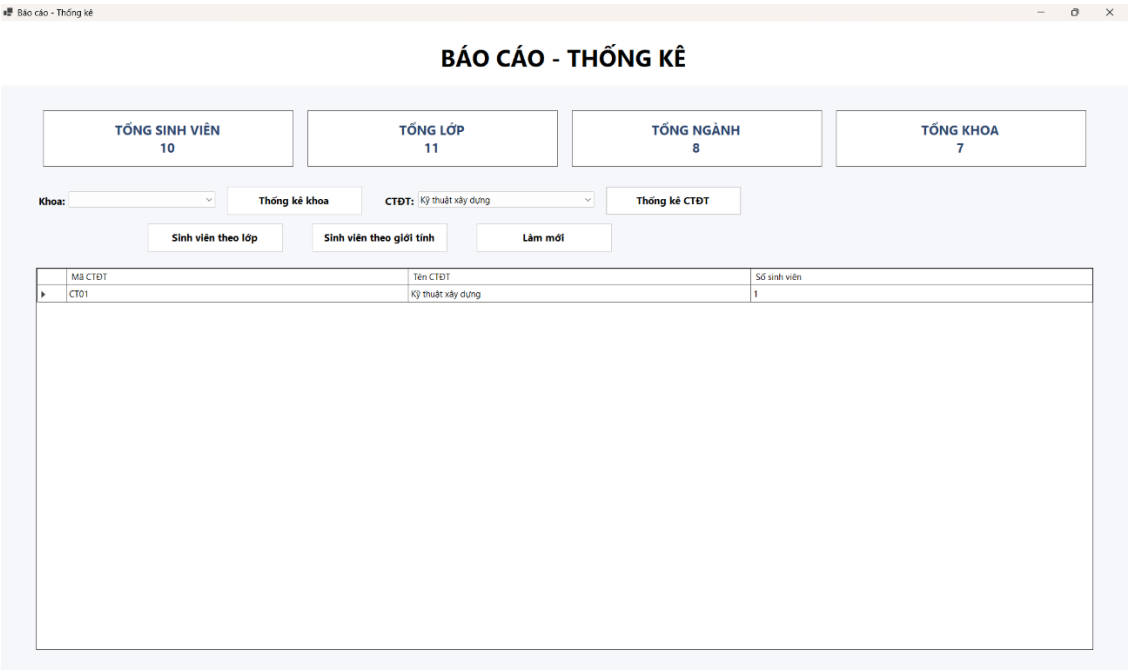
a) Giao diện

Bước 1: Chọn chương trình đào tạo cần xem số lượng sinh viên từ hộp lựa chọn CTĐT (ví dụ: Kỹ thuật xây dựng).

Bước 2: Nhấn vào nút Thống kê CTĐT để hệ thống thực thi yêu cầu xử lý.



Bước 3: Giao diện hiển thị bảng kết quả thống kê tương ứng với chương trình đào tạo đã chọn, bao gồm các thông tin chi tiết như Mã CTĐT, Tên CTĐT và Số sinh viên.



b) Code xử lý

1. BUS

```
        // Đếm sinh viên theo CTĐT
        1 reference
        public int DemSinhVienCTDT(string maCTDT)
        {
            if (string.IsNullOrWhiteSpace(maCTDT))
                return 0;

            return dal.DemSinhVienCTDT(
                maCTDT.Trim());
        }
    }
}
```

Nhiệm vụ: Tiếp nhận tham số maCTDT từ tầng giao diện, thực hiện kiểm tra tính hợp lệ của dữ liệu (kiểm tra maCTDT có bị trống hay không bằng string.IsNullOrWhiteSpace).

Nội dung thực thi: Nếu dữ liệu hợp lệ, tầng BUS sẽ gọi tiếp phương thức DemSinhVienCTDT ở tầng DAL (sau khi đã loại bỏ khoảng trắng thừa bằng Trim()) để thực hiện xử lý nghiệp vụ và trả về kết quả số lượng sinh viên cho giao diện.

2. DAL

```
        // Đếm sinh viên theo CTĐT bằng Function
        1 reference
        public int DemSinhVienCTDT(string maCTDT)
        {
            using (SqlConnection conn = Database.GetConnection())
            {
                conn.Open();

                string sql =
                    "SELECT dbo.fn_DemSVCTDT(@MaCTDT)";

                using (SqlCommand cmd =
                       new SqlCommand(sql, conn))
                {
                    cmd.Parameters.Add(
                        "@MaCTDT",
                        SqlDbType.VarChar,
                        10
                    ).Value = maCTDT;

                    return Convert.ToInt32(
                        cmd.ExecuteScalar());
                }
            }
        }
    }
}
```

Nhiệm vụ: Thiết lập kết nối cơ sở dữ liệu và gọi trực tiếp Function trong SQL Server để thực hiện truy vấn số lượng sinh viên theo chương trình đào tạo.

Nội dung thực thi:

- Khởi tạo kết nối thông qua Database.GetConnection() và mở kết nối bằng conn.Open().
- Chuẩn bị câu lệnh SQL gọi Function dạng chuỗi: "SELECT dbo.fn_DemSVCTDT(@MaCTDT)".
- Thêm tham số truyền vào @MaCTDT với kiểu dữ liệu VarChar(10) và gán giá trị maCTDT.

- Sử dụng phương thức cmd.ExecuteScalar() để thực thi và lấy về một giá trị duy nhất, sau đó chuyển đổi kết quả sang kiểu số nguyên (int) bằng Convert.ToInt32() để trả về cho tầng BUS.

3. SQL

```
989   -- Tạo function đếm số sinh viên thuộc một chương trình đào tạo
990   create function fn_DemSVCTDT
991   (
992       @MaCTDT varchar(10)
993   )
994   returns int
995   as
996   begin
997       declare @SoSinhVien int;
998
999       select @SoSinhVien = count(*)
1000      from ChuongTrinhDaoTao
1001      where MaCTDT = @MaCTDT;
1002
1003      return @SoSinhVien;
1004  end;
1005  go
```

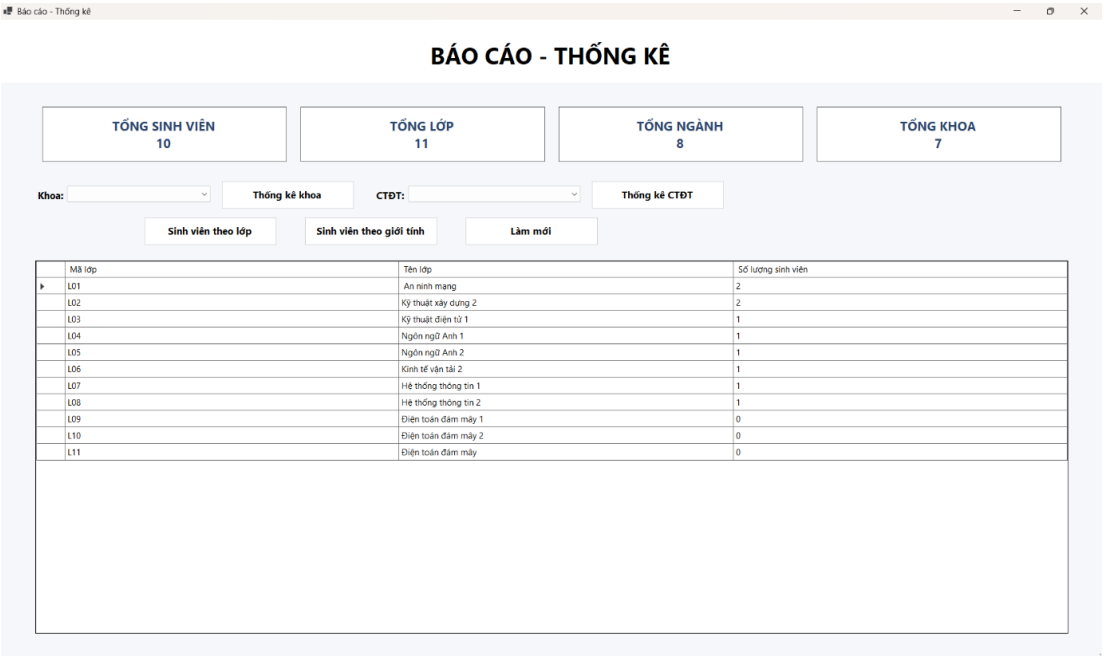
Nhiệm vụ: Xây dựng Scalar Function fn_DemSVCTDT trực tiếp trên cơ sở dữ liệu SQL Server để thực hiện logic đếm tổng số sinh viên thuộc một chương trình đào tạo dựa vào mã chương trình đào tạo truyền vào.

Nội dung thực thi:

- Nhận tham số đầu vào @MaCTDT (kiểu dữ liệu varchar(10)).
- Khai báo biến @SoSinhVien kiểu int để lưu trữ kết quả.
- Thực hiện câu lệnh SELECT kết hợp hàm COUNT(*) từ bảng ChuongTrinhDaoTao với điều kiện lọc MaCTDT = @MaCTDT.
- Trả về kết quả là một giá trị số nguyên duy nhất đại diện cho tổng số sinh viên thông qua lệnh RETURN @SoSinhVien.

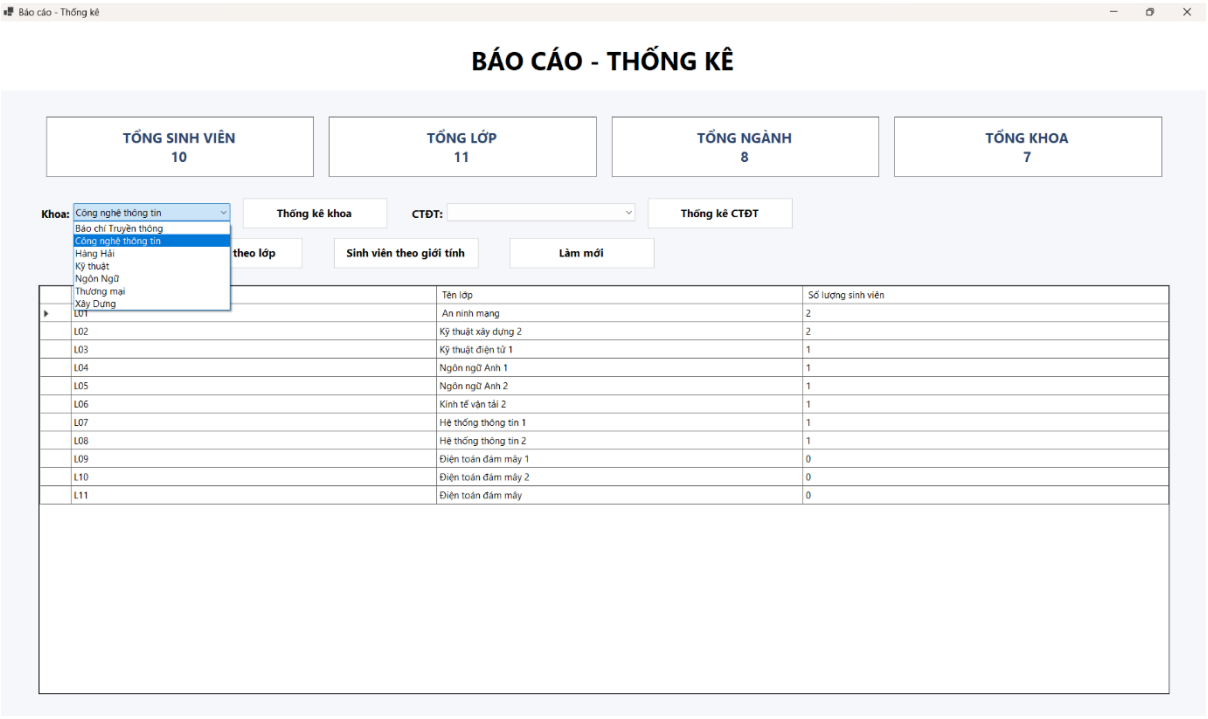
4. Tạo Function đếm số ngành trong khoa

a) Giao diện

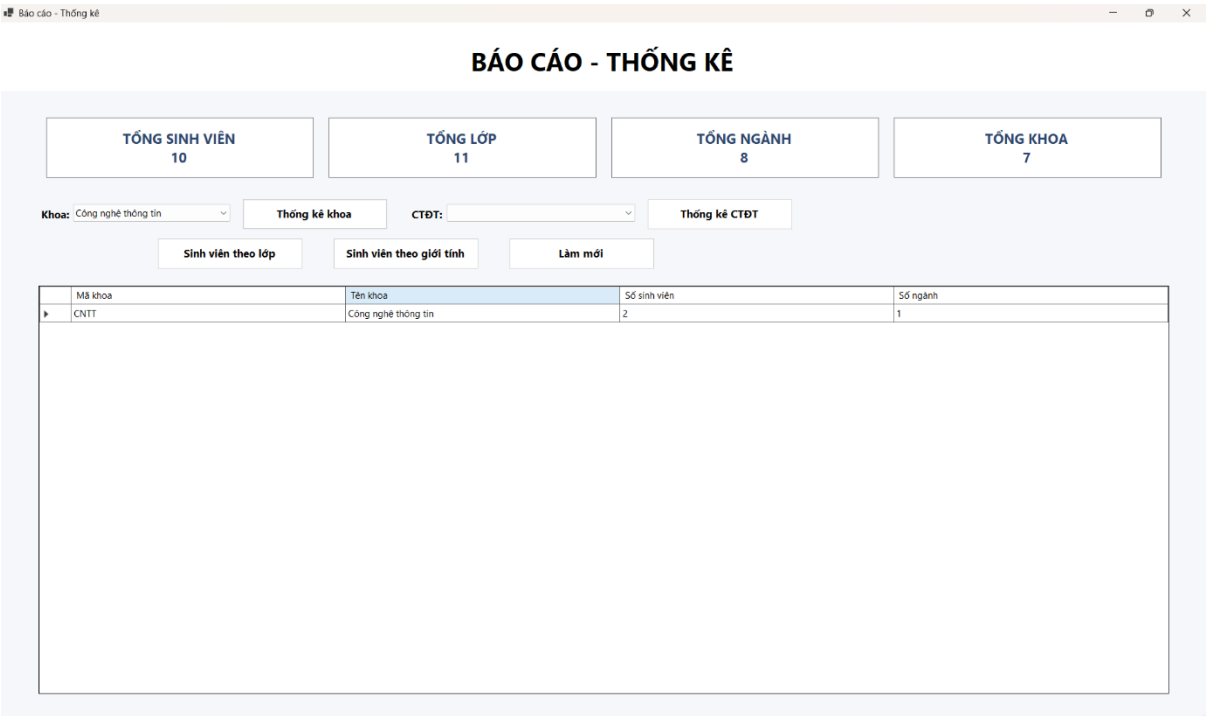


Bước 1: Chọn khoa cần xem số lượng sinh viên

Bước 2: Chọn “Thống kê khoa”



Bước 3: Hiển thị bảng kết quả thống kê tương ứng với khoa đã chọn, bao gồm các thông tin “Số ngành trong khoa”.



b) Code xử lý

1. BUS

```
20      // Đếm số ngành trong khoa
21      public int DemSoNganhKhoa(string maKhoa)
22      {
23          if (string.IsNullOrWhiteSpace(maKhoa))
24              return 0;
25
26          return dal.DemSoNganhKhoa(
27              maKhoa.Trim());
28      }
```

Nhiệm vụ: Tiếp nhận tham số maKhoa từ tầng giao diện, thực hiện kiểm tra tính hợp lệ dữ liệu (kiểm tra maKhoa có bị trống hay không bằng string.IsNullOrWhiteSpace).

Nội dung thực thi: Nếu dữ liệu hợp lệ, tầng BUS sẽ gọi tiếp phương thức tương ứng ở tầng DAL (cắt bỏ khoảng trắng thừa bằng Trim()) để lấy kết quả xử lý trả về cho giao diện.

3. DAL

```
33
34      // Đếm số ngành trong khoa bằng Function
35      public int DemSoNganhKhoa(string maKhoa)
36      {
37          using (SqlConnection conn = Database.GetConnection())
38          {
39              conn.Open();
40
41              string sql =
42                  "SELECT dbo.fn_DemSoNganhKhoa(@MaKhoa)";
43
44              using (SqlCommand cmd =
45                     new SqlCommand(sql, conn))
46              {
47                  cmd.Parameters.Add(
48                      "@MaKhoa",
49                      SqlDbType.VarChar,
50                      10
51                  ).Value = maKhoa;
52
53                  return Convert.ToInt32(
54                      cmd.ExecuteScalar());
55              }
56          }
57      }
```

Nhiệm vụ: Thiết lập kết nối cơ sở dữ liệu và gọi trực tiếp Function trong SQL Server để thực hiện truy vấn.

Nội dung thực thi:

- Khởi tạo kết nối thông qua Database.GetConnection() và mở kết nối bằng conn.Open().
- Chuẩn bị câu lệnh SQL gọi Function dạng chuỗi: "SELECT dbo.fn_DemSoNganhKhoa(@MaKhoa)".

- Thêm tham số truyền vào @MaLop (ở đây là @MaKhoa) với kiểu dữ liệu VarChar(10).
- Sử dụng phương thức cmd.ExecuteScalar() để thực thi và lấy về một giá trị duy nhất, sau đó chuyển đổi kết quả sang kiểu số nguyên (int) bằng Convert.ToInt32() để trả về cho tầng BUS.

4. SQL

```
-- Tạo function đếm số ngành trong khoa
create function fn_DemSoNganhKhoa
(
    @MaKhoa varchar(10)
)
returns int
as
begin
    declare @SoNganh int;

    select @SoNganh = count(*)
    from Nganh
    where MaKhoa = @MaKhoa;

    return @SoNganh;
end;
go
```

Nhiệm vụ: Xây dựng Scalar Function fn_DemSoNganhKhoa trực tiếp trên SQL Server để thực hiện logic đếm tổng số ngành thuộc một khoa dựa vào mã khoa được truyền vào.

Nội dung thực thi:

- Nhận tham số đầu vào @MaKhoa (kiểu dữ liệu varchar(10)).
- Khai báo biến @SoNganh kiểu int để lưu trữ kết quả.
- Thực hiện câu lệnh SELECT kết hợp hàm COUNT(*) từ bảng Nganh với điều kiện lọc MaKhoa = @MaKhoa.

- Trả về kết quả là một giá trị số nguyên duy nhất đại diện cho tổng số ngành của khoa thông qua lệnh RETURN @SoNganh.

2.4. TRIGGER

2.4.1. Khái niệm

Trigger là một đoạn thủ tục SQL được thực thi tự động khi 1 sự kiện cụ thể xảy ra trên 1 bảng (table) hoặc dạng xem (view).

2.4.2. Đặc điểm

Trigger thường được dùng để:

- Kiểm tra tính hợp lệ của dữ liệu.
- Đảm bảo các quy tắc nghiệp vụ (Business Logic)
- Tự động thực hiện một hành động khi dữ liệu thay đổi
- Ngăn chặn những thao tác không hợp lệ
- Đồng bộ hoặc cập nhật dữ liệu liên quan

Các sự kiện có thể kích hoạt Trigger bao gồm: Insert, Update, Delete, DDL (Data Definition Languague), DML (Data Manipulation Language)

=> Trigger kích hoạt khi DML (INSERT, UPDATE,DELETE) thay đổi thông tin trong 1 bảng hoặc 1 view HOẶC DDL (CREATE,ALTER,DROP) Thay đổi tới cấu trúc CSDL -> kích hoạt DDL Trigger khi đã tạo Trigger tương ứng.

Trong SQL Server còn có LOGON Trigger, được kích hoạt khi người dùng đăng nhập vào SQL Server.

Có thể dùng để kiểm soát hoặc giới hạn việc đăng nhập.

Trigger được chia thành 2 loại chính:

+ Trigger Before: Thực thi trước khi sự kiện xảy ra

+ trigger After: Thực thi sau khi sự kiện xảy ra

2.4.3. Cách sử dụng

Cú pháp tạo Trigger

```
CREATE TRIGGER trigger_name

ON table_name

FOR HOẶC AFTER { INSERT | UPDATE | DELETE}

AS

BEGIN

    -- CODE THỰC THI

END;
```

Các lớp Trigger trong SQL Server

- DDL Trigger: Sẽ kích hoạt khi những sự kiện bị thay đổi cấu trúc như việc tạo, sửa đổi, bỏ bảng. Cũng có thể xuất hiện trong những sự kiện liên quan tới server (sửa đổi bảo mật, cập nhật thống kê) -> kích hoạt khi chúng ta drop cái bảng
- DML Trigger: Kích hoạt chính là câu lệnh sửa đổi dữ liệu. Đó có thể là một câu lệnh chèn vào bảng, cập nhật hoặc xóa

Lưu ý:

- Trigger " AFTER" thường được sử dụng để thực hiện các hành động sau khi dữ liệu đã được thay đổi. Có thể truy cập vào dữ liệu đã được cập nhật và sử dụng nó trong các hành động khác -> Thực hiện sau khi dữ liệu đã thay đổi
- Trigger "FOR" thường được sử dụng để kiểm tra hoặc can thiệp vào dữ liệu trước khi lệnh INSERT, UPDATE, DELETE được thực hiện hoặc để kiểm tra và thay đổi dữ liệu trước khi cập nhật. Có khả năng ảnh hưởng đến dữ liệu trước khi nó được ghi vào cơ sở dữ liệu. Được dùng để kiểm tra và cản trở cập nhật dữ liệu nếu

cần -> Tức Trigger được thực thi sau khi thao tác INSERT, UPDATE hoặc DELETE xảy ra

- INSTEAD OF là dùng để thay thế lệnh INSERT, UPDATE hoặc DELETE ban đầu bằng hành động được viết trong trigger. Nó thường được dùng để kiểm tra hoặc xử lý dữ liệu trước khi cho phép thay đổi dữ liệu trong bảng hoặc view.

2.4.4. Ưu điểm và nhược điểm

Ưu điểm:

- Có thể bắt lỗi business logic ở mức CSDL
- là một cách khác thay thế việc thực hiện những công việc hẹn giờ theo lịch
- Rất hiệu quả khi sử dụng để kiểm những thay đổi của dữ liệu trong bảng

Nhược điểm:

- Là một phần mở rộng của việc kiểm tra tính hợp lệ của dữ liệu chứ không thay thế được hoàn toàn công việc này.
- Trigger hoạt động ngầm ở trong csdl, không hiển thị ở tầng giao diện. Do đó, khó chỉ ra được điều gì xảy ra ở tầng csdl.
- Trigger thực hiện các update lên bảng dữ liệu vì thế nó làm gia tăng lượng công việc lên csdl và làm cho hệ thống chạy chậm lại.

Trong DML Trigger của SQL Server có hai bảng logic đặc biệt: inserted (chứa dữ liệu mới) và deleted (chứa dữ liệu cũ hoặc dữ liệu bị xóa)

Đặc biệt Update có thể xài cả 2

- deleted → dữ liệu trước khi UPDATE
- inserted → dữ liệu sau khi UPDATE

Ngoài ra Trigger có thể xóa nhiều dòng (ví dụ cụ thể xóa sạch SV học môn Toán)

- Được sử dụng để hoàn tác thao tác đang thực hiện khi phát hiện dữ liệu không hợp lệ

Có ERROR Dùng để tạo thông báo lỗi từ SQL Server

2.4.5. Thực thi chức năng trên giao diện và code

Trigger là đối tượng CSDL tự động thực thi khi có sự kiện INSERT, UPDATE, hoặc DELETE xảy ra trên bảng.

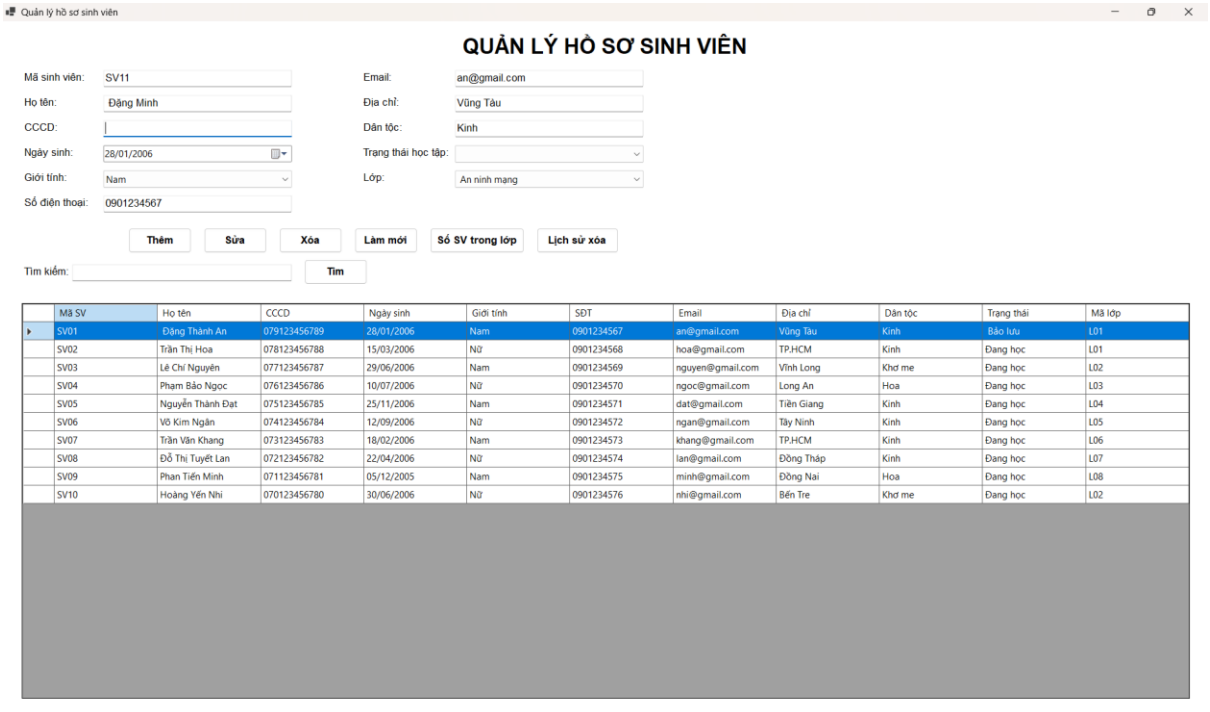
1. Tạo Trigger kiểm tra khi thêm sinh viên mới

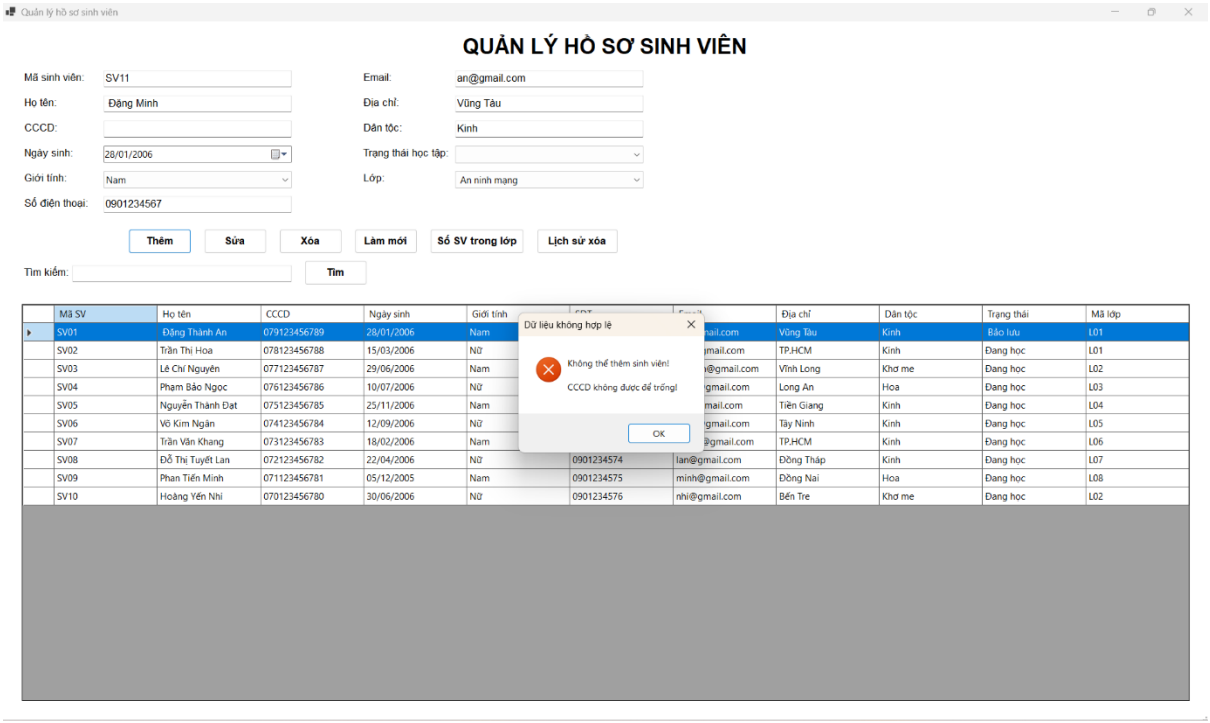
a) Giao diện

Bước 1: Nhập đầy đủ thông tin hồ sơ sinh viên mới vào các ô văn bản tương ứng trên form Quản lý hồ sơ sinh viên (để trống trường CCCD).

Bước 2: Nhấn vào nút Thêm để thực hiện thao tác lưu thông tin sinh viên mới vào hệ thống.

Bước 3: Hệ thống kiểm tra dữ liệu và hiển thị hộp thoại thông báo lỗi ("Không thể thêm sinh viên! CCCD không được để trống").





Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính			Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam			Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ			TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam			Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ			Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam			Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ			Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam			TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L02

b) SQL

```
1010    -- Tạo Trigger kiểm tra khi thêm sinh viên mới
1011    create trigger trg_KTThemSV
1012    ON dbo.SinhVien
1013    AFTER INSERT
1014    AS
1015    BEGIN
1016        SET NOCOUNT ON;
1017
1018        -- ======================================
1019        -- 1. Kiểm tra CCCD không được để trống
1020        -- ======================================
1021        IF EXISTS
1022        (
1023            SELECT 1
1024            FROM inserted
1025            WHERE CCCD IS NULL
1026               OR LTRIM(RTRIM(CCCD)) = ''
1027        )
1028        BEGIN
1029            -- Hủy thao tác thêm sinh viên
1030            ROLLBACK TRANSACTION;
1031
1032            -- Trả lỗi về cho ứng dụng C#
1033            THROW 50001,
1034                  N'CCCD không được để trống!',
1035                  1;
1036        END;
1037
            ------------------------------------
```

```
        -- ====================================
        -- 2. Kiểm tra mã lớp có tồn tại không
        -- ====================================
        IF EXISTS
        (
            SELECT 1
            FROM inserted i
            WHERE NOT EXISTS
            (
                SELECT 1
                FROM dbo.Lop l
                WHERE l.MaLop = i.MaLop
            )
        )
        BEGIN
            -- Hủy thao tác thêm sinh viên
            ROLLBACK TRANSACTION;

            -- Trả lỗi về cho ứng dụng C#
            THROW 50002,
                  N'Mã lớp không tồn tại!',
                  1;
        END;
    END;
    GO
```

Nhiệm vụ: Xây dựng Trigger trg_KTThemSV trên bảng SinhVien với sự kiện AFTER INSERT nhằm tự động kiểm tra tính hợp lệ của dữ liệu mới được thêm vào.

Nội dung thực thi:

- Sử dụng bảng ảo inserted để kiểm tra dữ liệu dòng vừa được thêm.
- **Kiểm tra 1 (CCCD):** Nếu phát hiện trường CCCD bị NULL hoặc để trống (chuỗi khoảng trắng), trigger sẽ thực hiện lệnh ROLLBACK TRANSACTION để hủy thao tác thêm và bắn lỗi về ứng dụng C# bằng lệnh THROW 50001 với thông báo "CCCD không được để trống!".

- **Kiểm tra 2 (Mã lớp):** Kiểm tra nếu MaLop của sinh viên mới thêm không tồn tại trong bảng Lop, trigger sẽ tiếp tục thực hiện ROLLBACK TRANSACTION và phát sinh lỗi qua lệnh THROW 50002 với thông báo “Mã lớp không tồn tại!”.

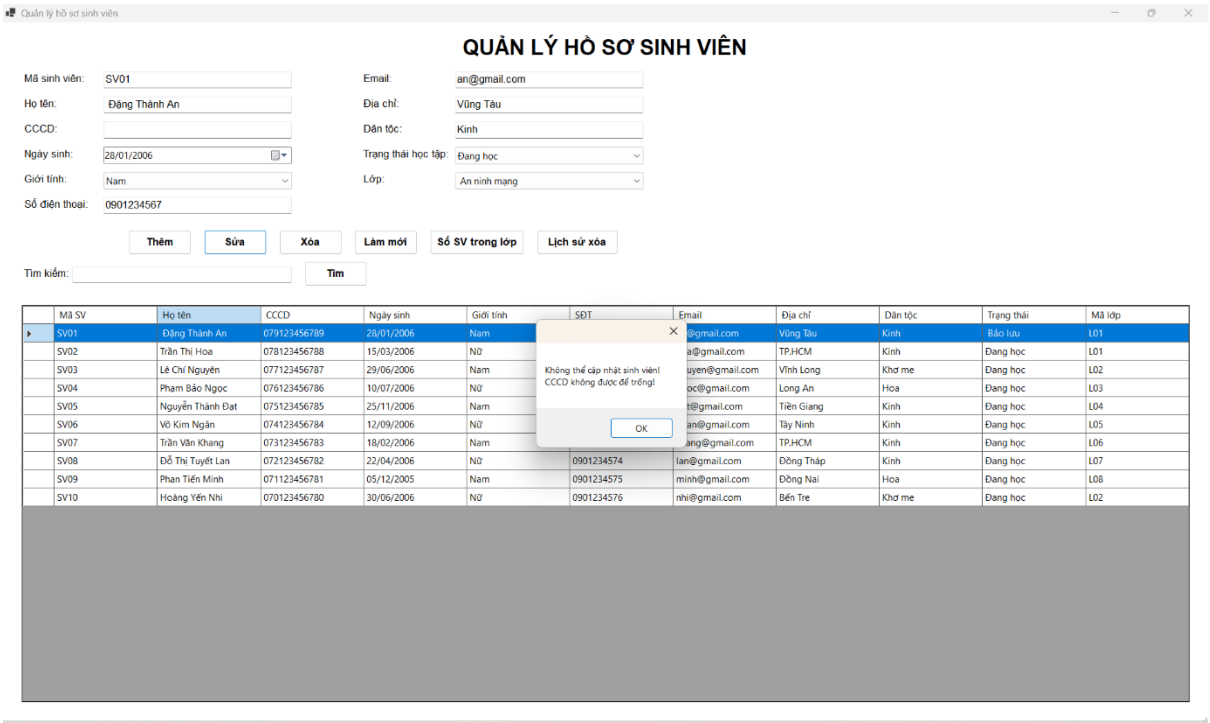
2. Tạo Trigger kiểm tra khi cập nhật CCCD của sinh viên

a) Giao diện

Bước 1: Chọn thông tin sinh viên cần cập nhật trên danh sách (ví dụ: sinh viên SV1 - Đặng Thành An) và chỉnh sửa lại trường thông tin CCCD trên form (để trống hoặc nhập khoảng trắng).

Bước 2: Nhấn vào nút Sửa để thực hiện cập nhật thông tin.

Bước 3: Hệ thống hiển thị hộp thoại thông báo lỗi từ Trigger kích hoạt khi cập nhật dữ liệu với nội dung cảnh báo (ví dụ: 'CCCD không được để trống!') và hủy thao tác lưu.



b) SQL

```
1065    -- Tạo trigger kiểm tra khi cập nhật CCCD của sinh viên
1066    create trigger trg_KTCapNhatSV
1067    ON dbo.SinhVien
1068    AFTER UPDATE
1069    AS
1070    BEGIN
1071        SET NOCOUNT ON;
1072
1073        -- Nếu CCCD bị NULL hoặc để trống
1074        IF EXISTS
1075        (
1076            SELECT 1
1077            FROM inserted
1078            WHERE CCCD IS NULL
1079               OR LTRIM(RTRIM(CCCD)) = ''
1080        )
1081        BEGIN
1082            -- Hủy thao tác UPDATE
1083            ROLLBACK TRANSACTION;
1084
1085            -- Trả thông báo lỗi về cho ứng dụng C#
1086            THROW 50001,
1087                  N'CCCD không được để trống!',
1088                  1;
1089        END
1090    END;
1091    GO
1092
```

Nhiệm vụ: Xây dựng Trigger trg_KTCapNhatSV trên bảng SinhVien với sự kiện AFTER UPDATE nhằm kiểm tra tính hợp lệ của dữ liệu khi sinh viên được cập nhật thông tin.

Nội dung thực thi:

- Sử dụng bảng ảo inserted để kiểm tra giá trị của bản ghi sau khi cập nhật.
- Kiểm tra điều kiện nếu trường CCCD bị NULL hoặc để trống (LTRIM(RTRIM(CCCD)) = '').

- Nếu vi phạm, trigger thực hiện lệnh ROLLBACK TRANSACTION để hủy bỏ toàn bộ thao tác cập nhật và dùng lệnh THROW 50001 để trả thông báo lỗi về ứng dụng C# với nội dung “CCCD không được để trống!”.

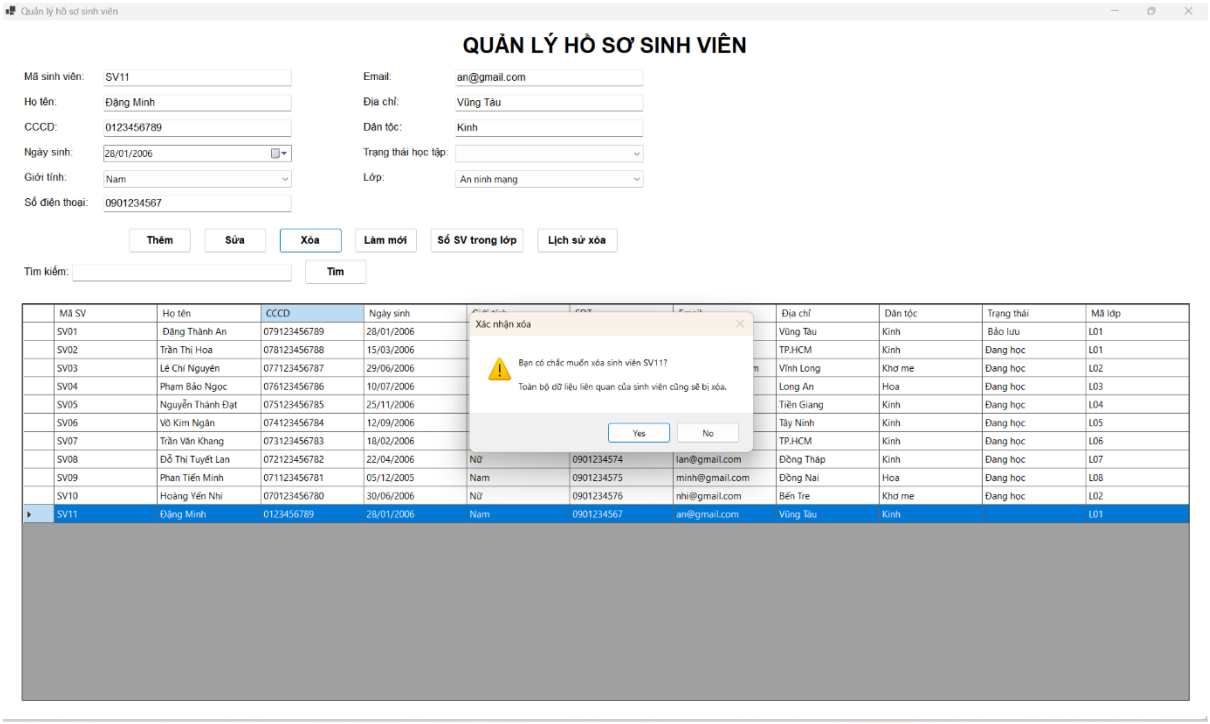
3. Tạo Trigger kiểm tra khi xóa sinh viên

a) Giao diện

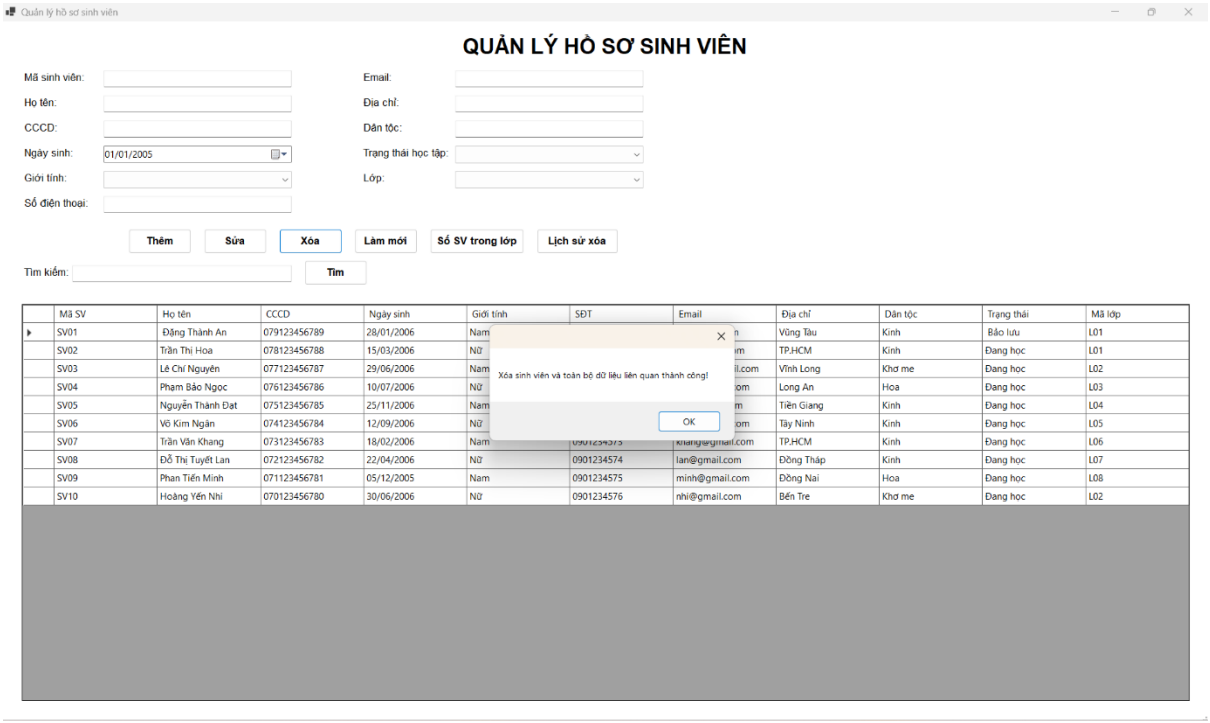
Bước 1: Chọn sinh viên cần xóa khỏi danh sách (ví dụ: SV11 - Đặng Minh).

Bước 2: Nhấn nút Xóa, hệ thống sẽ hiển thị hộp thoại xác nhận việc xóa thông tin sinh viên cùng cảnh báo toàn bộ dữ liệu liên quan sẽ bị xóa.

Bước 3: Nhấn Yes để đồng ý xóa, hệ thống sẽ thực thi và hiển thị thông báo thành công ("Xóa sinh viên và toàn bộ dữ liệu liên quan thành công!").



Mã SV	Họ tên	CCCD	Ngày sinh				Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006				Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006				TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006				Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006				Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006				Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006				Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006				TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L02
SV11	Đặng Minh	0123456789	28/01/2006	Nam	0901234567	an@gmail.com	Vũng Tàu	Kinh		L01



Mã SV	Họ tên	CCCD	Ngày sinh	Giới tính	SĐT	Email	Địa chỉ	Dân tộc	Trạng thái	Mã lớp
SV01	Đặng Thành An	079123456789	28/01/2006	Nam			Vũng Tàu	Kinh	Bảo lưu	L01
SV02	Trần Thị Hoa	078123456788	15/03/2006	Nữ			TP.HCM	Kinh	Đang học	L01
SV03	Lê Chí Nguyên	077123456787	29/06/2006	Nam			Vĩnh Long	Khơ me	Đang học	L02
SV04	Phạm Bảo Ngọc	076123456786	10/07/2006	Nữ			Long An	Hoa	Đang học	L03
SV05	Nguyễn Thành Đạt	075123456785	25/11/2006	Nam			Tiền Giang	Kinh	Đang học	L04
SV06	Võ Kim Ngân	074123456784	12/09/2006	Nữ			Tây Ninh	Kinh	Đang học	L05
SV07	Trần Văn Khang	073123456783	18/02/2006	Nam			TP.HCM	Kinh	Đang học	L06
SV08	Đỗ Thị Tuyết Lan	072123456782	22/04/2006	Nữ	0901234574	lan@gmail.com	Đồng Tháp	Kinh	Đang học	L07
SV09	Phan Tiến Minh	071123456781	05/12/2005	Nam	0901234575	minh@gmail.com	Đồng Nai	Hoa	Đang học	L08
SV10	Hoàng Yến Nhi	070123456780	30/06/2006	Nữ	0901234576	nhi@gmail.com	Bến Tre	Khơ me	Đang học	L02

b) SQL

```
1094    -- Tạo trigger kiểm tra khi xóa sinh viên
1095    create trigger trg_KTXoaSV
1096    ON dbo.SinhVien
1097    AFTER DELETE
1098    AS
1099    BEGIN
1100        SET NOCOUNT ON;
1101
1102        -- Bảng deleted chứa dữ liệu của sinh viên
1103        -- ngay trước khi bản ghi bị xóa
1104        INSERT INTO dbo.LichSuXoaSinhVien
1105        (
1106            MaSV,
1107            HoTen,
1108            CCCD,
1109            MaLop,
1110            ThoiGianXoa
1111        )
1112        SELECT
1113            MaSV,
1114            HoTen,
1115            CCCD,
1116            MaLop,
1117            GETDATE()
1118        FROM deleted;
1119    END;
1120    GO
1121
```

Nhiệm vụ: Xây dựng Trigger trg_KTXoaSV trên bảng SinhVien với sự kiện AFTER DELETE nhằm tự động ghi lại lịch sử các bản ghi bị xóa vào bảng nhật ký.

Nội dung thực thi:

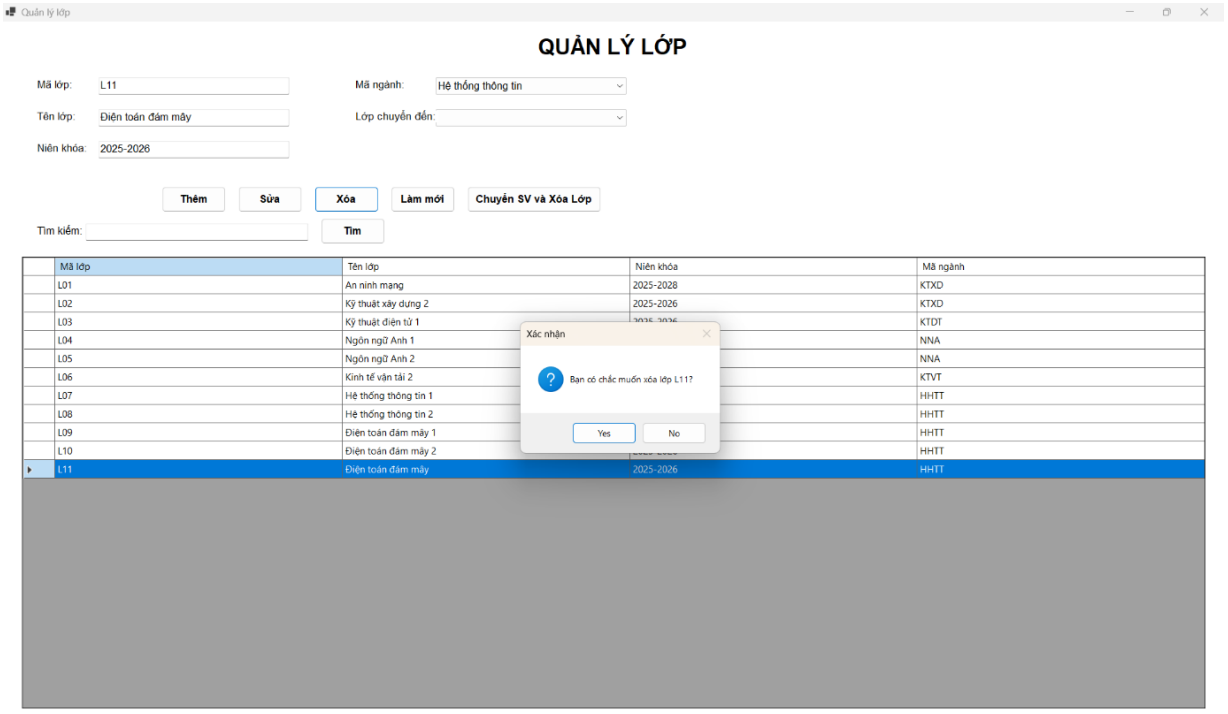
- Sử dụng bảng ảo deleted để truy xuất thông tin của sinh viên ngay trước khi bản ghi bị xóa khỏi cơ sở dữ liệu.
- Tự động thực hiện câu lệnh INSERT INTO dbo.LichSuXoaSinhVien để lưu trữ các thông tin gồm MaSV, HoTen, CCCD, MaLop và mốc thời gian xóa thông qua hàm GETDATE().

4. Tạo Trigger kiểm tra khi xóa lớp

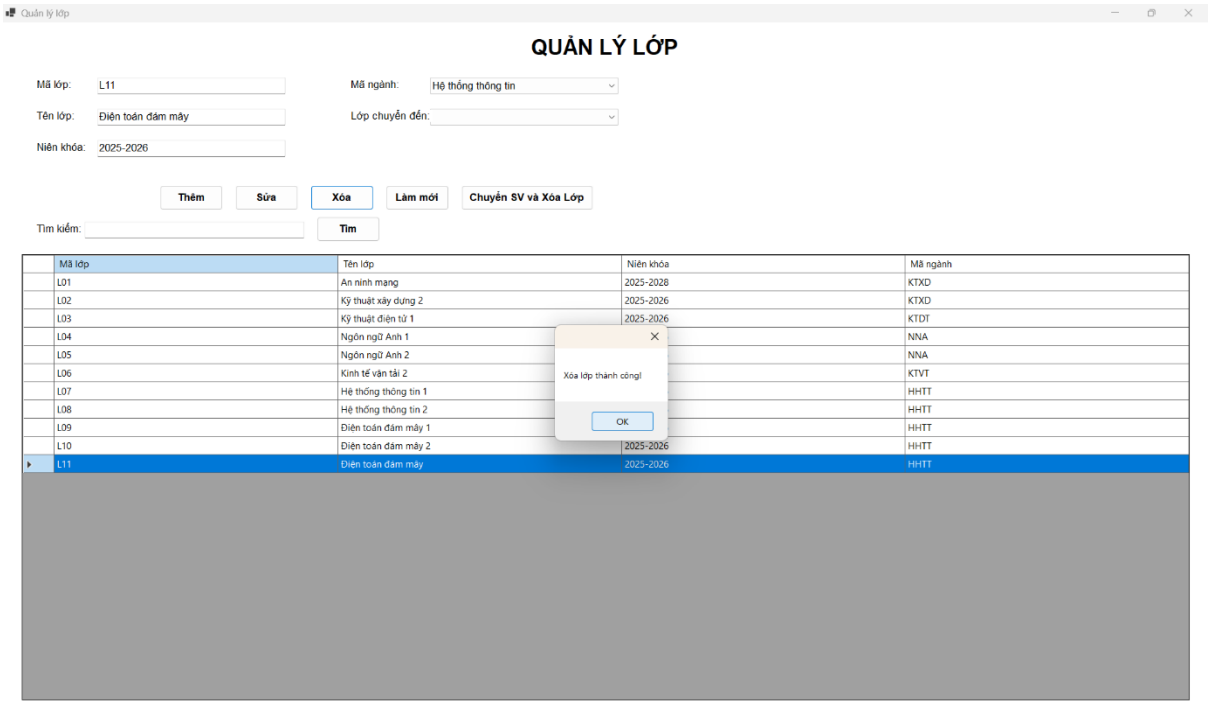
a) Giao diện

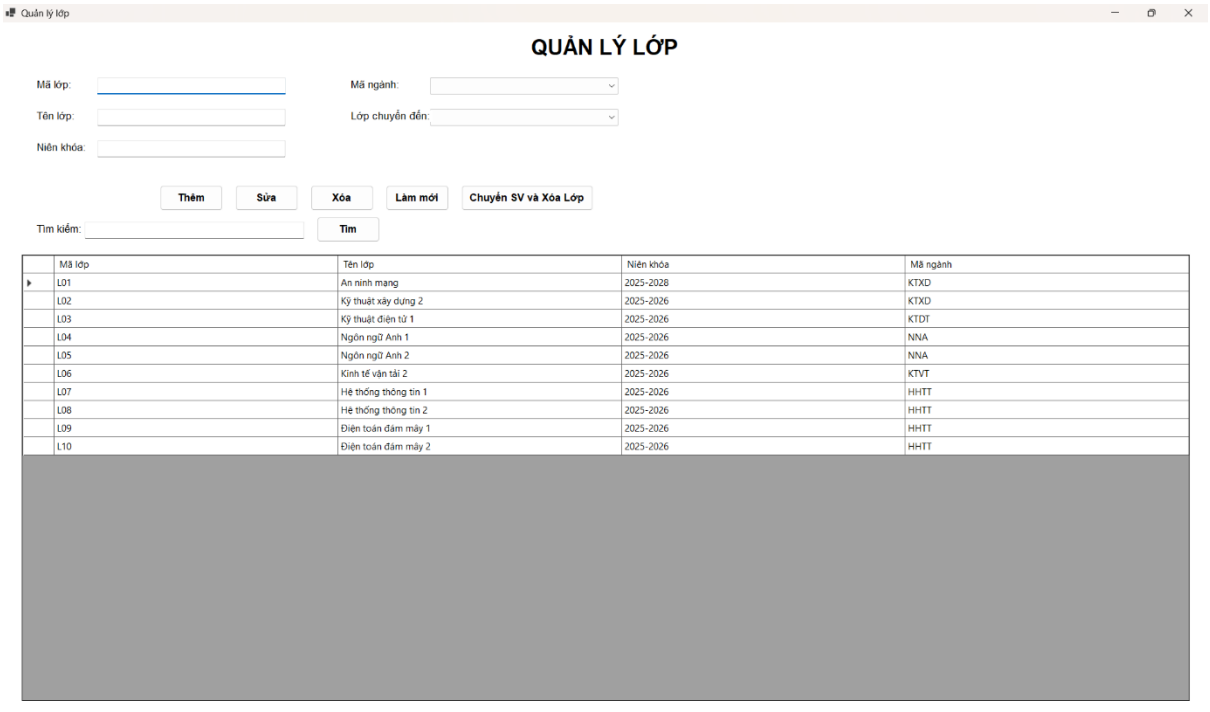
Bước 1: Chọn lớp học cần xóa trên danh sách giao diện quản lý lớp (ví dụ: lớp L11 - Điện toán đám mây).

Bước 2: Nhấn vào nút Xóa, hệ thống sẽ hiển thị bảng xác nhận yêu cầu người dùng quyết định có muốn xóa lớp đang chọn hay không.



Bước 3: Nhấn Yes để đồng ý xóa; hệ thống thực hiện thao tác xóa dữ liệu trong cơ sở dữ liệu và hiển thị thông báo thành công.





b) SQL

```
1123    -- Tạo trigger kiểm tra khi xóa lớp
1124    create trigger trg_KTXoaLop
1125    on Lop
1126    after delete
1127    as
1128    begin
1129        if exists
1130        (
1131            select * from deleted
1132        )
1133        begin
1134            print N'Đã xóa lớp thành công!';
1135        end
1136    end;
1137    go
1138
```

Nhiệm vụ: Xây dựng Trigger trg_KTXoaLop trên bảng Lop với sự kiện AFTER DELETE nhằm thông báo hoặc xử lý logic sau khi một lớp học bị xóa khỏi hệ thống.

Nội dung thực thi:

- Sử dụng bảng ảo deleted để kiểm tra các bản ghi vừa bị xóa khỏi bảng Lop.
- Kiểm tra điều kiện tồn tại bản ghi trong bảng deleted.
- Nếu thao tác xóa thành công, trigger sẽ chạy khối lệnh bên trong và sử dụng lệnh print N'Đã xóa lớp thành công!'; để in thông báo xác nhận.

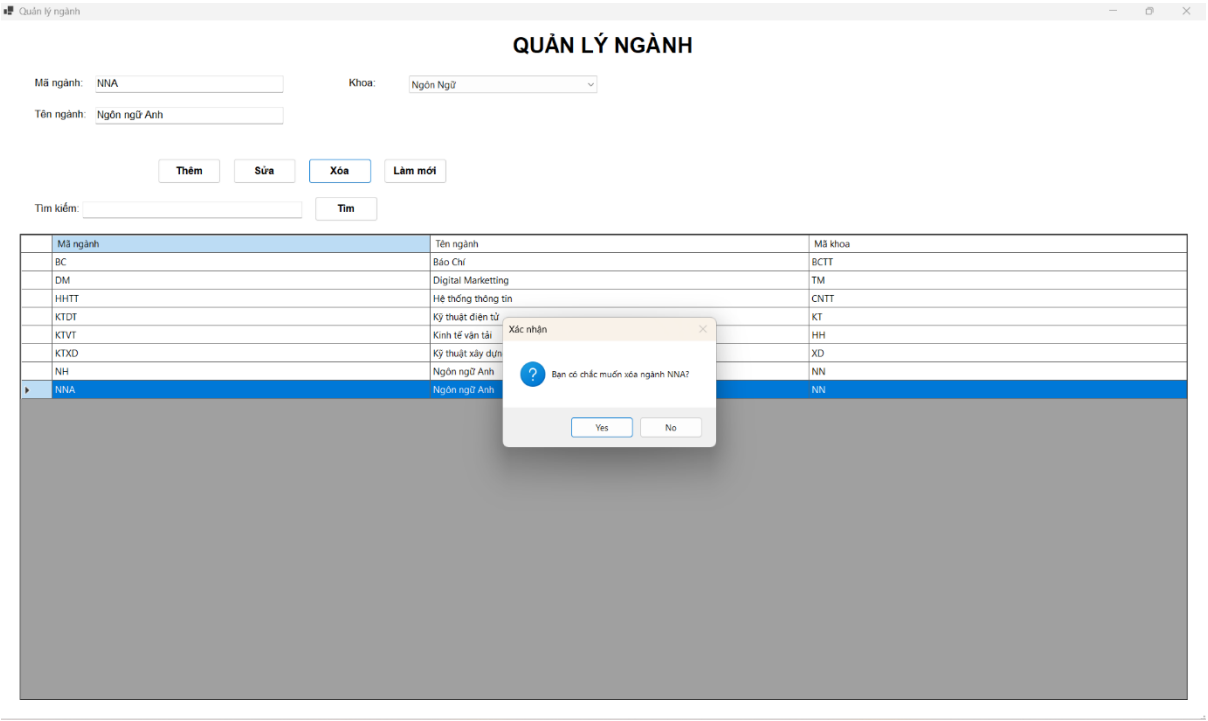
5. Tạo Trigger kiểm tra khi xóa ngành

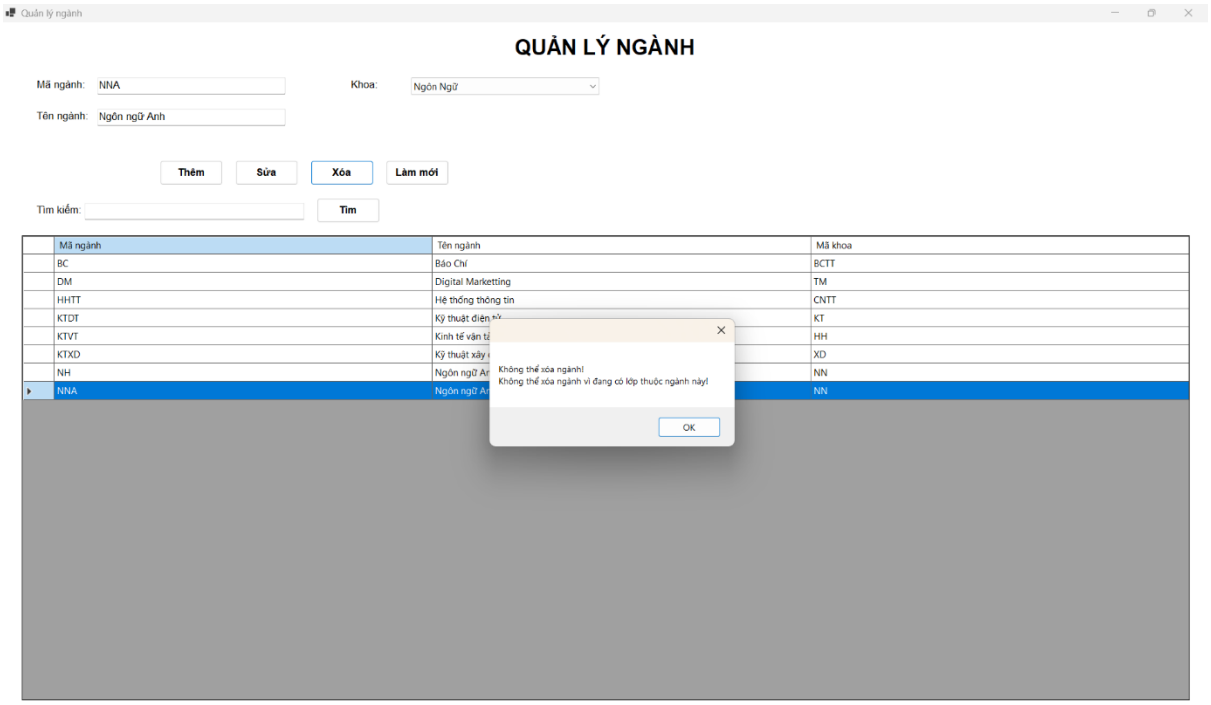
a) Giao diện

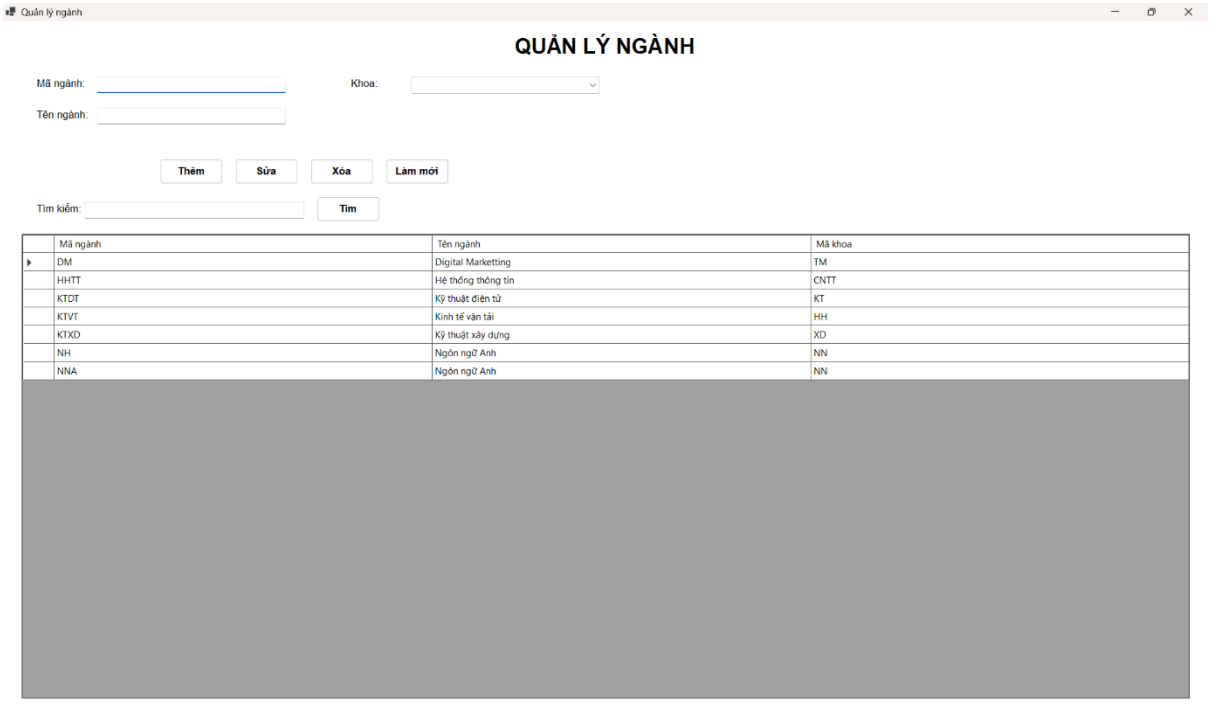
Bước 1: Chọn một ngành cần xóa trên danh sách giao diện quản lý ngành (ví dụ: ngành NNA - Ngôn ngữ Anh hoặc ngành BC - Báo Chí).

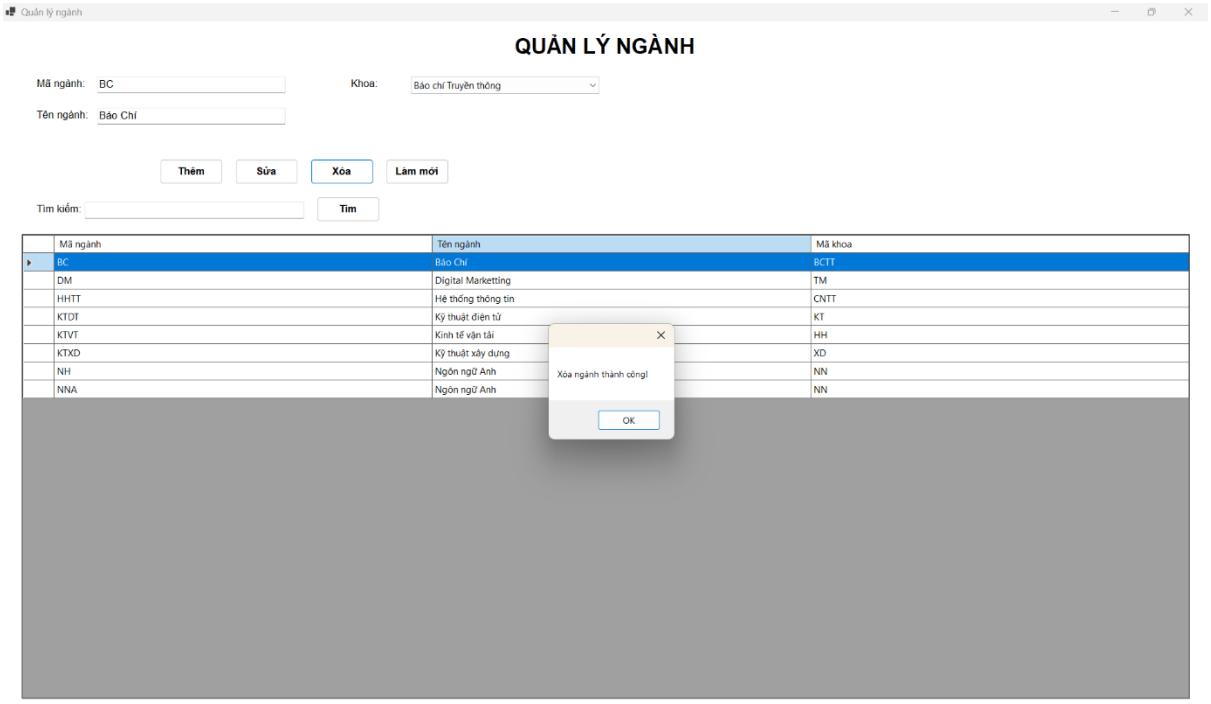
Bước 2: Nhấn vào nút Xóa và xác nhận thao tác trên hộp thoại cảnh báo của hệ thống.

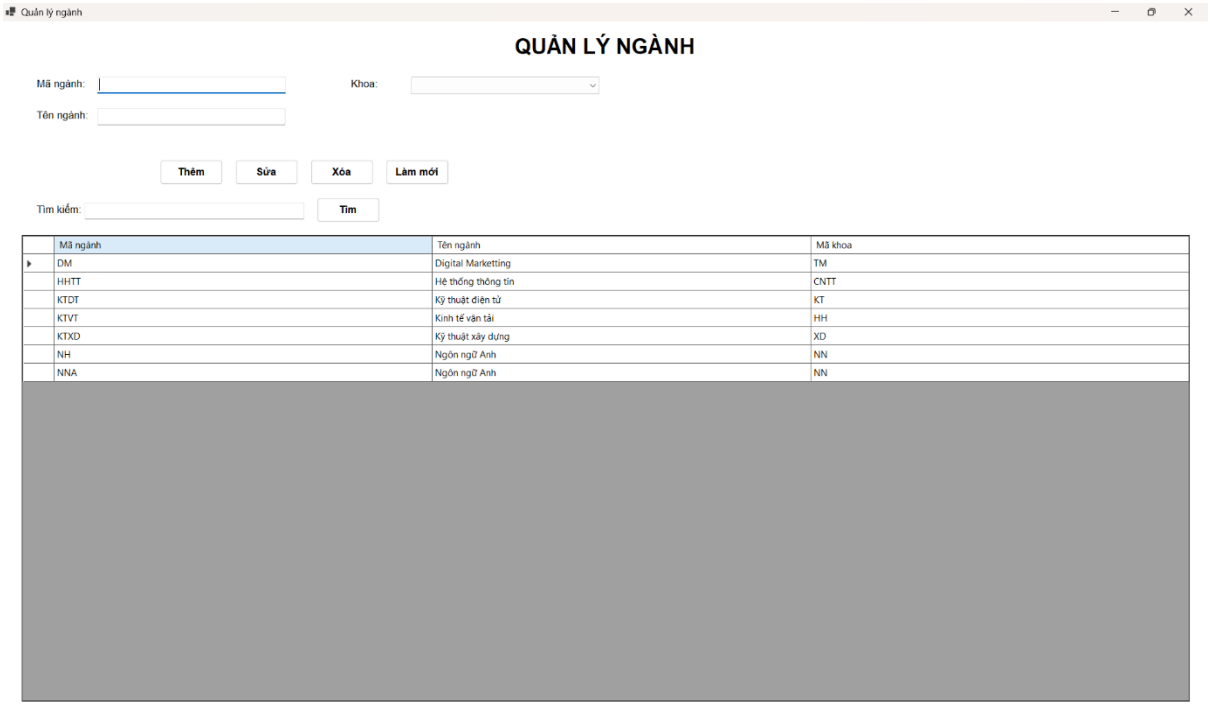
Bước 3: Hệ thống hiển thị hộp thoại thông báo kết quả: nếu ngành đang có lớp thuộc về nó, trigger sẽ chặn và hiển thị lỗi ("Không thể xóa ngành! Không thể xóa ngành vì đang có lớp thuộc ngành này!"); ngược lại nếu không có ràng buộc, hệ thống thông báo xóa thành công.




Quản lý ngành
QUẢN LÝ NGÀNH
Mã ngành: NNA
Khoa: Ngôn Ngữ
Tên ngành: Ngôn ngữ Anh
Thêm
Sửa
Xóa
Làm mới
Tìm kiếm:
Tìm
Mã ngành
Tên ngành
Mã khoa
BC
Báo Chí
BCTT
DM
Digital Marketting
TM
HHTT
Hệ thống thông tin
CNTT
KTDT
KT
KTVT
HH
KTXD
XD
NH
NN
NNA
NN
Không thể xóa ngành!
Không thể xóa ngành vì đang có lớp thuộc ngành này!
OK


Quản lý ngành
QUẢN LÝ NGÀNH
Mã ngành:
Khoa:
Tên ngành:
Thêm
Sửa
Xóa
Làm mới
Tìm kiếm:
Tìm
Mã ngành
Tên ngành
Mã khoa
DM
Digital Marketting
TM
HHTT
Hệ thống thông tin
CNTT
KTDT
Kỹ thuật điện tử
KT
KTVT
Kinh tế vận tải
HH
KTXD
Kỹ thuật xây dựng
XD
NH
Ngôn ngữ Anh
NN
NNA
Ngôn ngữ Anh
NN


Quản lý ngành
QUẢN LÝ NGÀNH
Mã ngành: BC
Khoa: Báo chí Truyền thông
Tên ngành: Báo Chí
Thêm
Sửa
Xóa
Làm mới
Tìm kiếm:
Tìm
Mã ngành
Tên ngành
Mã khoa
BC Báo Chí BCTT
DM Digital Marketting TM
HHTT Hệ thống thông tin CNTT
KTDT Kỹ thuật điện tử KT
KTVT Kinh tế vận tải HH
KTXD Kỹ thuật xây dựng XD
NH Ngôn ngữ Anh NN
NNA Ngôn ngữ Anh NN
Xóa ngành thành công!
OK


Quản lý ngành
QUẢN LÝ NGÀNH
Mã ngành:
Khoa:
Tên ngành:
Thêm
Sửa
Xóa
Làm mới
Tìm kiếm:
Tìm
Mã ngành
Tên ngành
Mã khoa
DM Digital Marketting TM
HHTT Hệ thống thông tin CNTT
KTDT Kỹ thuật điện tử KT
KTVT Kinh tế vận tải HH
KTXD Kỹ thuật xây dựng XD
NH Ngôn ngữ Anh NN
NNA Ngôn ngữ Anh NN

b) SQL

```
1140    -- Tạo trigger kiểm tra khi xóa ngành
1141    create trigger trg_KTXoaNganh
1142    on Nganh
1143    instead of delete
1144    as
1145    begin
1146        if exists
1147        (
1148            select * from deleted
1149            where MaNganh in (select MaNganh from Lop)
1150        )
1151        begin
1152            print N'Không thể xóa ngành đang có lớp!';
1153        end
1154        else
1155        begin
1156            delete from Nganh
1157            where MaNganh in (select MaNganh from deleted);
1158
1159            print N'Xóa ngành thành công!';
1160        end
1161    end;
1162    go
1163
```

Nhiệm vụ: Xây dựng Trigger trg_KTXoaNganh trên bảng Nganh với sự kiện INSTEAD OF DELETE nhằm kiểm tra ràng buộc dữ liệu trước khi thực hiện xóa.

Nội dung thực thi:

- Sử dụng bảng ảo deleted để xác định bản ghi ngành đang chuẩn bị bị xóa.
- Kiểm tra điều kiện xem MaNganh có đang tồn tại trong bảng Lop hay không (WHERE MaNganh IN (SELECT MaNganh FROM Lop)).
- Nếu tồn tại (đang có lớp thuộc ngành), trigger sẽ in ra thông báo hủy thao tác bằng lệnh print N'Không thể xóa ngành đang có lớp!';.
- Ngược lại, nếu không vi phạm ràng buộc, trigger sẽ thực hiện câu lệnh DELETE FROM Nganh dựa trên bảng deleted và thông báo thành công.

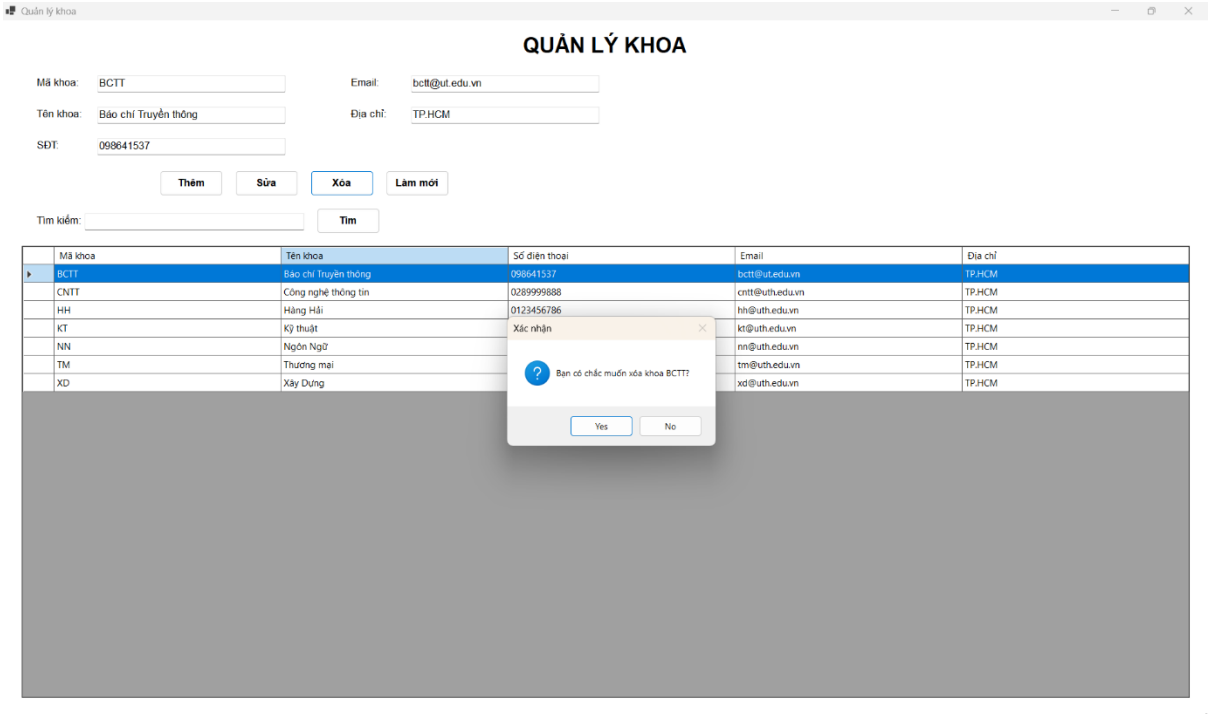
6. Tạo Trigger kiểm tra khi xóa khoa

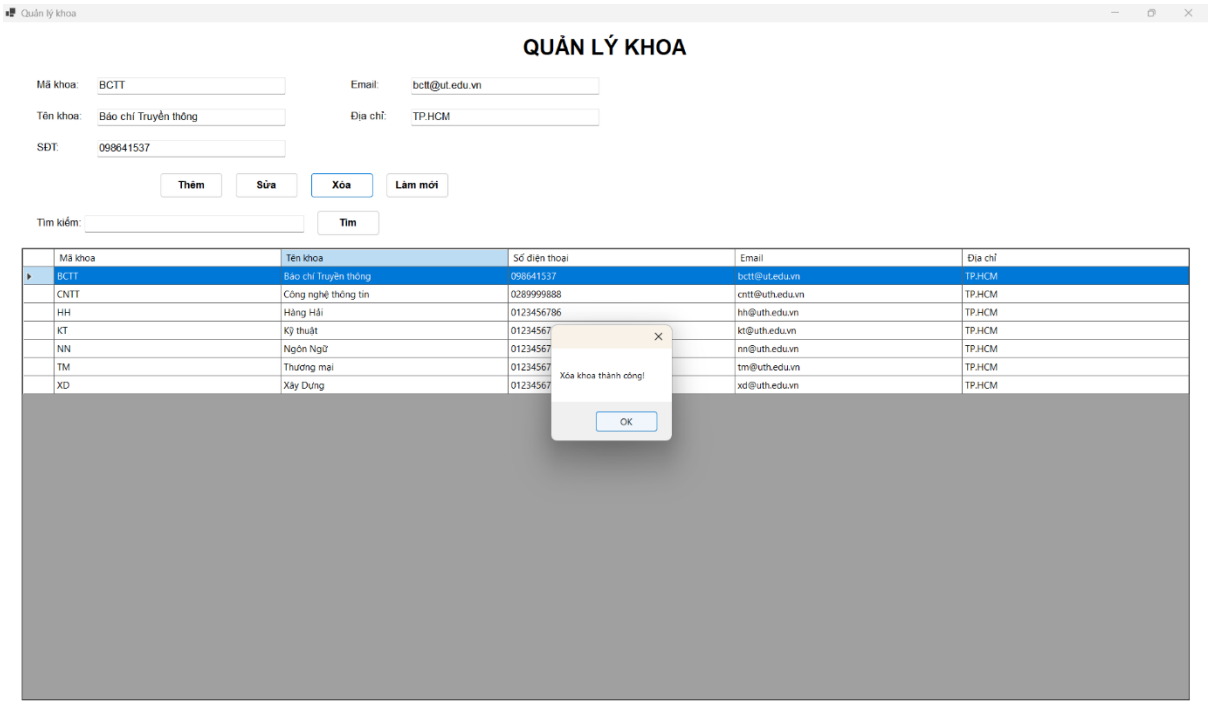
a) Giao diện

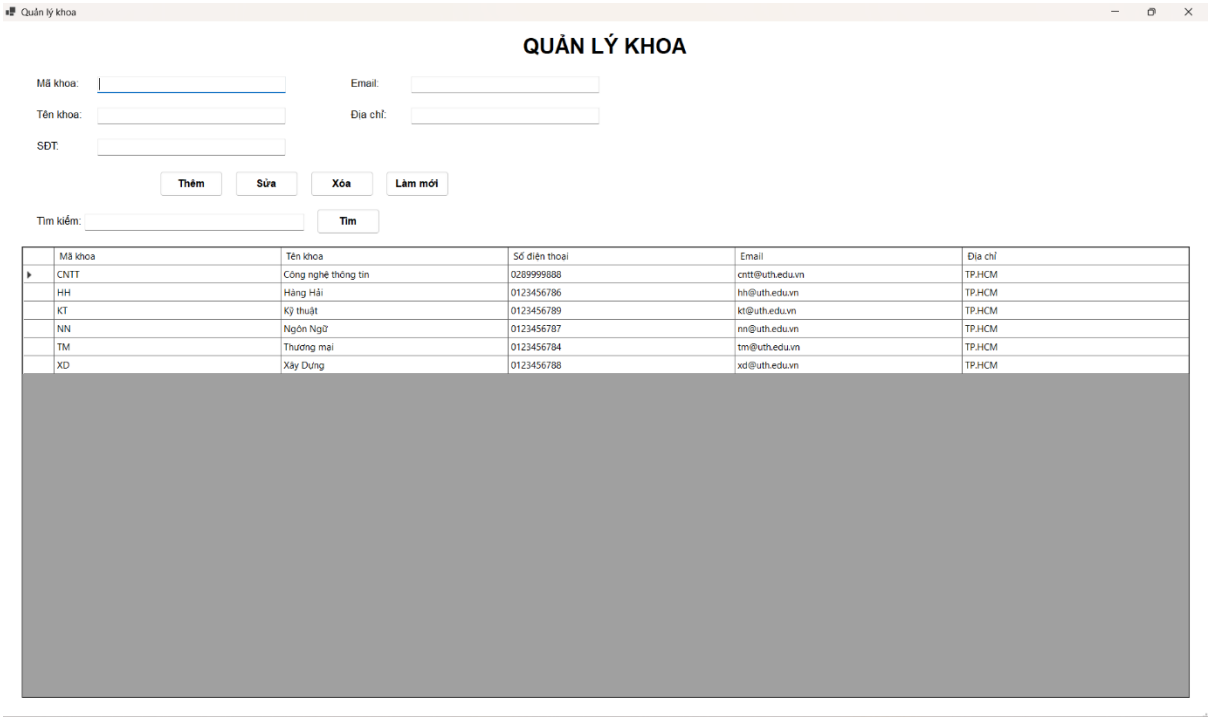
Bước 1: Chọn một khoa cần xóa trên danh sách giao diện quản lý khoa (ví dụ: khoa BCTT - Báo chí Truyền thông).

Bước 2: Nhấn vào nút Xóa và xác nhận thao tác trên hộp thoại cảnh báo của hệ thống.

Bước 3: Hệ thống hiển thị hộp thoại thông báo kết quả: nếu khoa đang có ngành thuộc về nó, trigger sẽ chặn và hiển thị lỗi, ngược lại nếu không có ràng buộc, hệ thống thông báo xóa thành công.




Quản lý khoa
QUẢN LÝ KHOA
Mã khoa: BCTT
Email: bctt@ut.edu.vn
Tên khoa: Báo chí Truyền thông
Địa chỉ: TP.HCM
SĐT: 098641537
Thêm
Sửa
Xóa
Làm mới
Tìm kiếm:
Tìm
Mã khoa | Tên khoa | Số điện thoại | Email | Địa chỉ
BCTT | Báo chí Truyền thông | 098641537 | bctt@ut.edu.vn | TP.HCM
CNTT | Công nghệ thông tin | 0289999888 | cntt@uth.edu.vn | TP.HCM
HH | Hàng Hải | 0123456786 | hh@uth.edu.vn | TP.HCM
KT | Kỹ thuật | 0123456 | kt@uth.edu.vn | TP.HCM
NN | Ngôn Ngữ | 0123456 | nn@uth.edu.vn | TP.HCM
TM | Thương mại | 0123456 | tm@uth.edu.vn | TP.HCM
XD | Xây Dựng | 0123456 | xd@uth.edu.vn | TP.HCM
Xóa khoa thành công!
OK


Quản lý khoa
QUẢN LÝ KHOA
Mã khoa:
Email:
Tên khoa:
Địa chỉ:
SĐT:
Thêm
Sửa
Xóa
Làm mới
Tìm kiếm:
Tìm
Mã khoa | Tên khoa | Số điện thoại | Email | Địa chỉ
CNTT | Công nghệ thông tin | 0289999888 | cntt@uth.edu.vn | TP.HCM
HH | Hàng Hải | 0123456786 | hh@uth.edu.vn | TP.HCM
KT | Kỹ thuật | 0123456789 | kt@uth.edu.vn | TP.HCM
NN | Ngôn Ngữ | 0123456787 | nn@uth.edu.vn | TP.HCM
TM | Thương mại | 0123456784 | tm@uth.edu.vn | TP.HCM
XD | Xây Dựng | 0123456788 | xd@uth.edu.vn | TP.HCM

b) SQL

```
1165    -- Tạo trigger kiểm tra khi xóa khoa
1166    create trigger trg_KTXoaKhoa
1167    on Khoa
1168    instead of delete
1169    as
1170    begin
1171        if exists
1172        (
1173            select * from deleted
1174            where MaKhoa in (select MaKhoa from Nganh)
1175        )
1176        begin
1177            print N'Không thể xóa khoa đang có ngành!';
1178        end
1179        else
1180        begin
1181            delete from Khoa
1182            where MaKhoa in (select MaKhoa from deleted);
1183
1184            print N'Xóa khoa thành công!';
1185        end
1186    end;
1187    go
1188
```

Nhiệm vụ: Xây dựng Trigger trg_KTXoaKhoa trên bảng Khoa với sự kiện INSTEAD OF DELETE nhằm kiểm tra ràng buộc dữ liệu trước khi thực hiện xóa.

Nội dung thực thi:

- Sử dụng bảng ảo deleted để xác định bản ghi khoa đang chuẩn bị bị xóa.
- Kiểm tra điều kiện xem MaKhoa có đang tồn tại trong bảng Nganh hay không (WHERE MaKhoa IN (SELECT MaKhoa FROM Nganh)).
- Nếu tồn tại (đang có ngành thuộc khoa), trigger sẽ in ra thông báo hủy thao tác bằng lệnh print N'Không thể xóa khoa đang có ngành!';.
- Ngược lại, nếu không vi phạm ràng buộc, trigger sẽ thực hiện câu lệnh DELETE FROM Khoa dựa trên bảng deleted và thông báo thành công.

CHƯƠNG 3: ĐIỀU KHIỂN TƯƠNG TRANH

3.1. Giới thiệu

Trong hệ quản trị cơ sở dữ liệu, nhiều giao tác có thể được thực hiện đồng thời và cùng truy cập đến một hoặc nhiều đơn vị dữ liệu. Nếu không có cơ chế kiểm soát phù hợp, các giao tác có thể ảnh hưởng lẫn nhau, làm dữ liệu bị sai lệch hoặc không đảm bảo tính nhất quán.

Điều khiển tương tranh là tập hợp các kỹ thuật được sử dụng để quản lý việc các giao tác truy cập đồng thời vào dữ liệu, nhằm hạn chế sự ảnh hưởng lẫn nhau giữa các giao tác và đảm bảo lịch giao tác có khả năng tuần tự.

Mục tiêu của điều khiển tương tranh là hạn chế các vấn đề thường xảy ra khi nhiều giao tác truy cập đồng thời vào cơ sở dữ liệu, bao gồm:

3.2. Mất dữ liệu cập nhật (Lost Update)

Mất dữ liệu cập nhật (**Lost Update**) xảy ra khi hai giao tác cùng đọc và cập nhật một dữ liệu, nhưng kết quả cập nhật của giao tác thực hiện trước bị giao tác thực hiện sau ghi đè lên. Vì vậy, một trong hai lần cập nhật bị mất.

Ví dụ: Giả sử tài khoản A có số dư 10.000.000 đồng. Hai giao tác T1 và T2 cùng muốn cập nhật số dư:

- T1 đọc số dư: 10.000.000 đồng.
- T2 cũng đọc số dư: 10.000.000 đồng.
- T1 trừ 2.000.000 đồng và ghi lại: 8.000.000 đồng.
- T2 trừ 3.000.000 đồng dựa trên dữ liệu cũ và ghi lại: 7.000.000 đồng.

Kết quả cuối cùng là 7.000.000 đồng, trong khi nếu thực hiện đúng cả hai giao dịch thì số dư phải là 5.000.000 đồng.

→ Dữ liệu cập nhật của T1 bị T2 ghi đè, làm cho CSDL không phản ánh chính xác các giao dịch đã xảy ra.

3.2.1. Mô phỏng hiện tượng Lost Update

Trong tình huống mô phỏng, Sinh viên và Chuyên viên nhập liệu cùng mở thông tin người thân **NT01** khi SDT hiện tại là **0901234568**.

Sinh viên cập nhật SDT thành **0901234111** và lưu thành công. Tuy nhiên, Chuyên viên vẫn đang giữ dữ liệu cũ có SDT = **0901234568**. Khi Chuyên viên sửa địa chỉ thành **Bình Dương** và lưu toàn bộ dữ liệu đang giữ, SDT cũ được ghi trở lại Database.

→ Kết quả: thay đổi SDT **0901234111** của Sinh viên bị mất.

Nguyên nhân: Hai Transaction cùng đọc dữ liệu ban đầu nhưng không giữ **khóa cập nhật** ngay từ bước đọc. Transaction thực hiện ghi sau sử dụng dữ liệu cũ và ghi đè lên kết quả của Transaction trước.

1.Cấu hình giao diện

Giao diện Lost Update được thiết kế theo mô hình **hai Transaction**, tương ứng với hai người dùng cùng thao tác trên một bản ghi. Giao diện gồm khu vực **Transaction 1** và **Transaction 2**, các nút thao tác theo từng bước và khu vực hiển thị kết quả thực tế trong Database.

Giao diện có hai chế độ **DEMO LOST UPDATE** và **KHẮC PHỤC LOST UPDATE**. Ở chế độ khắc phục, nội dung thao tác được thay đổi để thể hiện việc sử dụng **UPDLOCK**. Màu đỏ được sử dụng để biểu thị tình huống lỗi, trong khi màu xanh biểu thị trạng thái đã được khắc phục.

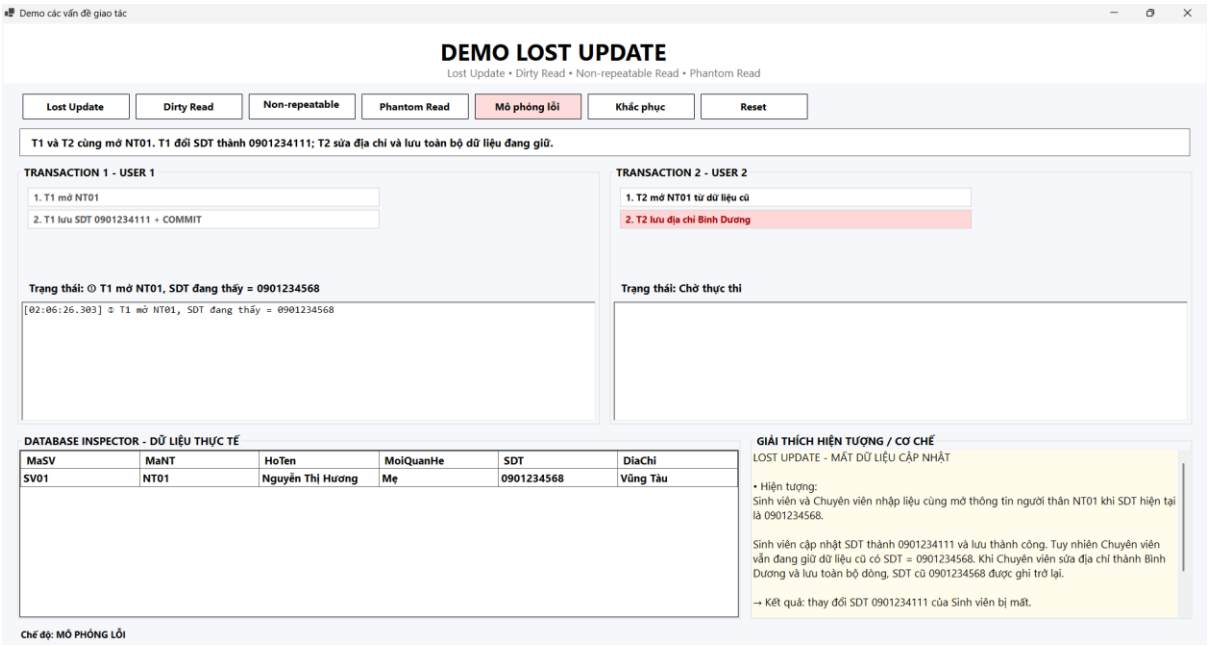
Bước 1 – Transaction 1 mở thông tin NT01

Ở bước đầu tiên, Transaction 1 mở thông tin người thân NT01 và đọc dữ liệu ban đầu:

SDT = 0901234568
Địa chỉ = Vũng Tàu

Trong chế độ mô phỏng lỗi, T1 chỉ đọc dữ liệu thông thường và chưa sử dụng **UPDLOCK**.

Sau khi thực hiện, giao diện ghi nhận T1 đã mở NT01 và cho phép Transaction 2 thực hiện bước tiếp theo.



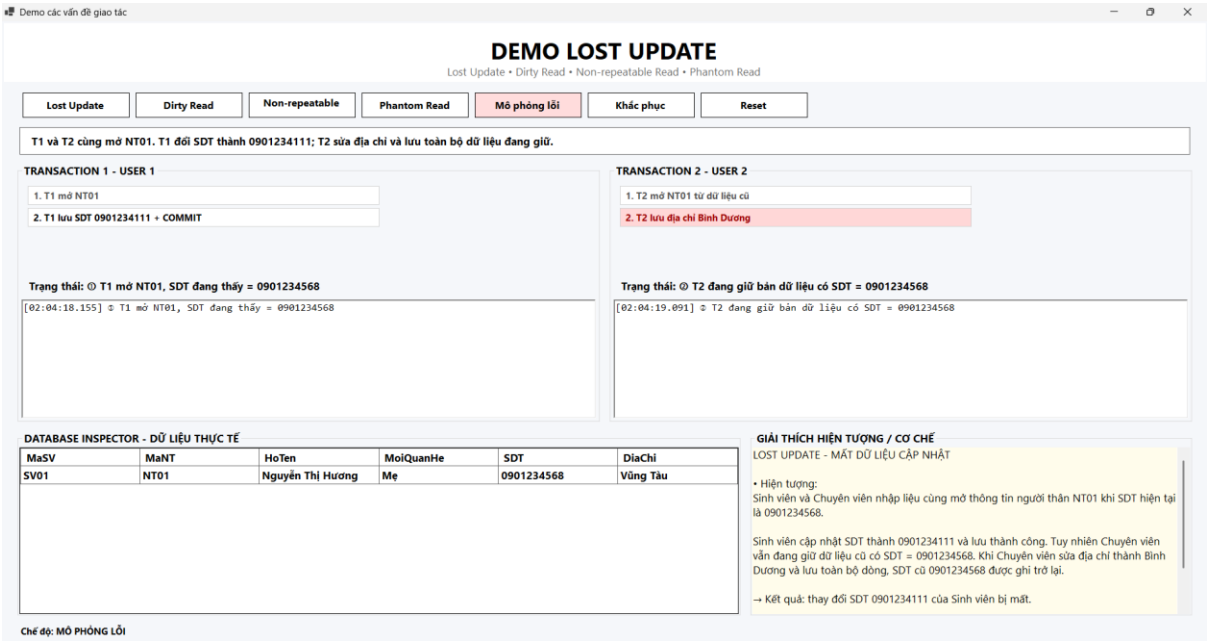
Màn hình UI của LostUpdate (ERROR)

Bước 2 – Transaction 2 mở NT01 từ dữ liệu cũ

Transaction 2 tiếp tục mở NT01 và nhận được cùng dữ liệu ban đầu:

SDT = 0901234568
Địa chỉ = Vũng Tàu

Lúc này, T1 và T2 đều đang dựa trên cùng một phiên bản dữ liệu cũ. Đây là điều kiện để Lost Update có thể xảy ra khi một Transaction cập nhật sau bằng dữ liệu đã lỗi thời.



Bước 3 – Transaction 1 cập nhật SDT và COMMIT

Transaction 1 thay đổi SDT:

0901234568 → 0901234111

Sau đó T1 thực hiện **COMMIT**.

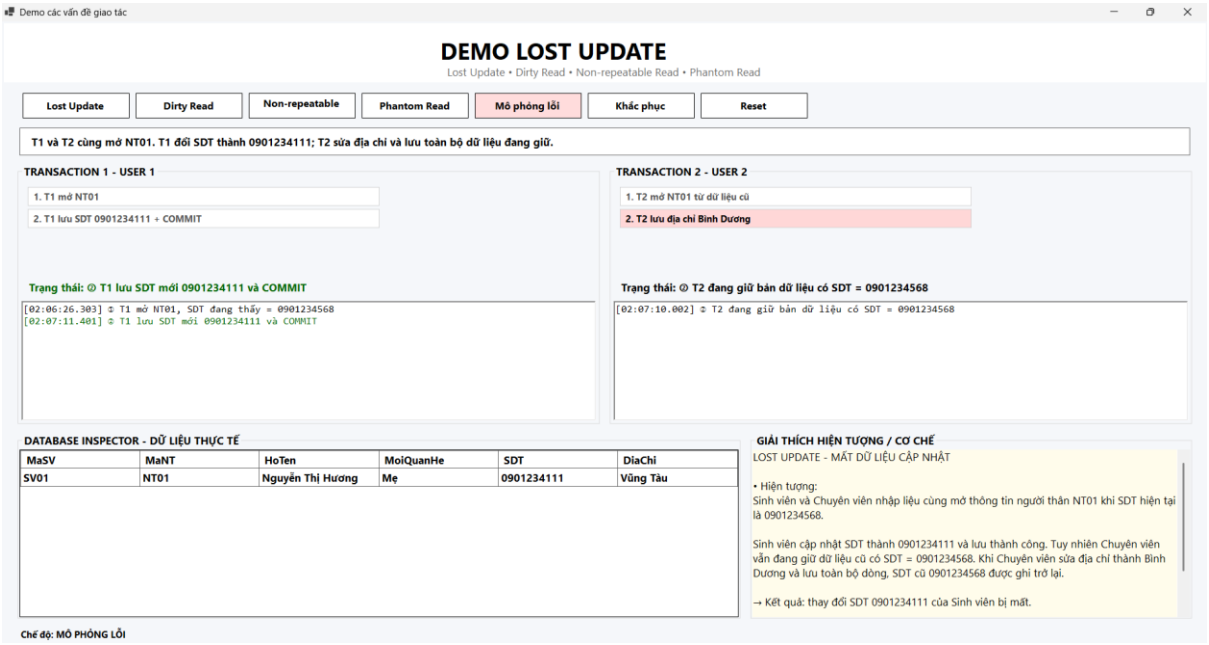
Database lúc này đã ghi nhận:

SDT = 0901234111
Địa chỉ = Vũng Tàu

Tuy nhiên, Transaction 2 vẫn đang giữ dữ liệu cũ:

SDT = 0901234568
Địa chỉ = Vũng Tàu

Do đó, thay đổi của T1 tuy đã được lưu thành công nhưng vẫn có nguy cơ bị ghi đè.



Bước 4 – Transaction 2 cập nhật địa chỉ

Transaction 2 thay đổi địa chỉ:

Vũng Tàu → Bình Dương

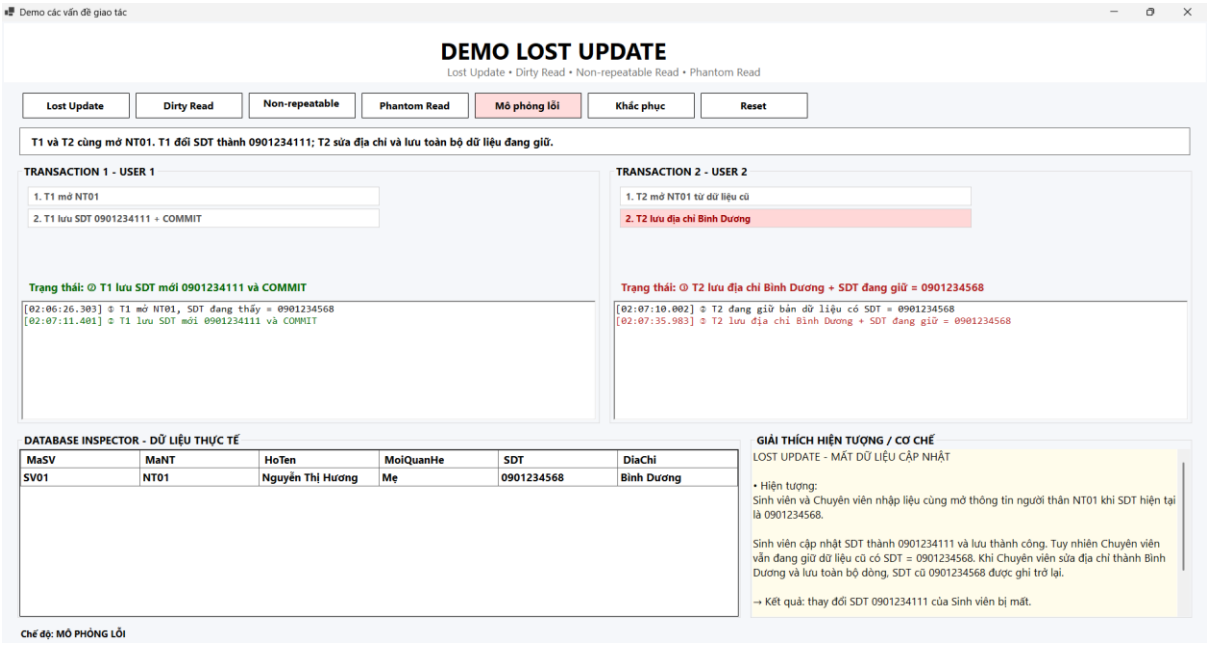
Tuy nhiên, T2 vẫn giữ SDT cũ là **0901234568**. Khi lưu toàn bộ dữ liệu, T2 ghi xuống Database:

SDT = 0901234568
Địa chỉ = Bình Dương

Trong khi trước đó Database đang có:

SDT = 0901234111
Địa chỉ = Vũng Tàu

→ SDT **0901234111** do T1 cập nhật đã bị T2 ghi đè. Đây chính là thời điểm xảy ra **Lost Update**.



Kết quả mô phỏng:

Sau khi hoàn thành các bước, Database có kết quả:

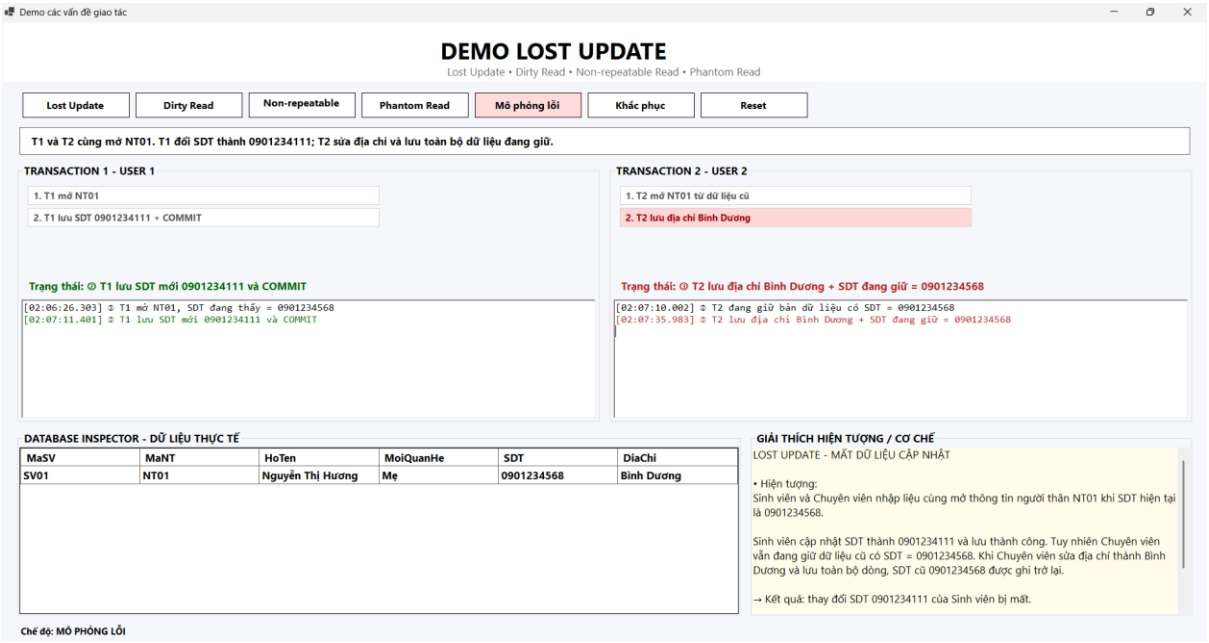
SDT = 0901234568

Địa chỉ = Bình Dương

Trong khi T1 trước đó đã cập nhật:

SDT = 0901234111

Như vậy, dữ liệu cập nhật của T1 đã bị T2 ghi đè bởi dữ liệu cũ. Kết quả này chứng minh tình trạng **Lost Update – mất dữ liệu cập nhật** khi các Transaction cùng thao tác trên một dữ liệu mà không có cơ chế kiểm soát tương tranh phù hợp.



2. Xử lý trình tự thực hiện các Transaction:

Sau khi người dùng thao tác trên giao diện, chương trình cần kiểm soát đúng thứ tự thực hiện giữa Transaction 1 và Transaction 2. Phần xử lý được thực hiện trong phương thức XuLyLUAsync(). Phương thức nhận tham số tag để xác định người dùng đang thực hiện bước nào, từ đó cập nhật trạng thái Transaction và điều khiển các nút trên giao diện.

Quá trình được xây dựng theo trình tự:

T1 mở dữ liệu → T2 mở dữ liệu → T1 cập nhật và COMMIT → T2 cập nhật.

Cách xử lý này giúp chương trình mô phỏng đúng điều kiện xảy ra Lost Update.

a. Xử lý Transaction 1 mở dữ liệu:

Khi người dùng thực hiện bước T1-1, chương trình gọi SetStepAsync() để ghi nhận Transaction 1 đã hoàn thành bước đầu tiên.

```
1317        private async Task XuLyLUAsync(
1318            string tag)
1319        {
1320            if (tag == "T1-1")
1321            {
1322                await bus.SetStepAsync(
1323                    runId, "T1", 1);
1324
1325                t1Buttons[0].Enabled = false;
1326                t2Buttons[0].Enabled = true;
1327
1328                await Task.Delay(250);
1329            }
```

Trong đoạn code trên, runId dùng để xác định lần chạy demo hiện tại, "T1" xác định Transaction 1 và 1 là bước thực hiện.

Sau khi T1 hoàn thành bước mở dữ liệu, nút của T1 được vô hiệu hóa bằng Enabled = false, đồng thời nút bước đầu tiên của T2 được bật lên. Điều này đảm bảo người dùng thực hiện đúng trình tự của kịch bản.

Task.Delay(250) tạo một khoảng trễ nhỏ để giao diện có thời gian cập nhật trạng thái trước khi chuyển sang bước tiếp theo.

b. Xử lý Transaction 2 mở dữ liệu:

Khi người dùng thực hiện T2-1, chương trình ghi nhận Transaction 2 đã mở dữ liệu.

```
1330        else if (tag == "T2-1")
1331        {
1332            t2Buttons[0].Enabled = false;
1333
1334            await bus.SetStepAsync(
1335                runId, "T2", 1);
1336
1337            // Ở FIX, T2 sẽ block ở UPDLOCK.
1338            // Phải cho phép T1 lưu để giải phóng khóa.
1339            t1Buttons[1].Enabled = true;
1340
1341            if (mode == "ERROR")
1342            {
1343                await Task.Delay(300);
1344            }
1345        }
```

Tương tự T1, nút thao tác của T2 được khóa sau khi thực hiện. Sau đó chương trình gọi SetStepAsync() để cập nhật trạng thái T2 = 1.

Đặc biệt, sau khi T2 đã mở dữ liệu, chương trình cho phép T1 thực hiện bước cập nhật bằng cách bật t1Buttons[1].

- Ở chế độ **ERROR**, T2 đọc dữ liệu mà không sử dụng khóa cập nhật. Đây là điều kiện để T1 và T2 có thể cùng giữ dữ liệu cũ và tạo ra tình huống Lost Update.

- Ở chế độ **FIX**, Transaction 2 được xử lý với cơ chế UPDLOCK ở phía SQL Server. Khi T1 đang giữ khóa, T2 sẽ phải chờ cho đến khi T1 hoàn thành và giải phóng khóa.

c. Xử lý Transaction 1 cập nhật và COMMIT

Khi T1 thực hiện bước T1-2, chương trình ghi nhận bước cập nhật của Transaction 1.

```
1346            else if (tag == "T1-2")
1347            {
1348                await bus.SetStepAsync(
1349                    runId, "T1", 2);
1350
1351                t1Buttons[1].Enabled = false;
1352
1353                if (mode == "ERROR")
1354                {
1355                    t2Buttons[1].Enabled = true;
1356                }
1357                else
1358                {
1359                    // Cho T2 thời gian nhận khóa và đọc dữ liệu mới.
1360                    await Task.Delay(450);
1361                    t2Buttons[1].Enabled = true;
1362                }
1363            }
```

Sau khi trạng thái T1 được cập nhật thành bước 2, nút thao tác của T1 được vô hiệu hóa.

Trong chế độ **ERROR**, chương trình cho phép T2 tiếp tục ngay sau khi T1 hoàn thành. Khi đó T2 vẫn có thể sử dụng dữ liệu cũ đã đọc trước đó và ghi đè lên dữ liệu mới của T1.

Trong chế độ **FIX**, chương trình chờ một khoảng thời gian trước khi cho phép T2 tiếp tục. Khoảng chờ này chỉ hỗ trợ việc mô phỏng trên giao diện; cơ chế khóa thực tế được thực hiện bởi SQL Server thông qua UPDLOCK.

d. Xử lý Transaction 2 cập nhật

Sau khi T1 hoàn thành, T2 thực hiện bước cuối T2-2.

```
1364            else if (tag == "T2-2")
1365            {
1366                await bus.SetStepAsync(
1367                    runId, "T2", 2);
1368
1369                t2Buttons[1].Enabled = false;
1370            }
1371        }
```

Chương trình cập nhật trạng thái Transaction 2 thành bước 2 và khóa nút thao tác sau khi hoàn thành.

Ở chế độ **ERROR**, bước này dẫn đến việc T2 ghi dữ liệu cũ xuống Database, từ đó có thể ghi đè lên thay đổi mà T1 đã thực hiện.

Ở chế độ **FIX**, T2 chỉ tiếp tục sau khi khóa của T1 được giải phóng và có thể đọc dữ liệu mới nhất. Vì vậy, khi T2 thay đổi địa chỉ, giá trị SDT mới của T1 vẫn được giữ lại.

e. BUS tiếp nhận yêu cầu từ giao diện

Sau khi giao diện xác định bước thực hiện, yêu cầu được chuyển xuống tầng BUS thông qua phương thức SetStepAsync().

```
16 references
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return dal.SetStepAsync(
        runId,
        giaoTac,
        step,
        action);
}
```

BUS không trực tiếp xử lý dữ liệu trong Database mà đóng vai trò trung gian giữa giao diện và DAL. Các thông tin gồm runId, giaoTac, step và action được truyền xuống DAL để thực hiện.

Cách tổ chức này giúp tách phần xử lý giao diện khỏi phần truy cập Database, đồng thời giữ đúng kiến trúc phân lớp của chương trình.

g. DAL gọi Stored Procedure

Tại tầng DAL, chương trình sử dụng ExecuteProcedureAsync() để gọi Stored Procedure sp_Demo_SetStep.

```
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return ExecuteProcedureAsync(
        "sp_Demo_SetStep",
        p =>
        {
            p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
            p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
            p.Add("@Step", SqlDbType.Int).Value = step;

            SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
            pa.Value = string.IsNullOrWhiteSpace(action)
                ? DBNull.Value
                : action;
        });
}
```

Đoạn code trên truyền các tham số từ chương trình C# xuống SQL Server:

- @RunId: xác định phiên demo.
- @GiaoTac: xác định Transaction T1 hoặc T2.
- @Step: xác định bước hiện tại.
- @Action: thông tin hành động bổ sung nếu có.

Trong trường hợp không truyền action, chương trình sử dụng DBNull.Value để biểu thị giá trị rỗng trong Database.

h. Luồng xử lý tổng quát

Như vậy, khi người dùng nhấn từng nút trên giao diện, chương trình xử lý theo mô hình:

Giao diện → XuLyLUAsync() → BUS → DAL → sp_Demo_SetStep → SQL Server

XuLyLUAsync() chịu trách nhiệm kiểm soát trình tự thao tác và giao diện. BUS tiếp nhận yêu cầu từ GUI, DAL thực hiện gọi Stored Procedure, còn SQL Server lưu trạng thái bước thực hiện của từng Transaction.

Việc chia thành các tầng giúp chương trình dễ quản lý và phù hợp với kiến trúc của hệ thống. Đồng thời, việc điều khiển trạng thái T1 và T2 theo từng bước giúp mô phỏng rõ ràng quá trình tương tranh dẫn đến **Lost Update** và quá trình khắc phục bằng **UPDLOCK**.

i. Xử lý Lost Update trên SQL Server

Để mô phỏng hiện tượng **Lost Update**, chương trình sử dụng hai Stored Procedure tương ứng với hai Transaction là sp_LU_T1_Runner và sp_LU_T2_Runner. Hai giao tác cùng thao tác trên bản ghi người thân NT01.

Dữ liệu được sử dụng trong kịch bản là:

Thuộc tính	Giá trị ban đầu
Mã người thân	NT01
Số điện thoại	0901234568
Địa chỉ	Vũng Tàu

- Trong chế độ **ERROR**, cả T1 và T2 đều đọc dữ liệu mà không sử dụng khóa cập nhật. T1 sau đó thay đổi SDT thành 0901234111, trong khi T2 vẫn giữ SDT cũ 0901234568. Khi T2 lưu toàn bộ dữ liệu, giá trị cũ sẽ ghi đè lên giá trị mới của T1.

- Trong chế độ **FIX**, SQL Server sử dụng UPDLOCK ngay từ bước đọc dữ liệu để đảm bảo chỉ một Transaction được quyền giữ khóa cập nhật trên bản ghi đang thao tác.

3. Khởi tạo dữ liệu Lost Update

Trước khi bắt đầu demo, Stored Procedure sp_Demo_Init đưa dữ liệu NT01 về trạng thái ban đầu.

```
549         IF @Scenario = 'LU'
550         BEGIN
551             UPDATE dbo.NguoiThan
552             SET SDT = '0901234568',
553                 DiaChi = N'Vũng Tàu'
554             WHERE MaNT = 'NT01';
555         END;
```

Đoạn xử lý trên đảm bảo mỗi lần chạy demo, dữ liệu đều được đưa về cùng một trạng thái:

SDT = 0901234568

DiaChi = Vũng Tàu

Sau đó, chương trình tạo một bản ghi trong DemoControl để lưu trạng thái của phiên demo.

```
563         INSERT INTO dbo.DemoControl
564         (
565             RunId, Scenario, Mode,
566             T1Step, T2Step,
567             T1Action, T2Action, Cancelled
568         )
569         VALUES
570         (
571             @RunId, @Scenario, @Mode,
572             0, 0, NULL, NULL, 0
573         );
574     END;
575     GO
```

T1Step và T2Step được sử dụng để đồng bộ thao tác giữa hai Transaction. Khi người dùng nhấn từng nút trên giao diện, C# sẽ cập nhật trạng thái này thông qua sp_Demo_SetStep.

a. Transaction 1 mở dữ liệu NT01

Stored Procedure sp_LU_T1_Runner trước tiên chờ giao diện báo rằng T1 đã thực hiện bước mở dữ liệu.

```
/* Chờ nút 1 */
WHILE 1 = 1
BEGIN
    SELECT @Step = T1Step, @Cancelled = Cancelled
    FROM dbo.DemoControl WITH (NOLOCK)
    WHERE RunId = @RunId;

    IF @Cancelled = 1 RETURN;
    IF @Step >= 1 BREAK;
    WAITFOR DELAY '00:00:00.150';
END;
```

Sau khi nhận được trạng thái bước 1, T1 bắt đầu Transaction.

BEGIN TRANSACTION;

Tùy theo chế độ chạy, cách đọc dữ liệu sẽ khác nhau.

Trong chế độ **ERROR**, T1 đọc dữ liệu thông thường:

```
ELSE
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan
    WHERE MaNT = 'NT01';
END;
```

Sau khi đọc, T1 ghi log:

```
SET @Msg =
    N'① T1 mở NT01, SDT đang thấy = ' +
    ISNULL(@OldSDT, 'NULL');

EXEC dbo.sp_Demo_Log
    @RunId, 'T1', 'INFO', @Msg;
```

Kết quả T1 nhìn thấy:

SDT = 0901234568

Địa chỉ = Vũng Tàu

b. Transaction 1 cập nhật SDT và COMMIT

Sau khi người dùng nhấn bước thứ hai, T1 tiếp tục thực hiện cập nhật.

```
991         UPDATE dbo.NguoiThan
992         SET SDT = '0901234111'
993         WHERE MaNT = 'NT01';
994
995         COMMIT TRANSACTION;
```

Tại thời điểm này, thay đổi của T1 đã được lưu chính thức vào Database.

SDT = 0901234111

Địa chỉ = Vũng Tàu

Chương trình ghi nhận kết quả bằng sp_Demo_Log:

```
997         EXEC dbo.sp_Demo_Log @RunId, 'T1', 'SUCCESS',
998             N'② T1 lưu SDT mới 0901234111 và COMMIT';
999     END;
000     GO
```

Trong chế độ ERROR, vấn đề nằm ở chỗ T2 đã có thể đọc và giữ dữ liệu cũ trước khi T1 COMMIT.

c. Transaction 2 mở dữ liệu

Transaction 2 được xử lý trong Stored Procedure sp_LU_T2_Runner.

Tương tự T1, T2 chờ giao diện cho phép thực hiện bước mở dữ liệu:

```
1017        WHILE 1 = 1
1018        BEGIN
1019            SELECT @Step = T2Step, @Cancelled = Cancelled
1020            FROM dbo.DemoControl WITH (NOLOCK)
1021            WHERE RunId = @RunId;
1022
1023            IF @Cancelled = 1 RETURN;
1024            IF @Step >= 1 BREAK;
1025            WAITFOR DELAY '00:00:00.150';
1026        END;
```

Sau đó T2 bắt đầu Transaction:

BEGIN TRANSACTION;

- Trường hợp xảy ra lỗi:

Ở chế độ ERROR, T2 đọc dữ liệu bình thường:

```
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan
    WHERE MaNT = 'NT01';
END;
```

T2 giữ lại giá trị:

- SDT = 0901234568
- Địa chỉ = Vũng Tàu
- Trong khi Database sau khi T1 COMMIT đã có:
- SDT = 0901234111
- Địa chỉ = Vũng Tàu

Đây chính là dữ liệu cũ mà T2 đang giữ.

d. Transaction 2 cập nhật địa chỉ và gây Lost Update

Sau khi người dùng nhấn nút lưu của T2, chương trình thực hiện:

```
/* Mô phỏng form lưu toàn bộ dòng từ dữ liệu đã mở trước đó */
UPDATE dbo.NguoiThan
SET SDT = @OldSDT,
    DiaChi = N'Bình Dương'
WHERE MaNT = 'NT01';

COMMIT TRANSACTION;
```

Điểm quan trọng của đoạn code trên là @OldSDT chính là giá trị mà T2 đã đọc trước đó.

Trong chế độ ERROR:

@OldSDT = 0901234568

Do đó câu lệnh thực tế tương đương:

UPDATE dbo.NguoiThan

SET SDT = '0901234568',

DiaChi = N'Bình Dương'

WHERE MaNT = 'NT01';

Kết quả cuối cùng:

- SDT = 0901234568
- Địa chỉ = Bình Dương

Trong khi T1 trước đó đã cập nhật:

SDT = 0901234111

→ Giá trị 0901234111 đã bị T2 ghi đè bởi 0901234568.

Chương trình xác định kết quả bằng điều kiện:

```
SET @LogType =
    CASE WHEN @Lan1 <> @Lan2
         THEN 'DANGER'
         ELSE 'SUCCESS'
    END;
```

Sau đó ghi lại nội dung thao tác:

```
SET @Msg =
    N'① T1 mở NT01, SDT đang thấy = ' +
    ISNULL(@OldSDT, 'NULL');

EXEC dbo.sp_Demo_Log
    @RunId, 'T1', 'INFO', @Msg;
```

Nhờ đó giao diện có thể lấy log từ Database và hiển thị cho người dùng biết Transaction nào đã gây ra lỗi.

3.2.2. Khắc Phục Lost Update

Phương pháp khắc phục:

Để khắc phục hiện tượng Lost Update, hệ thống sử dụng cơ chế khóa cập nhật của SQL Server thông qua UPDLOCK. Mục đích của phương pháp này là đảm bảo tại thời điểm một giao tác đang đọc dữ liệu để chuẩn bị cập nhật, giao tác khác không thể đồng thời lấy cùng dữ liệu cũ để thực hiện cập nhật.

Trong tình huống mô phỏng, hai giao tác T1 và T2 cùng thao tác trên bản ghi NT01 của bảng NguoiThan. T1 thực hiện thay đổi số điện thoại từ 0901234568 thành 0901234111, trong khi T2 thay đổi địa chỉ thành Bình Dương.

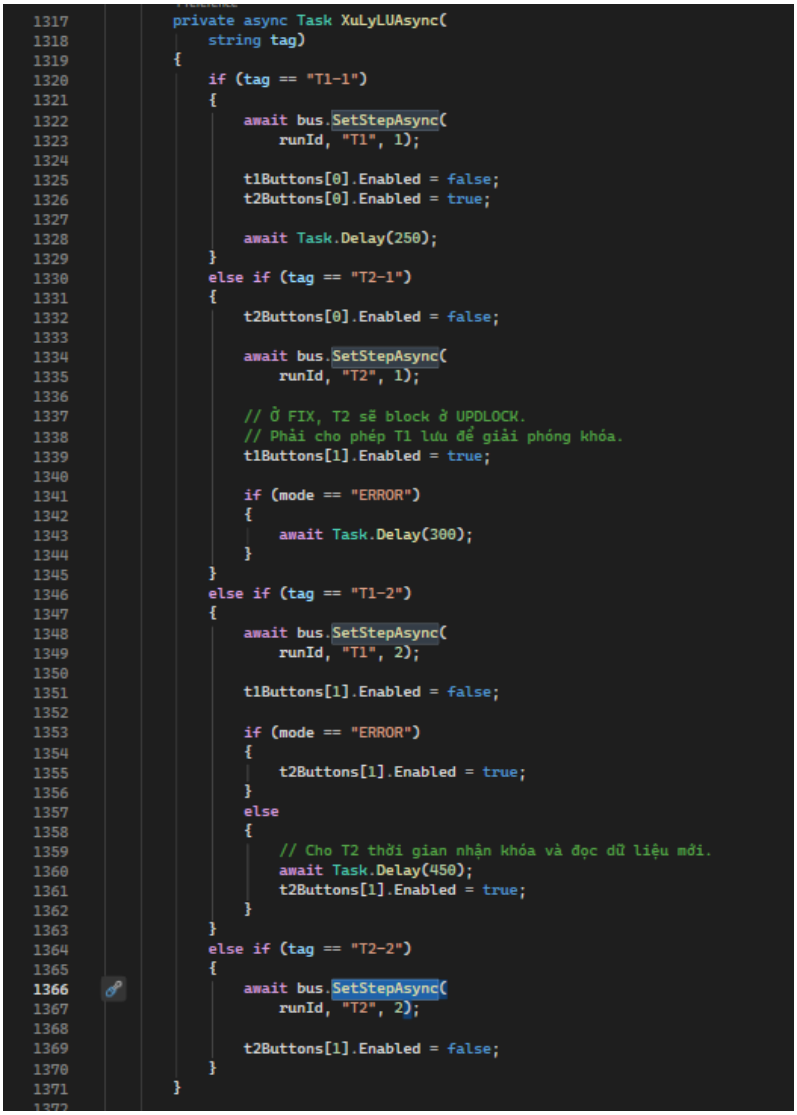
Ở chế độ khắc phục FIX, T1 và T2 đều thực hiện thao tác đọc dữ liệu bằng UPDLOCK. Khi T1 đã lấy khóa cập nhật trên bản ghi NT01, T2 sẽ phải chờ cho đến khi

T1 hoàn thành giao tác và thực hiện COMMIT. Sau đó T2 mới được tiếp tục đọc dữ liệu và nhận được giá trị SDT mới nhất do T1 cập nhật.

Nhờ đó, T2 không sử dụng lại dữ liệu cũ để ghi đè lên kết quả của T1, từ đó loại bỏ hiện tượng Lost Update.

a. Xử lý tại giao diện C#

Tại tầng giao diện, chương trình sử dụng hàm XuLyLUAsync() để điều khiển thứ tự thực hiện giữa hai giao tác. Giao diện không trực tiếp xử lý khóa dữ liệu mà chỉ gửi trạng thái của từng giao tác xuống tầng BUS và DAL.



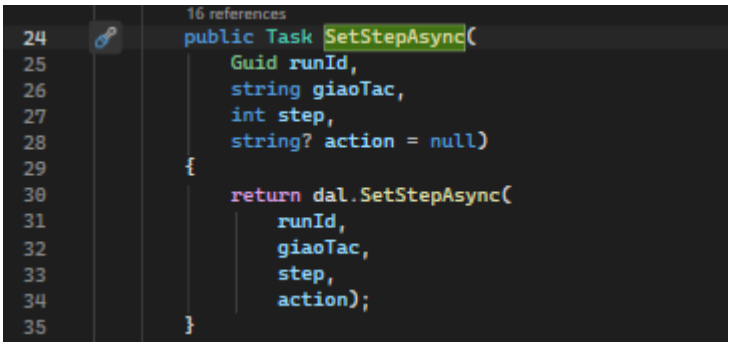
```
1317        private async Task XuLyLUAsync(
1318            string tag)
1319        {
1320            if (tag == "T1-1")
1321            {
1322                await bus.SetStepAsync(
1323                    runId, "T1", 1);
1324
1325                t1Buttons[0].Enabled = false;
1326                t2Buttons[0].Enabled = true;
1327
1328                await Task.Delay(250);
1329            }
1330            else if (tag == "T2-1")
1331            {
1332                t2Buttons[0].Enabled = false;
1333
1334                await bus.SetStepAsync(
1335                    runId, "T2", 1);
1336
1337                // Ở FIX, T2 sẽ block ở UPDLOCK.
1338                // Phải cho phép T1 lưu để giải phóng khóa.
1339                t1Buttons[1].Enabled = true;
1340
1341                if (mode == "ERROR")
1342                {
1343                    await Task.Delay(300);
1344                }
1345            }
1346            else if (tag == "T1-2")
1347            {
1348                await bus.SetStepAsync(
1349                    runId, "T1", 2);
1350
1351                t1Buttons[1].Enabled = false;
1352
1353                if (mode == "ERROR")
1354                {
1355                    t2Buttons[1].Enabled = true;
1356                }
1357                else
1358                {
1359                    // Cho T2 thời gian nhận khóa và đọc dữ liệu mới.
1360                    await Task.Delay(450);
1361                    t2Buttons[1].Enabled = true;
1362                }
1363            }
1364            else if (tag == "T2-2")
1365            {
1366                await bus.SetStepAsync(
1367                    runId, "T2", 2);
1368
1369                t2Buttons[1].Enabled = false;
1370            }
1371        }
1372
```

Hàm trên điều khiển bốn bước chính của tình huống Lost Update. T1 và T2 lần lượt mở dữ liệu, sau đó T1 thực hiện cập nhật và COMMIT, cuối cùng T2 thực hiện cập nhật địa chỉ.

Trong chế độ FIX, giao diện cho phép T2 tiếp tục sau một khoảng thời gian ngắn để việc mô phỏng dễ quan sát. Tuy nhiên, khoảng thời gian này chỉ phục vụ việc điều khiển giao diện, không phải cơ chế đảm bảo tính đúng đắn của dữ liệu. Cơ chế khắc phục thực sự được thực hiện tại SQL Server bằng UPDLOCK.

b. Xử lý tại tầng BUS và DAL

Sau khi giao diện xác định bước thực hiện của giao tác, thông tin được truyền qua tầng BUS:



Tầng BUS đóng vai trò trung gian, chuyển thông tin từ giao diện xuống DAL.

Tại DAL, chương trình gọi Stored Procedure sp_Demo_SetStep:

public Task SetStepAsync(

```
1 reference
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return ExecuteProcedureAsync(
        "sp_Demo_SetStep",
        p =>
        {
            p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
            p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
            p.Add("@Step", SqlDbType.Int).Value = step;

            SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
            pa.Value = string.IsNullOrWhiteSpace(action)
                ? DBNull.Value
                : action;
        });
}
```

Như vậy, quá trình xử lý được thực hiện theo mô hình:

WinForms - > XuLyLUAsync() - > BUS-> DAL - > sp_Demo_SetStep - > DemoControl

Cách tổ chức này giúp giao diện chỉ đảm nhận việc điều khiển mô phỏng, trong khi việc xử lý giao tác và khóa dữ liệu được thực hiện tại SQL Server.

c. Xử lý khóa cập nhật trong SQL Server

Cơ chế khắc phục chính được cài đặt trong các Stored Procedure thực hiện giao tác.

Đối với T1, khi chạy chế độ FIX, chương trình sử dụng:

```
IF @Mode = 'FIX'
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan WITH (UPDLOCK, ROWLOCK)
    WHERE MaNT = 'NT01';
END
ELSE
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan
    WHERE MaNT = 'NT01';
END;
```

UPDLOCK yêu cầu SQL Server đặt khóa cập nhật trên bản ghi được đọc. Vì thao tác này nằm trong một giao tác bắt đầu bằng BEGIN TRANSACTION, khóa được giữ cho đến khi giao tác kết thúc bằng COMMIT hoặc ROLLBACK.

Sau khi lấy được khóa, T1 thực hiện cập nhật:

```
991         UPDATE dbo.NguoiThan
992         SET SDT = '0901234111'
993         WHERE MaNT = 'NT01';
994
995         COMMIT TRANSACTION;
```

T1 thay đổi SDT thành 0901234111 và thực hiện COMMIT. Khi đó khóa được giải phóng và T2 mới có thể tiếp tục thực hiện thao tác cập nhật.

d. T2 chờ khóa và đọc dữ liệu mới

Đối với T2, chế độ FIX cũng sử dụng UPDLOCK:

```
IF @Mode = 'FIX'
BEGIN
    EXEC dbo.sp_Demo_Log @RunId, 'T2', 'WAIT',
        N'① T2 mở NT01 bằng UPDLOCK - sẽ chờ nếu T1 đang giữ khóa';

    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan WITH (UPDLOCK, ROWLOCK)
    WHERE MaNT = 'NT01';
END
ELSE
BEGIN
    SELECT @OldSDT = SDT, @OldDiaChi = DiaChi
    FROM dbo.NguoiThan
    WHERE MaNT = 'NT01';
END;
```

Nếu T1 vẫn đang giữ khóa trên NT01, câu lệnh SELECT của T2 sẽ bị chặn và phải chờ. Sau khi T1 thực hiện COMMIT, khóa được giải phóng và T2 tiếp tục thực hiện câu lệnh SELECT.

Lúc này, T2 đọc được giá trị SDT mới nhất:

- SDT = 0901234111
- thay vì giá trị cũ:
- SDT = 0901234568

Đây là điểm quan trọng giúp ngăn chặn Lost Update.

e. T2 cập nhật dữ liệu

Sau khi lấy được dữ liệu mới nhất, T2 thực hiện cập nhật địa chỉ:

```
/* Mô phỏng form lưu toàn bộ dòng từ dữ liệu đã mở trước đó */
UPDATE dbo.NguoiThan
SET SDT = @OldSDT,
    DiaChi = N'Bình Dương'
WHERE MaNT = 'NT01';

COMMIT TRANSACTION;
```

Trong chế độ FIX, giá trị @OldSDT lúc này đã được T2 đọc lại sau khi T1 COMMIT, do đó:

@OldSDT = 0901234111

T2 sẽ tương đương với việc thực hiện:

```
UPDATE dbo.NguoiThan
SET SDT = '0901234111'
WHERE MaNT = 'NT01';

COMMIT TRANSACTION;
```

Như vậy, T2 vẫn cập nhật được địa chỉ nhưng không làm mất thay đổi SDT của T1.

Kết quả sau khi khắc phục

Sau khi hoàn thành cả hai giao tác, hệ thống truy vấn lại bản ghi NT01:

SELECT

MaNT,

HoTen,

SDT,

DiaChi

FROM dbo.NguoiThan

WHERE MaNT = 'NT01';

Kết quả thu được:

Thuộc tính	Giá trị
MaNT	NT01
SDT	0901234111
DiaChi	Bình Dương

Kết quả cho thấy cả hai thay đổi đều được giữ lại. T1 đã cập nhật SDT thành 0901234111, sau đó T2 cập nhật địa chỉ thành Bình Dương nhưng vẫn giữ nguyên SDT mới của T1.

Có thể mô tả kết quả như sau:

T1:

SDT: 0901234568 → 0901234111 -> COMMIT -> Giải phóng khóa

T2:

Đọc SDT mới = 0901234111 -> Đổi địa chỉ = Bình Dương -> COMMIT

Do đó, hiện tượng Lost Update không còn xảy ra.

- **Đánh giá phương pháp khắc phục:**

Phương pháp sử dụng UPDLOCK giúp đảm bảo giao tác đang chuẩn bị cập nhật dữ liệu có quyền kiểm soát bản ghi trước khi giao tác khác có thể thực hiện thao tác tương tự. Khi T1 đang giữ khóa, T2 phải chờ thay vì tiếp tục đọc và sử dụng dữ liệu cũ.

Trong trường hợp mô phỏng, nguyên nhân gây Lost Update là T2 đọc dữ liệu cũ trước khi T1 hoàn thành cập nhật. Khi T2 lưu toàn bộ dòng dữ liệu, giá trị SDT cũ bị ghi đè lên giá trị mới của T1.

Sau khi áp dụng UPDLOCK, T2 không còn đọc được bản dữ liệu cũ trong quá trình cập nhật. T2 chỉ tiếp tục sau khi T1 hoàn thành giao tác và giải phóng khóa. Vì vậy, kết quả cuối cùng bảo toàn được cả thay đổi của T1 và T2.

Có thể tổng kết:

Nội dung	Trước khắc phục	Sau khắc phục
Cách đọc dữ liệu	Đọc bình thường	UPDLOCK
T2 khi T1 đang giữ khóa	Tiếp tục đọc	Chờ
T2 có dữ liệu cũ	Có	Không
SDT cuối cùng	0901234568	0901234111
Địa chỉ cuối cùng	Bình Dương	Bình Dương
Lost Update	Xảy ra	Không xảy ra

Như vậy, việc sử dụng transaction kết hợp với UPDLOCK tại SQL Server đã giải quyết được hiện tượng Lost Update trong tình huống mô phỏng.

3.3. Đọc dữ liệu rác (Dirty Read)

Đọc dữ liệu rác xảy ra khi một giao tác đọc dữ liệu mà giao tác khác đã thay đổi nhưng chưa COMMIT. Nếu giao tác thực hiện thay đổi sau đó bị ROLLBACK, dữ liệu mà giao tác đầu tiên đã đọc trở thành dữ liệu không hợp lệ.

Ví dụ:

- T1 cập nhật số dư tài khoản từ 10.000.000 thành 5.000.000 đồng nhưng chưa COMMIT.
- T2 đọc số dư và nhận được giá trị 5.000.000 đồng.
- Sau đó T1 gặp lỗi và thực hiện ROLLBACK, số dư trở lại 10.000.000 đồng.
- T2 đã sử dụng giá trị 5.000.000 đồng mặc dù giá trị này chưa bao giờ được xác nhận chính thức.

→ T2 sử dụng dữ liệu chưa được xác nhận, có thể dẫn đến các phép tính hoặc quyết định sai.

3.3.1. Mô phỏng hiện tượng Dirty Read

Hiện tượng: Phòng Đào tạo cập nhật trạng thái của SV01 từ "Đang học" thành "Bảo lưu" nhưng Transaction 1 chưa COMMIT. Trong thời điểm đó, Transaction 2 sử dụng READ UNCOMMITTED và vẫn đọc được trạng thái "Bảo lưu". Sau đó Transaction 1 ROLLBACK nên dữ liệu thật của SV01 trở lại "Đang học". → Như vậy giá trị "Bảo lưu" mà T2 đã đọc là dữ liệu chưa được xác nhận, tức dữ liệu rác.

Nguyên nhân: READ UNCOMMITTED cho phép một giao tác đọc dữ liệu đang được giao tác khác thay đổi nhưng chưa COMMIT.

Bước 1 – Transaction 1 bắt đầu giao tác

Nội dung thực hiện

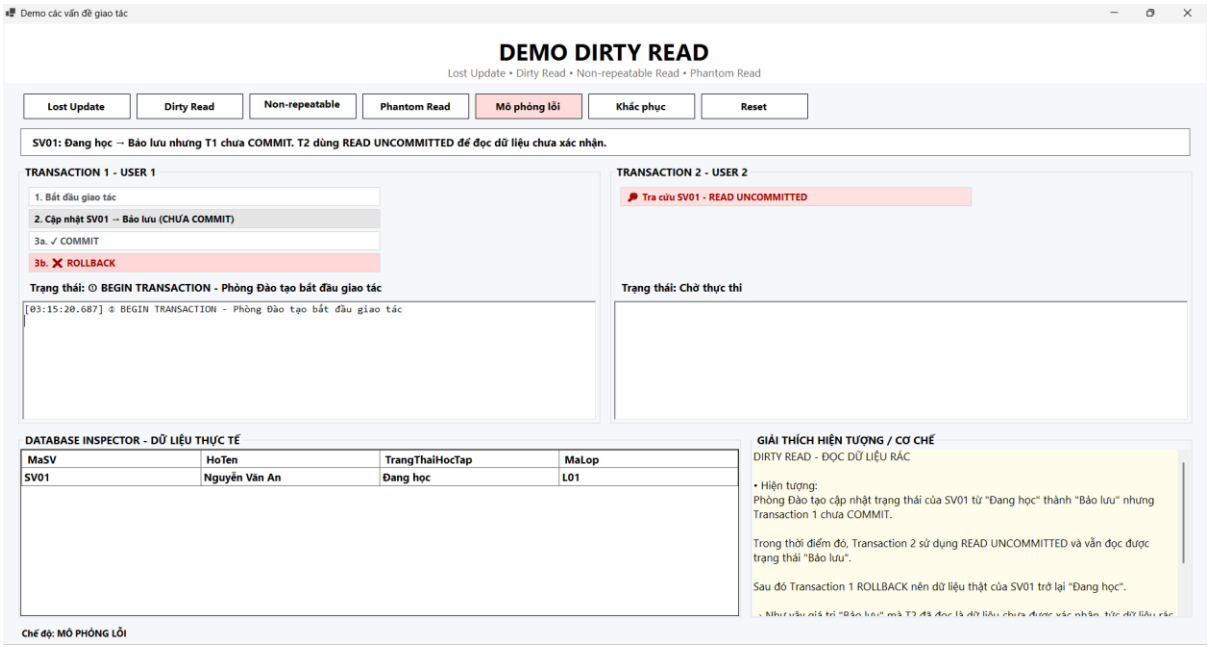
Ở bước đầu tiên, Transaction 1 thực hiện mở giao tác để chuẩn bị cập nhật trạng thái của sinh viên SV01. Tại thời điểm này, dữ liệu trong Database vẫn chưa thay đổi, trạng thái của SV01 vẫn là “Đang học”.

Trên giao diện, phần **TRANSACTION 1 – USER 1** hiển thị các bước:

- Bắt đầu giao tác
- Cập nhật SV01 → Bảo lưu (CHƯA COMMIT)
- 3a. COMMIT
- 3b. ROLLBACK

Trong đó, Bước 1 được thực hiện trước nhằm tạo giao tác T1. Sau bước này, T1 vẫn đang hoạt động và chưa thực hiện cập nhật dữ liệu.

Hình minh họa:



Code xử lý

a. GUI

Khi thực hiện Bước 1, sự kiện xử lý nút được gọi thông qua StepButton_Click():

```
1159        private async void StepButton_Click(
1160            object? sender,
1161            EventArgs e)
1162        {
1163            if (sender is not Button b ||
1164                b.Tag is not string tag)
1165                return;
1166
1167            try
1168            {
1169                if (scenario == "DR")
1170                    await XuLyDirtyAsync(tag);
1171                else if (scenario == "NRR")
1172                    await XuLyNRRAsync(tag);
1173                else if (scenario == "LU")
1174                    await XuLyLUAsync(tag);
1175                else
1176                    await XuLyPhantomAsync(tag);
1177
1178                await LoadInspectorAsync();
1179            }
1180            catch (Exception ex)
1181            {
1182                MessageBox.Show(
1183                    ex.Message,
1184                    "Lỗi thực thi",
1185                    MessageBoxButtons.OK,
1186                    MessageBoxIcon.Error);
1187            }
1188        }
1189
```

Vì kịch bản đang được chọn là Dirty Read (DR), chương trình chuyển sang:

await XuLyDirtyAsync(tag);

Sau đó, trong XuLyDirtyAsync(), khi tag là T1-1, chương trình gửi yêu cầu thiết lập Bước 1 cho Transaction 1:

```
1193            if (tag == "T1-1")
1194            {
1195                await bus.SetStepAsync(
1196                    runId, "T1", 1);
1197
1198                t1Buttons[0].Enabled = false;
1199                t1Buttons[1].Enabled = true;
1200            }
```

Ý nghĩa của đoạn code:

runId: xác định phiên mô phỏng hiện tại.

"T1": xác định giao tác đang được điều khiển là Transaction 1.

1: thông báo cho hệ thống rằng T1 đã thực hiện đến Bước 1.

Sau khi gửi yêu cầu thành công, nút Bước 1 được khóa và nút Bước 2 được mở để có thể thực hiện tiếp.

b. BUS

Từ FDemoGiaoTac.cs, lời gọi: bus.SetStepAsync(runId, "T1", 1); được chuyển sang lớp DemoGiaoTacBUS.cs.

Tại BUS, phương thức SetStepAsync() tiếp nhận các thông tin của bước hiện tại:

```
24      public Task SetStepAsync(
25          Guid runId,
26          string giaoTac,
27          int step,
28          string? action = null)
29      {
30          return dal.SetStepAsync(
31              runId,
32              giaoTac,
33              step,
34              action);
35      }
36
```

BUS không trực tiếp thao tác với Database mà tiếp tục chuyển yêu cầu xuống DAL.

c. DAL

Tại DemoGiaoTacDAL.cs, phương thức SetStepAsync() nhận yêu cầu từ BUS:

```
49      public Task SetStepAsync(
50          Guid runId,
51          string giaoTac,
52          int step,
53          string? action = null)
54      {
55          return ExecuteProcedureAsync(
56              "sp_Demo_SetStep",
57              p =>
58              {
59                  p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
60                  p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
61                  p.Add("@Step", SqlDbType.Int).Value = step;
62
63                  SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
64                  pa.Value = string.IsNullOrWhiteSpace(action)
65                      ? DBNull.Value
66                      : action;
67              });
68      }
```

Ở Bước 1, DAL truyền xuống Database các tham số:

@RunId = mã phiên hiện tại

@GiaoTac = "T1"

@Step = 1

DAL sử dụng:

```
        return ExecuteProcedureAsync(
            "sp_Demo_SetStep",
            p =>
            {
                p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
                p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
                p.Add("@Step", SqlDbType.Int).Value = step;

                SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
                pa.Value = string.IsNullOrWhiteSpace(action)
                    ? DBNull.Value
                    : action;
            });
    }
```

để gọi Stored Procedure sp_Demo_SetStep.

1.5 Phương thức thực hiện Stored Procedure

Trong SQL, Stored Procedure sp_Demo_SetStep được khai báo:

```
CREATE OR ALTER PROCEDURE dbo.sp_Demo_SetStep
    @RunId UNIQUEIDENTIFIER,
    @GiaoTac VARCHAR(5),
    @Step INT,
    @Action VARCHAR(20) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF @GiaoTac = 'T1'
    BEGIN
        UPDATE dbo.DemoControl
        SET T1Step = @Step,
            T1Action = COALESCE(@Action, T1Action)
        WHERE RunId = @RunId;
    END
    ELSE
    BEGIN
        UPDATE dbo.DemoControl
        SET T2Step = @Step,
            T2Action = COALESCE(@Action, T2Action)
        WHERE RunId = @RunId;
    END;
END;
GO
```

Khi @GiaoTac = 'T1', Stored Procedure thực hiện:

```
IF @GiaoTac = 'T1'
BEGIN
    UPDATE dbo.DemoControl
    SET T1Step = @Step,
        T1Action = COALESCE(@Action, T1Action)
    WHERE RunId = @RunId;
END
```

Tại Bước 1:

@GiaoTac = 'T1'

@Step = 1

nên Database cập nhật: DemoControl -> T1Step = 1

Giá trị này đóng vai trò như một tín hiệu điều khiển, cho biết Transaction 1 đã được phép thực hiện Bước 1.

1.6 Code hiển thị trên giao diện

Sau khi hoàn thành Bước 1, Transaction 1 đã được khởi động, nhưng dữ liệu của SV01 chưa bị thay đổi.

Database Inspector vẫn hiển thị:

MaSV	HoTen	TrangThaiHocTap	MaLop
SV01	Đặng Thành An	Đang học	L01

Điều này cho thấy Bước 1 chỉ khởi động Transaction 1, chưa phát sinh dữ liệu rác.

T1 lúc này đang ở trạng thái chờ thực hiện Bước 2 – Cập nhật SV01 từ “Đang học” sang “Bảo lưu” nhưng chưa COMMIT. Đây chính là điều kiện cần để ở bước tiếp theo Transaction 2 có thể sử dụng READ UNCOMMITTED và đọc được dữ liệu mà T1 chưa COMMIT.

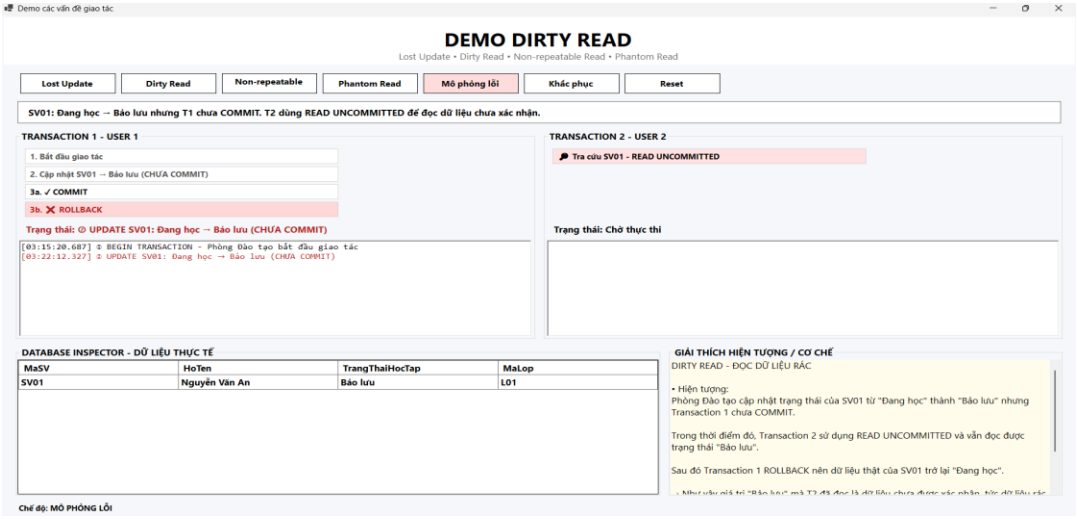
BƯỚC 2 – CẬP NHẬT SV01 THÀNH “BẢO LƯU” NHƯNG CHƯA COMMIT

Nội dung thực hiện

Sau khi Transaction 1 đã bắt đầu giao tác ở Bước 1, bước tiếp theo là cập nhật trạng thái học tập của sinh viên SV01 từ “Đang học” thành “Bảo lưu”. Tuy nhiên, điểm quan trọng của bước này là Transaction 1 chỉ cập nhật dữ liệu nhưng CHƯA COMMIT. Mục đích của việc chưa COMMIT là tạo ra một trạng thái dữ liệu chưa được xác nhận. Dữ liệu này sẽ được sử dụng ở bước tiếp theo để minh họa hiện tượng Dirty Read, khi Transaction 2 sử dụng mức cô lập READ UNCOMMITTED và đọc được dữ liệu mà Transaction 1 chưa COMMIT.

Cụ thể, trước khi cập nhật: **SV01 – Trạng thái học tập = Đang học**. Sau khi Transaction 1 thực hiện cập nhật: **SV01 – Trạng thái học tập = Bảo lưu**. Nhưng tại thời điểm này, **Transaction 1 CHƯA COMMIT**, do đó dữ liệu “Bảo lưu” chưa phải là dữ liệu cuối cùng được xác nhận trong giao tác.

Giao diện sau khi thực hiện Bước 2



Trong hình, tại khu vực **TRANSACTION 1 – USER 1**, bước 2 :Cập nhật SV01 → Bảo lưu (CHƯA COMMIT) đã được thực hiện.

Phần trạng thái hiển thị:

Trạng thái: ② Đang giữ giá trị mới = Bảo lưu (CHƯA COMMIT)

Đồng thời nhật ký của Transaction 1 xuất hiện dòng tương ứng với việc cập nhật dữ liệu. Trong khi đó, Transaction 2 vẫn đang ở trạng thái "Chờ thực thi". Điều này cho thấy Transaction 1 đã thay đổi dữ liệu nhưng Transaction 2 chưa đọc dữ liệu tại thời điểm này.

Code xử lý

a. GUI

Khi chọn nút: Cập nhật SV01 → Bảo lưu (CHƯA COMMIT)

sự kiện StepButton_Click() được gọi. Vì kịch bản hiện tại là Dirty Read:

```
if (scenario == "DR")
    await XuLyDirtyAsync(tag);
```

chương trình chuyển sang phương thức XuLyDirtyAsync(tag).

Tại đây, khi nhận được bước của Transaction 1, chương trình gửi tín hiệu:

```
await bus.SetStepAsync(
    runId, "T1", 1);
```

Trong đó:

runId: mã phiên mô phỏng.

"T1": Transaction 1.

2: Transaction 1 đang thực hiện Bước 2.

Sau khi gửi tín hiệu, giao diện chuyển sang trạng thái cho phép thực hiện bước tiếp theo:

```
else if (tag == "T1-2")
{
    await bus.SetStepAsync(
        runId, "T1", 2);

    t1Buttons[1].Enabled = false;
    t1Buttons[2].Enabled = true;
    t1Buttons[3].Enabled = true;
    t2Buttons[0].Enabled = true;

    await Task.Delay(250);
}
```

b. BUS

Từ FDemoGiaoTac.cs, yêu cầu được chuyển đến DemoGiaoTacBUS.cs thông qua:

```
24      public Task SetStepAsync(
25          Guid runId,
26          string giaoTac,
27          int step,
28          string? action = null)
29      {
30          return dal.SetStepAsync(
31              runId,
32              giaoTac,
33              step,
34              action);
35      }
```

BUS nhận thông tin:

runId → mã phiên

giaoTac → T1

step → 2

Sau đó chuyển tiếp xuống DAL bằng:

```
    return dal.SetStepAsync(
        runId,
        giaoTac,
        step,
        action);
}
```

c. DAL

Tại DemoGiaoTacDAL.cs, phương thức SetStepAsync() gọi Stored Procedure:

```
return ExecuteProcedureAsync(
    "sp_Demo_SetStep",
    p =>
    {
        p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
        p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
        p.Add("@Step", SqlDbType.Int).Value = step;

        SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
        pa.Value = string.IsNullOrWhiteSpace(action)
            ? DBNull.Value
            : action;
    });
```

Ở Bước 2, các tham số được truyền xuống là:

@RunId → mã phiên hiện tại

@GiaoTac → T1

@Step → 2

Sau đó DAL sử dụng ExecuteProcedureAsync() để thực hiện Stored Procedure trên SQL Server.

- Phương thức thực hiện Stored Procedure

Trong file Transaction.sql, Stored Procedure dbo.sp_Demo_SetStep xử lý tín hiệu của T1:

```
IF @GiaoTac = 'T1'
BEGIN
    UPDATE dbo.DemoControl
    SET T1Step = @Step,
        T1Action = COALESCE(@Action, T1Action)
    WHERE RunId = @RunId;
END
```

Với:

@GiaoTac = T1

@Step = 2

Database cập nhật: DemoControl -> T1Step = 2

Giá trị này được sử dụng để báo cho Runner của Transaction 1 rằng Bước 2 đã được cho phép thực hiện.

- Transaction 1 cập nhật SV01

Sau khi nhận được tín hiệu Bước 2, Runner của Transaction 1 tiếp tục thực hiện thao tác cập nhật.

Giá trị của SV01 được thay đổi: SV01 -> Đang học -> UPDATE -> Bảo lưu

Nhưng giao tác vẫn đang mở: BEGIN TRANSACTION -> UPDATE SV01 -> Bảo lưu -> CHƯA COMMIT

Trong Stored Procedure sp_DR_T1_Runner, phần cập nhật được thực hiện:

```
UPDATE dbo.SinhVien
SET TrangThaiHocTap = N'Bảo lưu'
WHERE MaSV = 'SV01';

EXEC dbo.sp_Demo_Log @RunId, 'T1', 'DANGER',
    N'② UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)';
```

Vì vậy, “Bảo lưu” là giá trị mới do T1 tạo ra nhưng chưa được xác nhận. Đây chính là dữ liệu mà Transaction 2 có thể đọc được nếu sử dụng mức cô lập READ UNCOMMITTED.

- Code thể hiện trên giao diện

Sau khi hoàn thành Bước 2, trạng thái của hệ thống là:

Thành phần	**Trạng thái**
Transaction 1	Đang thực hiện
SV01	Bảo lưu
Transaction 1 COMMIT	Chưa thực hiện
Transaction 2	Chưa đọc dữ liệu

Như vậy, Bước 2 chưa phải là thời điểm Transaction 2 đọc dữ liệu rác. Bước này chỉ tạo ra điều kiện để hiện tượng Dirty Read xảy ra ở bước tiếp theo.

BƯỚC 3 – TRANSACTION 2 THỰC HIỆN READ UNCOMMITTED ĐỂ ĐỌC DỮ LIỆU

Nội dung thực hiện

Sau khi Transaction 1 đã cập nhật trạng thái của sinh viên SV01 từ “Đang học” sang “Bảo lưu” nhưng chưa COMMIT, Transaction 2 thực hiện chức năng “Tra cứu SV01 - READ UNCOMMITTED”. Mục đích của bước này là kiểm tra xem Transaction 2 có thể đọc được dữ liệu mà Transaction 1 vừa thay đổi nhưng chưa được xác nhận hay không. Đây chính là bước tạo ra hiện tượng Dirty Read trong chế độ mô phỏng lỗi.

Giao diện sau khi thực hiện Bước 3

Sau khi thực hiện Bước 3, giao diện thể hiện rõ trạng thái của hai giao tác:

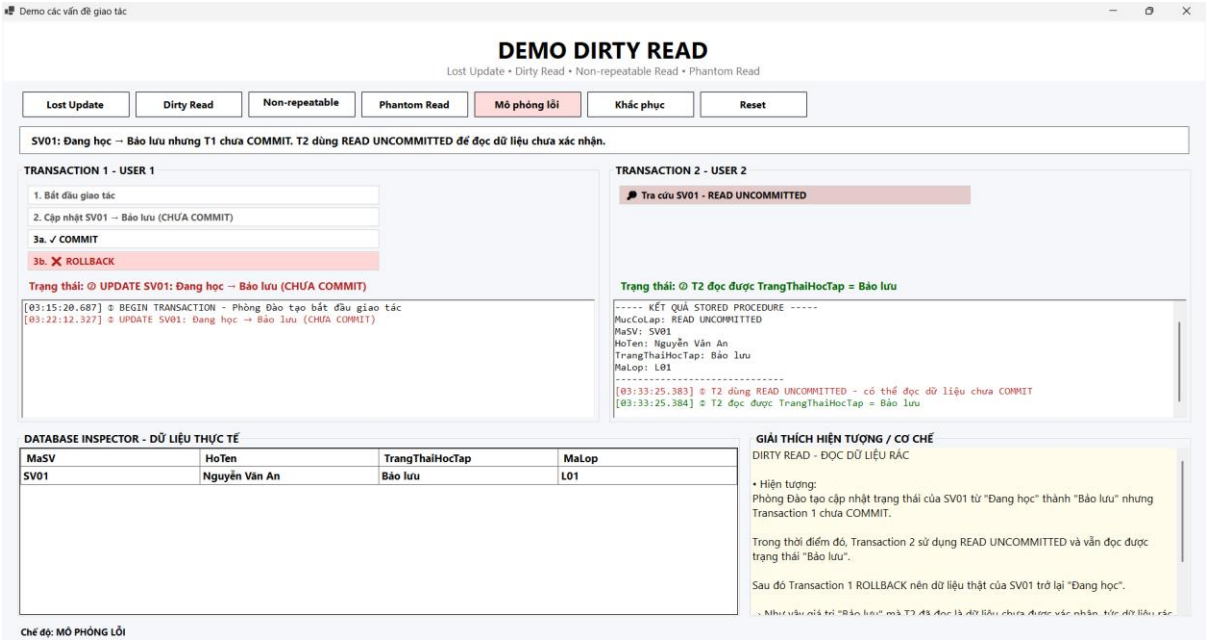
Transaction 1 – User 1 đang dừng tại bước "2. Cập nhật SV01 → Bảo lưu (CHƯA COMMIT)".

Dòng trạng thái của Transaction 1 cho biết "UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)".

Transaction 2 – User 2 thực hiện chức năng "Tra cứu SV01 - READ UNCOMMITTED".

Database Inspector hiển thị SV01 có trạng thái "Bảo lưu".

Điểm quan trọng là tại thời điểm này, Transaction 1 chưa COMMIT, nhưng Transaction 2 vẫn đọc được giá trị "Bảo lưu". Điều này chứng minh Transaction 2 đã đọc được dữ liệu chưa được xác nhận của Transaction 1, từ đó phát sinh hiện tượng Dirty Read.



Code xử lý

a. GUI

Khi người dùng thực hiện bước 3, sự kiện của nút tương ứng được xử lý trong file FDemoGiaoTac.cs, thông qua hàm StepButton_Click().

Đoạn code điều hướng đến xử lý Dirty Read:

```
1159        private async void StepButton_Click(
1160            object? sender,
1161            EventArgs e)
1162        {
1163            if (sender is not Button b ||
1164                b.Tag is not string tag)
1165                return;
1166
1167            try
1168            {
1169                if (scenario == "DR")
1170                    await XuLyDirtyAsync(tag);
1171                else if (scenario == "NRR")
1172                    await XuLyNRRAsync(tag);
1173                else if (scenario == "LU")
1174                    await XuLyLUAsync(tag);
1175                else
1176                    await XuLyPhantomAsync(tag);
1177
1178                await LoadInspectorAsync();
1179            }
1180            catch (Exception ex)
1181            {
1182                MessageBox.Show(
1183                    ex.Message,
1184                    "Lỗi thực thi",
1185                    MessageBoxButtons.OK,
1186                    MessageBoxIcon.Error);
1187            }
1188        }
```

Trong trường hợp này:

```
if (scenario == "DR")
    await XuLyDirtyAsync(tag);
```

cho biết hệ thống đang ở kịch bản Dirty Read (DR), vì vậy thao tác của người dùng sẽ được chuyển sang hàm XuLyDirtyAsync(tag).

Trong XuLyDirtyAsync(), khi người dùng thực hiện T2-1, chương trình gọi:

```
1235            else if (tag == "T2-1")
1236            {
1237                t2Buttons[0].Enabled = false;
1238
1239                DataTable dt =
1240                    await bus.RunAsync(
1241                        "sp_DR_T2_Read",
1242                        runId);
1243
1244                HienThiKetQua(
1245                    txtT2,
1246                    dt);
1247
1248                if (mode == "ERROR")
1249                    t2Buttons[0].Enabled = true;
1250            }
```

Đoạn code trên cho thấy Transaction 2 không tự thực hiện câu SQL trực tiếp mà gọi Stored Procedure sp_DR_T2_Read.

b. BUS

Đối với việc chạy Stored Procedure của Transaction 2, BUS sử dụng:

```
                8 references
52          public Task<DataTable> RunAsync(
53              string procedureName,
54              Guid runId)
55          {
56              return dal.RunAsync(
57                  procedureName,
58                  runId);
59          }
60      }
61  }
62
```

BUS không trực tiếp truy cập Database mà chuyển yêu cầu xuống DAL.

c. DAL

Trong file DemoGiaoTacDAL.cs, phương thức RunAsync() nhận tên Stored Procedure và chuyển yêu cầu đến hàm thực thi chung:

```
101     public Task<DataTable> RunAsync(
102         string procedureName,
103         Guid runId)
104     {
105         return ExecuteProcedureAsync(
106             procedureName,
107             p => p.Add(
108                 "@RunId",
109                 SqlDbType.UniqueIdentifier
110             ).Value = runId,
111             0);
112     }
113 }
114 }
```

Sau đó, ExecuteProcedureAsync() tạo kết nối đến SQL Server:

using SqlConnection conn = Database.GetConnection();

await conn.OpenAsync();

và cấu hình câu lệnh dưới dạng Stored Procedure:

```
using SqlCommand cmd = new SqlCommand(procedureName, conn);
cmd.CommandType = CommandType.StoredProcedure;
cmd.CommandTimeout = timeout;
```

Cuối cùng, Stored Procedure được thực thi thông qua:

```
using SqlDataReader reader = await cmd.ExecuteReaderAsync();
```

-Phương thức thực hiện Stored Procedure

Đối với kịch bản Dirty Read, Transaction 2 được thực hiện bởi Stored Procedure: sp_DR_T2_Read

Trong chế độ mô phỏng lỗi, Stored Procedure thiết lập:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

Sau đó đọc trạng thái của SV01:

```
SELECT @TrangThai = TrangThaiHocTap
FROM dbo.SinhVien
WHERE MaSV = 'SV01';
```

Giá trị đọc được được ghi vào log:

```
SET @Msg =
    N'② T2 đọc được TrangThaiHocTap = ' +
    ISNULL(@TrangThai, N'NULL');

EXEC dbo.sp_Demo_Log
    @RunId, 'T2', 'SUCCESS', @Msg;
```

Đây chính là phần tạo ra hiện tượng Dirty Read vì T2 đang đọc dữ liệu trong khi T1 chưa COMMIT.

Code thể hiện trên giao diện

Transaction 1:

Đang học → Bảo lưu -> CHƯA COMMIT

Transaction 2: READ UNCOMMITTED -> Đọc được: Bảo lưu

Như vậy, Transaction 2 đã đọc được dữ liệu "Bảo lưu" mặc dù Transaction 1 chưa COMMIT. Đây là biểu hiện trực tiếp của hiện tượng Dirty Read.

Điểm quan trọng hơn sẽ được thể hiện ở bước tiếp theo: Transaction 1 có thể thực hiện ROLLBACK, khi đó giá trị "Bảo lưu" bị hủy và SV01 trở lại "Đang học". Điều này chứng minh giá trị mà Transaction 2 vừa đọc thực sự là dữ liệu chưa được xác nhận.

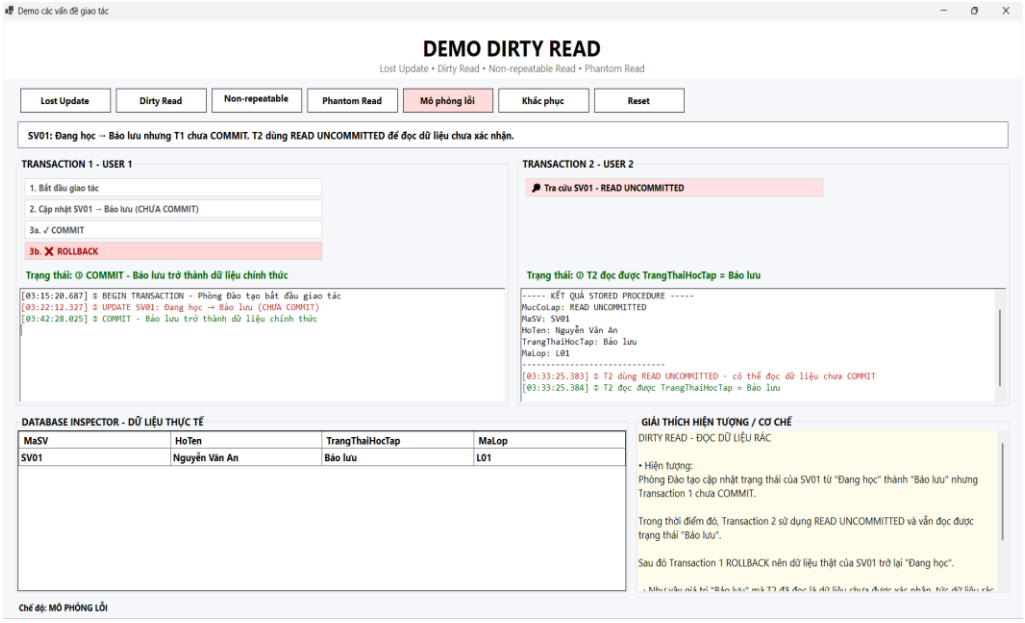
BƯỚC 4 – TRANSACTION 1 THỰC HIỆN COMMIT

Nội dung thực hiện

Sau khi Transaction 2 đã đọc được trạng thái "Bảo lưu" bằng READ UNCOMMITTED, bước tiếp theo là Transaction 1 thực hiện COMMIT. Mục đích của bước này là xác nhận chính thức thay đổi trước đó. Khi COMMIT thành công, giá trị "Bảo lưu" của SV01 không còn là dữ liệu tạm thời mà trở thành dữ liệu chính thức trong cơ sở dữ liệu.

Đây cũng là bước giúp phân biệt giữa dữ liệu chưa COMMIT ở bước trước và dữ liệu đã COMMIT ở bước này.

Giao diện sau khi thực hiện Bước 4



Qua giao diện có thể thấy Transaction 1 đã chuyển sang bước:

“3a. ✓ COMMIT”

Đồng thời dòng trạng thái hiển thị:

“③ COMMIT - Bảo lưu trở thành dữ liệu chính thức”

Trong phần nhật ký của Transaction 1 xuất hiện ba trạng thái liên tiếp:

① BEGIN TRANSACTION - Phòng Đào tạo bắt đầu giao tác

② UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)

③ COMMIT - Bảo lưu trở thành dữ liệu chính thức

Điều này cho thấy Transaction 1 đã hoàn tất giao tác và xác nhận thay đổi đối với SV01. Khác với bước trước, ở thời điểm này giá trị “Bảo lưu” đã được COMMIT, vì vậy đây là dữ liệu chính thức được lưu trong cơ sở dữ liệu.

Code xử lý

a. Tầng giao diện

Thao tác COMMIT được xử lý trong file FDemoGiaoTac.cs, thông qua sự kiện của các nút điều khiển từng bước.

Trong XuLyDirtyAsync(), khi người dùng thực hiện nút “3a. COMMIT”, hệ thống gửi yêu cầu cập nhật bước và hành động COMMIT:

```
await bus.SetStepAsync(
    runId,
    "T1",
    3,
    "ROLLBACK");
```

Ý nghĩa của đoạn code:

runId: xác định phiên demo hiện tại.

"T1": xác định Transaction 1.

3: chuyển Transaction 1 sang bước 3.

"COMMIT": yêu cầu Transaction 1 xác nhận giao tác.

Sau khi cập nhật bước, giao diện thay đổi trạng thái và khóa/mở các nút tương ứng để đảm bảo người dùng thực hiện đúng trình tự.

b. BUS

Yêu cầu COMMIT được chuyển xuống DemoGiaoTacBUS.cs thông qua phương thức SetStepAsync():

```
16 references
24      public Task SetStepAsync(
25          Guid runId,
26          string giaoTac,
27          int step,
28          string? action = null)
29      {
30          return dal.SetStepAsync(
31              runId,
32              giaoTac,
33              step,
34              action);
35      }
```

BUS nhận thông tin từ GUI rồi chuyển trực tiếp xuống:

dal.SetStepAsync(...)

Do đó, BUS đóng vai trò trung gian giữa giao diện GUI và DAL.

c. DAL

Trong file DemoGiaoTacDAL.cs, phương thức SetStepAsync() thực hiện gọi Stored Procedure:

```
1 reference
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return ExecuteProcedureAsync(
        "sp_Demo_SetStep",
        p =>
        {
            p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
            p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
            p.Add("@Step", SqlDbType.Int).Value = step;

            SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
            pa.Value = string.IsNullOrWhiteSpace(action)
                ? DBNull.Value
                : action;
        });
}
```

Ở đây, giá trị: action = "COMMIT" được truyền vào tham số: @Action

Sau đó DAL gọi Stored Procedure: sp_Demo_SetStep

- Phương thức thực hiện Stored Procedure

Trong file Transaction.sql, Stored Procedure sp_Demo_SetStep xử lý phần Transaction 1:

```
IF @GiaoTac = 'T1'
BEGIN
    UPDATE dbo.DemoControl
    SET T1Step = @Step,
        T1Action = COALESCE(@Action, T1Action)
    WHERE RunId = @RunId;
END
```

Cụ thể:

T1Step = @Step cập nhật Transaction 1 sang bước 3.

T1Action = COALESCE(@Action, T1Action) lưu hành động COMMIT.

WHERE RunId = @RunId đảm bảo chỉ cập nhật đúng phiên demo hiện tại.

Lúc này Database không trực tiếp COMMIT giao tác của Transaction 1 bằng sp_Demo_SetStep. Thay vào đó, Stored Procedure này ghi tín hiệu COMMIT vào bảng DemoControl. Transaction 1 đang chạy ở sp_DR_T1_Runner sẽ liên tục kiểm tra tín hiệu này.

Code SQL thực hiện COMMIT giao tác

Trong sp_DR_T1_Runner, Transaction 1 liên tục kiểm tra hành động:

```
SELECT @Action = T1Action, @Cancelled = Cancelled
FROM dbo.DemoControl WITH (NOLOCK)
WHERE RunId = @RunId;
```

Sau đó kiểm tra:

```
IF @Action IN ('COMMIT', 'ROLLBACK') BREAK;
```

Khi GUI đã gửi COMMIT, biến @Action nhận giá trị: COMMIT

Hệ thống đi vào nhánh:

```
IF @Action = 'COMMIT'
BEGIN
    COMMIT TRANSACTION;
    EXEC dbo.sp_Demo_Log @RunId, 'T1', 'SUCCESS',
        N'③ COMMIT – Bảo lưu trở thành dữ liệu chính thức';
END
```

Trong đó câu lệnh quan trọng nhất là: COMMIT TRANSACTION;

Câu lệnh này chính thức xác nhận toàn bộ thay đổi mà Transaction 1 đã thực hiện.

Code hiển thị trên giao diện

Sau khi COMMIT:

SV01: Đang học -> UPDATE->SV01: Bảo lưu-> COMMIT-> Bảo lưu trở thành dữ liệu chính thức

Do đó, giá trị "Bảo lưu" hiện tại không còn là dữ liệu chưa xác nhận. Nếu Transaction 2 thực hiện đọc lại dữ liệu sau thời điểm COMMIT, nó sẽ nhận được giá trị "Bảo lưu" như một dữ liệu hợp lệ.

Bước 4 hoàn tất quá trình xác nhận thay đổi của Transaction 1. Tuy nhiên, hiện tượng Dirty Read đã xảy ra ở Bước 3, vì Transaction 2 đã đọc được giá trị "Bảo lưu" trước khi Transaction 1 COMMIT.

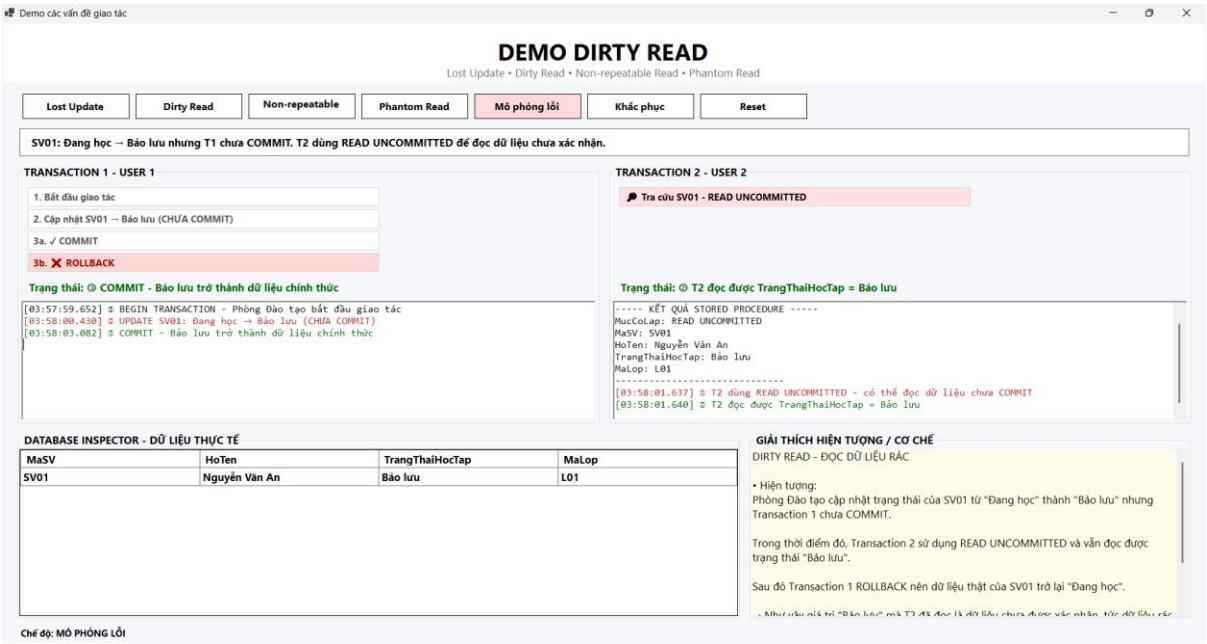
BƯỚC 5 – HOÀN TẤT KỊCH BẢN VÀ XÁC NHẬN KẾT QUẢ

Nội dung thực hiện

Sau khi Transaction 1 thực hiện COMMIT và Transaction 2 thực hiện READ UNCOMMITTED, hệ thống hoàn tất kịch bản mô phỏng Dirty Read. Tại thời điểm này, giá trị "Bảo lưu" của sinh viên SV01 đã được Transaction 1 xác nhận chính thức. Transaction 2 cũng đã đọc được giá trị này.

Bước này nhằm tổng hợp kết quả của quá trình thực hiện và xác nhận rằng hiện tượng Dirty Read đã xảy ra ở thời điểm Transaction 2 đọc dữ liệu trước khi Transaction 1 COMMIT.

Giao diện sau khi thực hiện Bước 5



Giao diện cho thấy Transaction 1 đã hoàn thành:

1. BEGIN TRANSACTION
2. UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)
3. COMMIT - Bảo lưu trở thành dữ liệu chính thức

Đồng thời Transaction 2 hiển thị:

1. T2 dùng READ UNCOMMITTED - có thể đọc dữ liệu chưa COMMIT
2. T2 đọc được TrangThaiHocTap = Bảo lưu

Như vậy, sau khi COMMIT, dữ liệu "Bảo lưu" vẫn được giữ lại trong cơ sở dữ liệu vì thay đổi của Transaction 1 đã được xác nhận.

Code xử lý

a. Tầng giao diện

Sau khi Transaction 2 thực hiện thao tác đọc, FDemoGiaoTac.cs gọi: await LoadInspectorAsync();

Hàm này lấy dữ liệu thực tế từ Database:

```
1583        private async Task LoadInspectorAsync()
1584        {
1585            DataTable dt =
1586                await bus.InspectorAsync(
1587                    scenario);
1588
1589            dgvInspector.DataSource = dt;
1590            dgvInspector.ClearSelection();
1591        }
```

Dữ liệu được chuyển qua BUS:

```
47      public Task<DataTable> InspectorAsync(string scenario)
48      {
49          return dal.InspectorAsync(scenario);
50      }
51
        8 references
52      public Task<DataTable> RunAsync(
53          string procedureName,
54          Guid runId)
55      {
56          return dal.RunAsync(
57              procedureName,
58              runId);
59      }
```

Sau đó BUS chuyển tiếp xuống DAL:

```
90          public Task<DataTable> InspectorAsync(string scenario)
91          {
92              return ExecuteProcedureAsync(
93                  "sp_Demo_Inspector",
94                  p => p.Add(
95                      "@Scenario",
96                      SqlDbType.VarChar,
97                      20
98                  ).Value = scenario);
99          }
```

b. Tầng DAL

Trong DemoGiaoTacDAL.cs, phương thức InspectorAsync() gọi Stored Procedure:

```
90          public Task<DataTable> InspectorAsync(string scenario)
91          {
92              return ExecuteProcedureAsync(
93                  "sp_Demo_Inspector",
94                  p => p.Add(
95                      "@Scenario",
96                      SqlDbType.VarChar,
97                      20
98                  ).Value = scenario);
99          }
```

Sau đó ExecuteProcedureAsync() mở kết nối SQL Server và thực hiện Stored Procedure:

```
17          using SqlConnection conn = Database.GetConnection();
18          await conn.OpenAsync();
19
20          using SqlCommand cmd = new SqlCommand(procedureName, conn);
21          cmd.CommandType = CommandType.StoredProcedure;
```

Kết quả được đọc bằng:

```
using SqlDataReader reader = await cmd.ExecuteReaderAsync();
```

và đưa vào DataTable để hiển thị trên dgvInspector.

- Phương thức thực hiện Stored Procedure

Sau khi các giao tác hoàn thành, hệ thống sử dụng:

sp_Demo_Inspector

để lấy dữ liệu thực tế của sinh viên SV01.

Kết quả trả về:

MaSV: SV01

HoTen: Đặng Thành An

TrangThaiHocTap: Bảo lưu

MaLop: L01

Điều này xác nhận rằng sau khi Transaction 1 COMMIT, giá trị "Bảo lưu" đã trở thành dữ liệu chính thức.

- Đánh giá hiện tượng Dirty Read

Điểm cần lưu ý là Dirty Read đã xảy ra trước khi COMMIT, ở thời điểm Transaction 2 thực hiện:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

và đọc:

```
SELECT @TrangThai = TrangThaiHocTap
FROM dbo.SinhVien
WHERE MaSV = 'SV01';
```

Đặc biệt:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

cho phép Transaction 2 đọc dữ liệu mà Transaction 1 chưa COMMIT.

Vì vậy, dù trong ảnh hiện tại kết quả cuối cùng là “Bảo lưu” và dữ liệu đã được COMMIT, bản chất của Dirty Read vẫn nằm ở lần đọc trước COMMIT.

BƯỚC 6 – NHÁNH 3b: TRANSACTION 1 THỰC HIỆN ROLLBACK

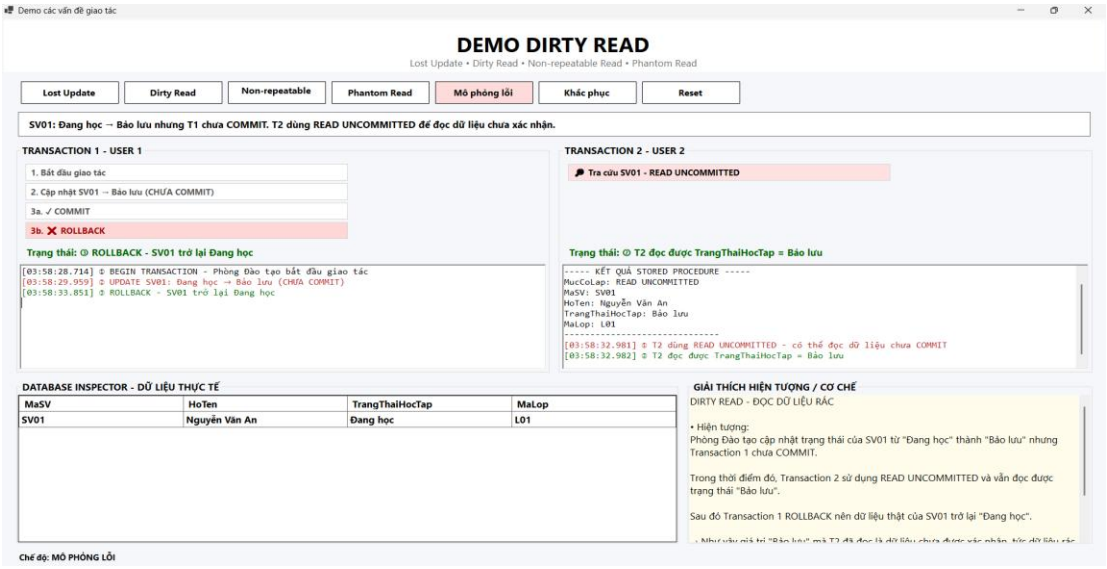
Nội dung thực hiện

Ngoài trường hợp Transaction 1 thực hiện COMMIT, chương trình còn cung cấp nhánh ROLLBACK để mô phỏng trường hợp thay đổi của Transaction 1 không được xác nhận. Ở nhánh này, Transaction 1 cập nhật trạng thái SV01 từ “Đang học” sang “Bảo lưu” nhưng sau đó thực hiện ROLLBACK. Khi đó, thay đổi “Bảo lưu” sẽ bị hủy và dữ liệu thực tế của SV01 được khôi phục về trạng thái ban đầu là “Đang học”.

Nhánh này có ý nghĩa quan trọng trong việc chứng minh rằng nếu Transaction 2 đọc được “Bảo lưu” trước khi Transaction 1 COMMIT, thì giá trị mà Transaction 2 đọc thực sự là dữ liệu chưa được xác nhận.

Lưu ý: Vì ảnh hiện tại đã thực hiện 3a. COMMIT, muốn trình bày nhánh 3b. ROLLBACK cần Reset/chạy lại kịch bản, sau đó thực hiện đến bước cập nhật SV01 nhưng chọn ROLLBACK thay vì COMMIT.

Giao diện sau khi thực hiện bước 6



Sau khi thực hiện ROLLBACK, trạng thái của Transaction 1 sẽ chuyển sang:

“ ROLLBACK - Hủy thay đổi, SV01 trở về Đang học”

Trong phần nhật ký, hệ thống sẽ thể hiện quá trình:

1. BEGIN TRANSACTION - Phòng Đào tạo bắt đầu giao tác
2. UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)
3. ROLLBACK - Hủy thay đổi

Như vậy, mặc dù trước đó Transaction 2 có thể đọc được giá trị “Bảo lưu” bằng READ UNCOMMITTED, nhưng sau khi Transaction 1 ROLLBACK thì giá trị này không còn tồn tại trong dữ liệu chính thức.

Code xử lý

a. GUI

Tại file FDemoGiaoTac.cs, khi thực hiện bước ROLLBACK, sự kiện StepButton_Click() vẫn chuyển yêu cầu đến hàm xử lý Dirty Read:

```
if (scenario == "DR")
    await XuLyDirtyAsync(tag);
```

Trong XuLyDirtyAsync(), nút ROLLBACK được xử lý tương tự nhánh COMMIT, nhưng hành động truyền xuống BUS là:

```
await bus.SetStepAsync(
    runId,
    "T1",
    3,
    "ROLLBACK");
```

Trong đó:

"T1": Transaction 1.

3: bước kết thúc giao tác.

"ROLLBACK": yêu cầu hủy bỏ các thay đổi chưa COMMIT.

Sau khi thực hiện, giao diện cập nhật trạng thái và tải lại Database Inspector.

b. BUS

Yêu cầu ROLLBACK được chuyển sang DemoGiaoTacBUS.cs thông qua:

```
16 references
24      public Task SetStepAsync(
25          Guid runId,
26          string giaoTac,
27          int step,
28          string? action = null)
29      {
30          return dal.SetStepAsync(
31              runId,
32              giaoTac,
33              step,
34              action);
35      }
```

BUS nhận:

```
await bus.SetStepAsync(
    runId,
    "T1",
    3,
    "ROLLBACK");
```

và chuyển yêu cầu xuống:

```
return dal.SetStepAsync(
```

c. DAL

Trong DemoGiaoTacDAL.cs, SetStepAsync() gọi Stored Procedure:

```
return ExecuteProcedureAsync(
    "sp_Demo_SetStep",
    p =>
    {
        p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
        p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
        p.Add("@Step", SqlDbType.Int).Value = step;

        SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
        pa.Value = string.IsNullOrWhiteSpace(action)
            ? DBNull.Value
            : action;
    });
```

Giá trị: ROLLBACK được truyền vào tham số: @Action

Sau đó DAL gọi: sp_Demo_SetStep để cập nhật trạng thái điều khiển của Transaction 1.

-Phương thức thực hiện Stored Procedure

Trong Transaction.sql, sp_Demo_SetStep cập nhật thông tin điều khiển của Transaction 1:

```
IF @GiaoTac = 'T1'
BEGIN
    UPDATE dbo.DemoControl
    SET T1Step = @Step,
        T1Action = COALESCE(@Action, T1Action)
    WHERE RunId = @RunId;
END
```

Với nhánh ROLLBACK, giá trị được lưu là: T1Action = 'ROLLBACK' Stored Procedure sp_DR_T1_Runner đang chạy sẽ nhận biết hành động này và thực hiện:

```
ELSE
BEGIN
    ROLLBACK TRANSACTION;
    EXEC dbo.sp_Demo_Log @RunId, 'T1', 'SUCCESS',
        N'③ ROLLBACK – SV01 trở lại Đang học';
END;
```

Lệnh quan trọng là: ROLLBACK TRANSACTION;

Lệnh này hủy toàn bộ thay đổi mà Transaction 1 thực hiện kể từ khi BEGIN TRANSACTION.

Qua hai nhánh COMMIT và ROLLBACK, chương trình cho thấy:

Nếu COMMIT → "Bảo lưu" trở thành dữ liệu chính thức.

Nếu ROLLBACK → "Bảo lưu" bị hủy, SV01 trở lại "Đang học".

Nhưng trong cả hai trường hợp, nếu T2 đọc được "Bảo lưu" khi T1 chưa COMMIT, thì hiện tượng Dirty Read đã xảy ra.

3.3.2. Khắc Phục Dirty Read

Phương pháp khắc phục

Để khắc phục hiện tượng Dirty Read, Transaction 2 không sử dụng READ UNCOMMITTED mà chuyển sang mức cô lập READ COMMITTED. Với mức cô lập này, Transaction 2 không được đọc dữ liệu đang được Transaction 1 thay đổi nhưng chưa COMMIT. Khi T1 vẫn đang giữ thay đổi "Bảo lưu", T2 phải chờ cho đến khi T1 thực hiện COMMIT hoặc ROLLBACK.

Hiện tượng sau khi khắc phục: T1 cập nhật SV01 từ "Đang học" thành "Bảo lưu" nhưng chưa COMMIT, T2 sử dụng READ COMMITTED nên không đọc được giá trị "Bảo lưu" chưa xác nhận và phải chờ T1 kết thúc. Nếu T1 COMMIT, T2 đọc được "Bảo lưu"; nếu T1 ROLLBACK, T2 đọc được "Đang học". → Như vậy T2 chỉ đọc được dữ liệu đã được xác nhận và không còn xảy ra Dirty Read.

Nguyên nhân được khắc phục: Thay READ UNCOMMITTED bằng READ COMMITTED giúp SQL Server không cho phép T2 đọc dữ liệu chưa COMMIT của T1.

Bước 1 – Khởi tạo kịch bản khắc phục

Nội dung thực hiện

Khi người dùng chọn chế độ **Khắc phục Dirty Read**, chương trình khởi tạo lại dữ liệu SV01 về trạng thái "Đang học". Đồng thời, hệ thống tạo một phiên mô phỏng mới với Mode = FIX. Việc tạo phiên mới giúp đảm bảo quá trình khắc phục được thực hiện độc lập với phiên mô phỏng lỗi trước đó.

Dữ liệu ban đầu:

SV01

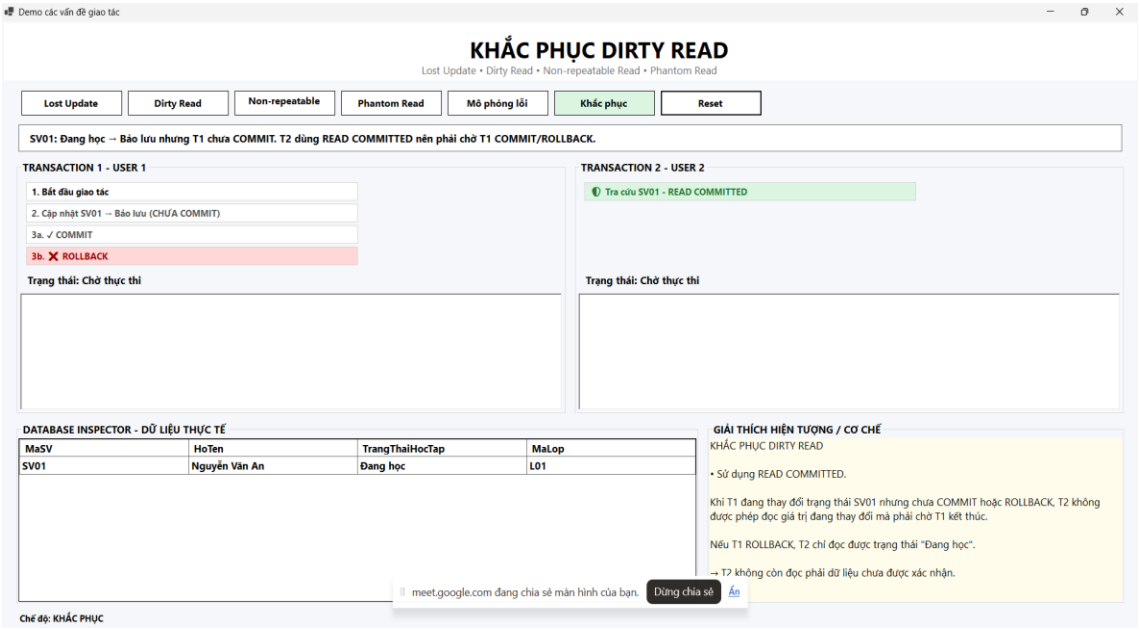
TrangThaiHocTap = Đang học

Giao diện chuyển sang tiêu đề:

KHẮC PHỤC DIRTY READ

và nút tra cứu của Transaction 2 chuyển từ:

Tra cứu SV01 - READ UNCOMMITTED sang: Tra cứu SV01 - READ COMMITTED



Code xử lý

a. GUI

Khi cấu hình chế độ khắc phục, giao diện kiểm tra biến mode:

```
812         private void CauHinhDirtyRead()
813         {
814             lblTieuDe.Text =
815                 mode == "ERROR"
816                 ? "DEMO DIRTY READ"
817                 : "KHẮC PHỤC DIRTY READ";
818
```

Nội dung mô tả tình huống cũng được thay đổi:

```
825         lblTinhHuong.Text =
826             "SV01: Đang học → Bảo lưu nhưng T1 chưa COMMIT. " +
827             (mode == "ERROR"
828                 ? "T2 dùng READ UNCOMMITTED để đọc dữ liệu chưa xác nhận."
829                 : "T2 dùng READ COMMITTED nên phải chờ T1 COMMIT/ROLLBACK.");
830
```

Nút tra cứu T2 được thiết lập theo chế độ:

```
853         t2Buttons[0].Text =
854             mode == "ERROR"
855             ? "🔎 Tra cứu SV01 – READ UNCOMMITTED"
856             : "🛡️ Tra cứu SV01 – READ COMMITTED";
857
```

b. BUS

Khi khởi tạo phiên mô phỏng, GUI gọi:

```
674             await bus.InitAsync(
675                 runId,
676                 scenario,
677                 mode);
678
```

BUS chuyển yêu cầu xuống DAL:

```
        public Task InitAsync(
            Guid runId,
            string scenario,
            string mode)
        {
            return dal.InitAsync(
                runId,
                scenario,
                mode);
        }
```

c. DAL

DAL gọi Stored Procedure khởi tạo:

```
    public Task InitAsync(
        Guid runId,
        string scenario,
        string mode)
    {
        return ExecuteProcedureAsync(
            "sp_Demo_Init",
            p =>
            {
                p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
                p.Add("@Scenario", SqlDbType.VarChar, 20).Value = scenario;
                p.Add("@Mode", SqlDbType.VarChar, 10).Value = mode;
            });
    }
```

7.4 Phương thức thực hiện Stored Procedure

Trong sp_Demo_Init, khi kịch bản là Dirty Read, dữ liệu SV01 được đưa về trạng thái ban đầu:

```
IF @Scenario = 'DR'
BEGIN
    UPDATE dbo.SinhVien
    SET TrangThaiHocTap = N'Đang học'
    WHERE MaSV = 'SV01';
END;
```

Sau đó hệ thống lưu chế độ chạy vào bảng DemoControl:

```
INSERT INTO dbo.DemoControl
(
    RunId, Scenario, Mode,
    T1Step, T2Step,
    T1Action, T2Action, Cancelled
)
VALUES
(
    @RunId, @Scenario, @Mode,
    0, 0, NULL, NULL, 0
);
```

Khi khởi tạo chế độ khắc phục:

Scenario = DR

Mode = FIX

Giá trị này sẽ được sp_DR_T2_Read sử dụng để quyết định mức cô lập của Transaction 2.

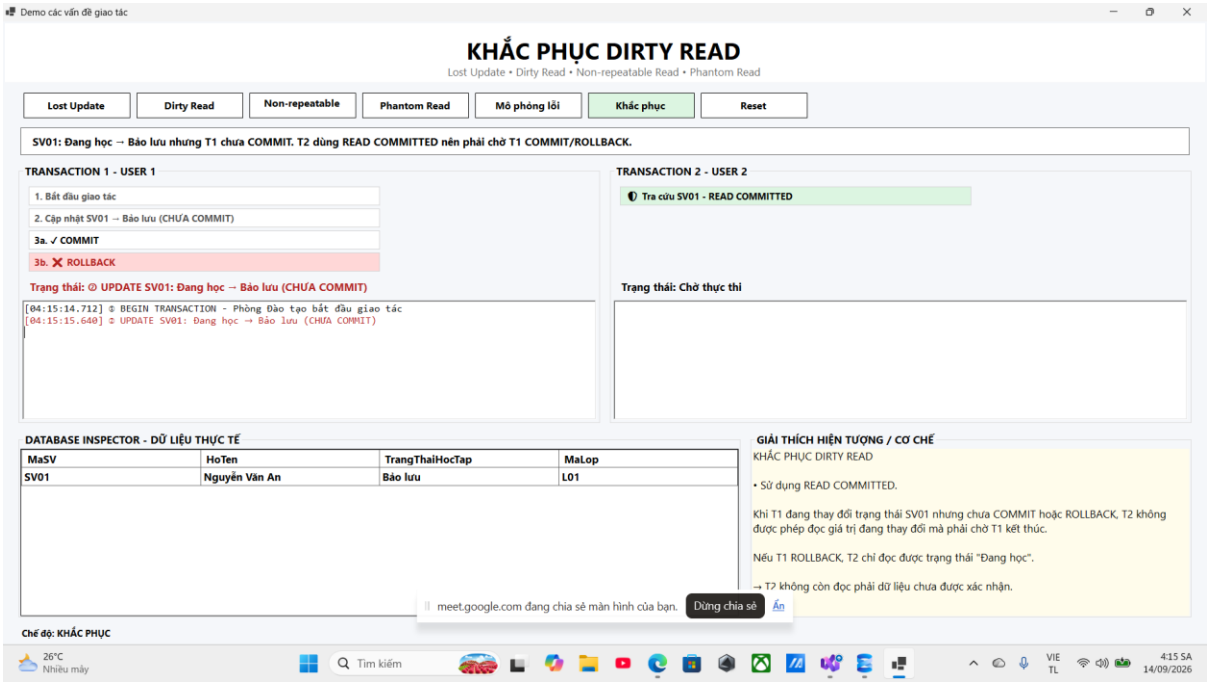
Bước 2 – Transaction 1 cập nhật SV01 nhưng chưa COMMIT

Nội dung thực hiện

Ở bước này, Transaction 1 vẫn thực hiện thao tác giống chế độ mô phỏng lỗi: bắt đầu giao tác và cập nhật SV01 từ "Đang học" thành "Bảo lưu" nhưng chưa COMMIT. Điểm khác biệt nằm ở Transaction 2. Trong chế độ khắc phục, T2 sẽ không sử dụng READ UNCOMMITTED mà sử dụng READ COMMITTED.

Trình tự:T1 Đang học -> UPDATE->Bảo lưu->CHƯA COMMIT

Tại thời điểm này, dữ liệu "Bảo lưu" vẫn chưa phải dữ liệu chính thức.



Code SQL thực hiện cập nhật

Transaction 1 vẫn sử dụng:

BEGIN TRANSACTION;

```
UPDATE dbo.SinhVien
SET TrangThaiHocTap = N'Bảo lưu'
WHERE MaSV = 'SV01';

EXEC dbo.sp_Demo_Log @RunId, 'T1', 'DANGER',
    N'② UPDATE SV01: Đang học → Bảo lưu (CHƯA COMMIT)';
```

Điểm quan trọng là T1 vẫn chưa thực hiện: COMMIT TRANSACTION;

Do đó T1 vẫn giữ thay đổi và tạo khóa trên dữ liệu tương ứng.

Bước 3 – Transaction 2 thực hiện READ COMMITTED

Nội dung thực hiện

Sau khi T1 cập nhật SV01 thành “Bảo lưu” nhưng chưa COMMIT, Transaction 2 thực hiện chức năng “Tra cứu SV01 - READ COMMITTED”. Đây là bước kiểm tra hiệu quả

của biện pháp khắc phục. Thay vì đọc trực tiếp dữ liệu chưa xác nhận như chế độ lỗi, T2 sử dụng READ COMMITTED và phải chờ Transaction 1 kết thúc trước khi đọc giá trị.

Hiện tượng: T1 đang giữ giá trị "Bảo lưu" chưa COMMIT, T2 thực hiện READ COMMITTED và không đọc được giá trị này ngay lập tức mà phải chờ T1 COMMIT hoặc ROLLBACK. → Điều này cho thấy dữ liệu chưa được xác nhận không còn được T2 đọc trực tiếp.

Code xử lý

a. GUI

Khi người dùng nhấn nút:

Tra cứu SV01 - READ COMMITTED

C# vẫn gọi cùng phương thức RunAsync():

```
1239            DataTable dt =
1240                await bus.RunAsync(
1241                    "sp_DR_T2_Read",
1242                    runId);
1243
1244            HienThiKetQua(
1245                txtT2,
1246                dt);
1247
```

Điểm khác biệt không nằm ở câu lệnh C# mà nằm ở giá trị Mode = FIX được lưu trong DemoControl.

b. BUS

BUS tiếp tục chuyển yêu cầu:

```
52          public Task<DataTable> RunAsync(
53              string procedureName,
54              Guid runId)
55          {
56              return dal.RunAsync(
57                  procedureName,
58                  runId);
59          }
60      }
61  }
62
```

c. DAL

DAL thực hiện:

```
100
101         public Task<DataTable> RunAsync(
102             string procedureName,
103             Guid runId)
104         {
105             return ExecuteProcedureAsync(
106                 procedureName,
107                 p => p.Add(
108                     "@RunId",
109                     SqlDbType.UniqueIdentifier
110                 ).Value = runId,
111                 0);
112         }
```

Sau đó Stored Procedure sp_DR_T2_Read được thực thi trên SQL Server.

Phương thức thực hiện Stored Procedure khắc phục

Trong sp_DR_T2_Read, hệ thống kiểm tra chế độ đang chạy:

```
SELECT @Mode = Mode
FROM dbo.DemoControl WITH (NOLOCK)
WHERE RunId = @RunId;
```

Nếu đang ở chế độ lỗi:

IF @Mode = 'ERROR'

```
IF @Mode = 'ERROR'
BEGIN
    EXEC dbo.sp_Demo_Log @RunId, 'T2', 'DANGER',
        N'① T2 dùng READ UNCOMMITTED – có thể đọc dữ liệu chưa COMMIT';

    SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

    SELECT @TrangThai = TrangThaiHocTap
    FROM dbo.SinhVien
    WHERE MaSV = 'SV01';

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
END
```

Nếu đang ở chế độ khắc phục:

```
ELSE
BEGIN
    EXEC dbo.sp_Demo_Log @RunId, 'T2', 'WAIT',
        N'① T2 dùng READ COMMITTED – đang chờ T1 kết thúc nếu còn khóa';

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    SELECT @TrangThai = TrangThaiHocTap
    FROM dbo.SinhVien
    WHERE MaSV = 'SV01';
END;
```

Câu lệnh quan trọng trong phần khắc phục là:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Với mức cô lập này, T2 không được đọc giá trị “Bảo lưu” chưa COMMIT của T1.

Bước 4 – Transaction 1 COMMIT trong chế độ khắc phục

Nội dung thực hiện

Sau khi T2 đã bắt đầu đọc bằng READ COMMITTED và phải chờ T1 kết thúc, Transaction 1 thực hiện COMMIT. Khi đó thay đổi "Bảo lưu" trở thành dữ liệu chính thức và T2 được phép tiếp tục đọc dữ liệu.

Điều này cho thấy T2 chỉ đọc được "Bảo lưu" sau khi T1 đã xác nhận dữ liệu.

Code xử lý COMMIT

GUI gửi tín hiệu:

```
1215                    await bus.SetStepAsync(
1216                        runId,
1217                        "T1",
1218                        3,
1219                        "COMMIT");
```

BUS chuyển tiếp:

```
30              return dal.SetStepAsync(
31                  runId,
32                  giaoTac,
33                  step,
34                  action);
```

DAL truyền:

@GiaoTac = T1

@Step = 3

@Action = COMMIT

đến:

sp_Demo_SetStep

Sau đó sp_DR_T1_Runner nhận được:

@Action = COMMIT

và thực hiện:

COMMIT TRANSACTION;

Kết quả sau khi khắc phục – Trường hợp COMMIT

Sau khi T1 COMMIT, T2 tiếp tục thực hiện câu lệnh READ COMMITTED và đọc được:

TrangThaiHocTap = Bảo lưu

Lúc này "Bảo lưu" đã là dữ liệu chính thức nên việc T2 đọc được giá trị này là hoàn toàn hợp lệ.

Kết quả: T2 không đọc "Bảo lưu" trong lúc T1 chưa COMMIT mà chỉ đọc được giá trị này sau khi T1 đã COMMIT. Vì vậy, Dirty Read không xảy ra.

BƯỚC 5 – KHẮC PHỤC VỚI NHÁNH ROLLBACK

Nội dung thực hiện

Để chứng minh rõ hơn hiệu quả của READ COMMITTED, chương trình tiếp tục kiểm tra trường hợp T1 ROLLBACK. T1 cập nhật SV01 thành "Bảo lưu" nhưng không xác nhận thay đổi. T2 thực hiện READ COMMITTED và phải chờ T1 kết thúc. Khi T1 ROLLBACK, thay đổi "Bảo lưu" bị hủy và T2 tiếp tục đọc được giá trị ban đầu là "Đang học".

Đây là điểm khác biệt quan trọng so với chế độ Dirty Read: trong chế độ lỗi, T2 có thể đọc "Bảo lưu" trước khi T1 ROLLBACK; trong chế độ khắc phục, T2 không nhìn thấy giá trị "Bảo lưu" chưa được xác nhận.

Code thực hiện ROLLBACK

GUI gửi:

```
1226                    await bus.SetStepAsync(
1227                        runId,
1228                        "T1",
1229                        3,
1230                        "ROLLBACK");
1231
```

DAL truyền hành động:

@Action = ROLLBACK

vào:

sp_Demo_SetStep

Sau đó sp_DR_T1_Runner thực hiện:

```
  ELSE
  BEGIN
      ROLLBACK TRANSACTION;
      EXEC dbo.sp_Demo_Log @RunId, 'T1', 'SUCCESS',
          N'③ ROLLBACK – SV01 trở lại Đang học';
  END;
```

Lệnh:

ROLLBACK TRANSACTION;

hủy thay đổi “Bảo lưu”.

T2 không đọc được dữ liệu “Bảo lưu” chưa xác nhận. Sau khi T1 ROLLBACK, T2 chỉ đọc được dữ liệu chính thức là “Đang học”.

SO SÁNH DIRTY READ TRƯỚC VÀ SAU KHI KHẮC PHỤC

Nội dung	**Mô phỏng lỗi**	**Khắc phục**
Mức cô lập của T2	READ UNCOMMITTED	READ COMMITTED
T1 UPDATE chưa COMMIT	T2 có thể đọc	T2 phải chờ
T2 đọc “Bảo lưu” trước COMMIT	Có	Không

T1 COMMIT	T2 đọc được “Bảo lưu”	T2 đọc được “Bảo lưu”
T1 ROLLBACK	T2 có thể đã đọc “Bảo lưu”	T2 đọc “Đang học”
Dữ liệu chưa xác nhận	Có thể bị đọc	Không được đọc
Dirty Read	Xảy ra	Không xảy ra

ĐÁNH GIÁ KẾT QUẢ KHẮC PHỤC

Ở chế độ mô phỏng lỗi, Transaction 2 sử dụng READ UNCOMMITTED nên có thể đọc được dữ liệu “Bảo lưu” ngay khi Transaction 1 chưa COMMIT. Nếu T1 ROLLBACK, giá trị “Bảo lưu” bị hủy nhưng T2 đã đọc giá trị này trước đó, từ đó phát sinh Dirty Read.

Ở chế độ khắc phục, Transaction 2 sử dụng READ COMMITTED. Khi T1 đang cập nhật SV01 nhưng chưa COMMIT, T2 phải chờ T1 kết thúc thay vì đọc giá trị chưa xác nhận. Nếu T1 COMMIT, T2 đọc được “Bảo lưu”; nếu T1 ROLLBACK, T2 đọc được “Đang học”. Như vậy, T2 luôn đọc dữ liệu đã được xác nhận.

Điểm quan trọng là **biện pháp khắc phục không nằm ở Task.Delay() của C#**. Task.Delay() chỉ được sử dụng để hỗ trợ trình tự trình diễn trên giao diện. Cơ chế thực sự ngăn Dirty Read là mức cô lập:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

KẾT LUẬN DIRTY READ

Như vậy, thông qua chương trình mô phỏng và cơ chế khắc phục bằng READ COMMITTED, có thể thấy mức cô lập của giao tác đóng vai trò quan trọng trong việc đảm bảo tính nhất quán và độ tin cậy của dữ liệu khi nhiều giao tác cùng truy cập cơ sở dữ liệu.

3.4. Không đọc lại được dữ liệu (Non-Repeatable Read)

Non-repeatable Read (đọc không lặp lại) là hiện tượng xảy ra khi một giao tác đọc cùng một dữ liệu nhiều lần, nhưng giữa các lần đọc có một giao tác khác thực hiện cập nhật dữ liệu và COMMIT. Khi giao tác ban đầu thực hiện đọc lại, kết quả nhận được có thể khác so với lần đọc trước.

Ví dụ:

- T1 đọc thông tin sản phẩm và thấy giá là 100.000 đồng
- T2 cập nhật giá sản phẩm thành 120.000 đồng và COMMIT
- T1 đọc lại giá sản phẩm
- Lần đọc thứ nhất: 100.000 đồng
- Lần đọc thứ hai: 120.000 đồng

Như vậy, trong cùng một giao tác T1, cùng một dữ liệu nhưng lại nhận được hai giá trị khác nhau. Kết quả xử lý của T1 có thể không nhất quán vì dữ liệu đã thay đổi trong khi giao tác vẫn đang thực hiện.

3.4.1. Mô phỏng hiện tượng Non-Repeatable Read

Trong chương trình mô phỏng, hiện tượng Non-repeatable Read được thực hiện thông qua hai giao tác. Transaction 1 (T1) thực hiện đọc số điện thoại của khoa Công nghệ thông tin hai lần, trong khi Transaction 2 (T2) thực hiện cập nhật số điện thoại và COMMIT giữa hai lần đọc của T1.

Dữ liệu ban đầu của khoa Công nghệ thông tin có số điện thoại: 0281234567

Transaction 2 sẽ cập nhật số điện thoại thành: 0289999888

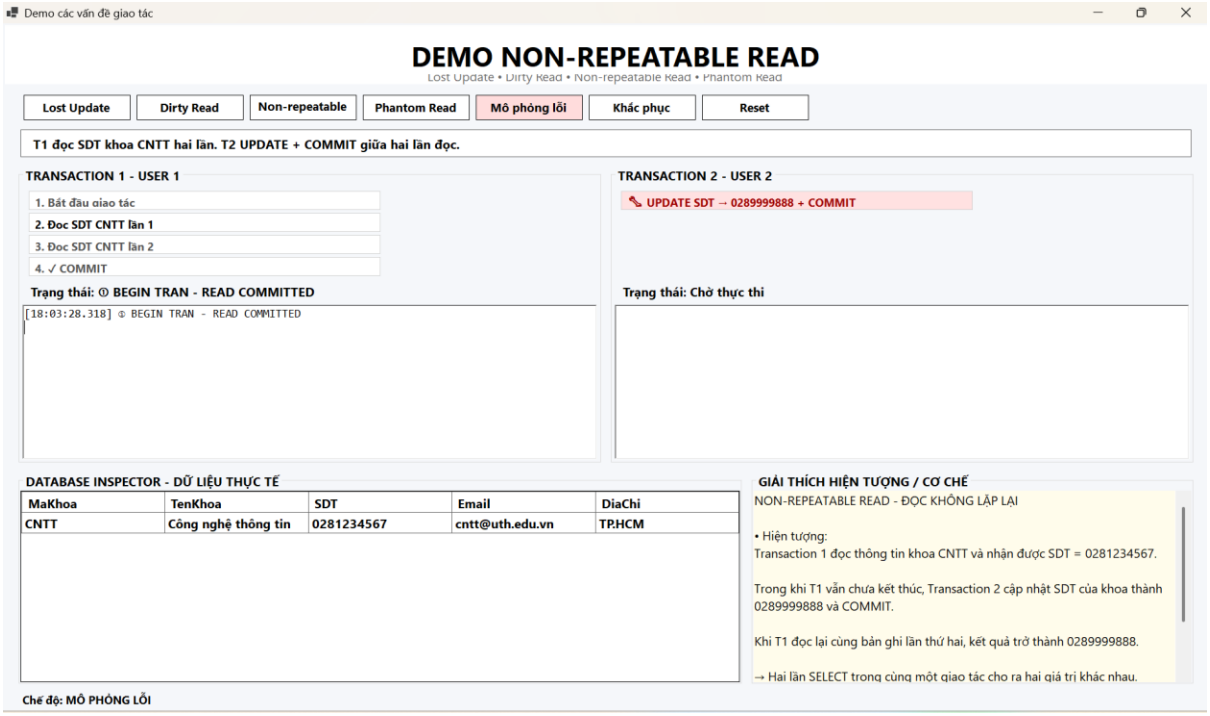
Quá trình mô phỏng hiện tượng được thực hiện theo trình tự:

T1 bắt đầu giao tác → T1 đọc SDT lần 1 → T2 cập nhật SDT → T2 COMMIT → T1 đọc SDT lần 2 → T1 COMMIT.

Khi T1 đọc lần thứ nhất nhận được 0281234567, nhưng sau khi T2 cập nhật và COMMIT, T1 đọc lần thứ hai nhận được 0289999888, hai kết quả đọc không giống nhau. Đây chính là hiện tượng **Non-repeatable Read**.

Bước 1 – Transaction 1 bắt đầu giao tác

1. Giao diện



Nội dụng của các bước thực hiện

Tại bước đầu tiên, hệ thống thực hiện mở Transaction 1 để chuẩn bị cho các thao tác đọc dữ liệu tiếp theo. Transaction 1 được thực hiện với mức cô lập READ COMMITTED do chương trình đang ở chế độ Mô phỏng lỗi.

Sau khi thực hiện bước **“**1. Bắt đầu giao tác”, giao diện hiển thị:

Trạng thái: ① BEGIN TRAN - READ COMMITTED

Đồng thời phần nhật ký của Transaction 1 hiển thị:

[18:03:28.318] ⓘ BEGIN TRAN - READ COMMITTED

Điều này cho thấy Transaction 1 đã được mở thành công và đang duy trì trạng thái thực hiện giao tác.

Tại thời điểm này, Transaction 1 **chưa đọc dữ liệu**, Transaction 2 **chưa thực hiện UPDATE**, vì vậy dữ liệu trong cơ sở dữ liệu vẫn giữ nguyên giá trị ban đầu:SDT = 0281234567

Do đó, hiện tượng Non-repeatable Read chưa xảy ra ở bước này. Đây là bước chuẩn bị để Transaction 1 thực hiện lần đọc dữ liệu thứ nhất ở bước tiếp theo.

2. Code xử lý

a. Tầng giao diện

Các nút thực hiện từng bước của Transaction được tạo trong phương thức: TaoNoiDungGiaoTac()

```
499             };
500
501         for (int i = 0; i < 4; i++)
502         {
503             Button b =
504                 TaoNutBuoc();
505
506             b.Tag = side + "-" + (i + 1);
507
508             b.Click +=
509                 StepButton_Click;
510
511             buttons[i] = b;
512
513             steps.Controls.Add(b);
514         }
515
```

Trong đó : b.Tag = side + "-" + (i + 1); dùng để xác định giao tác và số thứ tự của từng bước. Đối với nút đầu tiên của Transaction 1, Tag tương ứng là **T1-1**.

Khi người dùng thực hiện bước này, sự kiện Click của nút được chuyển đến phương thức: StepButton_Click()

Trong phương thức này, chương trình kiểm tra kịch bản đang được lựa chọn:

```
1166
1167            try
1168            {
1169                if (scenario == "DR")
1170                    await XuLyDirtyAsync(tag);
1171                else if (scenario == "NRR")
1172                    await XuLyNRRAsync(tag);
1173                else if (scenario == "LU")
1174                    await XuLyLUAsync(tag);
1175                else
1176                    await XuLyPhantomAsync(tag);
```

Do kịch bản hiện tại là NRR, chương trình chuyển sang phương thức:

await XuLyNRRAsync(tag); tiếp theo trong phương thức XuLyNRRAsync(), với tag = "T1-1", chương trình thực hiện:

```
            if (tag == "T1-1")
            {
                await bus.SetStepAsync(
                    runId, "T1", 1);
```

Đoạn code này xác định: runId: mã phiên mô phỏng hiện tại.

"T1": giao tác đang thực hiện là Transaction 1.

1: bước thực hiện là bước 1.

Như vậy, tầng giao diện không trực tiếp thực hiện câu lệnh BEGIN TRANSACTION, mà gửi yêu cầu xuống tầng BUS để xử lý.

b. Tầng BUS

Sau khi nhận yêu cầu từ giao diện, chương trình chuyển đến phương thức:
SetStepAsync()

```
16 references
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return dal.SetStepAsync(
        runId,
        giaoTac,
        step,
        action);
}
```

BUS không trực tiếp thao tác với cơ sở dữ liệu mà chuyển yêu cầu xuống đối tượng DAL thông qua:

```
{
    return dal.SetStepAsync(
        runId,
        giaoTac,
        step,
        action);
}
```

Vì vậy, luồng xử lý tiếp tục truyền xuống DAL

c. Tầng DAL

Tại tầng DAL, phương thức SetStepAsync() thực hiện gọi Stored Procedure sp_Demo_SetStep.

```
1 reference
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return ExecuteProcedureAsync(
        "sp_Demo_SetStep",
        p =>
        {
            p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
            p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
            p.Add("@Step", SqlDbType.Int).Value = step;

            SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
            pa.Value = string.IsNullOrWhiteSpace(action)
                ? DBNull.Value
                : action;
        });
}
```

Tại Bước 1, các tham số được truyền xuống Stored Procedure gồm:

Tham số	Giá trị	Ý Nghĩa
@RunId	Mã phiên hiện tại	Xác định phiên mô phỏng
@GiaoTac	T1	Xác định Transaction 1
@Step	1	Xác định bước 1
@Action	Null	Chưa có hành động COMMIT/ROLLBACK

DAL tiếp tục chuyển yêu cầu đến phương thức ExecuteProcedureAsync() để thực hiện Stored Procedure.

d. Phương thức thực thi Stored Procedure

Đầu tiên chương trình tạo và mở kết nối: using SqlConnection conn = Database.GetConnection();

await conn.OpenAsync();

Tiếp theo tạo đối tượng SqlCommand:

```
20          using SqlCommand cmd = new SqlCommand(procedureName, conn);
21          cmd.CommandType = CommandType.StoredProcedure;
22          cmd.CommandTimeout = timeout;
23
```

Thuộc tính: cmd.CommandType = CommandType.StoredProcedure; cho biết chương trình đang thực thi một Stored Procedure trên SQL Server

Sau đó Stored Procedure được thực thi bằng:

```
26          using SqlDataReader reader = await cmd.ExecuteReaderAsync();
27
```

e. Stored Procedure cập nhật trạng thái bước

Sau khi DAL gọi sp_Demo_SetStep, Stored Procedure này tiếp nhận các tham số:

@RunId, @GiaoTac = T1, @Step = 1, @Action = NULL

Phần khai báo:

```
80
81  CREATE OR ALTER PROCEDURE dbo.sp_Demo_SetStep
82      @RunId UNIQUEIDENTIFIER,
83      @GiaoTac VARCHAR(5),
84      @Step INT,
85      @Action VARCHAR(20) = NULL
86  AS
```

Sau đó chương trình sẽ kiểm tra giao tác: IF @GiaoTac = 'T1'

Vì Bước 1 thuộc Transaction 1 nên chương trình thực hiện:

```
91      BEGIN
92          UPDATE dbo.DemoControl
93          SET T1Step = @Step,
94              T1Action = COALESCE(@Action, T1Action)
95          WHERE RunId = @RunId;
96      END
97      ELSE
98      BEGIN
```

Với @Step = 1 hệ thống sẽ cập nhật T1Step = 1 Bảng **DemoControl** đóng vai trò lưu trạng thái điều khiển của phiên mô phỏng, giúp Stored Procedure Runner biết người dùng đã thực hiện đến bước nào.

f. Runner của Transaction 1 xử lý Bước 1

Sau khi T1Step được cập nhật thành 1, Stored Procedure: sp_NRR_T1_Runner nhận biết rằng Transaction 1 đã được phép thực hiện Bước 1.

Runner kiểm tra trạng thái bước:

```
237        /* Chờ nút 1 */
238        WHILE 1 = 1
239        BEGIN
240            SELECT @Step = T1Step, @Cancelled = Cancelled
241            FROM dbo.DemoControl WITH (NOLOCK)
242            WHERE RunId = @RunId;
243
244            IF @Cancelled = 1 RETURN;
245            IF @Step >= 1 BREAK;
246            WAITFOR DELAY '00:00:00.150';
247        END;
248
```

Với điều kiện: IF @Step >= 1 BREAK; khi T1Step = 1 điều kiện đúng, Runner thoát khỏi vòng chờ và tiếp tục thực hiện Transaction 1.

g. Thiết lập mức cô lập READ COMMITTED

Sau khi nhận được tín hiệu thực hiện Bước 1, chương trình kiểm tra chế độ mô phỏng.

Trong sp_NRR_T1_Runner chương trình thực hiện:

```
396
397        IF @Mode = 'FIX'
398            SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
399        ELSE
400            SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
401
402        BEGIN TRANSACTION;
403
```

Do giao diện hiện tại đang ở chế độ: Mô phỏng lỗi - ERROR nên chương trình thực hiện:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Mức cô lập này cho phép dữ liệu đã được giao tác khác COMMIT có thể được nhìn thấy ở những lần đọc tiếp theo. Đây là cơ sở để mô phỏng hiện tượng Non-repeatable Read trong các bước sau.

Bắt đầu Transaction 1

Sau khi thiết lập mức cô lập, chương trình thực hiện: BEGIN TRANSACTION; đây là câu lệnh SQL trực tiếp mở Transaction 1.

Sau đó chương trình tạo nội dung log:

```
403
404         SET @Msg =
405             CASE WHEN @Mode = 'FIX'
406                  THEN N'① BEGIN TRAN - REPEATABLE READ'
407                  ELSE N'ⓘ BEGIN TRAN - READ COMMITTED'
408             END;
409
```

Vì chế độ hiện tại là ERROR, nội dung log được tạo ra là: ① BEGIN TRAN - READ COMMITTED sau đó log được ghi nhận thông qua:

```
409
410         EXEC dbo.sp_Demo_Log
411             @RunId, 'T1', 'INFO', @Msg;
412
```

Log này được hệ thống lấy lên giao diện và hiển thị:[18:03:28.318] ⓘ BEGIN TRAN - READ COMMITTED do đó, nội dung hiển thị trên giao diện là kết quả của quá trình xử lý từ GUI → BUS → DAL → Stored Procedure → Database → Log → GUI.

- Bước 2 – Transaction 1 đọc SDT khoa CNTT lần 1

Nội dung thực hiện

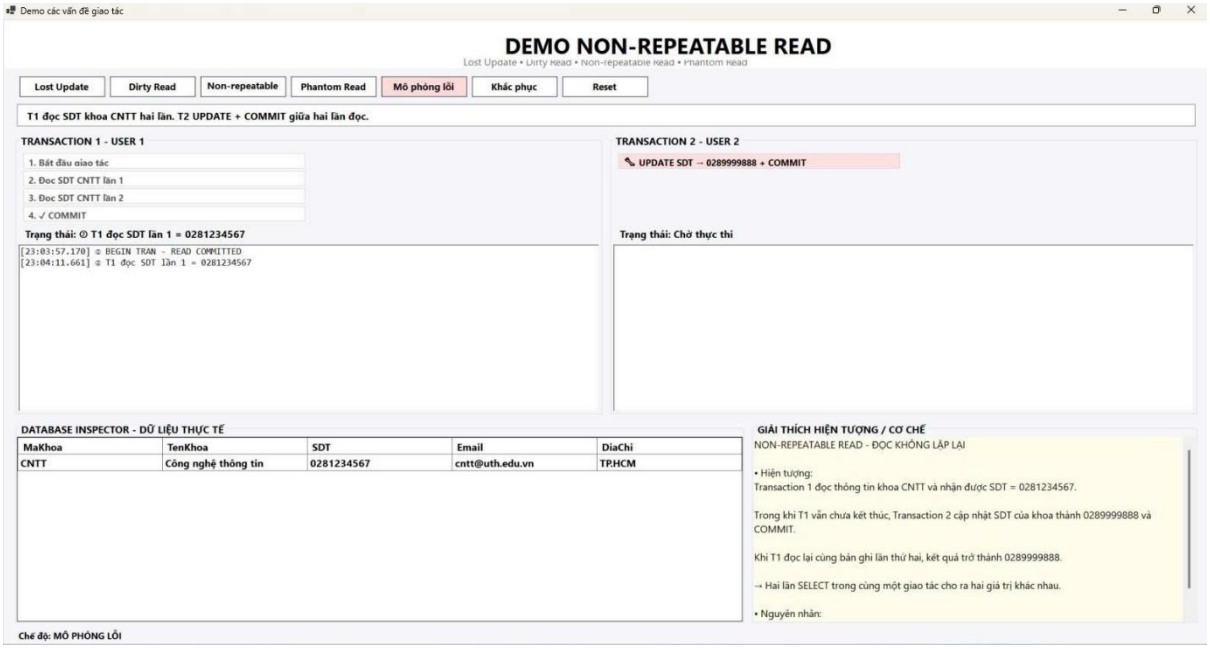
Sau khi Transaction 1 bắt đầu giao tác ở Bước 1, hệ thống thực hiện Bước 2: T1 đọc số điện thoại khoa CNTT lần thứ nhất.

Mục đích của bước này là lấy và ghi nhận giá trị ban đầu của số điện thoại trước khi Transaction 2 thực hiện cập nhật.

Theo giao diện mô phỏng, T1 đọc được: SDT = 0281234567

Trong khi đó, Transaction 2 vẫn ở trạng thái: Chờ thực thi.Điều này cho thấy tại thời điểm T1 đọc lần thứ nhất, T2 chưa thực hiện UPDATE, vì vậy dữ liệu trong cơ sở dữ liệu vẫn chưa thay đổi.

1 Giao diện sau khi thực hiện bước 2



Từ giao diện có thể quan sát được các nội dung sau:

Phần TRANSACTION 1 – USER 1 hiển thị 4 bước:

Bắt đầu giao tác.

Đọc SDT CNTT lần 1.

Đọc SDT CNTT lần 2.

COMMIT.

Trạng thái của T1 hiển thị:

T1 đọc SDT lần 1 = 0281234567

Phần log của T1 xuất hiện hai dòng:

BEGIN TRAN - READ COMMITTED

T1 đọc SDT lần 1 = 0281234567

Phần TRANSACTION 2 – USER 2 vẫn hiển thị:

Trạng thái: Chờ thực thi

Bảng DATABASE INSPECTOR – DỮ LIỆU THỰC TẾ vẫn hiển thị số điện thoại: 0281234567

Như vậy, giao diện đã thể hiện đúng trạng thái của hệ thống sau khi T1 hoàn thành lần đọc đầu tiên.

2. Code xử lý

a. Tầng giao diện:

Khi chọn nút “2. Đọc SDT CNTT lần 1”, sự kiện được xử lý thông qua phương thức:

```
1159        private async void StepButton_Click(
1160            object? sender,
1161            EventArgs e)
1162        {
1163            if (sender is not Button b ||
1164                b.Tag is not string tag)
1165                return;
1166
1167            try
1168            {
1169                if (scenario == "DR")
1170                    await XuLyDirtyAsync(tag);
1171                else if (scenario == "NRR")
1172                    await XuLyNRRAsync(tag);
1173                else if (scenario == "LU")
1174                    await XuLyLUAsync(tag);
1175                else
1176                    await XuLyPhantomAsync(tag);
1177
1178                await LoadInspectorAsync();
1179            }
1180            catch (Exception ex)
1181            {
1182                MessageBox.Show(
1183                    ex.Message,
1184                    "Lỗi thực thi",
1185                    MessageBoxButtons.OK,
1186                    MessageBoxIcon.Error);
1187            }
1188        }
1189
```

Khi hệ thống đang chạy kịch bản Non-repeatable Read, điều kiện: else if (scenario == "NRR")

await XuLyNRRAsync(tag);

được thục hiện. Sau đó chương trình chuyển sang phương thức XuLyNRRAsync() để xử lý cụ thể Bước 2 của T1.

Trong XuLyNRRAsync(), khi tag tương ứng với **T1-2**, chương trình gọi:

```
1203                    await bus.SetStepAsync(
1204                        runId, "T1", 2);
1205
```

Đoạn code này có nhiệm vụ thông báo cho hệ thống rằng Transaction 1 đã chuyển sang Bước 2. Các tham số được truyền vào gồm:

runId: xác định phiên mô phỏng.

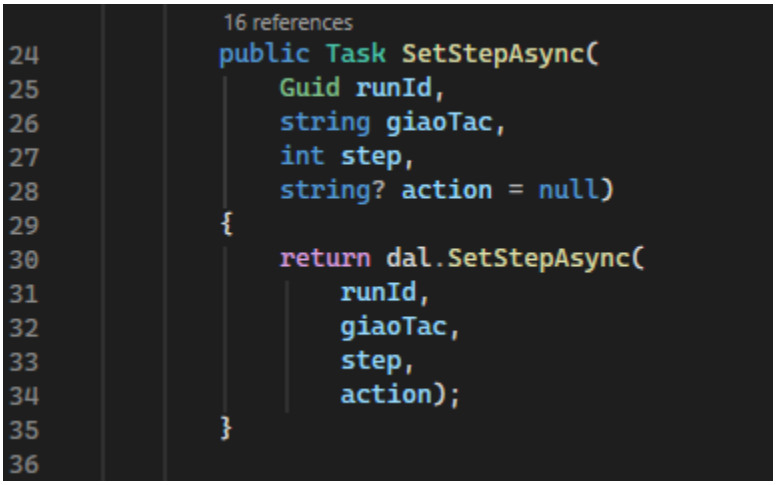
"T1": xác định Transaction 1.

2: xác định bước đang thực hiện.

Sau khi GUI gọi **bus.SetStepAsync()**, yêu cầu tiếp tục được chuyển xuống tầng BUS.

b. Tầng BUS

Tại tầng BUS, phương thức SetStepAsync() tiếp nhận yêu cầu từ GUI.



Tại đây, BUS không trực tiếp thực hiện câu lệnh SQL mà chuyển yêu cầu xuống DAL thông qua: dal.SetStepAsync(runId, giaoTac, step, action);

Với Bước 2, các giá trị được truyền xuống là:

runId = mã phiên hiện tại

giaoTac = "T1"

step = 2

Sau đó, yêu cầu tiếp tục đi đến DAL

c. Tầng DAL

Tại tầng DAL, phương thức SetStepAsync() nhận thông tin từ BUS.

```
1 reference
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return ExecuteProcedureAsync(
        "sp_Demo_SetStep",
        p =>
        {
            p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
            p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
            p.Add("@Step", SqlDbType.Int).Value = step;

            SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
            pa.Value = string.IsNullOrWhiteSpace(action)
                ? DBNull.Value
                : action;
        });
}
```

Tại đây, DAL xác định Stored Procedure cần gọi: "sp_Demo_SetStep"

và truyền vào:

@RunId → runId

@GiaoTac → "T1"

@Step → 2

@Action → NULL

Như vậy, DAL có nhiệm vụ **c**huyển yêu cầu từ tầng BUS xuống SQL Server thông qua Stored Procedure.

d. Phương thức thực thi Stored Procedure

Sau khi xác định Stored Procedure, DAL sử dụng phương thức ExecuteProcedureAsync() để thực thi.

```
16
17          using SqlConnection conn = Database.GetConnection();
18          await conn.OpenAsync();
19
20          using SqlCommand cmd = new SqlCommand(procedureName, conn);
21          cmd.CommandType = CommandType.StoredProcedure;
22          cmd.CommandTimeout = timeout;
```

Sau Stored Procedure được thực thi: using SqlDataReader reader = await cmd.ExecuteReaderAsync(); rồi qua SQL Server

e. Stored Procedure trong SQL

Stored Procedure khai báo sp_Demo_SetStep

```
80
81  CREATE OR ALTER PROCEDURE dbo.sp_Demo_SetStep
82      @RunId UNIQUEIDENTIFIER,
83      @GiaoTac VARCHAR(5),
84      @Step INT,
85      @Action VARCHAR(20) = NULL
86  AS
```

Sau đó chương trình cập nhật trạng thái bước của T1 trong bảng DemoControl:

```
90      IF @GiaoTac = 'T1'
91      BEGIN
92          UPDATE dbo.DemoControl
93          SET T1Step = @Step,
94              T1Action = COALESCE(@Action, T1Action)
95          WHERE RunId = @RunId;
96      END
```

Kết quả: T1Step = 2 Việc cập nhật T1Step = 2 đóng vai trò như một tín hiệu để Stored Procedure sp_NRR_T1_Runner biết rằng T1 đã được phép thực hiện Bước 2.

f. sp_NRR_T1_Runner thực hiện đọc dữ liệu

Stored Procedure xử lý Transaction 1 là: sp_NRR_T1_Runner

Trước khi thực hiện từng bước, chương trình kiểm tra trạng thái của T1 trong DemoControl:

```
240            SELECT @Step = T1Step, @Cancelled = Cancelled
241            FROM dbo.DemoControl WITH (NOLOCK)
242            WHERE RunId = @RunId;
243
```

Khi T1Step đã được cập nhật thành 2, Stored Procedure tiếp tục xử lý thao tác đọc dữ liệu lần thứ nhất.

Do chế độ đang được chọn trên giao diện là: chế độ mô phỏng lỗi nên bước 1 Transaction sử dụng:

```
333
334            SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Sau đó T1 thực hiện đọc số điện thoại khoa CNTT.

Kết quả đọc được là:0281234567

Giá trị này được ghi vào log và hiển thị trên giao diện: T1 đọc SDT lần 1 = 0281234567

Đây chính là dòng log nhìn thấy trong ảnh Bước 2.

g. Kết quả hiển thi trên giao diện

Sau khi Bước 2 hoàn thành, giao diện thể hiện:

Transaction 1

Trạng thái:

T1 đọc SDT lần 1 = 0281234567

Log:

[23:04:11.661] ① T1 đọc SDT lần 1 = 0281234567

Điều này chứng minh T1 đã thực hiện thành công lần đọc đầu tiên.

Transaction 2

Trạng thái:

Chờ thực thi

T2 chưa thực hiện thao tác UPDATE.

Như vậy, giá trị T1 đọc được hoàn toàn trùng với giá trị hiện tại trong Database.

- Bước 3 – Transaction 2 cập nhật SDT và COMMIT

Nội dung thực hiện:

Sau khi Transaction 1 đã đọc số điện thoại khoa CNTT lần thứ nhất với giá trị: 0281234567

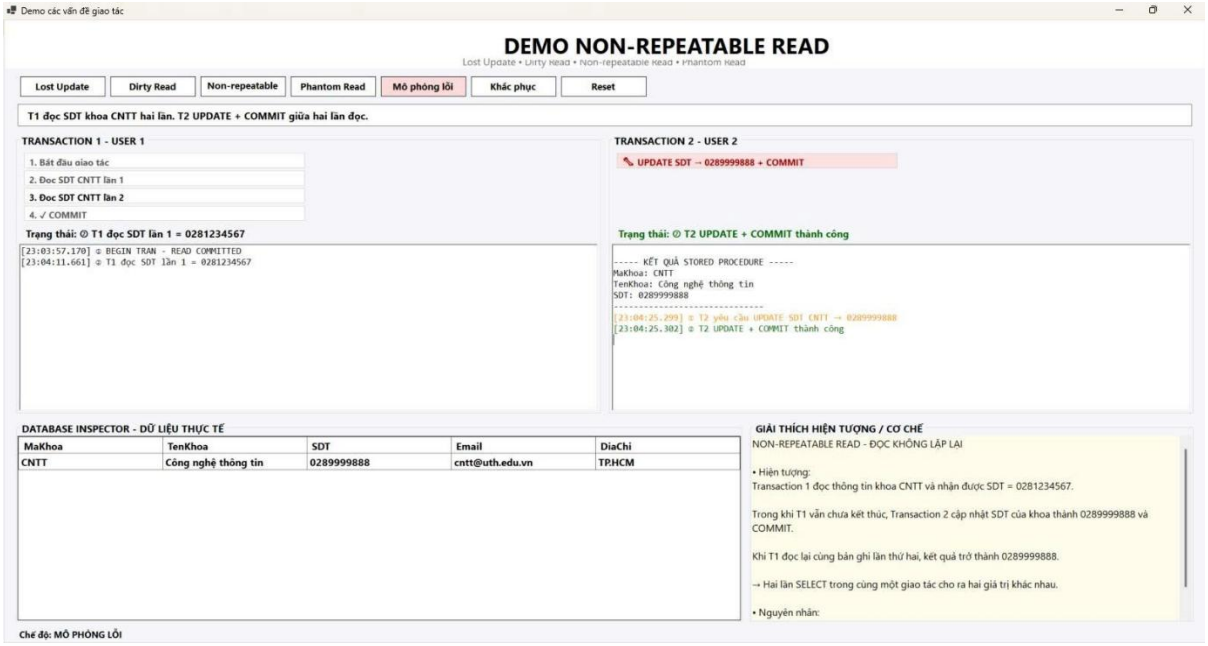
Transaction 2 được thực hiện để cập nhật số điện thoại của khoa CNTT thành 0289999888 và COMMIT.

Đây là bước quan trọng tạo ra điều kiện để xảy ra hiện tượng Non-repeatable Read.

Cụ thể, T1 vẫn đang trong giao tác và chưa COMMIT. Tuy nhiên, do kịch bản đang chạy ở chế độ MÔ PHỎNG LỖI – READ COMMITTED, sau khi câu lệnh SELECT của T1 kết thúc thì khóa đọc không còn được giữ cho toàn bộ giao tác. Vì vậy, T2 có thể thực hiện UPDATE dữ liệu.

Trình tự lúc này là: T1 đọc lần 1 = 0281234567 → T2 UPDATE = 0289999888 → T2 COMMIT → T1 đọc lại lần 2

1. Giao diện sau khi thực hiện Bước 3



Quan sát giao diện cho thấy:

Transaction 1

T1 vẫn hiển thị:

T1 đọc SDT lần 1 = 0281234567

Log của T1 vẫn giữ hai dòng:

① BEGIN TRAN - READ COMMITTED

② T1 đọc SDT lần 1 = 0281234567

Điều này cho thấy T1 chưa kết thúc giao tác và đang chờ bước đọc tiếp theo.

Transaction 2

Trạng thái của T2 chuyển thành:

T2 UPDATE + COMMIT thành công

Kết quả Stored Procedure hiển thị:

MaKhoa: CNTT

TenKhoa: Công nghệ thông tin

SDT: 0289999888

Đồng thời log hiển thị:

① T2 yêu cầu UPDATE SDT CNTT → 0289999888

② T2 UPDATE + COMMIT thành công

Database Inspector

Dữ liệu thực tế trong Database đã thay đổi từ: 0281234567 thành 0289999888

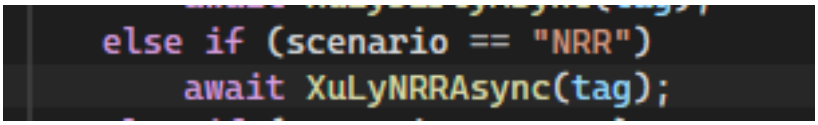
Đây là điểm quan trọng nhất của Bước 3: Database đã có giá trị mới, trong khi T1 vẫn chưa COMMIT.

2. Code xử lý

a. Tầng giao diện

Khi chọn nút: UPDATE SDT → 0289999888 + COMMIT sự kiện StepButton_Click() được gọi.

Trong phương thức này, nếu kịch bản là NRR, chương trình chuyển sang:



Sau đó chương trình đi vào XuLyNRRAsync() và xác định đây là thao tác của T2:

```
    else if (tag == "T2-1")
    {
        t2Buttons[0].Enabled = false;

        DataTable dt =
            await bus.RunAsync(
                "sp_DR_T2_Read",
                runId);

        HienThiKetQua(
            txtT2,
            dt);

        if (mode == "ERROR")
            t2Buttons[0].Enabled = true;
    }
}
```

Đoạn xử lý T2-1 nằm trong phần xử lý kịch bản NRR. Trong file code đã cung cấp, phần này thể hiện việc gọi sp_NRR_T2_Update, hiển thị kết quả và mở quyền cho T1 thực hiện bước đọc tiếp theo khi đang ở chế độ mô phỏng lỗi.

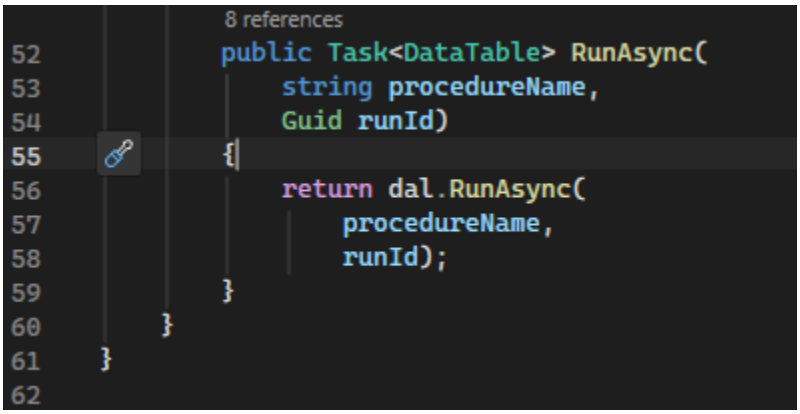
Ở đây có 3 việc chính:

- Thứ nhất, khóa nút T2: t2Buttons[0].Enabled = false; để tránh việc thực hiện cùng một thao tác nhiều lần.
- Thứ hai, gọi: await bus.RunAsync("sp_NRR_T2_Update", runId); để yêu cầu hệ thống thực hiện Stored Procedure cập nhật dữ liệu.
- Thứ ba, sau khi Stored Procedure hoàn thành, chương trình gọi: HienThiKetQua(txtT2, dt);để đưa kết quả UPDATE lên vùng hiển thị của Transaction 2.
- Cuối cùng: if (mode == "ERROR") t1Buttons[2].Enabled = true;cho phép T1 tiếp tục sang bước đọc lần thứ hai.

Như vậy, luồng từ GUI tiếp tục đi xuống tầng BUS:

b. Tầng BUS

Tại tầng BUS, GUI sử dụng phương thức RunAsync():



```
8 references
52          public Task<DataTable> RunAsync(
53              string procedureName,
54              Guid runId)
55          {
56              return dal.RunAsync(
57                  procedureName,
58                  runId);
59          }
60      }
61  }
62
```

Phương thức này có nhiệm vụ nhận tên Stored Procedure từ GUI rồi chuyển tiếp xuống DAL.

Đối với Bước 3, dữ liệu truyền xuống là:

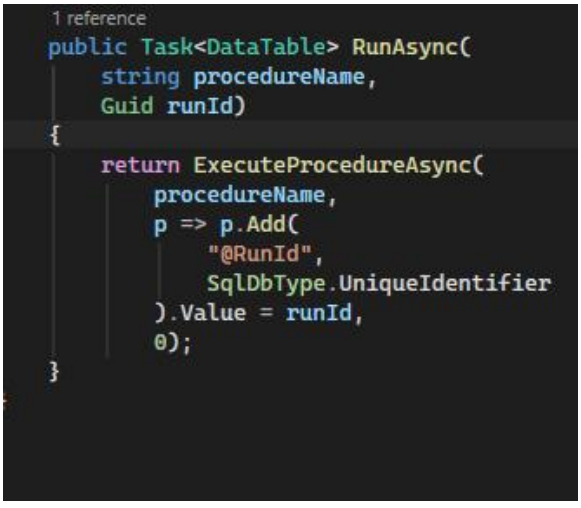
procedureName = "sp_NRR_T2_Update"

runId = mã phiên mô phỏng

Do đó, BUS không trực tiếp thực hiện UPDATE mà chuyển yêu cầu xuống DAL

c. Tầng ADL

Tại DAL, phương thức RunAsync() tiếp nhận tên Stored Procedure:



```
1 reference
public Task<DataTable> RunAsync(
    string procedureName,
    Guid runId)
{
    return ExecuteProcedureAsync(
        procedureName,
        p => p.Add(
            "@RunId",
            SqlDbType.UniqueIdentifier
        ).Value = runId,
        0);
}
```

Ở Bước 3: procedureName = "sp_NRR_T2_Update"

Nên DAL thực hiện Stored Procedure: sp_NRR_T2_Update

Và truyền vào: @RunId = runId

Sau đó DAL gọi ExecuteProcedureAsync() để thực thi Stored Procedure trên SQL Server.

d. Phương thức thực thi Stored Procedure

Trong DemoGiaoTacDAL.cs, phương thức ExecuteProcedureAsync() thực hiện kết nối đến SQL Server:

```
17          using SqlConnection conn = Database.GetConnection();
18          await conn.OpenAsync();
19
```

Sau đó tạo đối tượng SqlCommand:

```
20          using SqlCommand cmd = new SqlCommand(procedureName, conn);
21          cmd.CommandType = CommandType.StoredProcedure;
```

Cuối cùng Stored Procedure được thực thi:

```
26          using SqlDataReader reader = await cmd.ExecuteReaderAsync();
27
```

Như vậy, yêu cầu từ giao diện đã đi theo luồng: GUI → BUS → DAL → Stored Procedure

e. Stored Procedure trong SQL Server

Stored Procedure xử lý Transaction 2:

```
499   GO
500
501   CREATE OR ALTER PROCEDURE dbo.sp_NRR_T2_Update
502       @RunId UNIQUEIDENTIFIER
503   AS
504   BEGIN
505       SET NOCOUNT ON;
```

Trước khi UPDATE, chương trình ghi log: EXEC dbo.sp_Demo_Log @RunId, 'T2', 'WAIT',

N'① T2 yêu cầu UPDATE SDT CNTT → 0289999888';

Sau đó bắt đầu giao tác: BEGIN TRANSACTION;

Tiếp theo, T2 thực hiện UPDATE

```
511
512         UPDATE dbo.Khoa
513         SET SDT = '0289999888'
514         WHERE MaKhoa = 'CNTT';
515
```

Ý nghĩa:

- Tìm khoa có MaKhoa = 'CNTT'.
- Thay đổi cột SDT.
- Giá trị mới là 0289999888.

Sau khi UPDATE thành công, T2 thực hiện: COMMIT TRANSACTION;

Như vậy, thay đổi của T2 được xác nhận chính thức vào Database.

f. Ghi log T2 UPDATE + COMMIT thành công

Sau khi COMMIT, Stored Procedure tiếp tục ghi log: EXEC dbo.sp_Demo_Log @RunId, 'T2', 'SUCCESS',

N'② T2 UPDATE + COMMIT thành công';

Sau đó truy vấn lại dữ liệu:

```
519             N'② T2 UPDATE + COMMIT thà
520
521         SELECT MaKhoa, TenKhoa, SDT
522         FROM dbo.Khoa
523         WHERE MaKhoa = 'CNTT';
524     END;
```

Toàn bộ phần Stored Procedure sp_NRR_T2_Update

Kết quả trả về cho chương trình là:

- MaKhoa: CNTT
- TenKhoa: Công nghệ thông tin
- SDT: 0289999888

Đây chính là phần "KẾT QUẢ STORED PROCEDURE" đang hiển thị trong vùng Transaction 2 của ảnh.

g. Code hiển thị kết quả lên giao diện

Sau khi DAL trả kết quả về BUS rồi trở lại GUI, chương trình sử dụng: HienThiKetQua(txtT2,dt);

Phương thức HienThiKetQua() thực hiện đưa dữ liệu trả về từ Stored Procedure vào RichTextBox: box.AppendText(

```
Environment.NewLine +
"----- KẾT QUẢ STORED PROCEDURE -----" +
Environment.NewLine);
```

Sau đó chương trình duyệt qua từng dòng và từng cột để hiển thị:

```
box.AppendText(
    col.ColumnName +
    ": " +
    (row[col]?.ToString()
     ?? "") +
    Environment.NewLine);
```

Đoạn xử lý này được thể hiện trong phần HienThiKetQua()

Vì vậy, kết quả:

- MaKhoa: CNTT
- TenKhoa: Công nghệ thông tin
- SDT: 0289999888

được hiển thị trực tiếp ở khu vực của T2 như trong hình.

Bước 4 – Transaction 1 đọc SDT khoa CNTT lần 2

Nội dung thực hiện

Sau khi Transaction 2 đã cập nhật số điện thoại khoa CNTT từ: 0281234567 → 0289999888

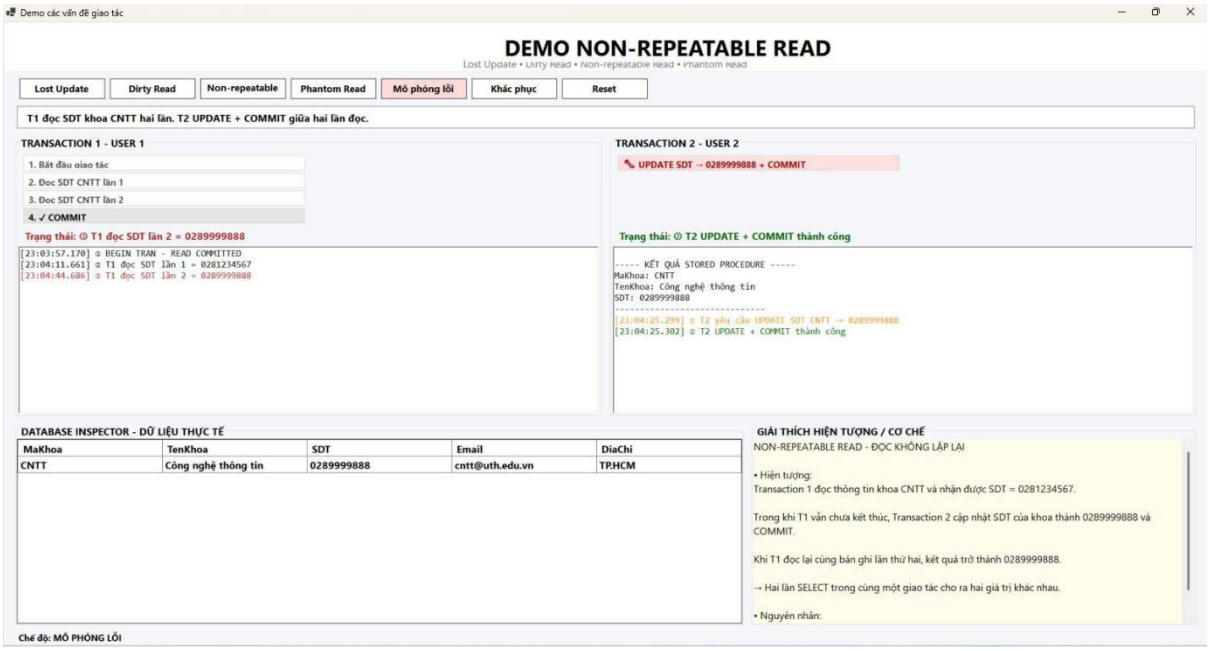
và thực hiện COMMIT, Transaction 1 tiếp tục thực hiện đọc số điện thoại lần thứ hai.

Kết quả lần đọc thứ hai là: 0289999888

Trong khi ở lần đọc thứ nhất, T1 nhận được: 0281234567 Như vậy, **c**ùng một Transaction T1, cùng một bản ghi và cùng một thao tác đọc nhưng nhận được hai giá trị khác nhau.

Đây chính là hiện tượng: NON-REPEATABLE READ – ĐỌC KHÔNG LẶP LẠI

1. Giao diện sau khi thực hiện bước 4



Quan sát giao diện cho thấy:

Transaction 1

Trạng thái của T1 chuyển thành:

T1 đọc SDT lần 2 = 0289999888

Phần log của T1 lúc này có 3 dòng:

1. BEGIN TRAN - READ COMMITTED
2. T1 đọc SDT lần 1 = 0281234567
3. T1 đọc SDT lần 2 = 0289999888

Điểm đáng chú ý là:

- Lần 1: 0281234567
- Lần 2: 0289999888

Hai lần đọc cho hai kết quả khác nhau.

Transaction 2

T2 vẫn hiển thị:

T2 UPDATE + COMMIT thành công

và kết quả Stored Procedure là:

- MaKhoa: CNTT
- TenKhoa: Công nghệ thông tin
- SDT: 0289999888

Hai lần đọc cho hai kết quả khác nhau.

Transaction 2

T2 vẫn hiển thị:

T2 UPDATE + COMMIT thành công

và kết quả Stored Procedure là:

- MaKhoa: CNTT
- TenKhoa: Công nghệ thông tin
- SDT: 0289999888

Database Inspector

Bảng dữ liệu thực tế cũng đang hiển thị:

SDT = 0289999888

Điều này chứng minh lần đọc thứ hai của T1 đã nhìn thấy giá trị mới đã được T2 COMMIT.

2. Code xử lý

a. Tầng giao diện

Khi thực hiện Bước 4 – Đọc SDT CNTT lần 2, sự kiện của nút vẫn được xử lý thông qua:

```
1 reference
private async void StepButton_Click(
    object? sender,
    EventArgs e)
{
    if (sender is not Button b ||
        b.Tag is not string tag)
        return;

    try
    {
        if (scenario == "DR")
            await XuLyDirtyAsync(tag);
        else if (scenario == "NRR")
            await XuLyNRRAsync(tag);
        else if (scenario == "LU")
            await XuLyLUAsync(tag);
        else
            await XuLyPhantomAsync(tag);

        await LoadInspectorAsync();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            ex.Message,
            "Lỗi thực thi",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}

1 reference
```

Vì kịch bản đang là: scenario = "NRR"

nên chương trình tiếp tục gọi: await XuLyNRRAsync(tag);

Sau đó XuLyNRRAsync() kiểm tra tag của nút T1-3 để xử lý lần đọc thứ hai.

T1 yêu cầu thực hiện Bước 3 trong giao tác

Tại phần xử lý T1 đọc lần thứ hai, chương trình gọi: await bus.SetStepAsync(runId, "T1" 3); các giá trị truyền xuống là:

runId = mã phiên mô phỏng

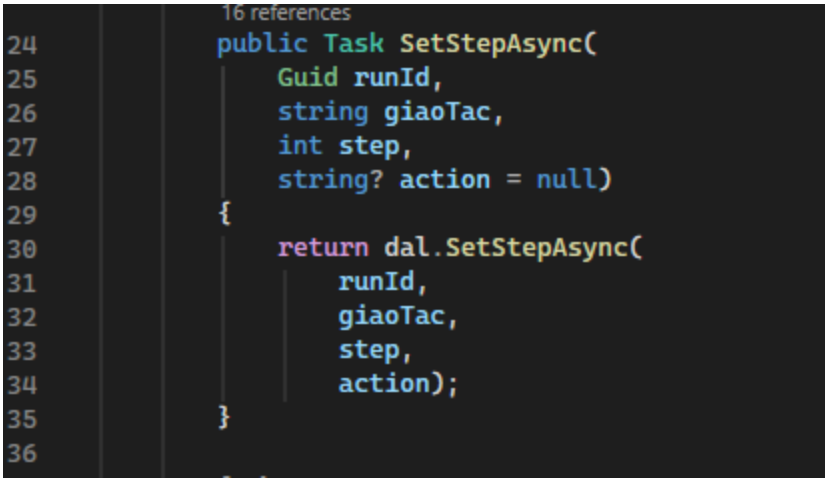
GiaoTac = "T1", Step = 3

Ý nghĩa của đoạn code này là thông báo cho SQL Server rằng Transaction 1 đã chuyển sang bước đọc dữ liệu lần thứ hai.

Sau khi GUI gọi BUS, luồng xử lý tiếp tục:

b. BUS

Tại tầng BUS, yêu cầu được tiếp nhận bởi phương thức SetStepAsync().

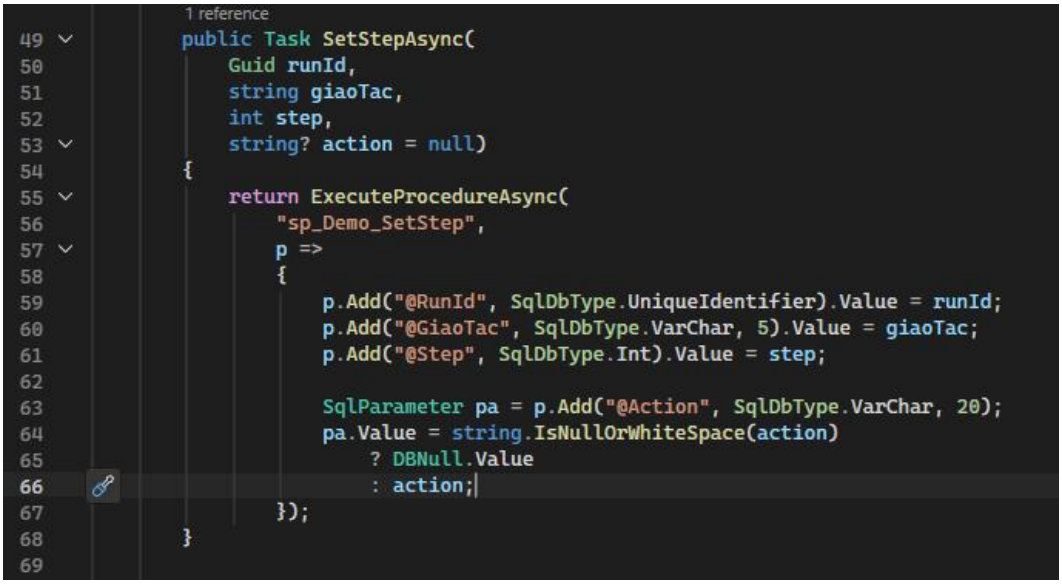


BUS nhận: GiaoTac = T1 , Step = 3

BUS không trực tiếp truy vấn dữ liệu mà chỉ đóng vai trò trung gian chuyển yêu cầu.

Sau đó chuyển yêu cầu xuống DAL: return dal.SetStepAsync(runId, giaoTac,step, action);

c. DAL



```
1 reference
public Task SetStepAsync(
    Guid runId,
    string giaoTac,
    int step,
    string? action = null)
{
    return ExecuteProcedureAsync(
        "sp_Demo_SetStep",
        p =>
        {
            p.Add("@RunId", SqlDbType.UniqueIdentifier).Value = runId;
            p.Add("@GiaoTac", SqlDbType.VarChar, 5).Value = giaoTac;
            p.Add("@Step", SqlDbType.Int).Value = step;

            SqlParameter pa = p.Add("@Action", SqlDbType.VarChar, 20);
            pa.Value = string.IsNullOrWhiteSpace(action)
                ? DBNull.Value
                : action;
        });
}
```

Với Bước 4, DAL truyền xuống SQL Server:

@RunId = runId

@GiaoTac = "T1"

@Step = 3

@Action = NULL

DAL tiếp tục gọi: sp_Demo_SetStep để cập nhật trạng thái bước của T1

Stored Procedure trong SQL Server

sp_Demo_SetSte nhận: @GiaoTac = 'T1'

@Step = 3

Stored Procedure thực hiện:

```
89
90      IF @GiaoTac = 'T1'
91      BEGIN
92          UPDATE dbo.DemoControl
93          SET T1Step = @Step,
94              T1Action = COALESCE(@Action, T1Action)
95          WHERE RunId = @RunId;
96      END
```

Kết quả: T1Step = 3 Đây là tín hiệu để sp_NRR_T1_Runner tiếp tục thực hiện lần đọc thứ hai.

sp_NRR_T1_Runner thực hiện lần đọc thứ hai

Sau khi T1Step được cập nhật thành 3, Stored Procedure sp_NRR_T1_Runner tiếp tục thực hiện bước tương ứng.

Điểm quan trọng là T1 vẫn đang trong giao tác trước đó.

Ở chế độ MÔ PHỎNG LỖI, mức cô lập của giao tác là:

```
334             SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

T1 đã đọc: Lần 1 → 0281234567

Sau đó T2 đã: UPDATE → 0289999888→ COMMIT

Do T2 đã COMMIT nên giá trị mới đã trở thành dữ liệu được xác nhận trong Database.

Khi T1 thực hiện SELECT lần thứ hai, T1 đọc được: 0289999888

Vì vậy log trên giao diện xuất hiện: T1 đọc SDT lần 2 = 0289999888

Code hiển thị trên giao diện

Sau khi Bước 4 hoàn thành, phần trạng thái của T1 hiển thị: T1 đọc SDT lần 2 = 0289999888

Log của T1 thể hiện đầy đủ quá trình:

[23:03:57.170] BEGIN TRAN - READ COMMITTED

[23:04:11.661] T1 đọc SDT lần 1 = 0281234567

[23:04:44.686] T1 đọc SDT lần 2 = 0289999888

Hai lần đọc trong cùng một giao tác T1 nhưng trả về: hai giá trị khác nhau

Nguyên nhân là giữa hai lần đọc, Transaction 2 đã thay đổi dữ liệu và COMMIT.

Vì vậy, kết quả đọc của T1 không còn lặp lại được như lần đọc trước .Đây chính là thời điểm hiện tượng Non-repeatable Read được thể hiện rõ ràng.

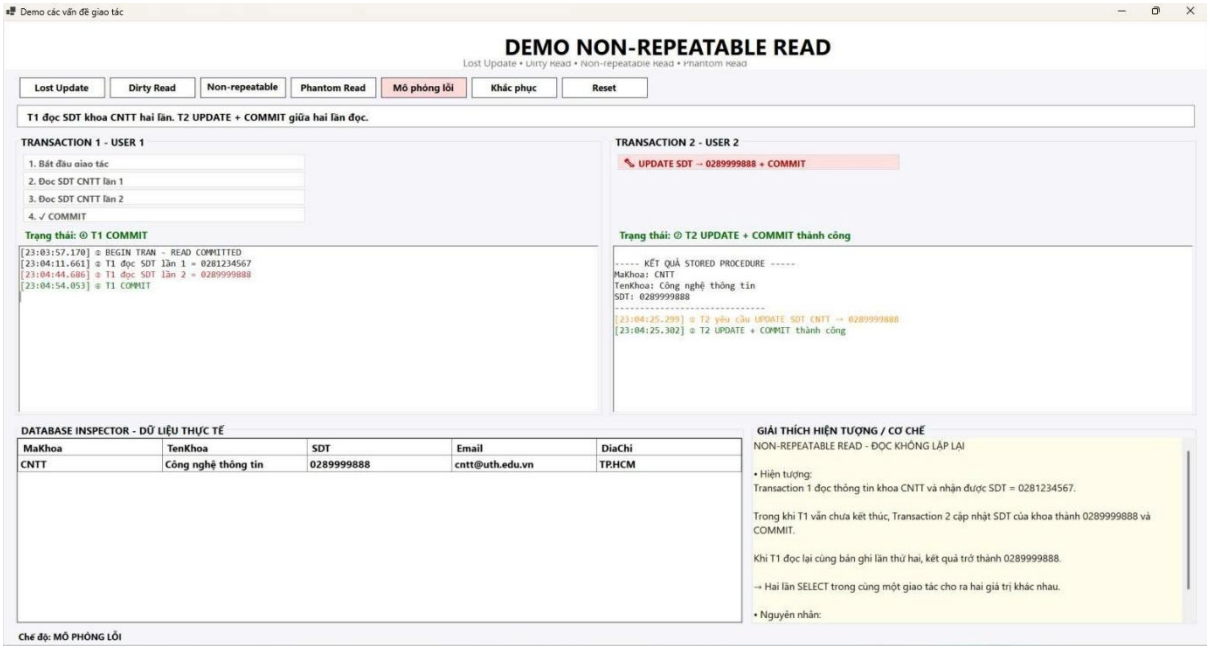
- Bước 5 – Transaction 1 thực hiện COMMIT

Nội dung thực hiện

Sau khi Transaction 1 hoàn thành lần đọc thứ hai và phát hiện hiện tượng dữ liệu bị thay đổi do Transaction 2 đã UPDATE và COMMIT ở Bước 3, người dùng thực hiện nhấn nút "4. ✓ COMMIT" để kết thúc giao tác Transaction 1.

Mục đích của bước này là giải phóng hoàn toàn tài nguyên và xác nhận việc đóng Transaction 1, hoàn tất toàn bộ quy trình mô phỏng lỗi Non-repeatable Read.

1. Giao diện sau khi thực hiện Bước 5



Từ giao diện có thể quan sát được các nội dung sau:

Transaction 1:

Trạng thái của T1 chuyển thành: Trạng thái: ④ T1 COMMIT.

Phần log của T1 xuất hiện đầy đủ 4 dòng nhật ký xử lý:

- [HH:mm:ss.fff] ① BEGIN TRAN - READ COMMITTED
- [HH:mm:ss.fff] ② T1 đọc SDT lần 1 = 0281234567
- [HH:mm:ss.fff] ③ T1 đọc SDT lần 2 = 0289999888
- [HH:mm:ss.fff] ④ T1 COMMIT

Tất cả các nút bấm của Transaction 1 đều bị vô hiệu hóa (Enabled = false) do giao tác đã kết thúc.

Transaction 2:

Trạng thái vẫn giữ nguyên kết quả ở Bước 3: T2 UPDATE + COMMIT thành công.

Kết quả Stored Procedure hiển thị thông tin Khoa CNTT với SDT: 0289999888.

Database Inspector – Dữ liệu thực tế:

Hiển thị chính thức dữ liệu hiện tại trong cơ sở dữ liệu với SDT = 0289999888.

2. Code xử lý

a.Tầng giao diện

Khi người dùng kích hoạt nút "4. ✓ COMMIT" trên giao diện (nút này ứng với tag T1-4), sự kiện StepButton_Click() được gọi và điều hướng tới phương thức XuLyNRRAsync(tag):

```
1303                await Task.Delay(250);
1304            }
1305            else if (tag == "T1-4")
1306            {
1307                await bus.SetStepAsync(
1308                    runId,
1309                    "T1",
1310                    4,
1311                    "COMMIT");
1312
1313                t1Buttons[3].Enabled = false;
1314            }
1315        }
1316
```

runId: Mã phiên mô phỏng hiện tại.

"T1" & 4: Xác định Transaction 1 đang chuyển sang Bước 4.

"COMMIT": Tham số hành động (Action) được gửi xuống để báo hiệu kết thúc giao tác.

Vô hiệu hóa nút t1Buttons[3] để tránh bấm lặp lại.

b. BUS

Tại tầng BUS, yêu cầu COMMIT của T1 được tiếp nhận và chuyển tiếp xuống tầng DAL.

BUS đóng vai trò trung gian giữa giao diện và DAL, không trực tiếp thực hiện câu lệnh SQL.

c. DAL

DAL tiếp nhận yêu cầu COMMIT và thực hiện gọi Stored Procedure tương ứng trên SQL Server.

Sau khi Stored Procedure thực hiện thành công, trạng thái của Transaction 1 được cập nhật thành hoàn thành.

Kết quả sau cùng được trả về giao diện để hiển thị trạng thái:

T1 COMMIT

Stored Procedure trong SQL Server

Ở bước này, hệ thống thực hiện thao tác COMMIT cho Transaction 1.

Khi Transaction 1 COMMIT:

Transaction T1 kết thúc.

Các tài nguyên và khóa của T1 được giải phóng.

T1 hoàn tất toàn bộ quá trình mô phỏng.

Trong trường hợp Mô phỏng lỗi, việc T2 đã UPDATE và COMMIT trước khi T1 đọc lần 2 mới là nguyên nhân làm cho T1 đọc được giá trị 0289999888. File Word cũng xác định đây là điểm thể hiện rõ Non-Repeatable Read.

Code hiển thị trên giao diện

Sau khi Bước 5 hoàn thành, phần trạng thái của T1 hiển thị:

T1 COMMIT

Log của T1: BEGIN TRAN - READ COMMITTED

T1 đọc SDT lần 1 = 0281234567

T1 đọc SDT lần 2 = 0289999888

T1 COMMIT

Hai lần đọc trong cùng một Transaction:

Lần 1 = 0281234567

Lần 2 = 0289999888

→ Hai giá trị khác nhau.

Nguyên nhân là T2 đã UPDATE và COMMIT giữa hai lần đọc của T1. Đây chính là hiện tượng Non-Repeatable Read được mô phỏng trong chương trình.

3.4.2. Khắc phục Non-Repeatable Read

- Transaction 1 bắt đầu giao tác với mức cô lập REPEATABLE READ

Nội dung thực hiện

Tại chế độ Khắc phục, khi chuyển sang bước đầu tiên của kịch bản Non-repeatable Read, hệ thống vẫn thực hiện mở Transaction 1 để chuẩn bị cho các thao tác đọc dữ liệu. Tuy nhiên, điểm khác biệt quan trọng là Transaction 1 sẽ được thiết lập với mức cô lập REPEATABLE READ (thay vì READ COMMITTED như ở chế độ mô phỏng lỗi)

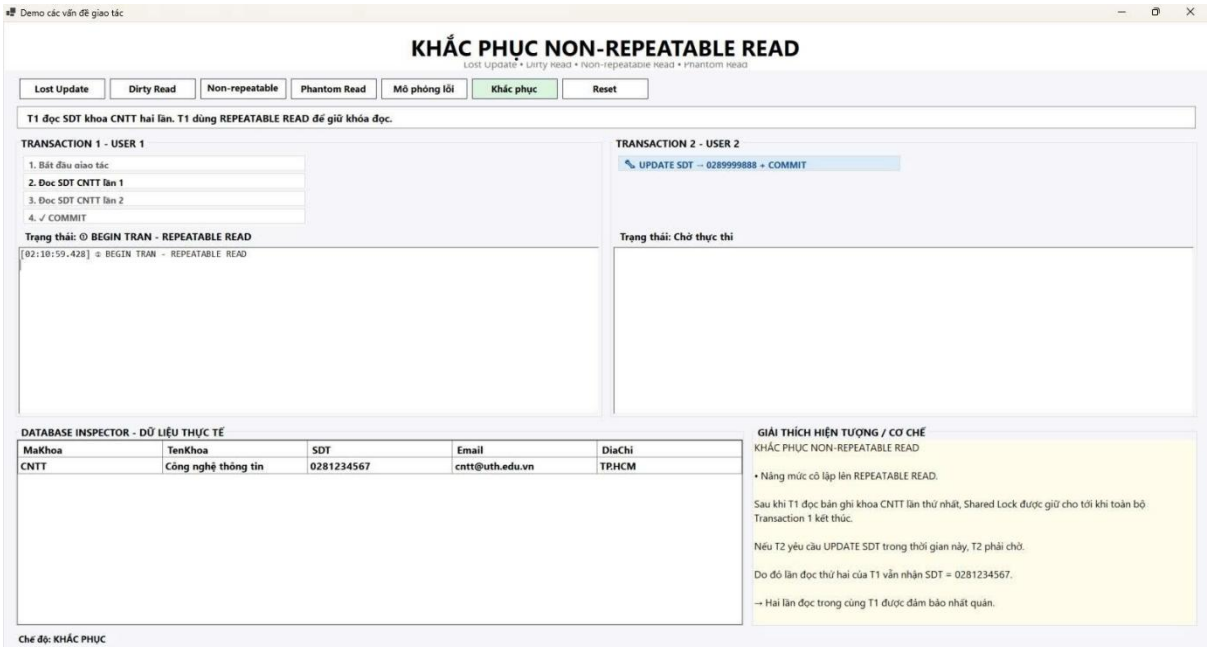
Sau khi thực hiện bước "1. Bắt đầu giao tác" ở chế độ Khắc phục, giao diện hiển thị:

Trạng thái: ① BEGIN TRAN - REPEATABLE READ

Nhật ký (Log) của Transaction 1: [18:05:12.415] ⓘ BEGIN TRAN - REPEATABLE READ

Điều này minh chứng rằng Transaction 1 đã khởi tạo thành công dưới cơ chế bảo vệ của mức cô lập cao hơn, chuẩn bị giữ các khóa đọc (Shared Lock) cần thiết nhằm ngăn chặn hiện tượng Non-repeatable Read xảy ra ở các bước tiếp theo.

1. Giao diện khi chọn chế độ Khắc phục



Trên giao diện, người dùng chọn: Non-repeatable Read → Khắc phục

Khi chuyển sang chế độ này, tiêu đề giao diện thay đổi thành:

KHẮC PHỤC NON-REPEATABLE READ

Phần mô tả tình huống hiển thị:

> T1 đọc SDT khoa CNTT hai lần. T1 dùng REPEATABLE READ để giữ khóa đọc.

Các bước của Transaction 1 vẫn gồm:

1. Bắt đầu giao tác
2. Đọc SDT CNTT lần 1
3. Đọc SDT CNTT lần 2
4. ✓ COMMIT

Transaction 2 có nút:

UPDATE SDT → 0289999888 + COMMIT

Điều này được cấu hình trực tiếp trong phương thức CauHinhNonRepeatable()

2. Code xử lý

a. Tầng giao diện

Khi người dùng đang ở chế độ Khắc phục và nhấn nút bắt đầu giao tác của T1 (Tag tương ứng T1-1), sự kiện StepButton_Click() được kích hoạt. Chương trình tiếp tục gọi phương thức xử lý kịch bản: await XuLyNRRAsync(tag);

Trong phương thức này, với tag = "T1-1", hệ thống gọi tiếp xuống tầng BUS thông qua: await bus.SetStepAsync(runId, "T1", 1);

b. BUS

Tầng GUI không trực tiếp thực hiện câu lệnh SQL.

GUI gọi: bus.SetStepAsync(runId, "T1", 1);

BUS tiếp tục chuyển yêu cầu xuống DAL thông qua: return dal.SetStepAsync(runId, giaoTac,step,action);

BUS có nhiệm vụ trung gian truyền thông tin từ GUI xuống DAL

c. DAL

Tại DAL, phương thức SetStepAsync() gọi Stored Procedure: sp_Demo_SetStep

Và truyền các tham số:

@RunId → runId

@GiaoTac → "T1"

@Step → 1

@Action → NULL

DAL sau đó thực hiện Stored Procedure thông qua ExecuteProcedureAsync()

Phương thức thực thi Stored Procedure

sp_Demo_SetStep cập nhật trạng thái của Transaction 1 trong bảng DemoControl.

Kết quả: T1Step = 1

Đây là tín hiệu để sp_NRR_T1_Runner biết rằng T1 đã được phép thực hiện Bước 1.

Thiết lập REPEATABLE READ

Trong sp_NRR_T1_Runner, chương trình kiểm tra chế độ hiện tại.

Nếu: @Mode = 'FIX'

thì chương trình thực hiện: SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

Sau đó mới thực hiện: BEGIN TRANSACTION;

Và log được tạo: ① BEGIN TRAN - REPEATABLE READ

Đây là điểm khác biệt quan trọng nhất giữa Mô phỏng lỗi và Khắc phục.

Trong chế độ lỗi: SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Còn trong chế độ khắc phục: SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-Khắc phục Bước 2 – Transaction 1 đọc SDT lần 1

Nội dung thực hiện

Sau khi T1 bắt đầu giao tác, người dùng chọn:

2. Đọc SDT CNTT lần 1

T1 thực hiện đọc số điện thoại của khoa CNTT.

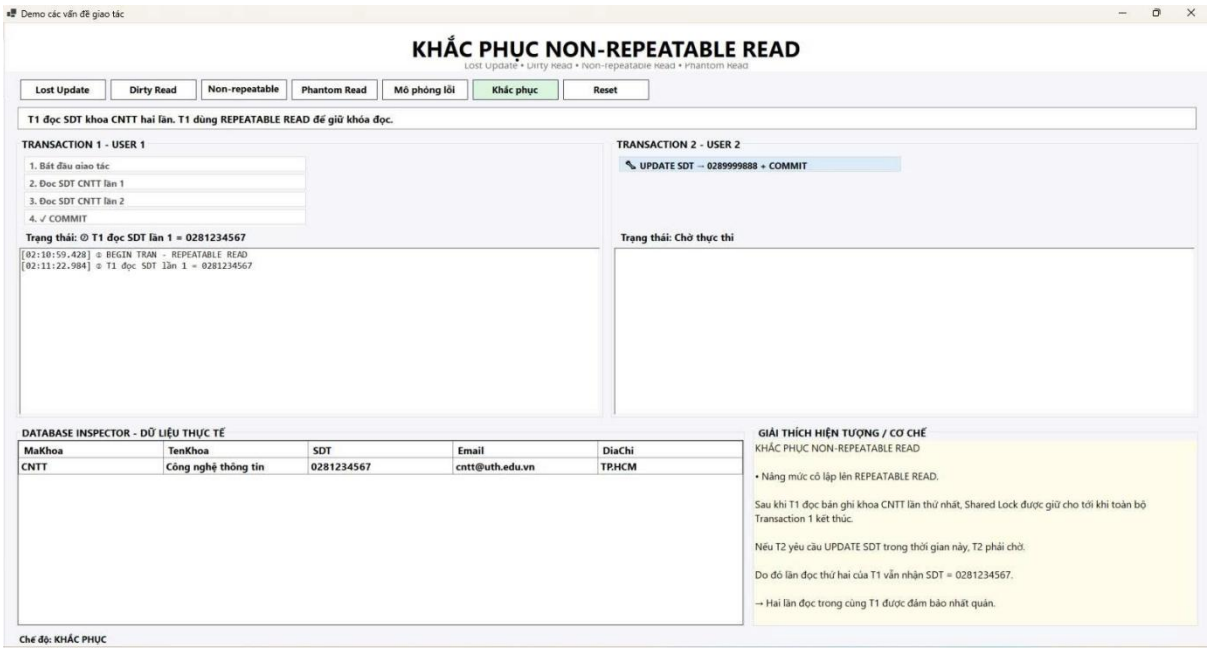
Kết quả:

SDT = 0281234567

Ngay sau khi đọc bản ghi này, trong chế độ REPEATABLE READ, Shared Lock của T1 được giữ lại.

Điều này có nghĩa là T2 không

1. Giao diện sau khi thực hiện Bước 2



Trên giao diện có thể thấy:

Transaction 1

Trạng thái:

T1 đọc SDT lần 1 = 0281234567

Log:

① BEGIN TRAN - REPEATABLE READ

② T1 đọc SDT lần 1 = 0281234567

Transaction 2

Vẫn đang ở trạng thái:

Chờ thực thi

Database Inspector

Dữ liệu vẫn là:

SDT = 0281234567

Như vậy, sau lần đọc thứ nhất, T1 đã giữ khóa đọc trên bản ghi khoa CNTT.

2. Code xử lý

a. Tầng giao diện

Khi chọn nút: Đọc SDT CNTT lần 1" sự kiện được xử lý thông qua phương thức: StepButton_Click()

Khi hệ thống đang chạy kịch bản Non-Repeatable Read, chương trình kiểm tra:

else if (scenario == "NRR") await XuLyNRRAsync(tag);

Sau đó chương trình chuyển sang phương thức: XuLyNRRAsync() để xử lý cụ thể Bước 2 của Transaction 1.

Trong XuLyNRRAsync(), khi tag tương ứng với: T1-2

chương trình gọi: await bus.SetStepAsync(runId, "T1", 2);

Đoạn code này có nhiệm vụ thông báo cho hệ thống rằng Transaction 1 đã chuyển sang Bước 2.

Các tham số được truyền vào gồm:

- runId: xác định phiên mô phỏng hiện tại.
- "T1": xác định giao tác đang thực hiện là Transaction 1.
- 2: xác định bước hiện tại là Bước 2.

Sau khi GUI gọi SetStepAsync(), yêu cầu tiếp tục được chuyển xuống tầng BUS. Cách truyền bước từ GUI xuống BUS/DAL được trình bày trong tài liệu của phần Mô phỏng lỗi.

b. BUS

Tại tầng BUS, phương thức: SetStepAsync() tiếp nhận yêu cầu từ GUI.

BUS không trực tiếp thực hiện câu lệnh SQL mà chuyển yêu cầu xuống DAL thông qua:

dal.SetStepAsync(runId,giaoTac, step,action);

Với Bước 2, các giá trị được truyền xuống là: runId = mã phiên hiện tại

giaoTac = "T1"

step = 2

Sau đó yêu cầu tiếp tục được chuyển đến tầng DAL.

c. DAL

Tại tầng DAL, phương thức: SetStepAsync() nhận thông tin từ BUS.

DAL xác định Stored Procedure cần gọi là: sp_Demo_SetStep và truyền vào các tham số:

@RunId → runId

@GiaoTac → "T1"

@Step → 2

@Action → NULL

Phương thức thực hiện Stored Procedure

sp_Demo_SetStep tiếp nhận thông tin Bước 2 của Transaction 1 và cập nhật trạng thái của Transaction 1 trong bảng điều khiển mô phỏng.

Kết quả:T1Step = 2

Giá trị T1Step = 2 đóng vai trò như một tín hiệu để Stored Procedure Runner của Transaction 1 biết rằng T1 đã được phép thực hiện thao tác đọc SDT lần thứ nhất.

sp_NRR_T1_Runner thực hiện đọc dữ liệu

Stored Procedure xử lý Transaction 1 là: sp_NRR_T1_Runner

Runner kiểm tra trạng thái bước của T1 trong bảng điều khiển.

Khi: T1Step = 2

Stored Procedure tiếp tục thực hiện thao tác đọc dữ liệu lần thứ nhất.

Do chương trình đang chạy ở chế độ Khắc phục, Transaction 1 sử dụng mức cô lập:

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

Sau đó T1 thực hiện đọc số điện thoại của khoa CNTT.

Kết quả đọc được:

0281234567

Giá trị này được ghi vào log và hiển thị trên giao diện:

T1 đọc SDT lần 1 = 0281234567

Điểm quan trọng ở đây là với REPEATABLE READ, Shared Lock của T1 đối với bản ghi đã đọc được giữ lại cho đến khi Transaction 1 kết thúc. Vì vậy, Transaction 2 không thể UPDATE bản ghi này trong thời gian T1 chưa COMMIT.

Kết quả hiển thị trên giao diện

Sau khi Bước 2 hoàn thành, giao diện thể hiện:

Transaction 1

Trạng thái:

T1 đọc SDT lần 1 = 0281234567

Log:

① BEGIN TRAN – REPEATABLE READ
② T1 đọc SDT lần 1 = 0281234567

Điều này chứng minh T1 đã thực hiện thành công lần đọc đầu tiên.

Transaction 2

Trạng thái:

Chờ thực thi

T2 chưa thể thực hiện UPDATE tại thời điểm này.

Database Inspector

Dữ liệu thực tế vẫn là:

SDT = 0281234567

Như vậy, sau Bước 2, T1 đã đọc được dữ liệu ban đầu và giữ Shared Lock trên bản ghi. Khi chuyển sang bước tiếp theo, nếu T2 yêu cầu UPDATE SDT thành 0289999888, T2 sẽ phải chờ cho đến khi T1 COMMIT.

→ Đây là điểm khác biệt quan trọng so với chế độ Mô phỏng lỗi READ COMMITTED và là cơ chế chính giúp khắc phục Non-Repeatable Read.

- Khắc phục – Bước 3 – Transaction 2 yêu cầu UPDATE SDT

Nội dung thực hiện

Sau khi Transaction 1 đã đọc số điện thoại khoa CNTT lần thứ nhất với giá trị: 0281234567

Transaction 2 được thực hiện để yêu cầu cập nhật số điện thoại của khoa Công nghệ thông tin thành: 0289999888

Tuy nhiên, khác với chế độ Mô phỏng lỗi – READ COMMITTED, ở chế độ Khắc phục – REPEATABLE READ, Transaction 1 đang giữ Shared Lock trên bản ghi đã đọc.

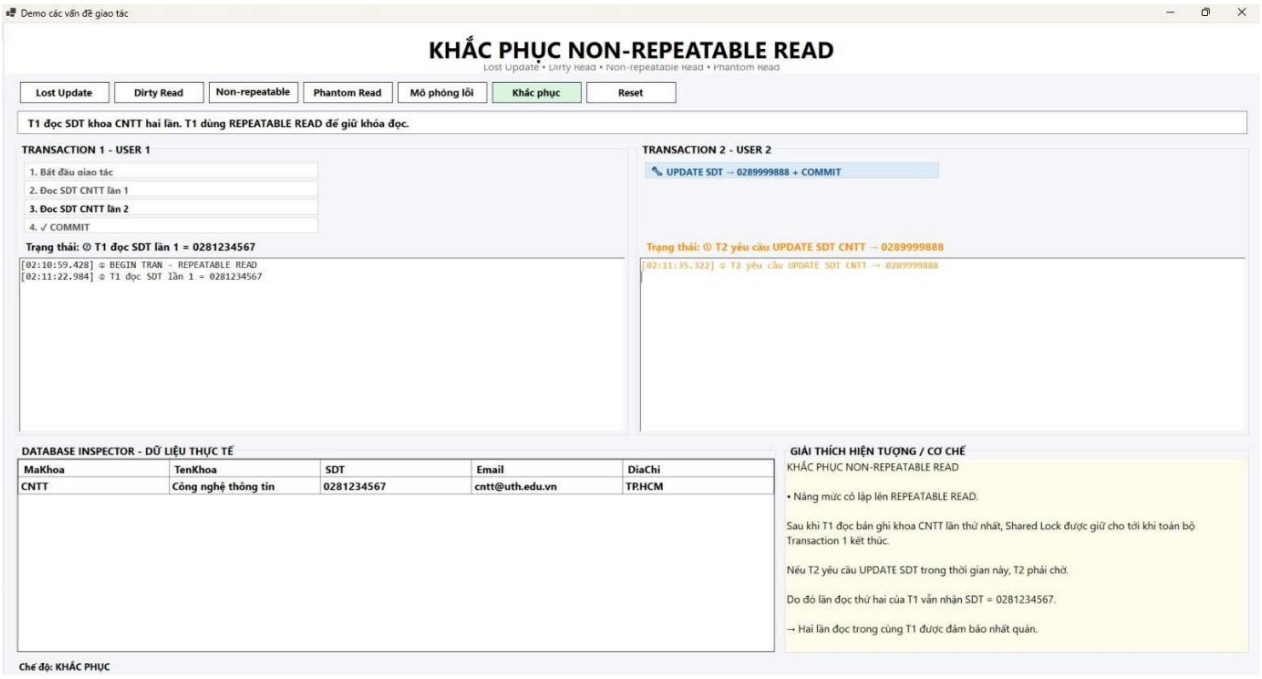
Vì vậy, khi Transaction 2 thực hiện yêu cầu UPDATE, T2 không thể cập nhật ngay mà phải chờ Transaction 1 kết thúc giao tác và giải phóng khóa.

Trình tự lúc này là:

T1 đọc lần 1 = 0281234567 → T1 giữ Shared Lock → T2 yêu cầu UPDATE = 0289999888 → T2 phải CHỜ → T1 đọc lần 2 → T1 COMMIT → T2 mới được UPDATE.

Đây chính là cơ chế quan trọng giúp khắc phục hiện tượng Non-Repeatable Read.

1. Giao diện sau khi thực hiện Bước 3



Quan sát giao diện cho thấy:

Transaction 1

T1 vẫn hiển thị:

T1 đọc SDT lần 1 = 0281234567

Phần log của T1 vẫn có:

① BEGIN TRAN – REPEATABLE READ

② T1 đọc SDT lần 1 = 0281234567

Điều này cho thấy T1 chưa kết thúc giao tác và vẫn đang giữ khóa đọc trên bản ghi khoa CNTT.

Transaction 2

Sau khi thực hiện nút:

UPDATE SDT → 0289999888 + COMMIT

trạng thái của T2 chuyển thành:

T2 yêu cầu UPDATE SDT CNTT → 0289999888

Log của T2 hiển thị:

① T2 yêu cầu UPDATE SDT CNTT → 0289999888

Tuy nhiên, T2 chưa hiển thị UPDATE + COMMIT thành công.

Nguyên nhân là T2 đang phải chờ Shared Lock do T1 đang giữ.

Database Inspector

Dữ liệu thực tế trong Database vẫn hiển thị:

MaKhoa: CNTT

TenKhoa: Công nghệ thông tin

SDT: 0281234567

Email: cntt@uth.edu.vn

DiaChi: TP.HCM

Như vậy, tại thời điểm T2 yêu cầu UPDATE, dữ liệu trong Database **chưa thay đổi.**

Đây là điểm quan trọng của Bước 3: T2 đã yêu cầu UPDATE nhưng chưa thể cập nhật dữ liệu vì T1 vẫn đang giữ khóa.

2. Code xử lý

a. Tầng giao diện

Khi chọn nút:

“UPDATE SDT → 0289999888 + COMMIT”

sự kiện StepButton_Click() được gọi.

Vì kịch bản đang là Non-repeatable Read, chương trình chuyển sang: await XuLyNRRAsync(tag);

Sau đó XuLyNRRAsync() xác định đây là thao tác của T2.

Ở bước này, chương trình gửi yêu cầu thực hiện Stored Procedure cập nhật SDT của T2.

Tuy nhiên, do đang ở chế độ KHẮC PHỤC, T2 sẽ không thể hoàn thành UPDATE ngay khi T1 vẫn còn giữ Shared Lock.

b. BUS

Từ GUI, yêu cầu được chuyển xuống tầng BUS.

BUS tiếp nhận yêu cầu thực hiện thao tác UPDATE của Transaction 2 và chuyển tiếp xuống DAL.

BUS đóng vai trò trung gian, không trực tiếp thực hiện câu lệnh SQL.

c. DAL

Tại DAL, yêu cầu được chuyển xuống SQL Server để thực hiện Stored Procedure xử lý Transaction 2.

Stored Procedure được sử dụng để thực hiện thao tác UPDATE SDT.

Tuy nhiên, do bản ghi đang bị Shared Lock của T1, thao tác UPDATE của T2 phải chờ cho đến khi khóa được giải phóng.

d. Cơ chế REPEATABLE READ trong Bước 3

Điểm quan trọng nhất ở Bước 3 là:

T1 đã đọc bản ghi → Shared Lock được giữ → T2 yêu cầu UPDATE → T2 phải chờ.

Do T1 sử dụng mức cô lập:

REPEATABLE READ

nên Shared Lock không được giải phóng ngay sau câu lệnh SELECT đầu tiên mà được giữ cho đến khi Transaction 1 kết thúc.

Vì vậy, dù T2 yêu cầu thay đổi:

0281234567 → 0289999888

Database vẫn chưa thay đổi.

→ T2 phải chờ T1 COMMIT trước khi có thể UPDATE.

Điều này đảm bảo rằng khi T1 thực hiện lần đọc thứ hai ở bước tiếp theo, T1 vẫn nhận được:

SDT = 0281234567

và hai lần đọc trong cùng Transaction T1 được đảm bảo nhất quán.

- Khắc phục – Bước 4 – Transaction 1 COMMIT

Nội dung thực hiện

Sau khi Transaction 1 đã đọc số điện thoại khoa CNTT hai lần và cả hai lần đều nhận được cùng một giá trị:

Lần 1: 0281234567

Lần 2: 0281234567

Transaction 1 thực hiện bước cuối cùng là COMMIT để kết thúc giao tác.

Trong thời gian T1 thực hiện hai lần đọc, Transaction 2 đã yêu cầu UPDATE SDT từ:

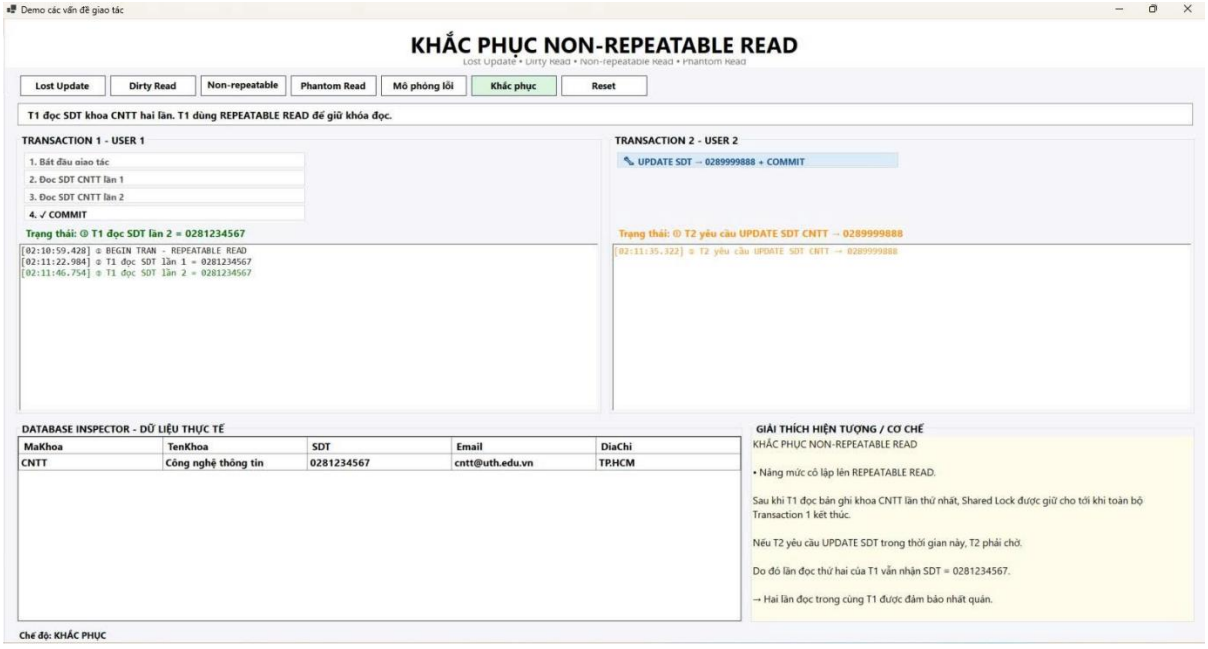
0281234567 → 0289999888

Tuy nhiên, do T1 sử dụng mức cô lập REPEATABLE READ, Shared Lock trên bản ghi khoa CNTT vẫn được giữ. Vì vậy T2 chưa thể thực hiện UPDATE và đang ở trạng thái chờ.

Khi T1 thực hiện COMMIT, giao tác T1 kết thúc và Shared Lock được giải phóng.

Sau đó Transaction 2 mới có thể tiếp tục thực hiện UPDATE SDT thành: 0289999888 và COMMIT.

1. Giao diện sau khi thực hiện bước 4



Quan sát giao diện sau khi chọn:

4. ✓ COMMIT

Transaction 1

Trạng thái của T1 chuyển sang trạng thái hoàn thành giao tác.

Phần log của T1 thể hiện đầy đủ quá trình:

1. BEGIN TRAN – REPEATABLE READ
2. T1 đọc SDT lần 1 = 0281234567
3. T1 đọc SDT lần 2 = 0281234567
4. T1 COMMIT

Điểm đáng chú ý là:

Lần 1: 0281234567

Lần 2: 0281234567

Hai lần đọc trong cùng một giao tác T1 cho cùng một kết quả.

Điều này chứng minh mức cô lập REPEATABLE READ đã giữ dữ liệu ổn định trong suốt thời gian T1 thực hiện giao tác.

Transaction 2

Trước khi T1 COMMIT, T2 đang ở trạng thái:

T2 yêu cầu UPDATE SDT CNTT → 0289999888

và phải chờ do T1 đang giữ Shared Lock.

Sau khi T1 COMMIT, T2 được giải phóng và tiếp tục thực hiện:

T2 UPDATE + COMMIT thành công

Kết quả cập nhật của T2 là:

MaKhoa: CNTT

TenKhoa: Công nghệ thông tin

SDT: 0289999888

Database Inspector

Sau khi T2 UPDATE + COMMIT thành công, dữ liệu thực tế trong Database được cập nhật:

SDT = 0289999888

Điểm quan trọng là mặc dù dữ liệu cuối cùng đã được T2 thay đổi thành 0289999888, nhưng trước thời điểm T1 COMMIT, cả hai lần đọc của T1 đều nhận được 0281234567.

2. Code xử lý

a. Tầng giao diện

Khi thực hiện Bước 4 – Đọc SDT CNTT lần 2, sự kiện của nút được xử lý thông qua:

await XuLyNRRAsync(tag);

Sau đó XuLyNRRAsync() xác định đây là thao tác T1 đọc SDT lần thứ hai.

T1 yêu cầu chuyển sang bước đọc tiếp theo trong giao tác.

Chương trình gọi: await bus.SetStepAsync(runId, "T1", 3);

Các giá trị được truyền xuống:

- runId = mã phiên mô phỏng.
- GiaoTac = "T1".
- Step = 3.

Ý nghĩa của đoạn code này là thông báo cho SQL Server rằng **T**ransaction 1 đã chuyển sang bước đọc SDT lần thứ hai. Cách xử lý này tương ứng với luồng Bước 4 trong phần Mô phỏng lỗi của tài liệu

b. BUS

Tại tầng BUS, yêu cầu được tiếp nhận bởi phương thức: SetStepAsync()

BUS nhận: GiaoTac = T1 , Step = 3

BUS không trực tiếp truy vấn dữ liệu mà chuyển yêu cầu xuống DAL:

return dal.SetStepAsync(runId, giaoTac, step, action);

c. DaL

Tại DAL, phương thức SetStepAsync() tiếp tục xử lý yêu cầu.

DAL truyền xuống SQL Server:

@RunId = runId

@GiaoTac = "T1"

@Step = 3

@Action = NULL

Sau đó DAL gọi: sp_Demo_SetStep để cập nhật trạng thái bước của T1.

Stored Procedure trong SQL Server

Stored Procedure sp_Demo_SetStep nhận: @GiaoTac = 'T1'

@Step = 3

Sau khi thực hiện, trạng thái được cập nhật: T1Step = 3 Đây là tín hiệu để sp_NRR_T1_Runner tiếp tục thực hiện thao tác đọc SDT lần thứ hai.

sp_NRR_T1_Runner thực hiện lần đọc thứ hai

Sau khi T1Step được cập nhật thành 3, sp_NRR_T1_Runner tiếp tục thực hiện bước đọc thứ hai.

Điểm quan trọng ở chế độ KHẮC PHỤC là Transaction 1 vẫn đang sử dụng mức cô lập:

REPEATABLE READ

T1 đã đọc lần đầu:

0281234567

Sau đó T2 yêu cầu:

UPDATE → 0289999888

Nhưng do T1 đang giữ Shared Lock nên T2 phải chờ.

Vì vậy, khi T1 thực hiện SELECT lần thứ hai, dữ liệu vẫn chưa bị T2 thay đổi.

T1 tiếp tục đọc được:

0281234567

Do đó log trên giao diện xuất hiện:

T1 đọc SDT lần 2 = 0281234567

Code hiển thị trên giao diện

Sau khi Bước 4 hoàn thành, phần trạng thái của T1 hiển thị:

T1 đọc SDT lần 2 = 0281234567

Log của T1 thể hiện đầy đủ quá trình:

1. BEGIN TRAN – REPEATABLE READ
2. T1 đọc SDT lần 1 = 0281234567
3. T1 đọc SDT lần 2 = 0281234567

Hai lần đọc trong cùng một giao tác T1 đều trả về:

0281234567

→ Hai lần đọc cùng một bản ghi cho cùng một giá trị.

Điều này chứng minh rằng Non-Repeatable Read đã được khắc phục nhờ sử dụng mức cô lập REPEATABLE READ.

- Khắc phục – Transaction 1 COMMIT

Nội dung thực hiện

Sau khi Transaction 1 đã thực hiện đọc SDT khoa CNTT lần thứ hai và vẫn nhận được: 0281234567

T1 tiếp tục thực hiện bước cuối cùng là:

COMMIT

để kết thúc giao tác.

Khác với chế độ Mô phỏng lỗi, ở chế độ KHẮC PHỤC – REPEATABLE READ, T1 đã giữ Shared Lock trên bản ghi trong suốt thời gian giao tác.

Do đó, trước khi T1 COMMIT:

T2 phải chờ

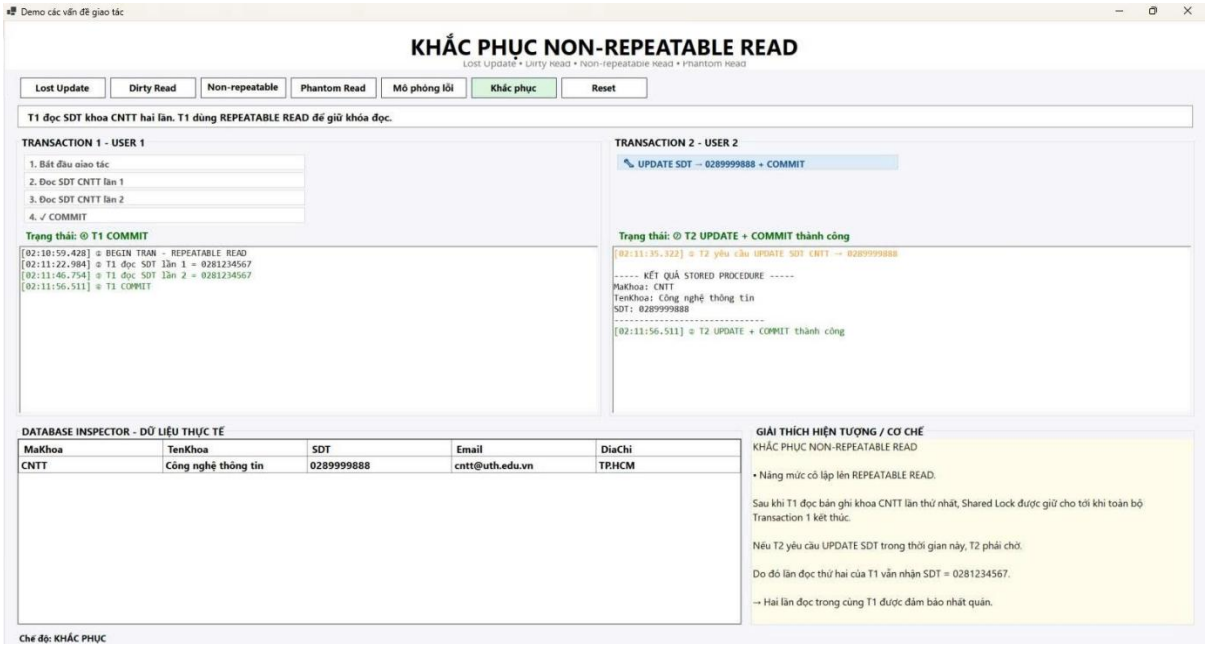
và chưa thể UPDATE dữ liệu.

Trình tự thực hiện là:

T1 đọc lần 1 = 0281234567 → T2 yêu cầu UPDATE → T2 chờ → T1 đọc lần 2 = 0281234567 → T1 COMMIT → T2 UPDATE = 0289999888 → T2 COMMIT.

Sau khi T1 COMMIT, Shared Lock được giải phóng. Lúc này T2 mới được phép tiếp tục thực hiện UPDATE và COMMIT.

1. Giao diện sau khi thực hiện Bước 5



Transaction 1

Sau khi chọn:

“4. COMMIT”

trạng thái của T1 chuyển thành:

T1 COMMIT

Log của T1 thể hiện: ① BEGIN TRAN - REPEATABLE READ

1. T1 đọc SDT lần 1 = 0281234567
2. T1 đọc SDT lần 2 = 0281234567
3. T1 COMMIT

Điểm quan trọng là cả hai lần đọc đều nhận được:

0281234567

Sau đó T1 mới COMMIT.

Transaction 2

Trước thời điểm T1 COMMIT, T2 đang ở trạng thái:

T2 yêu cầu UPDATE SDT CNTT → 0289999888

Sau khi T1 COMMIT và giải phóng Shared Lock, T2 tiếp tục thực hiện UPDATE.

Trạng thái T2 chuyển thành:

T2 UPDATE + COMMIT thành công

Kết quả Stored Procedure hiển thị: MaKhoa: CNTT

TenKhoa: Công nghệ thông tin

SDT: 0289999888

Điều này cho thấy T2 chỉ có thể thay đổi dữ liệu sau khi T1 kết thúc giao tác.

Database Inspector

Sau khi T2 UPDATE và COMMIT thành công, dữ liệu thực tế trong Database được cập nhật thành:

SDT = 0289999888

Các thông tin còn lại:

- MaKhoa: CNTT
- TenKhoa: Công nghệ thông tin
- Email: cntt@uth.edu.vn
- DiaChi: TP.HCM

2. Code xử lý

a. Tầng giao diện

Khi chọn nút:

“4. COMMIT”

sự kiện StepButton_Click() được gọi.

Do đang chạy kịch bản NRR, chương trình tiếp tục xử lý thông qua:await XuLyNRRAsync(tag); Tại đây, thao tác COMMIT của T1 được gửi xuống tầng BUS để cập nhật trạng thái giao tác.

b. BUS

BUS tiếp nhận yêu cầu COMMIT của T1 và chuyển yêu cầu xuống DAL.

BUS đảm nhiệm việc điều phối yêu cầu giữa giao diện và tầng truy cập dữ liệu.

c. DAL

DAL thực hiện yêu cầu trên SQL Server.

Khi T1 COMMIT thành công, Transaction 1 kết thúc và Shared Lock được giải phóng.

Việc giải phóng khóa cho phép Transaction 2 tiếp tục thực hiện thao tác UPDATE đang chờ.

Cơ chế khắc phục

Điểm quan trọng nhất ở Bước 5 là thứ tự thực hiện giữa T1 và T2.

Ở Mô phỏng lỗi:

T2 UPDATE + COMMIT trước → T1 đọc lần 2 → nhận 0289999888.

Ở Khắc phục:

T2 yêu cầu UPDATE → T2 chờ → T1 đọc lần 2 = 0281234567 → T1 COMMIT → T2 mới UPDATE + COMMIT.

Như vậy, việc sử dụng REPEATABLE READ đã ngăn không cho T2 thay đổi bản ghi giữa hai lần đọc của T1.

Code hiển thị trên giao diện

Sau khi hoàn thành Bước 5, log T1: BEGIN TRAN - REPEATABLE READ

T1 đọc SDT lần 1 = 0281234567

T1 đọc SDT lần 2 = 0281234567

T1 COMMIT

T2: T2 yêu cầu UPDATE SDT CNTT → 0289999888

T2 UPDATE + COMMIT thành công

Database cuối cùng:

SDT = 0289999888

Nhưng điểm quan trọng là:

T1 đọc lần 1 = 0281234567

T1 đọc lần 2 = 0281234567

→ Hai lần đọc trong cùng một Transaction **giống nhau.**

→ Hiện tượng Non-Repeatable Read đã được khắc phục.

→ Sau khi T1 COMMIT, T2 mới được phép UPDATE và COMMIT.

3.5. Hiện tượng bóng ma (Phantom Read)

Bóng ma (Phantom) xảy ra khi một giao tác thực hiện cùng một truy vấn nhiều lần nhưng giữa các lần truy vấn, một giao tác khác thêm hoặc xóa các bản ghi thỏa mãn điều kiện truy vấn. Khi truy vấn lại, số lượng bản ghi trả về thay đổi.

Ví dụ:

Giả sử T1 thực hiện truy vấn:

SELECT * FROM SinhVien WHERE MaKhoa = 'CNTT';

- Lần đầu T1 truy vấn và nhận được 50 sinh viên thuộc khoa CNTT
- T2 thêm một sinh viên mới thuộc khoa CNTT và COMMIT
- T1 thực hiện lại cùng truy vấn
- Lần này kết quả trả về 51 sinh viên

Sinh viên mới xuất hiện trong lần truy vấn thứ hai được gọi là bóng ma, vì bản ghi này không tồn tại trong tập kết quả của lần truy vấn đầu tiên. Số lượng hoặc tập bản ghi trong cùng một giao tác có thể thay đổi, gây ra sự không nhất quán khi xử lý dữ liệu.

Nhờ các cơ chế điều khiển tương tranh, hệ quản trị cơ sở dữ liệu có thể cho phép nhiều giao tác thực hiện đồng thời nhưng vẫn duy trì tính nhất quán và độ tin cậy của dữ liệu.

3.5.1. Mô phỏng hiện tượng Phantom Read

Transaction 1 bắt đầu giao tác với mức cô lập READ COMMITTED

Tại Bước 1, Transaction 1 bắt đầu một giao tác với mức cô lập READ COMMITTED.

Mức cô lập này chỉ đảm bảo Transaction 1 đọc được những dữ liệu đã được COMMIT. Tuy nhiên, nó không khóa phạm vi dữ liệu mà Transaction 1 sẽ truy vấn.

Do đó, trong thời gian Transaction 1 chưa kết thúc, Transaction 2 vẫn có thể thêm một bản ghi mới thỏa điều kiện truy vấn của Transaction 1.

Bước 1 – Transaction 1 bắt đầu giao tác

Mô phỏng lỗi – Transaction 1 bắt đầu giao tác

Nội dung thực hiện

Tại bước đầu tiên, hệ thống thực hiện mở Transaction 1 để chuẩn bị cho các thao tác truy vấn số lượng chương trình đào tạo thuộc ngành HHTT ở các bước tiếp theo.

Do chương trình đang ở chế độ MÔ PHỎNG LỖI**,** Transaction 1 sử dụng mức cô lập:

READ COMMITTED

Sau khi thực hiện bước:

"1. Bắt đầu giao tác"

Transaction 1 được mở và chờ các bước truy vấn tiếp theo.

Ở thời điểm này, Transaction 1 **chưa thực hiện truy vấn số lượng chương trình đào tạo.**

Transaction 2 cũng **chưa thực hiện INSERT CT99** vào ngành HHTT.

Vì vậy, dữ liệu trong Database vẫn giữ nguyên trạng thái ban đầu.

Theo giao diện mô phỏng, bảng **DATABASE INSPECTOR – DỮ LIỆU THỰC TẾ** đang hiển thị chương trình:

MaCTDT: CT05

TenCTDT: Hệ thống thông tin

TrìnhĐộ: Đại học

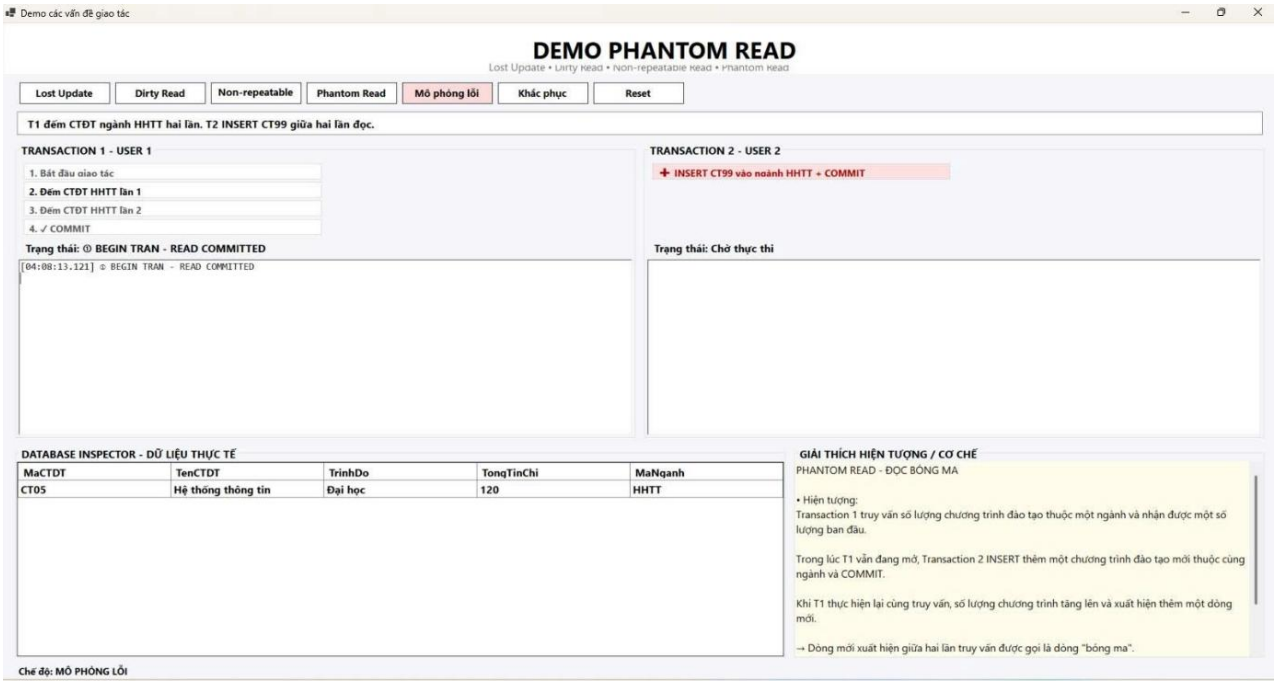
TổngTínChỉ: 120

MaNganh: HHTT

Như vậy, ở Bước 1 mới chỉ thực hiện việc mở Transaction 1, chưa xảy ra hiện tượng Phantom Read.

Đây là bước chuẩn bị để T1 thực hiện đếm số lượng CTĐT ngành HHTT lần thứ nhất ở Bước 2.

1. Giao diện sau khi thực hiện Bước 1



Từ giao diện có thể quan sát được các nội dung sau:

Transaction 1 – User 1

Phần TRANSACTION 1 – USER 1 hiển thị 4 bước:

1. Bắt đầu giao tác

2. Đếm CTĐT HHTT lần 1

3. Đếm CTĐT HHTT lần 2

4. ✓ COMMIT

Sau khi nhấn:

“1. Bắt đầu giao tác”

trạng thái của T1 hiển thị:

BEGIN TRAN - READ COMMITTED

Phần log của T1 xuất hiện: ① BEGIN TRAN - READ COMMITTED

Điều này cho thấy Transaction 1 đã được mở thành công và đang duy trì giao tác.

thực hiện thao tác INSERT.

Trạng thái của T2:

Chờ thực thi

Điều này cho thấy chưa có chương trình CT99 mới được thêm vào Database.

Database Inspector

Bảng dữ liệu thực tế vẫn hiển thị dữ liệu ban đầu, trong đó chương trình CT05 thuộc ngành:

HHTT

Chưa có CT99.

Như vậy, sau Bước 1, Database chưa thay đổi.

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn nút:"1. Bắt đầu giao tác"

sự kiện StepButton_Click() được gọi.

Trong phương thức này, chương trình kiểm tra kịch bản đang được lựa chọn.

Vì Phantom Read không thuộc các trường hợp Dirty Read, Non-repeatable Read hay Lost Update nên chương trình chuyển sang:

else

{

```
    await XuLyPhantomAsync(tag);
}
```

Đoạn code này xác định rằng yêu cầu hiện tại thuộc kịch bản Phantom Read.

Sau đó chương trình đi vào phương thức: private async Task XuLyPhantomAsync(string tag)

Tại đây chương trình kiểm tra: if (tag == "T1-1")

Khi tag = "T1-1", chương trình gọi: await bus.SetStepAsync(

```
    runId, "T1", 1);
```

Đoạn code này có nhiệm vụ thông báo cho hệ thống rằng:

runId: mã phiên mô phỏng hiện tại.

"T1": xác định Transaction 1.

1: xác định T1 đang thực hiện Bước 1.

Sau đó nút Bước 1 bị khóa: t1Buttons[0].Enabled = false;

và nút Bước 2 được mở: t1Buttons[1].Enabled = true;

Điều này cho phép người dùng tiếp tục thực hiện **“**2. Đếm CTĐT HHTT lần 1”

b. BUS

Sau khi GUI gọi: bus.SetStepAsync(

```
    runId, "T1", 1); yêu cầu được chuyển xuống tầng BUS.
```

BUS tiếp nhận thông tin: runId = mã phiên mô phỏng

giaoTac = T1

step = 1

BUS không trực tiếp thực hiện câu lệnh SQL mà chuyển yêu cầu tiếp xuống tầng DAL.

Luồng xử lý lúc này là:

GUI → BUS → DAL → Stored Procedure → Database

c. DAL

Tại DAL, yêu cầu cập nhật trạng thái Bước 1 của T1 được tiếp nhận.

DAL sử dụng phương thức SetStepAsync() để chuyển yêu cầu xuống SQL Server thông qua Stored Procedure quản lý trạng thái bước.

Các thông tin được truyền xuống gồm:

@RunId → runId

@GiaoTac → T1

@Step → 1

@Action → NULL

Mục đích là cập nhật trạng thái của Transaction 1 trong phiên mô phỏng.

Stored Procedure trong SQL Server

Sau khi DAL gửi yêu cầu xuống SQL Server, Stored Procedure quản lý trạng thái bước tiếp nhận:

Transaction = T1

Step = 1

Sau đó trạng thái của T1 trong bảng điều khiển được cập nhật thành:

T1Step = 1

Việc cập nhật T1Step = 1 đóng vai trò như tín hiệu cho Stored Procedure Runner biết rằng T1 đã được phép thực hiện Bước 1.

Sau khi nhận được tín hiệu này, Runner tiếp tục thực hiện Transaction 1.

Đối với chế độ MÔ PHỎNG LỖI, Transaction 1 sử dụng mức cô lập:

READ COMMITTED

Mức cô lập này được code SQL của phần mô phỏng sử dụng cho kịch bản lỗi; tài liệu cũng mô tả rằng READ COMMITTED cho phép dữ liệu đã được giao tác khác COMMIT có thể được nhìn thấy ở các lần đọc tiếp theo

Code hiển thị trên giao diện

Sau khi thực hiện xong Bước 1, giao diện hiển thị:

T1 Trạng thái: ① BEGIN TRAN - READ COMMITTED

Log: ① BEGIN TRAN - READ COMMITTED

T2 Trạng thái: Chờ thực thi

Database

Dữ liệu vẫn chưa thay đổi.

Chưa xuất hiện CT99 mới trong ngành HHTT.

- Mô phỏng lỗi Bước 2 – Transaction 1 đếm CTĐT ngành HHTT lần 1

Sau khi Transaction 1 bắt đầu giao tác ở Bước 1, hệ thống thực hiện Bước 2: T1 đếm số lượng chương trình đào tạo thuộc ngành HHTT lần thứ nhất.

Người dùng chọn nút:

2. Đếm CTĐT HHTT lần 1

Transaction 1 thực hiện truy vấn số lượng chương trình đào tạo có MaNganh = HHTT.

Theo dữ liệu ban đầu trong Database Inspector, ngành HHTT hiện có:

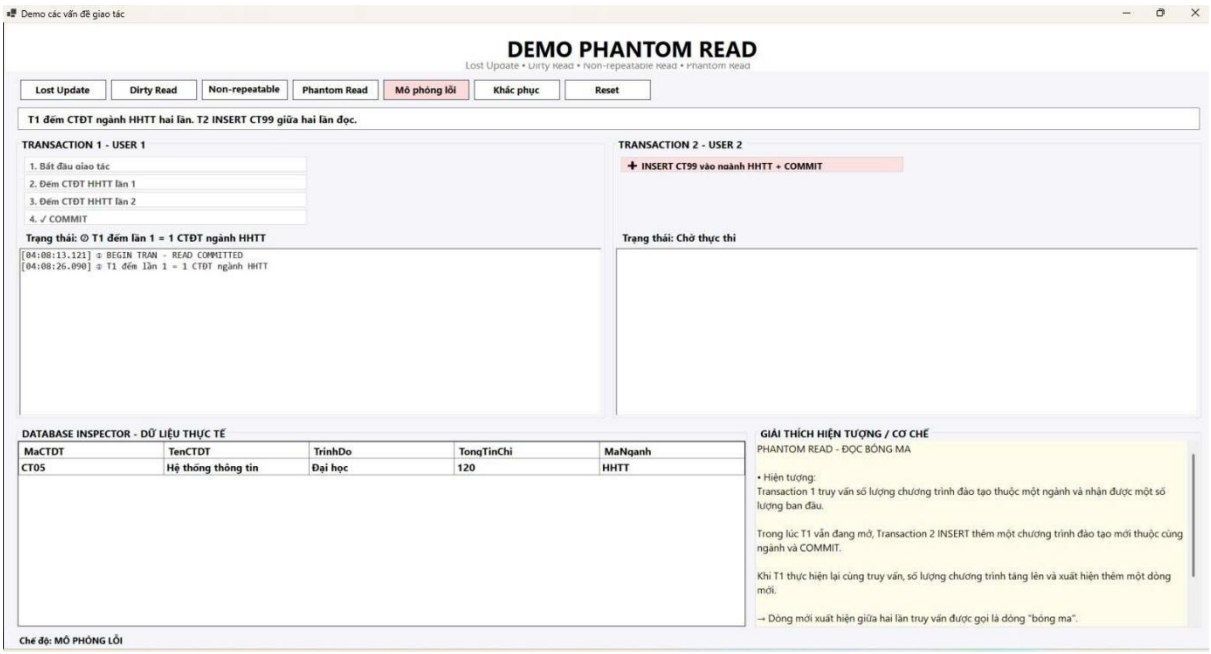
1 CTĐT

Kết quả được ghi nhận vào log:

T1 đếm lần 1 = 1 CTĐT ngành HHTT

Tại thời điểm này, Transaction 2 vẫn chưa thực hiện thao tác INSERT nên dữ liệu trong Database chưa thay đổi.

1. Giao diện sau khi thực hiện Bước 2



Trên giao diện có thể quan sát:

Transaction 1

Trạng thái:

T1 đếm lần 1 = 1 CTĐT ngành HHTT

Log:

① BEGIN TRAN - READ COMMITTED
② T1 đếm lần 1 = 1 CTĐT ngành HHTT

Transaction 2

Trạng thái:

Chờ thực thi

T2 chưa thực hiện thao tác:

INSERT CT99 vào ngành HHTT + COMMIT

Database Inspector

Dữ liệu vẫn gồm:

CT05

Hệ thống thông tin

Đại học

120 tín chỉ

HHTT

Như vậy, sau Bước 2, Transaction 1 đã ghi nhận số lượng CTĐT ban đầu là 1. Transaction 2 được mở để thực hiện INSERT CT99 giữa hai lần đọc của T1.

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn nút "2. Đếm CTĐT HHTT lần 1", sự kiện được xử lý thông qua phương thức:

StepButton_Click()

Do kịch bản đang chạy là Phantom Read nên chương trình chuyển đến:

await XuLyPhantomAsync(tag);

Trong XuLyPhantomAsync(), khi tag = "T1-2", chương trình gọi:

await bus.SetStepAsync(runId, "T1", 2);

Đoạn code này có nhiệm vụ thông báo cho hệ thống rằng Transaction 1 đã chuyển sang Bước 2.

Các tham số được truyền vào gồm:

runId: mã phiên mô phỏng hiện tại.

"T1": xác định giao tác đang thực hiện là Transaction 1.

2: xác định bước hiện tại là Bước 2.

Sau khi thực hiện xong, nút Bước 2 của T1 được khóa và nút thao tác của T2 được cho phép thực hiện.

b. BUS

Tại tầng BUS, phương thức SetStepAsync() tiếp nhận yêu cầu từ GUI.

BUS không trực tiếp thực hiện câu lệnh SQL mà chuyển yêu cầu xuống DAL thông qua:

dal.SetStepAsync(runId, giaoTac, step, action);

Với Bước 2, các giá trị được truyền xuống là:

runId = mã phiên hiện tại

giaoTac = "T1"

step = 2

action = NULL

c. DAL

Tại tầng DAL, phương thức SetStepAsync() xác định Stored Procedure cần gọi là:

sp_Demo_SetStep

Các tham số được truyền vào:

@RunId → runId

@GiaoTac → "T1"

@Step → 2

@Action → NULL

Sau đó DAL thực hiện Stored Procedure trên SQL Server.

Phương thức thực thi Stored Procedure

sp_Demo_SetStep tiếp nhận thông tin Bước 2 của Transaction 1 và cập nhật trạng thái trong bảng điều khiển mô phỏng.

Kết quả:

T1Step = 2

Giá trị T1Step = 2 đóng vai trò như tín hiệu để Stored Procedure Runner của Transaction 1 biết rằng T1 đã được phép thực hiện thao tác đếm CTĐT lần thứ nhất. Cách truyền Bước 2 qua GUI → BUS → DAL → Stored Procedure được thể hiện theo cấu trúc trong tài liệu mẫu

sp_PR_T1_Runner thực hiện đếm dữ liệu

Stored Procedure xử lý Transaction 1 của Phantom Read là:

sp_PR_T1_Runner

Khi nhận được tín hiệu T1Step = 2, Runner tiếp tục thực hiện truy vấn đếm số lượng CTĐT thuộc ngành HHTT.

Ở chế độ Mô phỏng lỗi, giao tác sử dụng mức cô lập:

READ COMMITTED

Sau đó T1 thực hiện truy vấn:

Đếm số CTĐT có MaNganh = HHTT

Kết quả:

1 CTĐT

Giá trị này được ghi vào log và hiển thị trên giao diện:

T1 đếm lần 1 = 1 CTĐT ngành HHTT

Việc sử dụng READ COMMITTED tạo điều kiện để T1 có thể nhìn thấy dữ liệu mới được T2 INSERT và COMMIT ở lần truy vấn tiếp theo, từ đó mô phỏng hiện tượng Phantom Read

Code hiển thi trên giao diện

T1 đếm CTĐT ngành HHTT lần 1 → 1 CTĐT.

T1 sử dụng READ COMMITTED.

T2 đang ở trạng thái chờ thực thi.

Sau đó T2 có thể INSERT CT99 vào ngành HHTT và COMMIT.

→ Đây là tiền đề để T1 nhìn thấy thêm một dòng ở lần đọc thứ hai.

Mô phỏng lỗi Bước 3 – Transaction 1 đếm CTĐT ngành HHTT lần 2

Nội dung thực hiện

Sau khi Transaction 1 đã thực hiện lần đếm đầu tiên và Transaction 2 đã INSERT CT99 vào ngành HHTT + COMMIT, hệ thống chuyển sang Bước 3.

Người dùng chọn:

3. Đếm CTĐT HHTT lần 2

Transaction 1 thực hiện lại cùng truy vấn đếm số lượng chương trình đào tạo có MaNganh = HHTT.

Ở lần đọc đầu tiên, T1 nhận được:

1 CTĐT

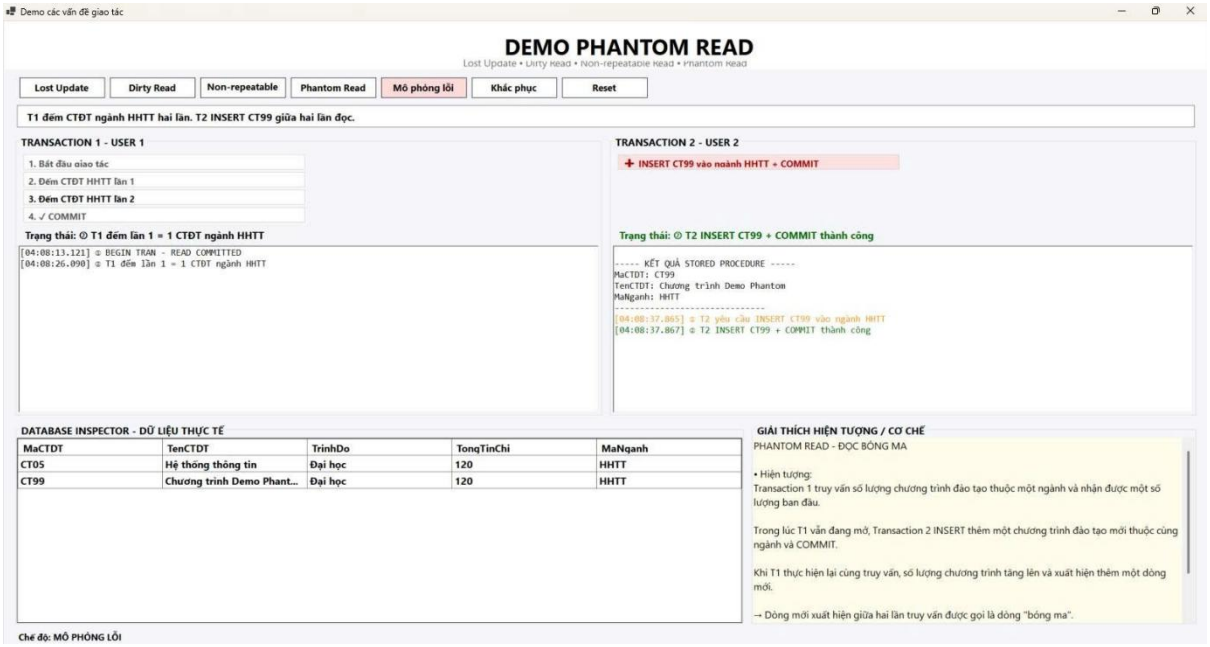
Trong thời gian T1 vẫn đang mở giao tác, T2 đã thêm CT99 vào cùng ngành HHTT và COMMIT thành công. Vì vậy, khi T1 thực hiện truy vấn lần thứ hai, dữ liệu đã thay đổi.

Kết quả lần đếm thứ hai:

2 CTĐT ngành HHTT

Như vậy, cùng một truy vấn trong cùng Transaction 1 nhưng kết quả thay đổi từ 1 lên 2 và xuất hiện thêm CT99. Đây chính là hiện tượng Phantom Read – đọc bóng ma.

1. Giao diện sau khi thực hiện Bước 3



Transaction 1

Trạng thái:

T1 đếm lần 2 = 2 CTĐT ngành HHTT

Log thể hiện quá trình:

① BEGIN TRAN - READ COMMITTED
② T1 đếm lần 1 = 1 CTĐT ngành HHTT
③ T1 đếm lần 2 = 2 CTĐT ngành HHTT

Điểm quan trọng là số lượng CTĐT đã tăng từ 1 → 2.

Transaction 2

Trạng thái:

T2 INSERT CT99 + COMMIT thành công

Log của T2 thể hiện:

T2 yêu cầu INSERT CT99 vào ngành HHTT
T2 INSERT CT99 + COMMIT thành công

Database Inspector

Sau khi T2 COMMIT, dữ liệu thực tế xuất hiện thêm:

CT05 – Hệ thống thông tin – HHTT

CT99 – Chương trình Demo Phantom – HHTT

Như vậy, CT99 đã trở thành một dòng dữ liệu mới trong phạm vi mà T1 đang truy vấn.

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn nút:

“3. Đếm CTĐT HHTT lần 2”

sự kiện StepButton_Click() được kích hoạt.

Do scenario = "PR" nên chương trình chuyển sang:

await XuLyPhantomAsync(tag);

Với tag = "T1-3", chương trình gọi:

await bus.SetStepAsync(runId, "T1", 3);

Đoạn code này thông báo cho hệ thống rằng Transaction 1 đã chuyển sang Bước 3.

Sau đó:

Nút Đếm CTĐT HHTT lần 2 được khóa.

Nút COMMIT của T1 được cho phép thực hiện.

b. BUS

BUS tiếp nhận yêu cầu:

SetStepAsync(runId, "T1", 3);

Sau đó chuyển yêu cầu xuống DAL:

dal.SetStepAsync(runId, giaoTac, step, action);

BUS có nhiệm vụ trung gian truyền thông tin từ GUI xuống DAL.

c. DAL

DAL gọi Stored Procedure:

sp_Demo_SetStep

với các tham số:

@RunId → runId

@GiaoTac → "T1"

@Step → 3

@Action → NULL

Sau đó yêu cầu được thực thi trên SQL Server.

Stored Procedure cập nhật trạng thái

sp_Demo_SetStep cập nhật:

T1Step = 3

Giá trị này là tín hiệu để sp_PR_T1_Runner tiếp tục thực hiện thao tác đếm dữ liệu lần thứ hai.

sp_PR_T1_Runner thực hiện lần đếm thứ hai

Khi nhận được T1Step = 3, sp_PR_T1_Runner thực hiện lại truy vấn đếm số lượng CTĐT ngành HHTT.

Ở chế độ Mô phỏng lỗi, T1 sử dụng:

READ COMMITTED

Do T2 đã INSERT CT99 và COMMIT trước đó nên T1 nhìn thấy dữ liệu mới.

Kết quả:

T1 đếm lần 2 = 2 CTĐT ngành HHTT

So sánh:

Lần 1: 1 CTĐT
Lần 2: 2 CTĐT

→ Số lượng thay đổi và xuất hiện thêm dòng CT99, tạo ra hiện tượng Phantom Read. Code của giao diện cũng xác định rõ đây là trường hợp T1 đọc số lượng lần hai sau khi T2 INSERT.

- Mô phỏng lỗi – Bước 4 – Transaction 1 COMMIT

Nội dung thực hiện

Sau khi Transaction 1 đã thực hiện hai lần đếm CTĐT ngành HHTT và phát hiện số lượng thay đổi:

Lần 1: 1 CTĐT

Lần 2: 2 CTĐT

Transaction 1 thực hiện bước cuối cùng là:

4. ✓ COMMIT

Ở chế độ Mô phỏng lỗi, T1 sử dụng mức cô lập READ COMMITTED.

Trước đó, Transaction 2 đã INSERT CT99 vào ngành HHTT và COMMIT thành công. Vì vậy, CT99 đã được ghi chính thức vào cơ sở dữ liệu.

Khi T1 COMMIT, giao tác kết thúc và các tài nguyên mà T1 đang sử dụng được giải phóng.

Kết quả cuối cùng trong Database vẫn có:

CT05 – Hệ thống thông tin – HHTT

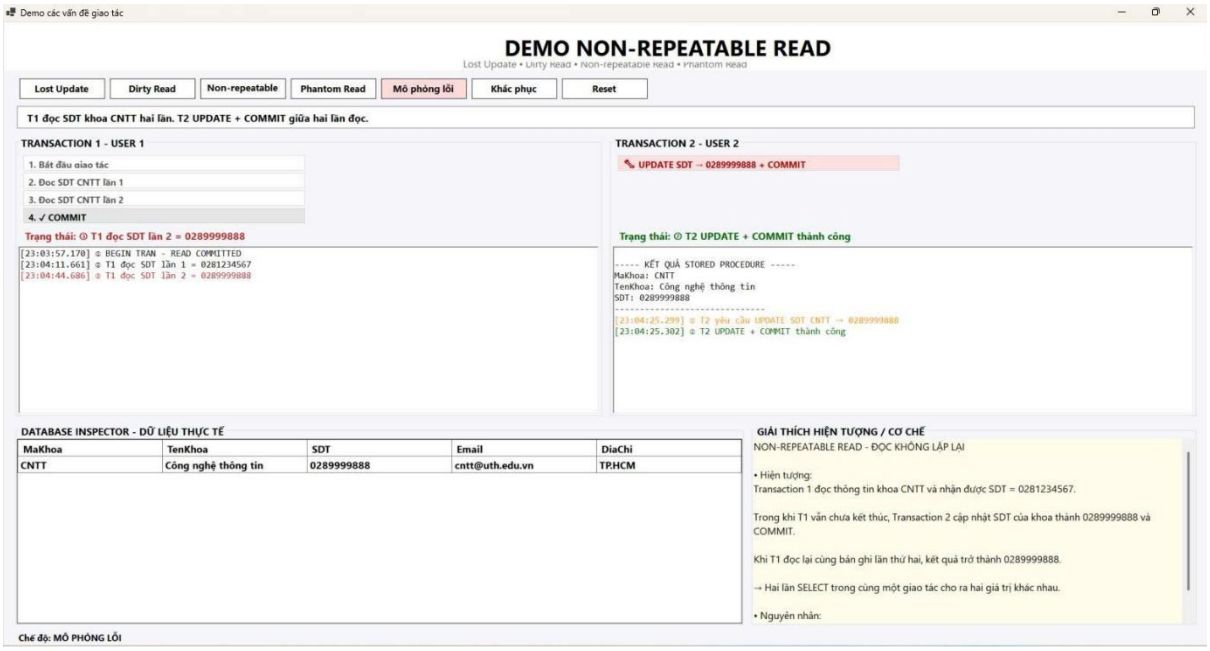
CT99 – Chương trình Demo Phantom – HHTT

Như vậy, quá trình mô phỏng Phantom Read kết thúc với kết quả:

T1 đọc lần 1 = 1 → T2 INSERT + COMMIT → T1 đọc lần 2 = 2

→ Xuất hiện dòng bóng ma CT99 giữa hai lần truy vấn của T1.

1. Giao diện sau khi thực hiện Bước 4



Trên giao diện, trạng thái của T1 chuyển thành:

T1 COMMIT

Log của T1 thể hiện đầy đủ:

- BEGIN TRAN - READ COMMITTED
- T1 đếm lần 1 = 1 CTĐT ngành HHTT
- T1 đếm lần 2 = 2 CTĐT ngành HHTT
- T1 COMMIT

Transaction 2 vẫn giữ kết quả:

T2 INSERT CT99 + COMMIT thành công

Database Inspector hiển thị 2 chương trình đào tạo thuộc ngành HHTT:

CT05
CT99

Điều này cho thấy CT99 đã được T2 thêm thành công và trở thành dữ liệu thực tế trong Database.

Sau khi T1 COMMIT, các nút thao tác của T1 được vô hiệu hóa vì giao tác đã kết thúc.

2. Code xử lý

a. Tầng giao diện

Khi người dùng kích hoạt nút:

"4. ✓ COMMIT"

nút này có tag = "T1-4".

Sự kiện StepButton_Click() được gọi và chuyển đến:

await XuLyPhantomAsync(tag);

Trong XuLyPhantomAsync(), chương trình xử lý:

tag == "T1-4"

và gọi:

await bus.SetStepAsync(runId, "T1", 4, "COMMIT");

Các giá trị truyền xuống gồm:

runId: mã phiên mô phỏng.

"T1": xác định Transaction 1.

4: xác định Bước 4.

"COMMIT": hành động kết thúc giao tác.

Sau đó nút COMMIT của T1 được vô hiệu hóa.

b. BUS

BUS tiếp nhận yêu cầu COMMIT:

SetStepAsync(runId, "T1", 4, "COMMIT");

Sau đó chuyển tiếp xuống DAL.

BUS không trực tiếp thực hiện câu lệnh SQL mà đóng vai trò trung gian giữa GUI và DAL.

c. DAL

DAL gọi Stored Procedure:

sp_Demo_SetStep

với các tham số:

@RunId = runId

@GiaoTac = "T1"

@Step = 4

@Action = "COMMIT"

Sau đó SQL Server nhận tín hiệu T1 đã thực hiện COMMIT.

Stored Procedure trong SQL Server

sp_Demo_SetStep cập nhật trạng thái của T1 thành:

T1Step = 4

và ghi nhận hành động:

COMMIT

Runner của Phantom Read tiếp tục xử lý giao tác và kết thúc Transaction 1.

- Mô phỏng lỗi – Bước 5 – Transaction 1 thực hiện COMMIT

Nội dung thực hiện

Sau khi Transaction 1 thực hiện lần đếm thứ hai và nhận được:

T1 đếm lần 2 = 2 CTĐT ngành HHTT

người dùng chọn nút:

"4. ✓ COMMIT"

để kết thúc Transaction 1.

Ở chế độ Mô phỏng lỗi, Transaction 1 sử dụng mức cô lập READ COMMITTED.

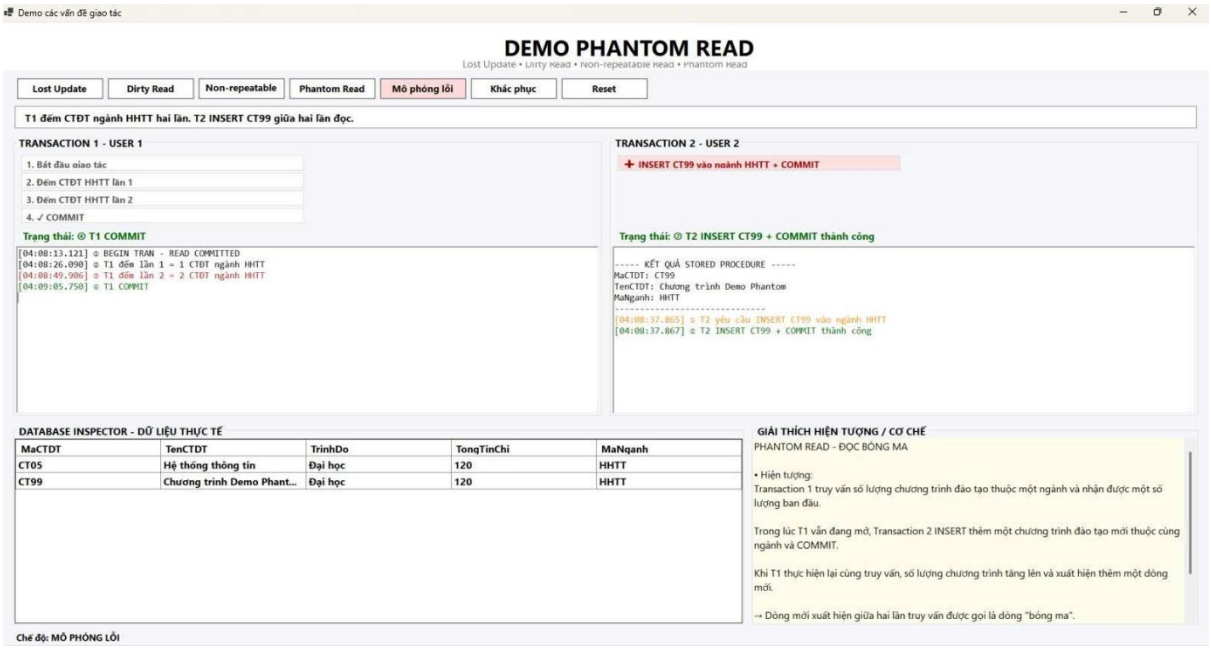
Trước đó, Transaction 2 đã INSERT CT99 vào ngành HHTT và COMMIT thành công. Vì vậy, dữ liệu CT99 đã được ghi chính thức vào Database.

Trình tự thực hiện:

T1 đếm lần 1 = 1
→ T2 INSERT CT99 + COMMIT
→ T1 đếm lần 2 = 2
→ T1 COMMIT

Mục đích của bước này là kết thúc Transaction 1 và hoàn tất quá trình mô phỏng hiện tượng Phantom Read.

1. Giao diện sau khi thực hiện Bước 5



Transaction 1

Trạng thái của T1 chuyển thành:

④ T1 COMMIT

Log hiển thị đầy đủ:

BEGIN TRAN - READ COMMITTED

T1 đếm lần 1 = 1 CTĐT ngành HHTT

T1 đếm lần 2 = 2 CTĐT ngành HHTT

T1 COMMIT

Tất cả các nút của Transaction 1 được vô hiệu hóa vì giao tác đã kết thúc.

Transaction 2

Trạng thái:

T2 INSERT CT99 + COMMIT thành công

Log:

T2 yêu cầu INSERT CT99 vào ngành HHTT

T2 INSERT CT99 + COMMIT thành công

Database Inspector

Dữ liệu thực tế trong Database gồm:

CT05 – Hệ thống thông tin – HHTT

CT99 – Chương trình Demo Phantom – HHTT

Như vậy, CT99 đã được thêm thành công vào cơ sở dữ liệu.

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn nút:

"4. ✓ COMMIT"

sự kiện StepButton_Click() được gọi.

Do đang chạy kịch bản Phantom Read, chương trình chuyển sang:

await XuLyPhantomAsync(tag);

Với tag = "T1-4", chương trình gọi:

await bus.SetStepAsync(runId, "T1", 4, "COMMIT");

Trong đó:

runId: mã phiên mô phỏng.

"T1": xác định Transaction 1.

4: xác định bước COMMIT trên giao diện.

"COMMIT": hành động kết thúc giao tác.

Sau khi thực hiện, nút COMMIT của T1 được vô hiệu hóa.

b. BUS

BUS tiếp nhận yêu cầu COMMIT từ giao diện thông qua:

SetStepAsync()

Sau đó chuyển yêu cầu xuống DAL:

dal.SetStepAsync(runId, giaoTac, step, action);

BUS không trực tiếp thực hiện câu lệnh SQL mà đóng vai trò trung gian giữa GUI và DAL.

c. DAL

DAL nhận các thông tin:

@RunId = runId

@GiaoTac = "T1"

@Step = 4

@Action = "COMMIT"

Sau đó gọi Stored Procedure:

sp_Demo_SetStep

để cập nhật trạng thái của Transaction 1.

Stored Procedure trong SQL Server

sp_Demo_SetStep nhận tín hiệu T1 đã thực hiện COMMIT.

Sau đó Transaction 1 kết thúc và giải phóng các tài nguyên mà giao tác đang sử dụng.

Kết quả cuối cùng của chế độ Mô phỏng lỗi là:

T1 lần 1 = 1
T1 lần 2 = 2

→ Hai lần truy vấn cho kết quả khác nhau.

→ Phantom Read xảy ra.

3.5.2. Khắc phục Phantom Read

- Bước 1: Bắt đầu giao tác

Nội dung thực hiện

Tại Bước 1, Transaction 1 bắt đầu giao tác để chuẩn bị thực hiện hai lần đếm số lượng chương trình đào tạo thuộc ngành HHTT.

Trong chế độ KHẮC PHỤC, Transaction 1 sử dụng mức cô lập:

SERIALIZABLE

Mục đích của việc nâng mức cô lập lên SERIALIZABLE là ngăn Transaction 2 chèn thêm dữ liệu mới vào phạm vi mà Transaction 1 đang truy vấn.

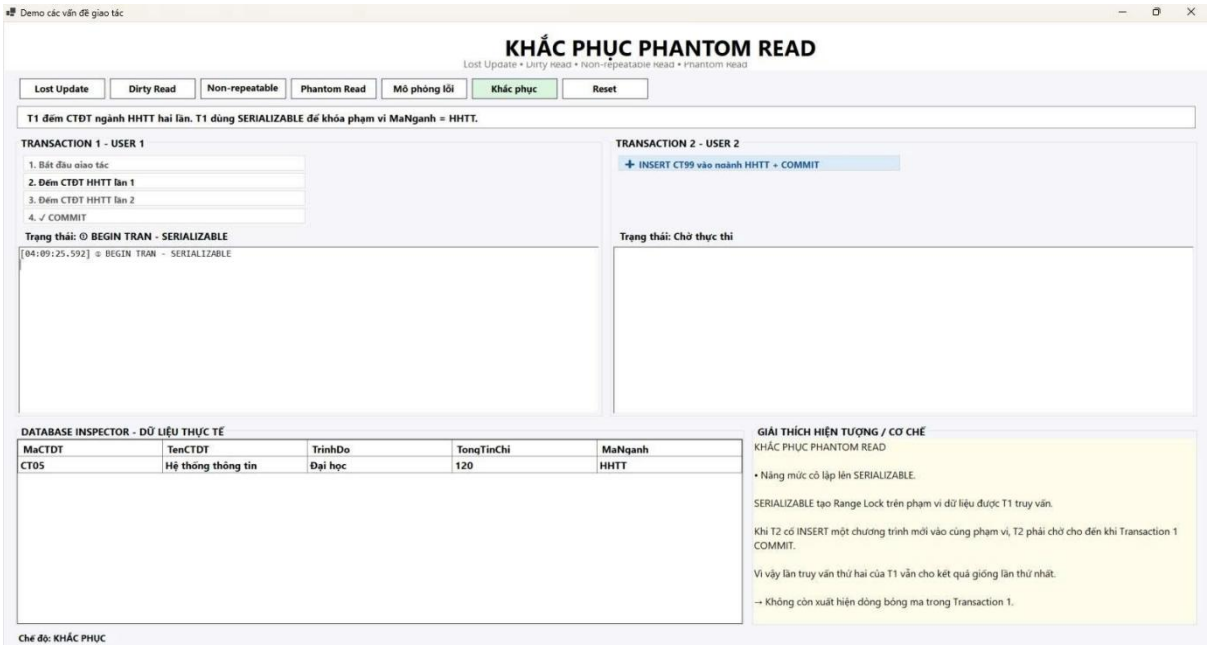
Trong trường hợp này, phạm vi dữ liệu cần bảo vệ là:

MaNganh = HHTT

Sau khi người dùng thực hiện “1. Bắt đầu giao tác”, Transaction 1 được mở với mức cô lập SERIALIZABLE.

Lúc này Transaction 2 vẫn chưa thực hiện thao tác INSERT.

1. Giao diện sau khi thực hiện Bước 1



Transaction 1 – User 1

Ở phía bên trái, Transaction 1 gồm 4 bước:

1. Bắt đầu giao tác
2. Đếm CTĐT HHTT lần 1
3. Đếm CTĐT HHTT lần 2
4. COMMIT

Sau khi thực hiện Bước 1, trạng thái hiển thị:

Trạng thái: ⓪ BEGIN TRAN - SERIALIZABLE

Log hiển thị: [04:09:25.592] ⓪ BEGIN TRAN - SERIALIZABLE

Điều này cho thấy Transaction 1 đã bắt đầu thành công với mức cô lập SERIALIZABLE.

Transaction 2 – User 2

Ở phía bên phải, Transaction 2 có thao tác:

INSERT CT99 vào ngành HHTT + COMMIT

Tuy nhiên tại thời điểm này T2 chưa được thực hiện.

Trạng thái của T2 vẫn là:

Trạng thái: Chờ thực thi

Như vậy, chưa có chương trình CT99 được thêm vào Database.

2. Code xử lý

a. Tầng giao diện

Khi người dùng nhấn:

“1. Bắt đầu giao tác”

sự kiện của nút được xử lý trong StepButton_Click().

Do đang ở kịch bản Phantom Read, chương trình gọi phương thức:
XuLyPhantomAsync(tag);

Trong XuLyPhantomAsync(), khi tag tương ứng với: T1-1
chương trình gọi: await bus.SetStepAsync(runId, "T1", 1);

Các tham số có ý nghĩa:

- runId: mã phiên mô phỏng.
- "T1": xác định Transaction 1.
- 1: xác định T1 đang thực hiện Bước 1.

Sau khi thực hiện thành công, chương trình khóa nút Bước 1 và cho phép thực hiện Bước 2.

b. BUS

Sau khi GUI gọi: bus.SetStepAsync(runId, "T1", 1);yêu cầu được chuyển xuống tầng BUS.

BUS tiếp nhận thông tin của Transaction 1 và Bước 1, sau đó chuyển tiếp yêu cầu xuống DAL.

BUS không trực tiếp thực hiện câu lệnh SQL mà đóng vai trò trung gian giữa giao diện và DAL.

c. DAL

Tại DAL, phương thức SetStepAsync() tiếp nhận yêu cầu từ BUS.

Các thông tin được truyền xuống SQL Server gồm:

@RunId → mã phiên hiện tại

@GiaoTac → T1

@Step → 1

@Action → NULL

DAL sau đó gọi Stored Procedure để cập nhật trạng thái Bước 1 của Transaction 1.

Thiết lập mức cô lập SERIALIZABLE

Sau khi Transaction 1 được phép thực hiện, hệ thống thiết lập mức cô lập:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Đây là điểm quan trọng trong phần khắc phục Phantom Read.

SERIALIZABLE giúp bảo vệ phạm vi dữ liệu mà T1 sẽ truy vấn.

Trong trường hợp này, T1 sẽ truy vấn các chương trình thuộc: MaNganh = HHTT

Khi T1 thực hiện truy vấn ở Bước 2, SQL Server sẽ tạo cơ chế khóa phạm vi dữ liệu.

Nếu T2 cố gắng: INSERT CT99 vào ngành HHTT thì T2 sẽ phải chờ cho đến khi T1 thực hiện:COMMIT

Code hiển thị sau khi thực hiện Bước 1

Sau khi nhấn **"**1. Bắt đầu giao tác**"**, phía Transaction 1 hiển thị:

Trạng thái: ① BEGIN TRAN - SERIALIZABLE

Trong khung log hiển thị: [04:09:25.592] ① BEGIN TRAN - SERIALIZABLE

Phía Transaction 2 vẫn hiển thị: Trạng thái: Chờ thực thi

Database Inspector vẫn giữ nguyên dữ liệu: CT05 | Hệ thống thông tin | Đại học | 120 | HHTT

Kết quả: Transaction 1 đã bắt đầu giao tác với mức cô lập SERIALIZABLE, đồng thời dữ liệu chưa thay đổi và Transaction 2 chưa được thực hiện.

→ Đây là bước chuẩn bị để ở bước tiếp theo, khi T1 thực hiện truy vấn MaNganh = HHTT, hệ thống sẽ bảo vệ phạm vi dữ liệu và ngăn T2 INSERT thêm bản ghi mới vào cùng phạm vi.

Khắc phục – Bước 2 Transaction 1 đếm CTĐT ngành HHTT lần 1

Nội dung thực hiện

Sau khi Transaction 1 bắt đầu giao tác ở Bước 1, hệ thống thực hiện Bước 2:

T1 đếm CTĐT HHTT lần 1

Người dùng chọn:

2. Đếm CTĐT HHTT lần 1

T1 thực hiện truy vấn số lượng chương trình đào tạo thuộc ngành HHTT.

Kết quả ban đầu:

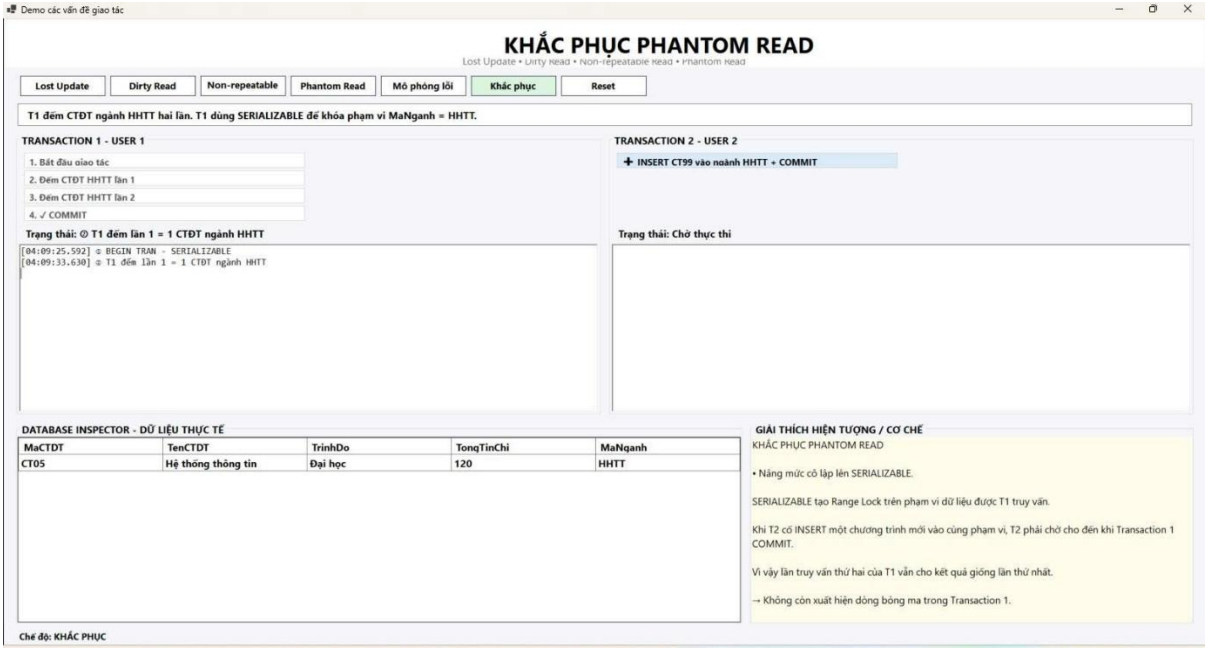
1 CTĐT

Tuy nhiên, điểm khác biệt so với chế độ Mô phỏng lỗi là Transaction 1 sử dụng mức cô lập:

SERIALIZABLE

Mức cô lập này tạo Range Lock trên phạm vi dữ liệu mà T1 đang truy vấn. Vì vậy, khi Transaction 2 cố gắng INSERT thêm CT99 vào cùng phạm vi MaNganh = HHTT, T2 phải chờ cho đến khi T1 COMMIT.

1. Giao diện sau khi thực hiện Bước 2



Trên giao diện có thể thấy:

Transaction 1

Trạng thái:

T1 đếm lần 1 = 1 CTĐT ngành HHTT

Log:

① BEGIN TRAN - SERIALIZABLE
② T1 đếm lần 1 = 1 CTĐT ngành HHTT

Transaction 2

Trạng thái:

Chờ thực thi

T2 chưa thực hiện:

INSERT CT99 vào ngành HHTT + COMMIT

Database Inspector

Dữ liệu vẫn là:

CT05 – Hệ thống thông tin – Đại học – 120 – HHTT

Như vậy, sau Bước 2, T1 đã lấy được số lượng CTĐT ban đầu là 1 và đồng thời giữ khóa phạm vi dữ liệu MaNganh = HHTT.

Khi T2 cố gắng INSERT CT99 vào cùng phạm vi, T2 sẽ phải chờ cho đến khi T1 COMMIT

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn nút “2. Đếm CTĐT HHTT lần 1”, sự kiện được xử lý thông qua:

StepButton_Click()

Sau đó chương trình chuyển sang:

await XuLyPhantomAsync(tag);

Trong XuLyPhantomAsync(), với tag = "T1-2", chương trình gọi:

await bus.SetStepAsync(runId, "T1", 2);

Sau khi gửi tín hiệu Bước 2, nút Đếm CTĐT HHTT lần 1 được khóa và nút INSERT CT99 vào ngành HHTT + COMMIT của T2 được cho phép thực hiện.

b. BUS

BUS tiếp nhận yêu cầu:

SetStepAsync(runId, "T1", 2);

Sau đó chuyển yêu cầu xuống DAL thông qua:

dal.SetStepAsync(runId, giaoTac, step, action);

c. DAL

DAL gọi Stored Procedure:

sp_Demo_SetStep

với các tham số:

- @RunId → runId
- @GiaoTac → "T1"
- @Step → 2
- @Action → NULL

Stored Procedure trong SQL Server

sp_Demo_SetStep cập nhật trạng thái:

T1Step = 2

Đây là tín hiệu để sp_PR_T1_Runner tiếp tục thực hiện thao tác đếm CTĐT lần thứ nhất.

sp_PR_T1_Runner thực hiện đếm dữ liệu

Do đang ở chế độ Khắc phục, Transaction 1 sử dụng:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Sau đó T1 thực hiện truy vấn đếm CTĐT thuộc ngành HHTT.

Kết quả:

1 CTĐT

Giá trị được ghi vào log:

T1 đếm lần 1 = 1 CTĐT ngành HHTT

Điểm quan trọng là SERIALIZABLE tạo Range Lock trên phạm vi dữ liệu mà T1 đang truy vấn. Vì vậy, Transaction 2 không thể INSERT thêm một dòng mới vào cùng phạm vi trước khi T1 COMMIT.

Code hiển thị trên giao diện

T1 đếm CTĐT ngành HHTT lần 1 → 1 CTĐT.

T1 sử dụng SERIALIZABLE.

Khi T1 thực hiện truy vấn, phạm vi MaNganh = HHTT được khóa bằng Range Lock.

→ T2 muốn INSERT CT99 vào cùng phạm vi phải chờ T1 COMMIT, nên T1 không xuất hiện dòng bóng ma ở lần đọc thứ hai.

- Khắc phục – Bước 3 – Transaction 1 đếm CTĐT ngành HHTT lần 2

Nội dung thực hiện

Trong chế độ Khắc phục, Transaction 1 sử dụng mức cô lập:

SERIALIZABLE

Ở Bước 2, T1 đã đếm CTĐT ngành HHTT lần đầu và giữ Range Lock trên phạm vi dữ liệu MaNganh = HHTT.

Sau đó Transaction 2 yêu cầu:

INSERT CT99 vào ngành HHTT + COMMIT

Tuy nhiên, do T1 đang giữ Range Lock nên T2 phải chờ và chưa thể hoàn tất INSERT.

Khi người dùng chọn:

3. Đếm CTĐT HHTT lần 2

T1 thực hiện lại cùng truy vấn.

Vì T2 chưa thể INSERT và COMMIT nên dữ liệu trong phạm vi HHTT vẫn chưa thay đổi.

Kết quả:

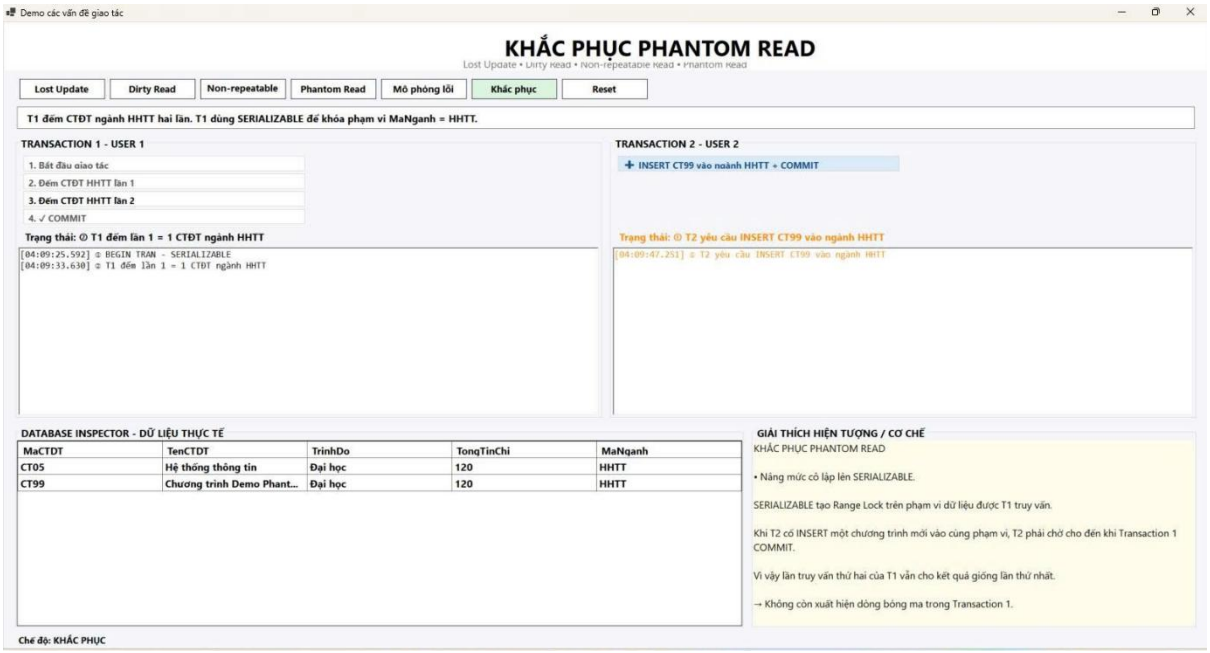
T1 đếm lần 2 = 1 CTĐT ngành HHTT

Như vậy:

Lần 1 = 1 CTĐT
Lần 2 = 1 CTĐT

→ Hai lần truy vấn cho cùng một kết quả, không xuất hiện dòng bóng ma. Cơ chế này đúng với phần giải thích của chương trình: SERIALIZABLE tạo Range Lock và T2 phải chờ đến khi T1 COMMIT.

1. Giao diện sau khi thực hiện Bước 3



Transaction 1

Trạng thái:

T1 đếm lần 2 = 1 CTĐT ngành HHTT

Log:

① BEGIN TRAN - SERIALIZABLE
② T1 đếm lần 1 = 1 CTĐT ngành HHTT
③ T1 đếm lần 2 = 1 CTĐT ngành HHTT

Hai lần đếm cho cùng kết quả.

Transaction 2

Trạng thái:

T2 yêu cầu INSERT CT99 vào ngành HHTT

Log hiển thị trạng thái chờ, vì T2 chưa thể hoàn thành INSERT trong khi T1 chưa COMMIT.

Database Inspector

Trong thời điểm T1 thực hiện lần đếm thứ hai, dữ liệu trong phạm vi HHTT vẫn chưa có CT99.

→ Vì vậy T1 vẫn nhận được 1 CTĐT.

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn:

"3. Đếm CTĐT HHTT lần 2"

chương trình xử lý qua:

StepButton_Click()

→ XuLyPhantomAsync(tag)

Với tag = "T1-3":

await bus.SetStepAsync(runId, "T1", 3);

Sau đó chương trình khóa nút Bước 3 và cho phép T1 thực hiện bước COMMIT.

b. BUS

BUS nhận:

- GiaoTac = "T1"
- Step = 3

Sau đó chuyển yêu cầu xuống DAL.

c. DAL

DAL gọi:

sp_Demo_SetStep

với:

- @RunId = runId
- @GiaoTac = "T1"
- @Step = 3
- @Action = NULL

Stored Procedure trong SQL Server

sp_Demo_SetStep cập nhật:

T1Step = 3

Đây là tín hiệu để sp_PR_T1_Runner tiếp tục xử lý lần đếm thứ hai.

sp_PR_T1_Runner thực hiện lần đếm thứ hai

T1 vẫn đang ở mức cô lập:

SERIALIZABLE

Do T1 đang giữ Range Lock trên phạm vi MaNganh = HHTT, T2 không thể thêm CT99 vào phạm vi này trước khi T1 COMMIT.

Vì vậy, khi T1 thực hiện lại truy vấn:

T1 đếm lần 2 = 1 CTĐT ngành HHTT

Kết quả lần 2 giống lần 1.

→ Không xuất hiện dòng bóng ma trong Transaction 1.

- Khắc phục - Bước 4 – Transaction 1 COMMIT

Nội dung thực hiện

Trong chế độ Khắc phục, Transaction 1 sử dụng mức cô lập:

SERIALIZABLE

Ở Bước 2, T1 đã tạo Range Lock trên phạm vi MaNganh = HHTT.

Khi Transaction 2 yêu cầu:

INSERT CT99 vào ngành HHTT + COMMIT

T2 không thể hoàn thành thao tác ngay mà phải chờ T1 COMMIT.

Sau Bước 3, T1 thực hiện lại truy vấn và nhận được:

- Lần 1: 1 CTĐT
- Lần 2: 1 CTĐT

Sau đó T1 thực hiện:

4. ✓ COMMIT

Khi T1 COMMIT, Range Lock được giải phóng.

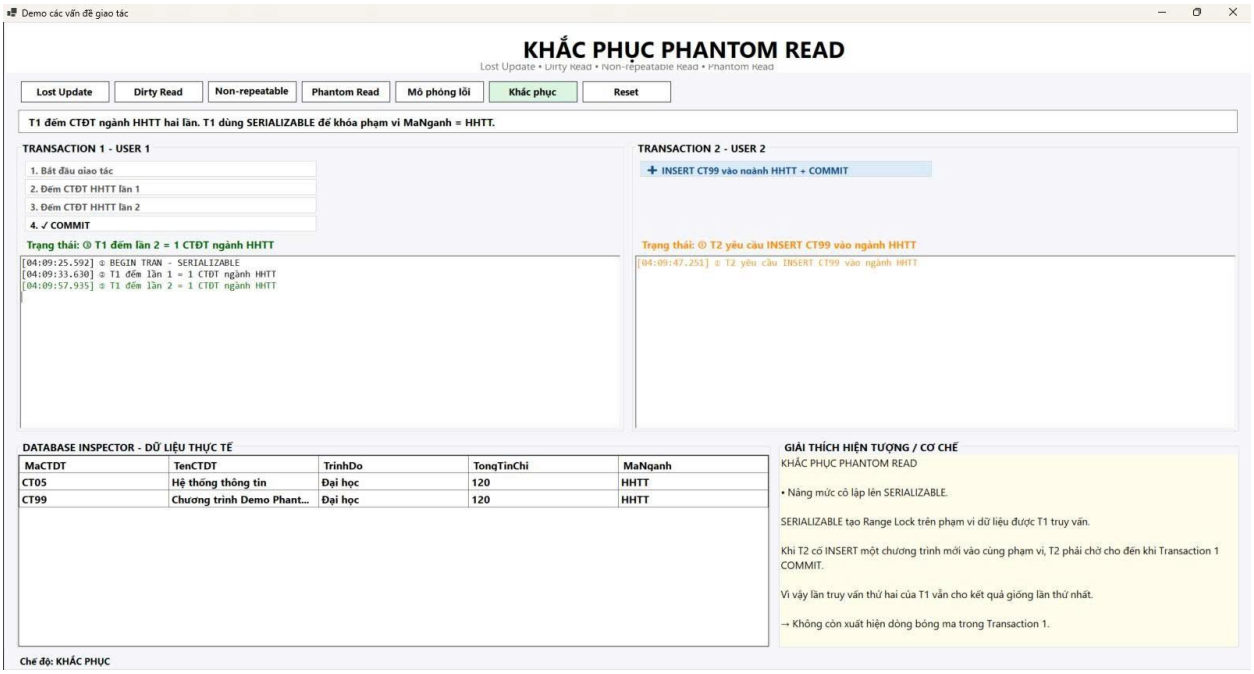
Lúc này Transaction 2 mới được phép tiếp tục INSERT CT99 và COMMIT thành công.

Kết quả cuối cùng:

T1 không xuất hiện dòng bóng ma, còn CT99 chỉ được thêm vào Database sau khi T1 kết thúc.

Cơ chế này đúng với phần giải thích của chương trình: SERIALIZABLE tạo Range Lock và T2 phải chờ đến khi Transaction 1 COMMIT.

1. Giao diện sau khi thực hiện Bước 4



Transaction 1

Trạng thái:

④ T1 COMMIT

Log:

① BEGIN TRAN - SERIALIZABLE
② T1 đếm lần 1 = 1 CTĐT ngành HHTT
③ T1 đếm lần 2 = 1 CTĐT ngành HHTT
④ T1 COMMIT

Điểm quan trọng:

Hai lần đếm đều bằng 1.

Transaction 2

Trước khi T1 COMMIT:

T2 yêu cầu INSERT CT99 vào ngành HHTT

→ T2 đang chờ.

Sau khi T1 COMMIT:

T2 INSERT CT99 + COMMIT thành công

Database Inspector sau đó xuất hiện:

- CT05
- CT99

Như vậy CT99 vẫn được thêm vào Database, nhưng không xuất hiện trong lần đọc thứ hai của T1.

2. Code xử lý

a. Tầng giao diện

Khi người dùng chọn:

“4. ✓ COMMIT”

chương trình nhận tag = "T1-4" và xử lý thông qua:

await XuLyPhantomAsync(tag);

Sau đó gọi:

await bus.SetStepAsync(runId, "T1", 4, "COMMIT");

Đồng thời vô hiệu hóa nút COMMIT để tránh người dùng thực hiện lại thao tác.

b. BUS

BUS nhận:

- GiaoTac = "T1"
- Step = 4
- Action = "COMMIT"

Sau đó chuyển yêu cầu xuống DAL.

c. DAL

DAL gọi:

sp_Demo_SetStep

và truyền:

- @RunId = runId
- @GiaoTac = "T1"
- @Step = 4
- @Action = "COMMIT"

Stored Procedure trong SQL Server

SQL Server nhận tín hiệu T1 COMMIT.

Transaction 1 kết thúc và Range Lock được giải phóng.

Sau đó Transaction 2 đang chờ được giải phóng và tiếp tục thực hiện:

INSERT CT99 → COMMIT thành công

Kết quả là CT99 được ghi vào Database sau khi T1 đã kết thúc.

- Khắc phục – Bước 5 – Transaction 1 thực hiện COMMIT

Nội dung thực hiện

Ở chế độ Khắc phục, Transaction 1 sử dụng mức cô lập:

SERIALIZABLE

Trong quá trình thực hiện hai lần đếm, SERIALIZABLE tạo Range Lock trên phạm vi dữ liệu MaNganh = HHTT.

Do đó khi Transaction 2 yêu cầu:

INSERT CT99 vào ngành HHTT + COMMIT

T2 phải chờ cho đến khi Transaction 1 kết thúc.

Ở Bước 3, T1 thực hiện lần đếm thứ hai và vẫn nhận được:

1 CTĐT ngành HHTT

Sau đó người dùng chọn:

"4. ✓ COMMIT"

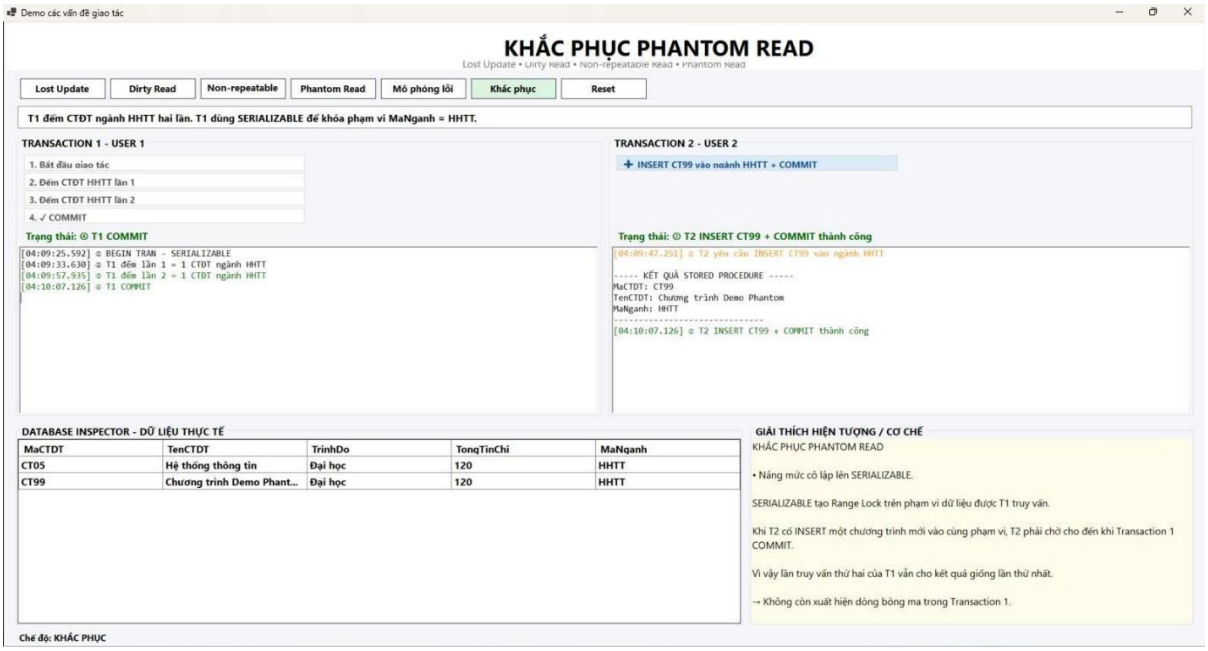
để kết thúc Transaction 1.

Trình tự thực hiện:

T1 đếm lần 1 = 1
→ T2 yêu cầu INSERT → chờ
→ T1 đếm lần 2 = 1
→ T1 COMMIT
→ T2 INSERT CT99 + COMMIT

Sau khi T1 COMMIT, Range Lock được giải phóng và T2 mới có thể tiếp tục thực hiện INSERT.

1. Giao diện sau khi thực hiện Bước 5



Transaction 1

Trạng thái:

ⓓ T1 COMMIT

Log:

1. BEGIN TRAN - SERIALIZABLE
2. T1 đếm lần 1 = 1 CTĐT ngành HHTT
3. T1 đếm lần 2 = 1 CTĐT ngành HHTT
4. T1 COMMIT

Điểm quan trọng là:

Lần 1 = 1 CTĐT

Lần 2 = 1 CTĐT

Hai lần truy vấn cho cùng một kết quả.

Transaction 2

Trước khi T1 COMMIT:

T2 yêu cầu INSERT CT99 vào ngành HHTT

→ T2 đang chờ.

Sau khi T1 COMMIT:

T2 INSERT CT99 + COMMIT thành công

Database Inspector

Sau khi T2 được giải phóng và INSERT thành công, Database xuất hiện:

- CT05 – Hệ thống thông tin – HHTT
- CT99 – Chương trình Demo Phantom – HHTT

Điểm quan trọng là CT99 chỉ được thêm sau khi T1 đã COMMIT, nên nó không xuất hiện trong lần đếm thứ hai của T1.

2. Code xử lý

a. Tầng giao diện

Khi chọn:

"4. ✓ COMMIT"

sự kiện StepButton_Click() được gọi và chuyển sang:

await XuLyPhantomAsync(tag);

Với tag = "T1-4", chương trình gửi:

await bus.SetStepAsync(runId, "T1", 4, "COMMIT");

Sau đó nút COMMIT được vô hiệu hóa để tránh thực hiện lại thao tác.

b. BUS

BUS nhận yêu cầu COMMIT của T1 và chuyển xuống DAL.

c. DAL

DAL thực hiện yêu cầu trên SQL Server thông qua:

sp_Demo_SetStep

với:

- @GiaoTac = "T1"
- @Step = 4
- @Action = "COMMIT"

Cơ chế trong SQL Server

Khi T1 COMMIT:

Range Lock được giải phóng.

Transaction 2 đang chờ được tiếp tục thực hiện:

INSERT CT99 → COMMIT thành công

Cơ chế SERIALIZABLE và Range Lock được chương trình mô tả trực tiếp là cách ngăn T2 INSERT vào cùng phạm vi trong khi T1 chưa COMMIT.

Cơ chế khắc phục

Điểm quan trọng nhất của Bước 5 là thứ tự thực hiện giữa T1 và T2.

Mô phỏng lỗi

T1 đọc lần 1 = 1
→ T2 INSERT + COMMIT
→ T1 đọc lần 2 = 2
→ T1 COMMIT

→ Xuất hiện dòng bóng ma CT99.

Khắc phục

T1 đọc lần 1 = 1
→ T2 yêu cầu INSERT → chờ
→ T1 đọc lần 2 = 1
→ T1 COMMIT
→ T2 INSERT + COMMIT

→ Không xuất hiện dòng bóng ma trong Transaction 1.

Như vậy, việc sử dụng SERIALIZABLE giúp khóa phạm vi dữ liệu T1 đang truy vấn, ngăn Transaction 2 chèn thêm dòng mới vào phạm vi đó trước khi T1 kết thúc.

3.6. DEADLOCK

Deadlock xảy ra khi các giao tác chờ khóa của nhau, khiến không giao tác nào có thể tiếp tục thực hiện và hệ thống rơi vào trạng thái tắc nghẽn.

Ví dụ:

- T1 đang giữ khóa Y
- T2 đang giữ khóa X
- T1 muốn khóa X nhưng X đang do T2 giữ
- T2 muốn khóa Y nhưng Y đang do T1 giữ

Hai giao tác chờ nhau và không giao tác nào có thể tiếp tục → Deadlock.

Một số giải pháp xử lý Deadlock

- Yêu cầu giao tác xin tất cả các khóa cùng lúc
- Cấp khóa theo một thứ tự nhất định
- Phát hiện Deadlock và thực hiện phục hồi

3.6.1. Mô phỏng DEADLOCK

Tổng quan:

Bước 1 - Trạng thái kẹt:

- T1 giữ L01 và chờ SV01.
- T2 giữ SV01 và chờ L01.

=> Xuất hiện vòng chờ: T1 → T2 → T1.

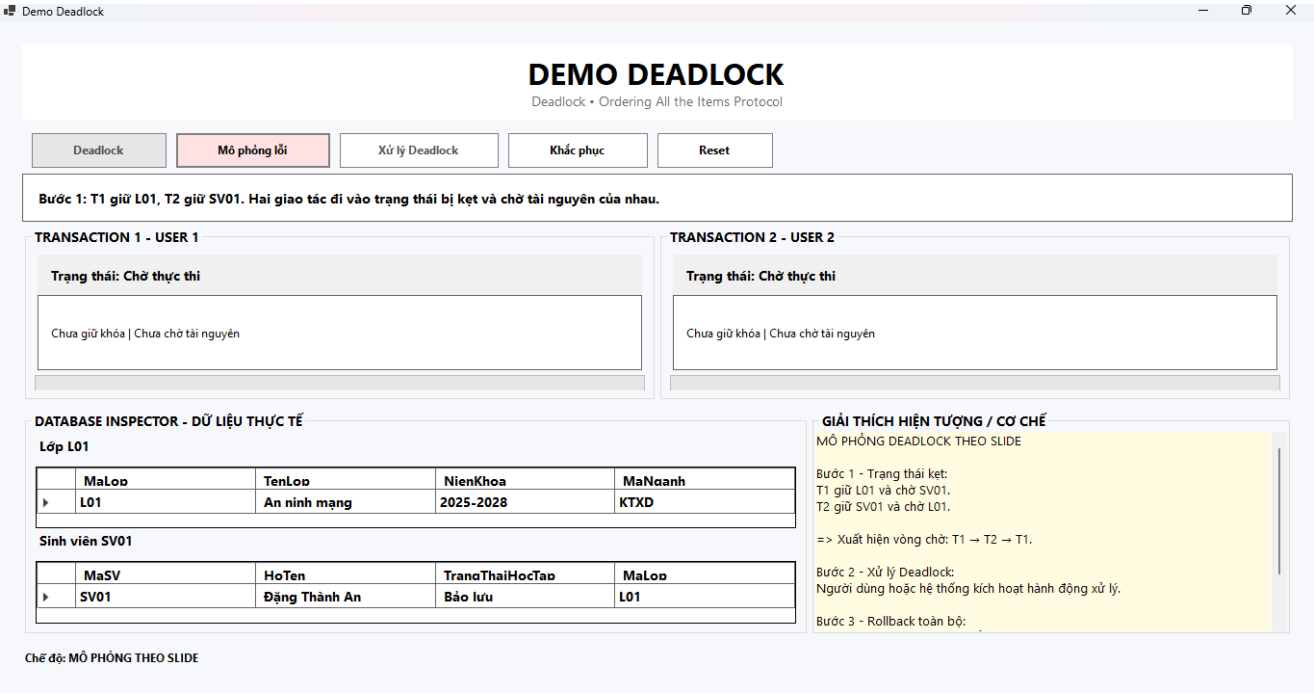
Bước 2 - Xử lý Deadlock:

Người dùng hoặc hệ thống kích hoạt hành động xử lý.

Bước 3 - Rollback toàn bộ: Cả T1 và T2 được rollback để minh họa.

Lưu ý: SQL Server thực tế thường tự chọn một deadlock victim, không rollback cả hai.

a) Giao diện



b) Luồng xử lý

- Bước 1: Bấm Mô phỏng lỗi

```
1 reference
private async void btnMoPhong_Click(
    object sender,
    EventArgs e)
{
    string loi =
        dal.KiemTraDuLieuDemo();

    if (!string.IsNullOrWhiteSpace(loi))
    {
        MessageBox.Show(
            loi,
            "Thiếu dữ liệu demo",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        return;
    }

    timerKhacPhuc.Stop();
    HienThiGiaiThichMoPhong();

    SetDangChay(true);

    try
    {
        XoaLog();

        string[] ketQua =
            await dal
                .BatDauMoPhongTheoSlideAsync();

        GhiLog(txtLogT1, ketQua[0]);
        GhiLog(txtLogT2, ketQua[1]);

        thoiDiemBatDau =
            DateTime.Now;
```

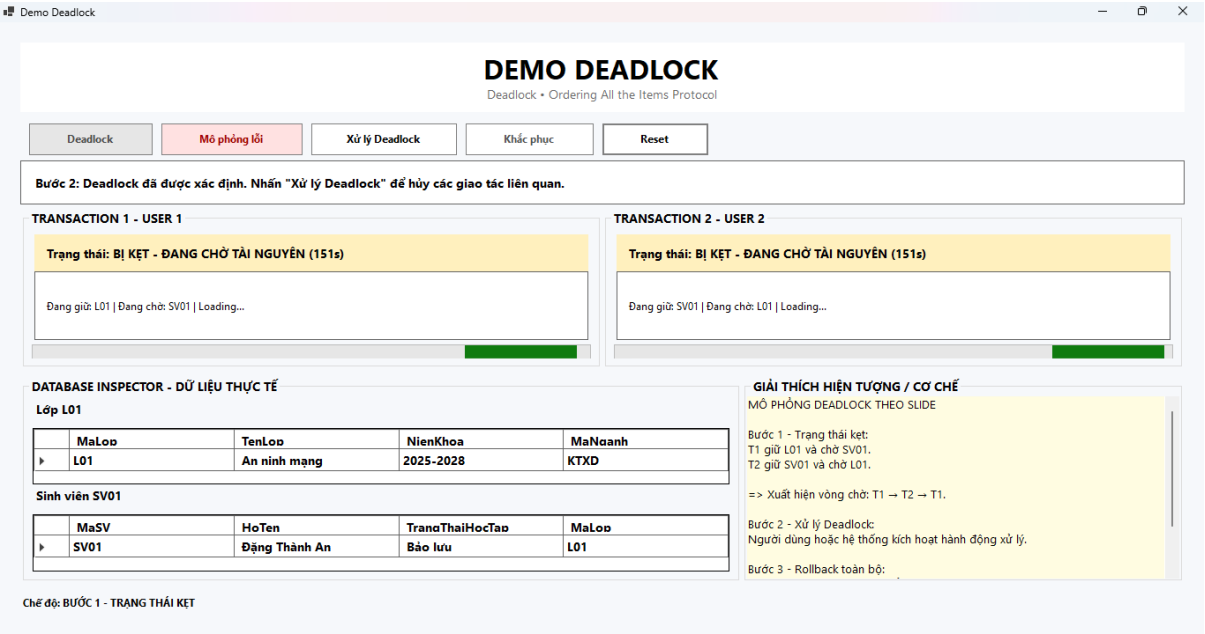
```
        lblMoTa.Text =
            "Bước 1: Hai giao tác đang bị kẹt. T1 chờ SV01 và T2 chờ L01.";

        lblCheDo.Text =
            "Chế độ: BƯỚC 1 – TRẠNG THÁI KẸT";

        prgT1.Style =
            ProgressBarStyle.Marquee;

        prgT2.Style =
            ProgressBarStyle.Marquee;

        timerKet.Start();

        SetTrangThaiKet(
            lblTrangThaiT1,
            lblChiTietT1,
            "Đang giữ: L01 | Đang chờ: SV01");

        SetTrangThaiKet(
            lblTrangThaiT2,
            lblChiTietT2,
            "Đang giữ: SV01 | Đang chờ: L01");

        GhiLog(
            txtLogT1,
            "T1 bị kẹt: giữ L01, chờ SV01.");

        GhiLog(
            txtLogT2,
            "T2 bị kẹt: giữ SV01, chờ L01.");

        btnXuLyDeadlock.Enabled = false;
    }
```

```
        catch (Exception ex)
        {
            await dal.RollbackCaHaiAsync();

            MessageBox.Show(
                "Không thể bắt đầu mô phỏng!\n" +
                ex.Message);
        }
        finally
        {
            btnReset.Enabled = true;
        }
    }
```

- Mô phỏng lỗi Deadlock:

Khi người dùng chọn chức năng **Mô phỏng lỗi**, chương trình kiểm tra dữ liệu trước khi bắt đầu mô phỏng. Sau đó hai transaction được thực hiện đồng thời với thứ tự khóa tài nguyên khác nhau.

- Transaction 1 khóa tài nguyên thứ nhất và chờ tài nguyên do Transaction 2 đang giữ.
- Transaction 2 đồng thời khóa tài nguyên thứ hai và chờ tài nguyên do Transaction 1 đang giữ.
- Hai transaction tạo thành vòng chờ lẫn nhau, từ đó phát sinh Deadlock.



Có thể minh họa ngắn:

Transaction 1- >Khóa A->Chờ B->Transaction 2->Khóa B->Chờ A

→ T1 chờ T2 và T2 chờ T1 → Deadlock

Nguyên nhân xảy ra Deadlock

Deadlock xảy ra khi hai transaction cùng giữ tài nguyên mà transaction còn lại đang cần và không transaction nào có thể tiếp tục thực hiện.

Trong chương trình, Transaction 1 và Transaction 2 thực hiện thao tác trên các tài nguyên theo thứ tự khác nhau. Khi Transaction 1 giữ tài nguyên A và chờ tài nguyên B, đồng thời Transaction 2 giữ tài nguyên B và chờ tài nguyên A, hệ quản trị cơ sở dữ liệu phát hiện vòng chờ giữa hai transaction. Đây là tình trạng Deadlock.

4 điều kiện Coffman: Tình huống mô phỏng thỏa các điều kiện cần thiết để xảy ra Deadlock gồm: độc quyền tài nguyên, giữ và chờ, không thể cưỡng chế tài nguyên và chờ vòng.

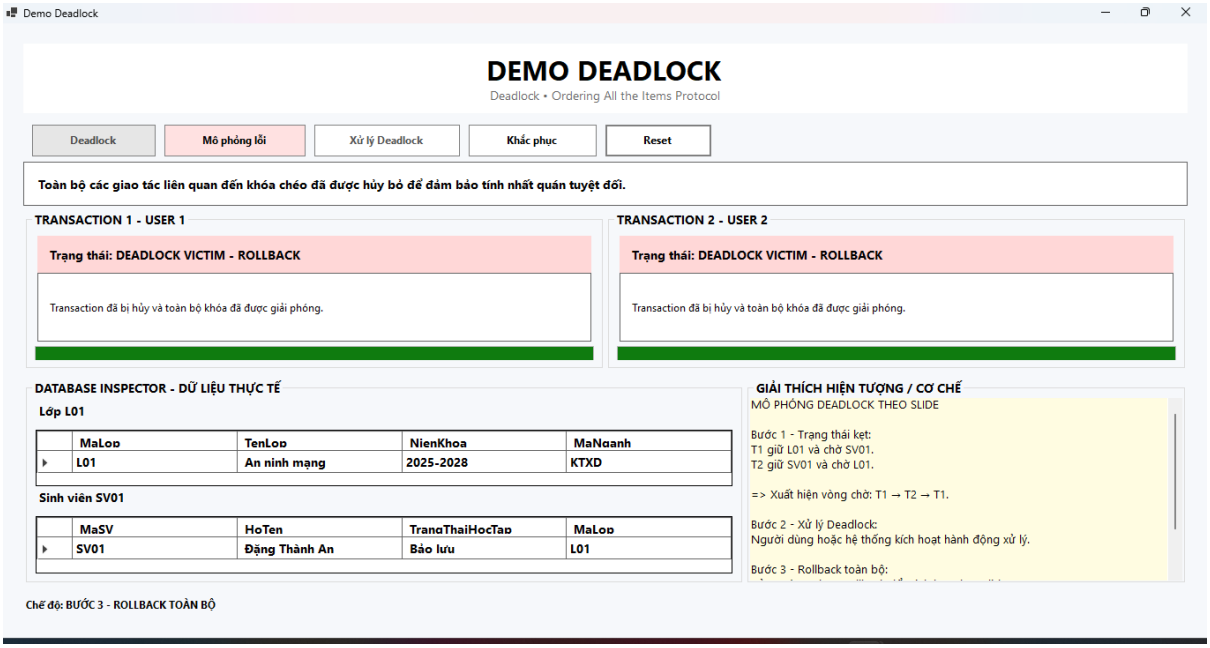
- Bước 2: Bấm Xử lý Deadlock

```
1 reference
private async void btnXuLyDeadlock_Click(
    object sender,
    EventArgs e)
{
    timerKet.Stop();

    btnXuLyDeadlock.Enabled = false;

    lblMoTa.Text =
        "Bước 3: Hệ thống đang rollback toàn bộ các giao tác liên quan đến khóa chéo.";

    lblCheDo.Text =
        "Chế độ: BƯỚC 3 – ROLLBACK TOÀN BỘ";

    lblTrangThaiT1.Text =
        "Trạng thái: Đang ROLLBACK...";

    lblTrangThaiT2.Text =
        "Trạng thái: Đang ROLLBACK...";

    await dal.RollbackCaHaiAsync();

    prgT1.Style =
        ProgressBarStyle.Blocks;

    prgT2.Style =
        ProgressBarStyle.Blocks;

    prgT1.Value = 100;
    prgT2.Value = 100;

    SetRollback(
        lblTrangThaiT1,
        lblChiTietT1);
```

```
    SetRollback(
        lblTrangThaiT2,
        lblChiTietT2);

    const string thongBao =
        "Toàn bộ các giao tác liên quan đến khóa chéo đã được hủy bỏ để đảm bảo tính nhất quán tuyệt đối.";

    GhiLog(
        txtLogT1,
        "T1: DEADLOCK VICTIM – ROLLBACK.");

    GhiLog(
        txtLogT2,
        "T2: DEADLOCK VICTIM – ROLLBACK.");

    GhiLog(
        txtLogT1,
        thongBao);

    GhiLog(
        txtLogT2,
        thongBao);

    lblMoTa.Text =
        thongBao;

    LoadData();
    SetDangChay(false);
}
```

Khi Deadlock được xác định, người dùng nhấn **Xử lý Deadlock**. GUI dừng bộ đếm và gọi phương thức RollbackCaHaiAsync() tại DAL để hủy các transaction đang giữ khóa

chéo. Sau khi rollback, trạng thái của Transaction 1 và Transaction 2 được cập nhật trên giao diện và dữ liệu được tải lại để đảm bảo tính nhất quán.



3.6.2. Khắc phục DEADLOCK - ORDERING ALL THE ITEMS PROTOCOL

Khi bấm nút Khắc phục Deadlock trên Giao diện:

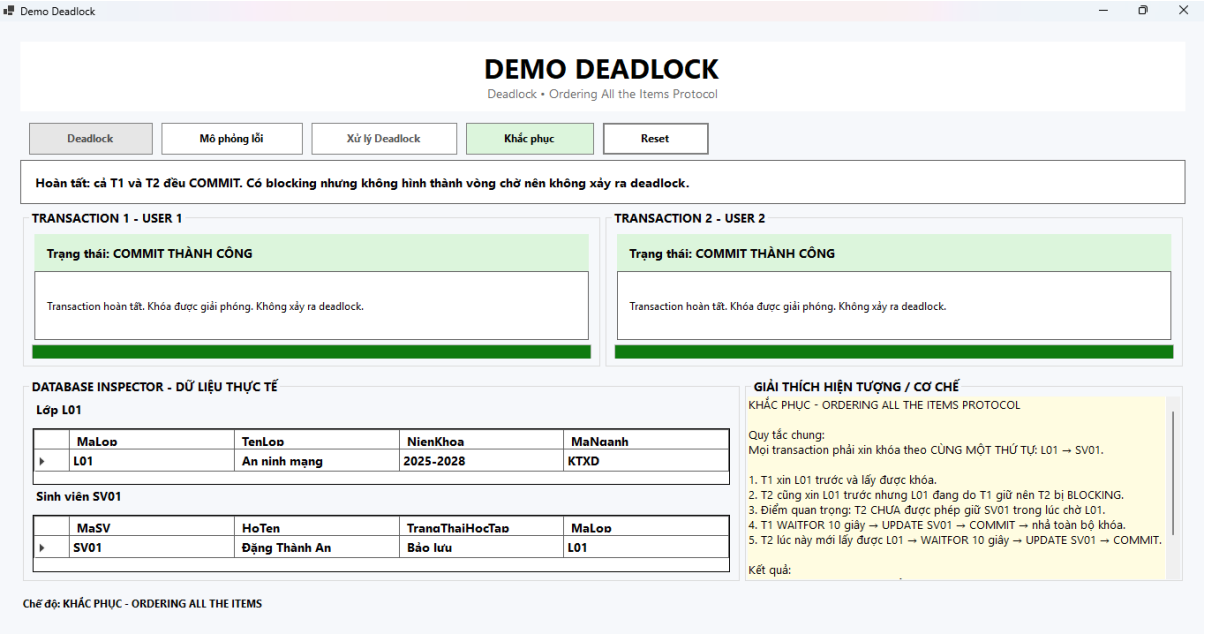
```
private async void btnKhacPhuc_Click(
    object sender,
    EventArgs e)
{
    timerKet.Stop();

    await dal.RollbackCaHaiAsync();

    ChonCheDoKhacPhuc();
    HienThiGiaiThichKhacPhuc();

    SetDangChay(true);
    XoaLog();

    lblCheDo.Text =
        "Chế độ: KHẮC PHỤC – ORDERING ALL THE ITEMS";

    lblMoTa.Text =
        "T1 và T2 cùng tuân theo một thứ tự khóa: L01 → SV01. T2 chờ L01 thay vì giữ SV01 rồi chờ ngược lại.";

    thoiDiemKhacPhuc =
        DateTime.Now;

    BatTrangThaiKhacPhuc();

    timerKhacPhuc.Start();

    GhiLog(
        txtLogT1,
        "BEGIN TRANSACTION → yêu cầu khóa L01.");

    GhiLog(
        txtLogT2,
        "BEGIN TRANSACTION → cũng yêu cầu khóa L01.");
```

```
    try
    {
        string[] ketQua =
            await dal
                .ChayNganDeadlockAsync();

        timerKhacPhuc.Stop();

        CapNhatKetQuaKhacPhuc(
            lblTrangThaiT1,
            lblChiTietT1,
            prgT1,
            ketQua[0]);

        CapNhatKetQuaKhacPhuc(
            lblTrangThaiT2,
            lblChiTietT2,
            prgT2,
            ketQua[1]);

        GhiLog(
            txtLogT1,
            ketQua[0]);

        GhiLog(
            txtLogT2,
            ketQua[1]);

        lblMoTa.Text =
            "Hoàn tất: cả T1 và T2 đều COMMIT. Có blocking nhưng không hình thành vòng chờ nên không xảy ra deadlock.";

        LoadData();
    }
    catch (Exception ex)
    {
        timerKhacPhuc.Stop();

        MessageBox.Show(
            "Lỗi khi chạy khắc phục!\n" +
```

```
                ex.Message);
    }
    finally
    {
        SetDangChay(false);
    }
}

1 reference
private void BatTrangThaiKhacPhuc()
{
    prgT1.Style =
        ProgressBarStyle.Blocks;

    prgT2.Style =
        ProgressBarStyle.Blocks;

    prgT1.Value = 5;
    prgT2.Value = 5;

    lblTrangThaiT1.Text =
        "Trạng thái: T1 ĐÃ LẤY KHÓA L01";

    lblTrangThaiT1.BackColor =
        Color.FromArgb(255, 245, 205);

    lblChiTietT1.Text =
        "BEGIN → UPDATE L01 thành công → đang giữ X-Lock trên L01 → WAITFOR 10 giây → chưa yêu cầu SV01.";

    lblTrangThaiT2.Text =
        "Trạng thái: T2 BLOCKING – ĐANG CHỜ L01";

    lblTrangThaiT2.BackColor =
        Color.FromArgb(255, 245, 205);

    lblChiTietT2.Text =
        "BEGIN → UPDATE L01 chưa thực hiện được vì T1 đang giữ khóa → T2 chưa giữ SV01 → không tạo khóa chéo.";
}
```

Chương trình thực hiện lại thao tác theo phương án phòng tránh Deadlock.

Các transaction được thực hiện theo cùng một thứ tự truy cập tài nguyên. Nhờ đó, các transaction không tạo ra vòng chờ lẫn nhau và hạn chế xảy ra Deadlock.



Giải thích hiện tượng:

Quy tắc chung:

Mọi transaction phải xin khóa theo CÙNG MỘT THỨ TỰ: L01 → SV01.

1. T1 xin L01 trước và lấy được khóa.

2. T2 cũng xin L01 trước nhưng L01 đang do T1 giữ nên T2 bị BLOCKING.

3. Điểm quan trọng: T2 CHƯA được phép giữ SV01 trong lúc chờ L01.

4. T1 WAITFOR 10 giây → UPDATE SV01 → COMMIT → nhả toàn bộ khóa.

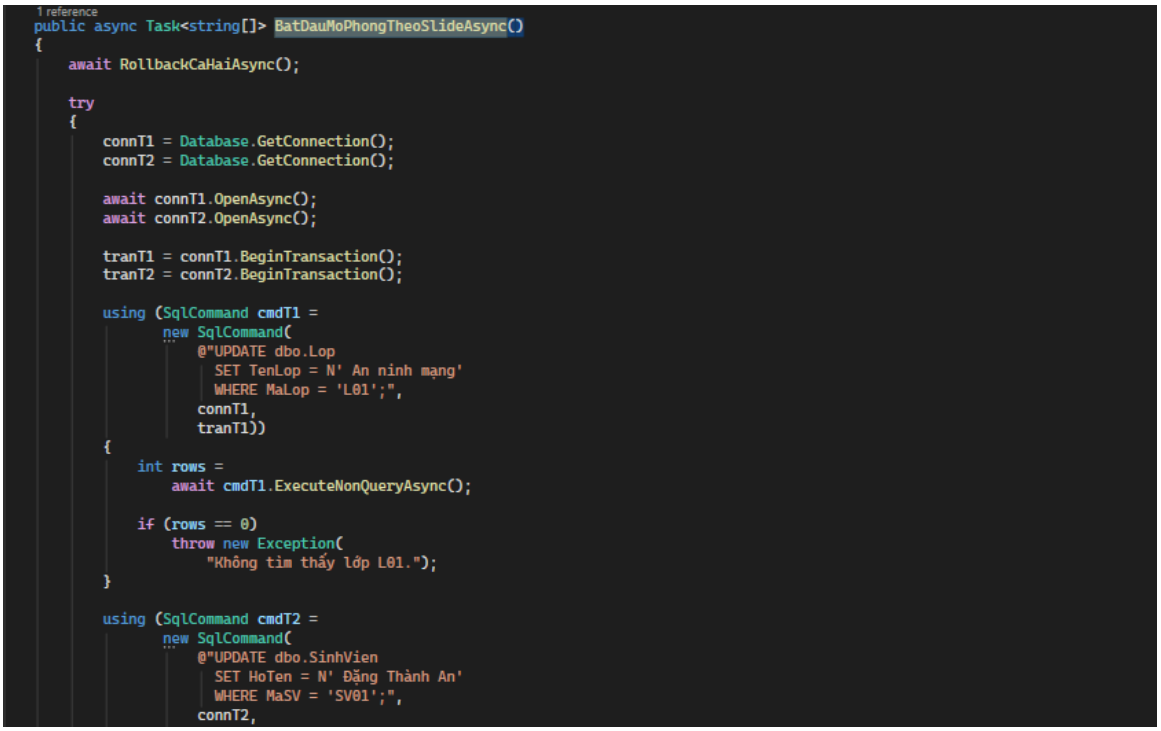
5. T2 lúc này mới lấy được L01 → WAITFOR 10 giây → UPDATE SV01 → COMMIT.

Kết quả:

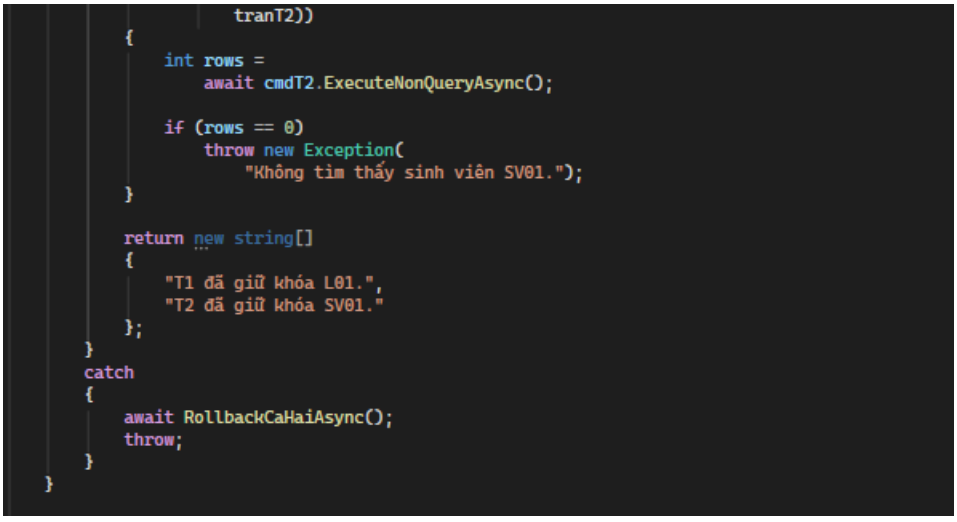
- Có chờ (blocking), nhưng KHÔNG có vòng chờ.
- Không tồn tại T1 chờ T2 đồng thời T2 chờ T1.

=> Không xảy ra deadlock, cả hai transaction đều COMMIT.

DAL



```
1 reference
public async Task<string[]> BatDauMoPhongTheoSlideAsync()
{
    await RollbackCaHaiAsync();

    try
    {
        connT1 = Database.GetConnection();
        connT2 = Database.GetConnection();

        await connT1.OpenAsync();
        await connT2.OpenAsync();

        tranT1 = connT1.BeginTransaction();
        tranT2 = connT2.BeginTransaction();

        using (SqlCommand cmdT1 =
               new SqlCommand(
                   @"UPDATE dbo.Lop
                     SET TenLop = N' An ninh mạng'
                     WHERE MaLop = 'L01';",
                   connT1,
                   tranT1))
        {
            int rows =
                await cmdT1.ExecuteNonQueryAsync();

            if (rows == 0)
                throw new Exception(
                    "Không tìm thấy lớp L01.");
        }

        using (SqlCommand cmdT2 =
               new SqlCommand(
                   @"UPDATE dbo.SinhVien
                     SET HoTen = N' Đặng Thành An'
                     WHERE MaSV = 'SV01';",
                   connT2,
```



```
                   tranT2))
        {
            int rows =
                await cmdT2.ExecuteNonQueryAsync();

            if (rows == 0)
                throw new Exception(
                    "Không tìm thấy sinh viên SV01.");
        }

        return new string[]
        {
            "T1 đã giữ khóa L01.",
            "T2 đã giữ khóa SV01."
        };
    }
    catch
    {
        await RollbackCaHaiAsync();
        throw;
    }
}
```

Tầng DAL tạo hai kết nối cơ sở dữ liệu độc lập tương ứng với Transaction 1 và Transaction 2. Mỗi kết nối bắt đầu một transaction riêng. Transaction 1 thực hiện cập nhật lớp L01 và giữ khóa, trong khi Transaction 2 cập nhật sinh viên SV01 và giữ khóa. Sau đó hai transaction được đặt vào trạng thái chờ tài nguyên của nhau để mô phỏng Deadlock.

DAL - Rollback

```
7 references
public Task RollbackCaHaiAsync()
{
    try
    {
        tranT1?.Rollback();
    }
    catch
    {
    }

    try
    {
        tranT2?.Rollback();
    }
    catch
    {
    }

    tranT1?.Dispose();
    tranT2?.Dispose();

    connT1?.Dispose();
    connT2?.Dispose();

    tranT1 = null;
    tranT2 = null;

    connT1 = null;
    connT2 = null;

    return Task.CompletedTask;
}
```

Phương thức RollbackCaHaiAsync() thực hiện rollback cho cả Transaction 1 và Transaction 2. Sau khi rollback, các transaction và connection được giải phóng và đặt lại về trạng thái ban đầu. Cơ chế này được sử dụng để hủy trạng thái Deadlock và đưa cơ sở dữ liệu về trạng thái nhất quán.

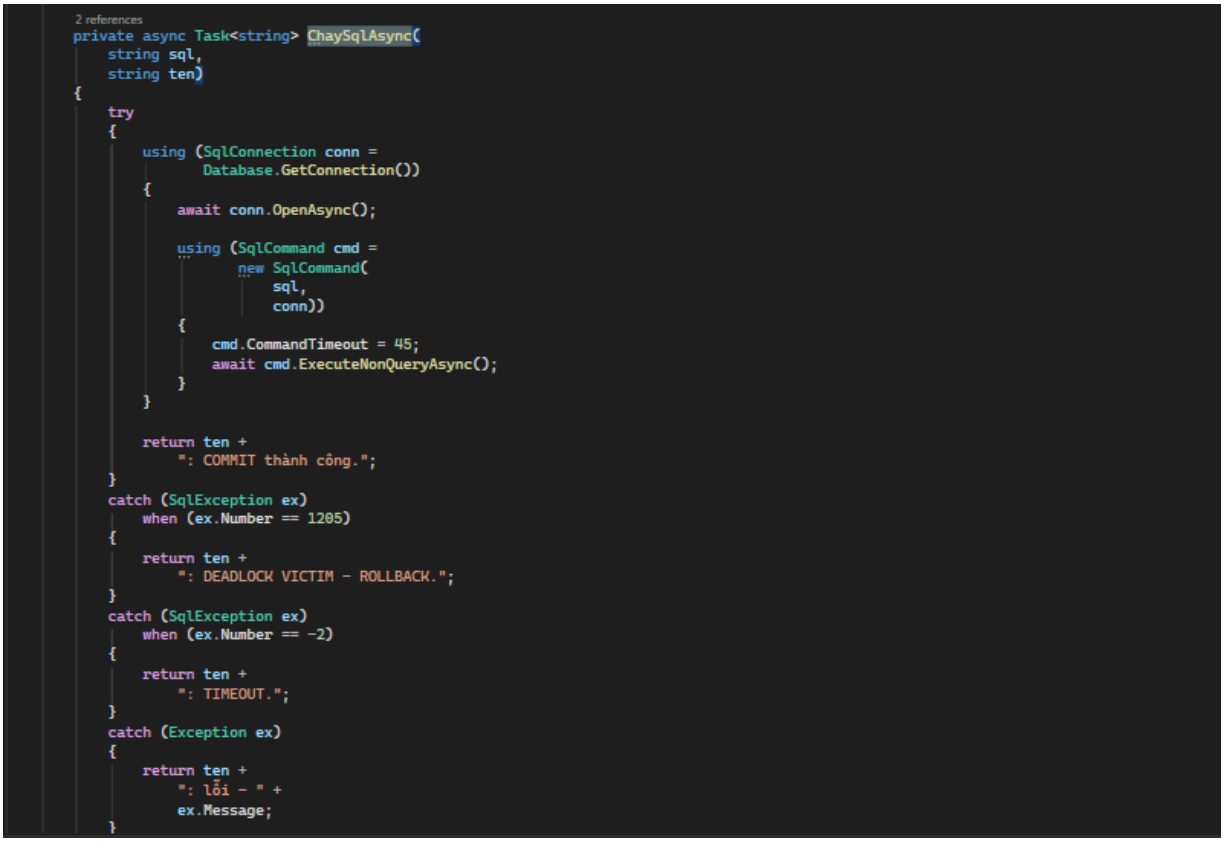
DAL - Khắc phục

```
1 reference
public async Task<string[]> ChayNganDeadlockAsync()
{
    Task<string> t1 =
        ChaySqlAsync(
            @"
            BEGIN TRANSACTION;

            UPDATE dbo.Lop
            SET TenLop = N' An ninh mạng'
            WHERE MaLop = 'L01';

            WAITFOR DELAY '00:00:10';

            UPDATE dbo.SinhVien
            SET TrangThaiHocTap = N' Bảo lưu'
            WHERE MaSV = 'SV01';

            COMMIT TRANSACTION;",
            "T1");
```

```
    Task<string> t2 =
        ChaySqlAsync(
            @"
            BEGIN TRANSACTION;

            UPDATE dbo.Lop
            SET NienKhoa = N'2025-2028'
            WHERE MaLop = 'L01';

            WAITFOR DELAY '00:00:10';

            UPDATE dbo.SinhVien
            SET HoTen = N' Đặng Thành An'
            WHERE MaSV = 'SV01';

            COMMIT TRANSACTION;",
            "T2");

    return await Task.WhenAll(t1, t2);
}
```

Phương thức ChayNganDeadlockAsync() thực hiện hai transaction đồng thời nhưng áp dụng cùng một thứ tự khóa. Cả T1 và T2 đều yêu cầu khóa bảng Lop với mã lớp L01 trước khi tiếp tục thao tác trên bảng SinhVien. Việc thống nhất thứ tự lấy khóa giúp loại bỏ vòng chờ giữa hai transaction và ngăn ngừa Deadlock.

DAL - Bắt Deadlock



```
2 references
private async Task<string> ChaySqlAsync(
    string sql,
    string ten)
{
    try
    {
        using (SqlConnection conn =
               Database.GetConnection())
        {
            await conn.OpenAsync();

            using (SqlCommand cmd =
                   new SqlCommand(
                       sql,
                       conn))
            {
                cmd.CommandTimeout = 45;
                await cmd.ExecuteNonQueryAsync();
            }
        }

        return ten +
            ": COMMIT thành công.";
    }
    catch (SqlException ex)
        when (ex.Number == 1205)
    {
        return ten +
            ": DEADLOCK VICTIM - ROLLBACK.";
    }
    catch (SqlException ex)
        when (ex.Number == -2)
    {
        return ten +
            ": TIMEOUT.";
    }
    catch (Exception ex)
    {
        return ten +
            ": lỗi - " +
            ex.Message;
    }
```

Trong quá trình thực thi SQL, DAL bắt lỗi SqlException với mã lỗi 1205, là lỗi Deadlock của SQL Server. Khi một transaction bị SQL Server chọn làm Deadlock Victim, hệ thống trả về thông báo **DEADLOCK VICTIM - ROLLBACK**. Ngoài ra, DAL cũng xử lý trường hợp Timeout để tránh chương trình bị treo khi giao tác chờ quá lâu.

SQL

```sql
-- T1. KHÓA LỚP
----------------
begin transaction;

-- Khóa Lớp 'L01'
update Lop
set TenLop = N' An ninh mạng'
where MaLop = 'L01';

-- Giữ khóa 10 giây
waitfor delay '00:00:10';

-- Sau đó mới khóa SinhVien 'SV01' để cập nhật trạng thái
update SinhVien
set TrangThaiHocTap = N' Bảo lưu'
where MaSV = 'SV01';

commit transaction;

```

Transaction 1 bắt đầu giao tác và khóa bản ghi lớp L01 thông qua câu lệnh UPDATE. Sau đó sử dụng WAITFOR DELAY để giữ khóa trong một khoảng thời gian trước khi tiếp tục yêu cầu khóa sinh viên SV01. Trong thời gian này, T1 giữ tài nguyên L01 và chờ tài nguyên SV01.

```sql
----------------------
-- T2. KHÓA SINH VIÊN
----------------------
begin transaction;

-- Khóa Sinh Viên 'SV01'
update SinhVien
set HoTen = N' Đặng Thành An'
where MaSV = 'SV01';

-- Giữ khóa 10 giây
waitfor delay '00:00:10';

-- Sau đó T2 khóa 'L01' để điều chỉnh niên khóa
update Lop
set NienKhoa = N'2025-2028'
where MaLop = 'L01';

commit transaction;

```

Transaction 2 bắt đầu giao tác và khóa bản ghi sinh viên SV01 trước. Sau đó sử dụng WAITFOR DELAY để giữ khóa trước khi tiếp tục yêu cầu khóa lớp L01. Do T1 đang giữ L01 và T2 đang giữ SV01, hai transaction chờ tài nguyên của nhau, tạo thành vòng chờ và dẫn đến Deadlock.

Khắc phục

```
-- NGĂN NGỪA DEADLOCK THEO GIAO THỨC
-- Ordering All the Items Protocol
----------------------------------------

----------------
-- T1. KHÓA LỚP
----------------
begin transaction;

-- Khóa Lớp 'L01'
update Lop
set TenLop = N' An ninh mạng'
where MaLop = 'L01';

-- Giữ khóa 10 giây
waitfor delay '00:00:10';

-- Sau đó mới khóa SinhVien 'SV01' để cập nhật trạng thái
update SinhVien
set TrangThaiHocTap = N' Bảo lưu'
where MaSV = 'SV01';

commit transaction;
```

```

----------------------
-- T2. KHÓA SINH VIÊN
----------------------
begin transaction;

-- T2 cũng phải khóa Lớp 'L01' trước
update Lop
set NienKhoa = N'2025-2028'
where MaLop = 'L01';

-- Giữ khóa 10 giây
waitfor delay '00:00:10';

-- Sau đó mới khóa SinhVien 'SV01' để điều chỉnh họ tên
update SinhVien
set HoTen = N' Đặng Thành An'
where MaSV = 'SV01';

commit transaction;
```

Để ngăn ngừa Deadlock, hệ thống áp dụng giao thức **Ordering All the Items**, trong đó tất cả các transaction phải truy cập tài nguyên theo cùng một thứ tự. Cả T1 và T2 đều khóa L01 trước, sau đó mới thực hiện thao tác trên SV01. Vì không còn trường hợp T1 giữ L01 chờ SV01 trong khi T2 giữ SV01 chờ L01 nên vòng chờ được loại bỏ và Deadlock không xảy ra.

TỔNG KẾT

Qua quá trình nghiên cứu và thực hiện đồ án về **Quản lý Hồ Sơ Sinh viên**, nhóm đã tích lũy và đúc kết được nhiều kinh nghiệm thực tiễn quan trọng:

- **Về mặt kiến thức chuyên môn:** Đồ án là cơ hội để em vận dụng tổng hợp các kiến thức đã học vào một bài toán thực tế. Cụ thể, giúp nhóm nắm vững hơn về quy trình phân tích bài toán, thiết kế mô hình dữ liệu, cũng như cách ứng dụng thành thạo các đối tượng nâng cao trong cơ sở dữ liệu như View, Stored Procedure, Function và Trigger để tối ưu hóa hiệu năng và bảo mật dữ liệu.
- **Về kỹ năng xử lý vấn đề phức tạp:** Đặc biệt thông qua chương quản lý giao tác và điều khiển tương tranh, em đã hiểu rõ sâu sắc bản chất và cách khắc phục các hiện tượng tranh chấp dữ liệu thực tế như Lost Update, Dirty Read, Non-Repeatable Read, Phantom Read và Deadlock. Điều này giúp nhóm có tư duy cẩn trọng hơn trong việc thiết kế hệ thống có tính đồng thời cao.
- **Về kỹ năng lập trình và phát triển phần mềm:** Quá trình hiện thực hóa các tính năng từ cơ sở dữ liệu lên giao diện code đã giúp trong quá trình học, rèn luyện tư duy tổ chức mã nguồn, quản lý lỗi và hoàn thiện quy trình xây dựng một ứng dụng hoàn chỉnh.

Mặc dù đồ án vẫn còn những điểm cần cải thiện do giới hạn về thời gian và kiến thức thực tế, những gì học hỏi được qua đề tài này là hành trang quý giá, tạo nền tảng vững chắc cho quá trình học tập và phát triển nghề nghiệp của bản thân trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Trường Đại học Giao thông Vận tải TP.HCM - Viện Đào tạo Chất lượng cao. Bài giảng Hệ quản trị Cơ sở dữ liệu, tài liệu học phần Cơ sở dữ liệu.

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.

[3] Date, C. J. (2003). *An introduction to database systems* (8th ed.). Addison-Wesley.

[4] Microsoft. (n.d.). *Transact-SQL (T-SQL) reference*. Microsoft Learn. Truy cập từ https://learn.microsoft.com/sql/t-sql/language-reference

[5] Microsoft. (n.d.). *Cursors (Transact-SQL)*. Microsoft Learn. Truy cập từ https://learn.microsoft.com/sql/t-sql/language-elements/cursors-transact-sql

[6] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of database systems* (7th ed.). Pearson.

[7] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed., Chương 17 "Transactions" & Chương 18 "Concurrency Control"). McGraw-Hill Education.

[8] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of database systems* (7th ed., chương "Introduction to Transaction Processing Concepts and Theory"). Pearson.

[9] Ramakrishnan, R., & Gehrke, J. (2003). *Database management systems* (3rd ed., chương "Concurrency Control"). McGraw-Hill.

[10] VOER. (n.d.). *Giao dịch cơ sở dữ liệu (Transaction)*. Tài liệu dịch/diễn giải từ Elmasri-Navathe.

[11] Wikipedia. (n.d.). *Serializability*. Truy cập từ https://en.wikipedia.org/wiki/Serializability

[12] Microsoft. (n.d.). *Transact-SQL reference (BEGIN TRANSACTION, COMMIT TRANSACTION, ROLLBACK TRANSACTION)*. Microsoft Learn.

[13] Microsoft. (n.d.). *Views - SQL Server*. Microsoft Learn. Truy cập từ https://learn.microsoft.com/en-us/sql/relational-databases/views/views

[14] Microsoft. (n.d.). *CREATE VIEW (Transact-SQL)*. Microsoft Learn. Truy cập từ https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql

[15] Microsoft. (n.d.). *ALTER VIEW (Transact-SQL)*. Microsoft Learn. Truy cập từ https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-view-transact-sql

[16] Microsoft. (n.d.). *Create indexed views*. Microsoft Learn. Truy cập từ https://learn.microsoft.com/en-us/sql/relational-databases/views/create-indexed-views

[17] Thanh Huyen. (2026). *PROCEDURE (Thủ tục) trong SQL Server*. QuanTriMang.com. https://quantrimang.com/hoc/procedure-thu-tuc-trong-sql-server-159768

[18] YouTube. (2026). *Video hướng dẫn về Function trong SQL Server*. https://www.youtube.com/watch?v=7nReuUzyqP0&feature=youtu.be

[19] TITV. (2023). *Triggers trong SQL Server* [Video]. YouTube. https://www.youtube.com/watch?v=wtGOsoFzU_I

[20] Phạm, A. T. (2018). *Sử dụng trigger trong SQL qua ví dụ cơ bản*. Viblo.

https://viblo.asia/p/su-dung-trigger-trong-sql-qua-vi-du-co-ban-aWj538APK6m

[21] TopDev. (n.d.). *Trigger trong SQL: Cú pháp và cách tạo Trigger SQL*. TopDev.

https://topdev.vn/blog/trigger-trong-sql/#cac-loai-trigger-trong-sql

[22] Microsoft. (n.d.). *CREATE FUNCTION (Transact-SQL)*. Microsoft Learn.

https://learn.microsoft.com/en-us/sql/t-sql/statements/create-function-transact-sql?view=sql-server-ver17

[23] Kteam. (2021). *Function trong SQL Server* [Video]. YouTube.

https://howkteam.vn/course/truy-van-voi-dieu-kien-trong-sql-server/function-trong-sql-server-1411

[24] Microsoft. (2026). *Transaction locking and row versioning guide*. Microsoft Learn.

https://learn.microsoft.com/fil-ph/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver17

[25] Microsoft. (2026). *Deadlocks guide*. Microsoft Learn.

https://learn.microsoft.com/vi-vn/sql/relational-databases/sql-server-guides?view=sql-server-ver17

[26] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2006). *The two-phase locking protocol* [Slide presentation]. Database System Concepts, 5th ed., McGraw-Hill.

https://www.db-book.com/Previous-editions/db5/slide-dir/ch16.pdf